# SENTIMENT ANALYSIS BASED ON APPRAISAL THEORY AND FUNCTIONAL LOCAL GRAMMARS

BY

KENNETH BLOOM

Submitted in partial fulfillment of the
requirements for the degree of
Doctor of Philosophy in Computer Science
in the Graduate College of the
Illinois Institute of Technology

Approved _____
Advisor

Chicago, Illinois
December 2011

© Copyright by

KENNETH BLOOM

December 2011

ACKNOWLEDGMENT

I am thankful to God for having given me the ability to complete this thesis, and for providing me with the many insights that I present in this thesis. All of a person's ability to achieve anything in the world is only granted by the grace of God, as it is written "and you shall remember the Lord your God, because it is he who gives you the power to succeed." (Deuteronomy 8:18)

I am thankful to my advisor Dr. Shlomo Argamon, for suggesting that I attend IIT in the first place, for all of the discussions about concepts and techniques in sentiment analysis (and for all of the rides to and from IIT where we discussed these things), for all of the drafts he's reviewed, and for the many other ways that he's helped that I have not mentioned here.

I am thankful to the members of both my proposal and thesis committees, for their advice about my research: Dr. Kathryn Riley, Dr. Ophir Frieder, Dr. Nazli Goharian, Dr. Xiang-Yang Li, Dr. Mustafa Bilgic, and Dr. David Grossman.

I am thankful to my colleagues — the other students in my lab, and elsewhere in the computer science department — with whom I have worked closely over the last 6 years, and had many opportunities to discuss research ideas and software development techniques for completing this thesis: Navendu Garg and Dr. Casey Whitelaw (whose 2005 paper "Using Appraisal Taxonomies for Sentiment Analysis" is the basis for many ideas in this dissertation), Mao-jian Jiang (who proposed a project related to my own as his own thesis research), Sterling Stein, Paul Chase, Rodney Summerscales, Alana Platt, and Dr. Saket Mengle. I am also thankful to Michael Fabian, whom I trained to annotate the IIT sentiment corpus, and through the training process helped to clarify the annotation guidelines for the corpus.

I am thankful to Rabbi Avraham Rockmill and Rabbi Michael Azose, who at a particularly difficult time in my graduate school career advised me not to give up; to come back to Chicago and finish my doctorate. I am thankful to all of my friends in Chicago who have helped me to make it to the end of this process. I will miss you all.

Lastly, I am thankful to my parents for their support, particularly my father, Dr. Jeremy Bloom, for his very valuable advice about managing my workflow to complete this thesis.

TABLE OF CONTENTS

Page

ACKNOWLEDGEMENT . . . . . iii

LIST OF TABLES . . . . . viii

LIST OF FIGURES . . . . . x

LIST OF ALGORITHMS . . . . . xii

ABSTRACT . . . . . xiii

CHAPTER

1. INTRODUCTION . . . . . 1

1.1. Sentiment Classification versus Sentiment Extraction . . . 3
1.2. Structured Opinion Extraction . . . . . 6
1.3. Evaluating Structured Opinion Extraction . . . . . 9
1.4. FLAG: Functional Local Appraisal Grammar Extractor . . 11
1.5. Appraisal Theory in Sentiment Analysis . . . . . 14
1.6. Structure of this dissertation . . . . . 16

2. PRIOR WORK . . . . . 17

2.1. Applications of Sentiment Analysis . . . . . 17
2.2. Evaluation and other kinds of subjectivity . . . . . 18
2.3. Review Classification . . . . . 20
2.4. Sentence classification . . . . . 22
2.5. Structural sentiment extraction techniques . . . . . 25
2.6. Opinion lexicon construction . . . . . 31
2.7. The grammar of evaluation . . . . . 33
2.8. Local Grammars . . . . . 42
2.9. Barnbrook's COBUILD Parser . . . . . 47
2.10. FrameNet labeling . . . . . 50
2.11. Information Extraction . . . . . 51

3. FLAG'S ARCHITECTURE . . . . . 57

3.1. Architecture Overview . . . . . 57
3.2. Document Preparation . . . . . 59

CHAPTER Page

4. THEORETICAL FRAMEWORK . . . . . 63

4.1. Appraisal Theory . . . . . 63
4.2. Lexicogrammar . . . . . 71
4.3. Summary . . . . . 76

5. EVALUATION RESOURCES . . . . . 78

5.1. MPQA 2.0 Corpus . . . . . 79
5.2. UIC Review Corpus . . . . . 84
5.3. Darmstadt Service Review Corpus . . . . . 89
5.4. JDPA Sentiment Corpus . . . . . 93
5.5. IIT Sentiment Corpus . . . . . 99
5.6. Summary . . . . . 105

6. LEXICON-BASED ATTITUDE EXTRACTION . . . . . 106

6.1. Attributes of Attitudes . . . . . 106
6.2. The FLAG appraisal lexicon . . . . . 109
6.3. Baseline Lexicons . . . . . 115
6.4. Appraisal Chunking Algorithm . . . . . 116
6.5. Sequence Tagging Baseline . . . . . 118
6.6. Summary . . . . . 122

7. THE LINKAGE EXTRACTOR . . . . . 124

7.1. Do All Appraisal Expressions Fit in a Single Sentence? . . 124
7.2. Linkage Specifications . . . . . 128
7.3. Operation of the Associator . . . . . 132
7.4. Example of the Associator in Operation . . . . . 134
7.5. Summary . . . . . 138

8. LEARNING LINKAGE SPECIFICATIONS . . . . . 139

8.1. Hunston and Sinclair's Linkage Specifications . . . . . 139
8.2. Additions to Hunston and Sinclair's Linkage Specifications 140
8.3. Sorting Linkage Specifications by Specificity . . . . . 140
8.4. Finding Linkage Specifications . . . . . 147
8.5. Using Ground Truth Appraisal Expressions as Candidates . 150
8.6. Heuristically Generating Candidates from Unannotated Text 152
8.7. Filtering Candidate Appraisal Expressions . . . . . 153
8.8. Selecting Linkage Specifications by Individual Performance 155
8.9. Selecting Linkage Specifications to Cover the Ground Truth 157
8.10. Summary . . . . . 157

CHAPTER Page

9. DISAMBIGUATION OF MULTIPLE INTERPRETATIONS . . 159

9.1. Ambiguities from Earlier Steps of Extraction . . . . . 159
9.2. Discriminative Reranking . . . . . 162
9.3. Applying Discriminative Reranking in FLAG . . . . . 164
9.4. Summary . . . . . 167

10. EVALUATION OF PERFORMANCE . . . . . 168

10.1. General Principles . . . . . 168
10.2. Attitude Group Extraction Accuracy . . . . . 173
10.3. Linkage Specification Sets . . . . . 178
10.4. Does Learning Linkage Specifications Help? . . . . . 181
10.5. The Document Emphasizing Processes and Superordinates 186
10.6. The Effect of Attitude Type Constraints and Rare Slots . . 187
10.7. Applying the Disambiguator . . . . . 188
10.8. The Disambiguator Feature Set . . . . . 190
10.9. End-to-end extraction results . . . . . 193
10.10. Learning Curve . . . . . 197
10.11. The UIC Review Corpus . . . . . 201

11. CONCLUSION . . . . . 204

11.1. Appraisal Expression Extraction . . . . . 204
11.2. Sentiment Extraction in Non-Review Domains . . . . . 205
11.3. FLAG's Operation . . . . . 206
11.4. FLAG's Best Configuration . . . . . 208
11.5. Directions for Future Research . . . . . 209

APPENDIX . . . . . 212

A. READING A SYSTEM DIAGRAM IN SYSTEMIC FUNCTIONAL LINGUISTICS . . . . . 212

A.1. A Simple System . . . . . 213
A.2. Simultaneous Systems . . . . . 214
A.3. Entry Conditions . . . . . 215
A.4. Realizations . . . . . 216

B. ANNOTATION MANUAL FOR THE IIT SENTIMENT CORPUS 217

B.1. Introduction . . . . . 218
B.2. Attitude Groups . . . . . 218
B.3. Comparative Appraisals . . . . . 228
B.4. The Target Structure . . . . . 232

APPENDIX Page

B.5. Evaluator . . . . . 239
B.6. Which Slots are Present in Different Attitude Types? . . . 244
B.7. Using Callisto to Tag . . . . . 247
B.8. Summary of Slots to Extract . . . . . 248
B.9. Tagging Procedure . . . . . 248

BIBLIOGRAPHY . . . . . 250

LIST OF TABLES


| Table | | Page |
|---|---|---|
| 2.1 | Comparison of reported results from past work in structured opinion extraction | 27 |
| 5.1 | Mismatch between Hu and Liu's reported corpus statistics, and what's actually present. | 89 |
| 6.1 | Manually and Automatically Generated Lexicon Entries. | 114 |
| 6.2 | Accuracy of SentiWordNet at Recreating the General Inquirer's Positive and Negative Word Lists. | 117 |
| 10.1 | Accuracy of Different Methods for Finding Attitude Groups on the IIT Sentiment Corpus. | 175 |
| 10.2 | Accuracy of Different Methods for Finding Attitude Groups on the Darmstadt Corpus. | 175 |
| 10.3 | Accuracy of Different Methods for Finding Attitude Groups on the JDPA Corpus. | 175 |
| 10.4 | Accuracy of Different Methods for Finding Attitude Groups on the MPQA Corpus. | 176 |
| 10.5 | Performance of Different Linkage Specification Sets on the IIT Sentiment Corpus. | 182 |
| 10.6 | Performance of Different Linkage Specification sets on the Darmstadt and JDPA Corpora. | 182 |
| 10.7 | Performance of Different Linkage Specification Sets on the MPQA Corpus. | 185 |
| 10.8 | Comparison of Performance when the Document Focusing on Appraisal Expressions with Superordinates and Processes is Omitted. | 186 |
| 10.9 | The Effect of Attitude Type Constraints and Rare Slots in Linkage Specifications on the IIT Sentiment Corpus. | 187 |
| 10.10 | The Effect of Attitude Type Constraints and Rare Slots in Linkage Specifications on the Darmstadt, JDPA, and MPQA Corpora. | 188 |
| 10.11 | Performance with the Disambiguator on the IIT Sentiment Corpus. | 189 |
| 10.12 | Performance with the Disambiguator on the Darmstadt Corpus. | 189 |
| 10.13 | Performance with the Disambiguator on the JDPA Corpora. | 190 |

Table Page

10.14 Performance with the Disambiguator on the IIT Sentiment Corpus. 191

10.15 Performance with the Disambiguator on the Darmstadt Corpus. . . 191

10.16 Performance with the Disambiguator on the JDPA Corpus. . . . . 192

10.17 Incidence of Extracted Attitude Types in the IIT, JDPA, and Darmstadt Corpora. . . . . 193

10.18 End-to-end Extraction Results on the IIT Sentiment Corpus . . . 194

10.19 End-to-end Extraction Results on the Darmstadt and JDPA Corpora 195

10.20 FLAG's results at finding evaluators and targets compared to similar NTCIR subtasks. . . . . 197

10.21 Accuracy at finding distinct product feature mentions in the UIC review corpus . . . . . 202

B.1 How to tag multiple appraisal expressions with conjunctions. . . . 248

LIST OF FIGURES


| Figure | | Page |
|---|---|---|
| 2.1 | Types of attitudes in the MPQA corpus version 2.0 | 34 |
| 2.2 | Examples of patterns for evaluative language in Hunston and Sinclair's [72] local grammar. | 37 |
| 2.3 | Evaluative parameters in Bednarek's theory of evaluation | 40 |
| 2.4 | Opinion Categories in Asher et. al's theory of opinion in discourse | 41 |
| 2.5 | A dictionary entry in Barnbrook's local grammar | 45 |
| 3.1 | FLAG system architecture | 57 |
| 3.2 | Different kinds of dependency parses used by FLAG. | 62 |
| 4.1 | The Appraisal system | 65 |
| 4.2 | Martin and White's subtypes of Affect versus Bednarek's | 69 |
| 4.3 | The Engagement system | 70 |
| 5.1 | Types of attitudes in the MPQA corpus version 2.0 | 80 |
| 5.2 | An example review from the UIC Review Corpus. The left column lists the product features and their evaluations, and the right column gives the sentences from the review. | 86 |
| 5.3 | Inconsistencies in the UIC Review Corpus | 88 |
| 6.1 | An intensifier increases the force of an attitude group. | 107 |
| 6.2 | The *attitude type* taxonomy used in FLAG's appraisal lexicon. | 110 |
| 6.3 | A sample of entries in the attitude lexicon. | 111 |
| 6.4 | Shallow parsing the attitude group "not very happy". | 118 |
| 6.5 | Structure of the MALLET CRF extraction model. | 119 |
| 7.1 | Three example linkage specifications | 129 |
| 7.2 | Dependency parse of the sentence "It was an interesting read." | 135 |
| 7.3 | Phrase structure parse of the sentence "It was an interesting read." | 135 |
| 7.4 | Appraisal expression candidates found in the sentence "It was an interesting read." | 138 |

Figure Page


8.1 "The Matrix is a good movie" matches two different linkage specifications . . . . . 141

8.2 Finite state machine for comparing two linkage specifications $a$ and $b$ within a strongly connected component. . . . . 143

8.3 Three isomorphic linkage specifications. . . . . 145

8.4 Word correspondences in three isomorphic linkage specifications. . 145

8.5 Final graph for sorting the three isomorphic linkage specifications. 145

8.6 Operation of the linkage specification learner when learning from ground truth annotations . . . . . 151

8.7 The patterns of appraisal components that can be put together into an appraisal expression by the unsupervised linkage learner. . . . 154

8.8 Operation of the linkage specification learner when learning from a large unlabeled corpus . . . . . 154

9.1 Ambiguity in word-senses for the word 'good' . . . . . 160

9.2 Ambiguity in word-senses for the word 'devious' . . . . . 161

9.3 "The Matrix is a good movie" under two different linkage patterns 161

9.4 WordNet hypernyms of interest in the reranker. . . . . 165

10.1 Learning curve on the IIT sentiment corpus . . . . . 198

10.2 Learning curve on the Darmstadt corpus . . . . . 199

10.3 Learning curve on the IIT sentiment corpus with the disambiguator 200

B.1 Attitude Types that you will be tagging are marked in bold, with the question that defines each attitude type. . . . . 223

LIST OF ALGORITHMS

Algorithm Page

7.1 Algorithm for turning attitude groups into appraisal expression candidates . . . . . 132

8.1 Algorithm for topologically sorting linkage specifications . . . . . 142

8.2 Algorithm for learning a linkage specification from a candidate appraisal expression. . . . . 149

8.3 Covering algorithm for scoring appraisal expressions . . . . . 158

# ABSTRACT

Much of the past work in structured sentiment extraction has been evaluated in ways that summarize the output of a sentiment extraction technique for a particular application. In order to get a true picture of how accurate a sentiment extraction system is, however, it is important to see how well it performs at finding individual mentions of opinions in a corpus.

Past work also focuses heavily on mining opinion/product-feature pairs from product review corpora, which has lead to sentiment extraction systems assuming that the documents they operate on are review-like — that each document concerns only one topic, that there are lots of reviews on a particular product, and that the product features of interest are frequently recurring phrases.

Based on existing linguistics research, this dissertation introduces the concept of an appraisal expression, the basic grammatical unit by which an opinion is expressed about a target. The IIT sentiment corpus, intended to present an alternative to both of these assumptions that have pervaded sentiment analysis research, consists of blog posts annotated with appraisal expressions to enable the evaluation of how well sentiment analysis systems find individual appraisal expressions.

This dissertation introduces FLAG, an automated system for extracting appraisal expressions. FLAG operates using a three step process: (1) identifying attitude groups using a lexicon-based shallow parser, (2) identifying potential structures for the rest of the appraisal expression by identifying patterns in a sentence's dependency parse tree, (3) selecting the best appraisal expression for each attitude group using a discriminative reranker. FLAG achieves an overall accuracy of 0.261 $F_1$ at identifying appraisal expressions, which is good considering the difficulty of the task.

# CHAPTER 1
# INTRODUCTION

Many traditional data mining tasks in natural language processing focus on extracting data from documents and mining it according to topic. In recent years, the natural language community has recognized the value in analyzing opinions and emotions expressed in free text. Sentiment analysis is the task of having computers automatically extract and understand the opinions in a text.

Sentiment analysis has become a growing field for commercial applications, with at least a dozen companies offering products and services for sentiment analysis, with very different sets of goals and capabilities. Some companies (like tweetfeel.com and socialmention.com) are focused on searching particular social media to find to find posts about a particular query and categorizing the posts as positive or negative. Other companies (like Attensity and Lexalytics) have more sophisticated offerings that recognize opinions and the entities that those opinions are about. The Attensity Group [10] lays out a number of important dimensions of sentiment analysis that their offering covers, among them identifying opinions in text, identifying the "voice" of the opinions, discovering the specific topics that a corporate client will be interested in singling out related to their brand or product, identifying current trends, and predicting future trends.

Early applications of sentiment analysis focused on classifying movie reviews or product reviews as positive or negative or identifying positive and negative sentences, but many recent applications involve opinion mining in ways that require a more detailed analysis of the sentiment expressed in texts. One such application is to use opinion mining to determine areas of a product that need to be improved by summarizing product reviews to see what parts of the product are generally considered good or bad by users. Another application requiring a more detailed analysis of

sentiment is to understand where political writers fall on the political spectrum, something that can only be done by looking at support or opposition to specific policies. A couple other of applications, like allowing politicians who want a better understanding of how their constituents view different issues, or predicting stock prices based on opinions that people have about the companies and resources involved the marketplace, can similarly take advantage of structured representations of opinion. These applications can be tackled with a structured approach to opinion extraction.

Sentiment analysis researchers are currently working on creating the techniques to handle these more complicated problems, defining the structure of opinions and the techinques to extract the structure of opinions. However, many of these efforts have been lacking. The techniques used to extract opinions have become dependent on certain assumptions that stem from the fact that researchers are testing their techniques on corpora of product reviews. These assumptions mean that these techniques won't work as well on other genres of opinionated texts. Additionally, the representation of opinions that most researchers have been assuming is too coarse grained and inflexible to capture all of the information that's available in opinions, which has led to inconsistencies in how human annotators tag the opinions in the most commonly used sentiment corpora.

The goal of this disseration is to redefine the problem of structured sentiment analysis, to recognize and eliminate the assumptions that have been made in previous research, and to analyze opinions in a fine-grained way that will allow more progress to be made in the field. The problems currently found in sentiment analysis, and the approach introduced in this disseration are described more fully in the following sections.

## 1.1 Sentiment Classification versus Sentiment Extraction

To understand the additional information that can be obtained by identifying structured representations of opinions, consider an example of a classification task, typical of the kinds of opinion summarization applications performed today — movie review classification. In movie review classification, the goal is to determine whether the reviewer liked the movie based on the text of the review. This task was a popular starting point for sentiment analysis research, since it was easy to construct corpora from product review websites and movie review websites by turning the number of stars on the review into class labels indicating that the review conveyed overall positive or negative sentiment. Pang et al. [134] achieved 82.9% accuracy at classifying movie reviews as positive or negative using Support Vector Machine classification with a simple bag-of-words feature set. In a bag-of-words technique, the classifier identifies single-word opinion clues and weights them according to their ability to help classify reviews as positive or negative.

While 82.9% accuracy is a respectable result for this task, there are many aspects of sentiment that the bag-of-words representation cannot cover. It cannot account for the effect of the word "not," which turns formerly important indicators of positive sentiment into indicators of negative sentiment. It also cannot account for comparisons between the product being reviewed and other products. It cannot account for other contextual information about the opinions in a review, like recognizing that the sentence "The Lost World was a good book, but a bad movie" contributes a negative opinion clue when it appears in a movie review of the Steven Spielberg movie, but contributes a positive clue when it appears in a review of the Michael Crichton novel. It cannot account for opinion words set off with modality or a subjunctive (e.g. "I would have liked it if this camera had aperture control.") In order to work with these aspects of sentiment and enable more complicated sentiment

tasks, it is necessary to use structured approaches to sentiment that can capture these kinds of things.

One seeking to understand sentiment in political texts, for example, needs to understand not just whether a positive opinion is being conveyed, but also what that opinion is about. Consider, for example, this excerpt from a New York Times editorial about immigration laws [127]:

> The Alabama Legislature opened its session on March 1 on a note of humility and compassion. In the Senate, a Christian pastor asked God to grant members "wisdom and discernment" to do what is right. "Not what's right in their own eyes," he said, "but what's right according to your word." Soon after, both houses passed, and the governor signed, the country's cruelest, most unforgiving immigration law.
>
> The law, which takes effect Sept. 1, is so inhumane that four Alabama church leaders — an Episcopal bishop, a Methodist bishop and a Roman Catholic archbishop and bishop — have sued to block it, saying it criminalizes acts of Christian compassion. It is a sweeping attempt to terrorize undocumented immigrants in every aspect of their lives, and to make potential criminals of anyone who may work or live with them or show them kindness.
>
> ...
>
> Congress was once on the brink of an ambitious bipartisan reform that would have enabled millions of immigrants stranded by the failed immigration system to get right with the law. This sensible policy has been abandoned. We hope the church leaders can waken their fellow Alabamans to the moral damage done when forgiveness and justice are so ruthlessly denied. We hope Washington and the rest of the country will also listen.

The first part of this editorial speaks negatively about an immigration law passed by the state of Alabama, while the latter part speaks positively about a failed attempt by the United States Congress to pass a law about immigration. There is a lot of specific opinion information available in this editorial. In the first and second paragraphs, there are several negative evaluations of Alabama's immigration law ("the country's cruelest", "most unforgiving", "inhumane"), as well as information ascribing a particular emotional reaction ("terrorizes") to the law's victims. In the

last paragraph, there is a positive evaluation about a proposed federal immigration law ("sensible policy"), as well a negative evaluation of the current "failed immigration system", and a negative evaluation of of Alabama's law ascribed to "church leaders."

With this information, it's possible to solve many more complicated sentiment tasks. Consider a particular application where the goal is to determine which political party the author of the editorial aligns himself with. Actors across the political spectrum have varying opinions on both laws in this editorial, so it is not enough to determine that there is positive or negative sentiment in the editorial. Even when combined with topical text classification to determine the subject of the editorial (immigration law), a bag-of-words technique cannot reveal that the negative opinion is about a state immigration law and the positive opinion is about the proposed federal immigration law. If the opinions had been reversed, there would still be positive and negative sentiment in the document, and there would still be topical information about immigration law. Even breaking down the document at the paragraph or sentence level and performing text classification to determine the topic and sentiment of these smaller units of text does not isolate the opinions and topics in a way that clearly correlates opinions with topics. Using structured sentiment information to discover that the negative sentiment is about the Alabama law, and that the positive sentiment is about the federal law does tell us (presuming that we're versed in United States politics) that the author of this editorial is likely aligned with the Democratic Party.

It is also possible to use these structured opinions to separate out opinions about the federal immigration reform, and opinions about the Alabama state law and compare them. Structured sentiment extraction techniques give us the ability to make these kinds of determinations from text.

## 1.2 Structured Opinion Extraction

The goal of structured opinion extraction is to extract individual opinions in text and break down those opinions into parts, so that those parts can be used in sentiment analysis applications. To perform structured opinion extraction, there are a number of tasks that one must tackle. First, one must define the scope of the sentiments to be identified, and the structure of the sentiments to identify. Defining the scope of the task can be particularly challenging as one must balance the idea of finding everything that expresses an opinion (no matter how indirectly it does so) with the idea of finding just things that are clearly opinionated and that a lot of people can agree that they understand the opinion the same way.

After defining the structured opinion extraction task, one must tackle the technical aspects of the problem. Opinions need to be identified, and ambiguities need to be resolved. The orientation of the opinion (positive or negative) needs to be determined. If they are part of the structure defined for the task, targets (what the opinion is about) and evaluators (the person whose opinion it is) need to be identified and matched up with the opinions that were extracted. There are tradeoffs to be made between identifying all opinions at the cost of finding false positives, or identifying only the opinions that one is confident about at the cost of missing many opinions. Depending on the scope of the opinions, there may be challenges in adapting the technique for use on different genres of text, or developing resources for different genres of text. Lastly, for some domains of text there are more general text-processing challenges that arise from the style of the text written in the domain. (For example when analyzing Twitter posts, the 140-character length limit for a posting, the informal nature of the medium, and the conventions for hash tags, retweets, and replies can really challenge the text parsers that have been trained on other domains.)

The predominant way of thinking about structured opinion extraction in the

academic sentiment analysis community has been defined by the task of identifying product features and opinions about those product features. The results of this task have been aimed at product review summarization applications that enable companies to quickly identify what parts of a product need improvement and consumers to quickly identify whether the parts of a product that are important to them work correctly. This task consists of finding two parts of an opinion: an *attitude* conveying the nature of the opinion, and a *target* which the opinion is about. The guidelines for this task usually require the target to be a compact noun phrase that concisely names a part of the product being reviewed. The decision to focus on these two parts of an opinion has been made based on the requirements of the applications that will use the extracted opinions, but really it is not a principled way to understand opinions, as several examples will show. (These examples are all drawn from the corpora that discussed in Chapter 5, and demonstrate very real, common problems in these corpora that stem from the decision to focus on only these two parts of an opinion.)

(1) This setup using the CD target was about as easy as learning how to open a refrigerator door for the first time.

In example 1, there is an attitude expressed by the word "easy". A human annotator seeking to determine the target of this attitude has a difficult choice to make in deciding whether to use "setup" or "CD" as the target. Additionally, the comparison "learning how to open a refrigerator door for the first time" needs to be included in the opinion somehow, because this choice of comparison says something very different than if the comparison was with "learning how to fly the space shuttle," the former indicating an easy setup process, and the latter indicating a very difficult setup process. A correct understanding would recognize "setup" as the target, and "using the CD" as an aspect of the setup (a context in which the evaluation applies), to differentiate this evaluation from an evaluation of of setup using a web interface,

for example.

(2) There are a few extremely sexy new features in Final Cut Pro 7.

In example 2, there is an attitude expressed by the phrase "extremely sexy". A human annotator seeking to determine the target of this attitude must choose between the phrases "new features" and "Final Cut Pro 7." In this sentence, it's a bit clearer that the words "extremely sexy" are talking directly about "new features", but there is an implied evaluation of "Final Cut Pro 7". Selecting "new features" as the target of the evaluation loses this information, but selecting "Final Cut Pro 7" as the target of this evaluation isn't really a correct understanding of the opinion conveyed in the text. A correct understanding of this opinion would recognize "new features" as the target of the evaluation, and "in Final Cut Pro 7" as an aspect.

(3) It is much easier to have it sent to your inbox.

(4) Luckily, eGroups allows you to choose to moderate individual list members...

In examples 3 and 4, it isn't the need to ramrod different kinds of information into a single target annotation that causes problems — it's the requirement that the target be a compact noun phrase naming a product feature. The words "easier" and "luckily" both evaluate propositions expressed as clauses, but the requirement that the target be a compact noun phrase leads annotators of these sentences to incorrectly annotate the target. In the corpus these sentences were drawn from, the annotators selected the dummy pronoun "it" at the beginning of example 3 as the target of "easier", and the verb "choose" in example 4 as the target of "luckily." Neither of these examples is the correct way to annotate a proposition, and the decision made on these sentences is inconsistent between the two sentences. The annotators were forced to choose these incorrect annotations as a result of annotation instructions that did not capture the full range of possible opinion structures.

I introduce here the concept of an *appraisal expression*, a basic grammatical structure expressing a single evaluation, based on linguistic analyses of evaluative language [20, 21, 72, 110], to correctly capture the full complexity of opinion expressions. In an appraisal expression, in addition to the evaluator (the person to whom the opinion is attributed), attitude, and target, other parts may also be present, such as a superordinate when the target is evaluated as a member of a class or an aspect when the evaluation only applies in a specific context (see examples 5 thru 7).

(5) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

(6) $_{\text{evaluator}}$ I $_{\text{attitude}}$ hate it $_{\text{target}}$ when people talk about me rather than to me.

(7) $_{\text{evaluator}}$ He opened with $_{\text{expressor}}$ greetings of gratitude and $_{\text{attitude}}$ peace.

I view extracting appraisal expressions as a fundamental subtask in sentiment analysis, which needs to be studied on its own terms. Appraisal expression extraction must be considered as an independent subtask in sentiment analysis because it can be used by many higher level applications.

In this dissertation, I introduce the FLAG[1] appraisal expression extraction system, and the IIT Sentiment Corpus, designed to evaluate performance at the task of appraisal expression extraction.

## 1.3 Evaluating Structured Opinion Extraction

In addition to the problems posed by trying to cram a complicated opinion structure into an annotation scheme that only recognizes attitudes and targets, much of the work that's been performed in structured opinion extraction has not been

[1] FLAG is an acronym for "Functional Local Appraisal Grammar", and the technologies that motivate this name will be discussed shortly.

evaluated in ways that are suited for finding the best appraisal expression extraction technique. Many researchers have used appraisal expression extraction implicitly as a means to accomplishing their chosen application, while giving short shrift to the appraisal extraction task itself. This makes it difficult to tell whether the accuracy of someone's software at a particular application is due to the accuracy of their appraisal extraction technique, or whether it's due to other steps that are performed after appraisal extraction in order to turn the extracted appraisal expressions into the results for the application. For example Archak et al. [5], who use opinion extraction to predict how product pricing is driven by consumer sentiment, devote only a couple sentences to describing how their sentiment extractor works, with no citation to any other paper that describes the process in more detail.

Very recently, there has been some work on evaluating appraisal expression extraction on its own terms. Some new corpora annotated with occurrences of appraisal expressions have been developed [77, 86, 192], but the research using most of these corpora has not advanced to the point of evaluating an appraisal expression extraction system from end to end.

These corpora have been limited, however, by the assumption that the documents in question are review-like. They focus on identifying opinions in product reviews, and they often assume that the only targets of interest are product features, and the only opinions of interest are those that concern the product features. This focus on finding opinions about product features in product reviews has influenced both evaluation corpus construction and the software systems that extract opinions from these corpora. Typical opinion corpora contain lots of reviews about a particular product or a particular type of product. Sentiment analysis systems targeted for these corpora take advantage of this homogeneity to identify the names of common product features based on lexical redundancy in the corpus. These techniques then

find opinion words that describe the product features that have already been found.

The customers of sentiment analysis applications are interested in mining a broader range of texts such as blogs, chat rooms, message boards, and social networking sites [10, 98]. They're interested in finding favorable and unfavorable comparisons of their product in reviews of other products. They're interested in mining perceptions of a their brand, just as much as they're interested in mining perceptions of a their company's products. For these reasons, sentiment analysis needs to move beyond the assumption that all texts of interest are review-like.

The assumption that the important opinions in a document are evaluations of product features breaks down completely when performing sentiment analysis on blog posts or tweets. In these domains, it may be difficult to curate a large collection of text on a single narrowly-defined topic, or the users of a sentiment analysis technique may not be interested in operating on only a single narrowly-defined topic. O'Hare et al. [131], for example, observed that in the domain of financial blogs, 30% of the documents encountered are relevant to at least one stock, but each of those documents is relevant to three different stocks on average. This would make the assumption of lexical redundancy for opinion targets unsupportable.

To enable a fine-grained evaluation of appraisal expression extraction systems in these more general sentiment analysis domains, I have created the IIT Sentiment Corpus, a corpus of blog posts annotated with all of the appraisal expressions that were there to be found, regardless of topic.

## 1.4 FLAG: Functional Local Appraisal Grammar Extractor

To move beyond the review-centric view of appraisal extraction that others in sentiment analysis research have been working with, I have developed FLAG, an appraisal expression extractor that doesn't rely on domain-dependent features to find

appraisal expressions accurately.

FLAG's operation is inspired by appraisal theory and local grammar techniques. Appraisal theory [110] is a theoretical framework within Systemic Functional linguistics (SFL) [64] for classifying different kinds of evaluative language. In the SFL tradition, it treats meaning as a series of choices that the speaker or writer makes and it characterizes how these choices are reflected in the lexicon and syntactic structure of evaluative text. Syntactic structure is complicated, affected by many other overlapping concerns outside the scope of appraisal theory, but it can be treated uniformly through the lens of a local grammar. Local grammars specify the patterns used by linguistic phenomena which can be found scattered throughout a text, expressed using a diversity of different linguistic resources. Together, appraisal theory and local grammars describe the behavior of an appraisal expression.

FLAG demonstrates that the use of appraisal theory and local grammars can be an effective method for sentiment analysis, and can provide significantly more information about the extracted sentiments than has been available using other techniques.

Hunston and Sinclair [72] describe a general set of steps for local grammar parsing, and they study the application of these steps to evaluative language. In their formulation, parsing a local grammar consists of three steps. A parser must (1) detect which regions of a free text should be parsed using the local grammar, then it should (2) determine which local grammar pattern to use to parse the text. Finally, it should (3) parse the text, using the pattern it has selected. With machine learning techniques and the information supplied by appraisal theory, I contend that this process should be modified to make selection of the correct pattern the last step, because then a machine learning algorithm can select the best pattern based on the consistency of the parses themselves. This idea is inspired by reranking techniques in

probabilistic parsing [33], machine translation [150], and question answering [141]. In this way, FLAG adheres to the principle of least commitment [107, 118, 162], putting off decisions about which patterns are correct until it has as much information as possible about the text each pattern identifies.

**H1:** The three step process of finding attitude groups, identifying the potential appraisal expression structures for each attitude group, and then selecting the best one can accurately extract targets in domains such as blogs, where one can't take advantage of redundancy to create or use domain-specific resources as part of the appraisal extraction process.

The first step in FLAG's operation is to detect ranges of text which are candidates for parsing. This is done by finding opinion phrases which are constructed from opinion head words and modifiers listed in a lexicon. The lexicon lists positive and negative opinion words and modifiers with the options they realize in the ATTITUDE system. This lexicon is used to locate opinion phrases, possibly generating multiple possible interpretations of the same phrase.

The second step in FLAG's extraction process is to determine a set of potential appraisal expression instances for each attitude group, using a set of *linkage specifications* (patterns in a dependency parse of the sentence that represent patterns in the local grammar of evaluation) to identify the targets, evaluators, and other parts of each potential appraisal expression instance. Using these linkage specifications, FLAG is expected, in general, to find several patterns for each attitude found in the first step.

It is time consuming to develop a list of patterns, and a relatively unintuitive task for any developer who would have to develop this list. Therefore, I have developed a supervised learning technique that can learn these local grammar patterns from an

annotated corpus of opinionated text.

**H2:** Target linkage patterns can be automatically learned, and when they are they are more effective than hand-constructed linkage patterns at finding opinion targets and evaluators.

The third step in FLAG's extraction is to select the correct combination of local grammar pattern and appraisal attributes for each attitude group from among the candidates extracted by the previous steps. This is accomplished using supervised support vector machine reranking to select the most grammatically consistent appraisal expression for each attitude group.

**H3:** Machine learning can be used to effectively determine which linkage pattern finds the correct appraisal expression for a given attitude group.

## 1.5 Appraisal Theory in Sentiment Analysis

FLAG brings two new ideas to the task of sentiment analysis, based on the work of linguists studying the evaluative language.

Most existing work and corpora in sentiment analysis have considered only three parts of an appraisal expression: attitudes, evaluators, and targets, as these are the most obviously useful pieces of information and they are the parts that most commonly appear in appraisal expressions. However, Hunston and Sinclair's [72] local grammar of evaluation demonstrated the existence of other parts of an appraisal expression that provide useful information about the opinion when they are identified. These parts include superordinates, aspects, processes, and expressors. Superordinates, for example indicate that the target is being evaluated relative to some class that it is a member of. (An example of some of these parts is shown in example,

sentence 8. All of these parts are defined, with numerous examples, in Section 4.2 and in Appendix B.)

(8) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

By analyzing existing sentiment corpora against the rubric of this expanded local grammar of appraisal, I test the following hypotheses:

**H4:** Including superordinates, aspects, processes, and expressors in an appraisal annotation scheme makes it easier to develop sentiment corpora that are annotated consistently, preventing many of the errors and inconsistencies that occurred frequently when existing sentiment corpora were annotated.

**H5:** Identifying superordinates, aspects, processes, and expressors in an appraisal expression improves the ability of an appraisal expression extractor to identify targets and evaluators as well.

Additionally, FLAG incorporates ideas from Martin and White's [110] ATTITUDE system, recognizing that there are different types of attitudes that are realized using different local grammar patterns. These different attitude types are closely related to the lexical meanings of the attitude words. FLAG recognizes three main attitude types: affect (which conveys emotions, like the word "hate"), judgment (which evaluates a person's behavior in a social context, like the words "idiot" or "evil"), and appreciation (which evaluates the intrinsic qualities of an object, like the word "beautiful").

**H6:** Determining whether an attitude is an example of affect, appreciation, or judgment improves accuracy at determining an attitude's structure compared to performing the same task without determining the attitude types.

**H7:** Restricting linkage specifications to specific attitude types improves accuracy compared to not restricting linkage specifications by attitude type.

## 1.6 Structure of this dissertation

In Chapter 2, I survey the field of sentiment analysis, as well as other research related to FLAG's operation. In Chapter 3, I describe FLAG's overall organization. In Chapter 4, I present an overview of appraisal theory, and introduce my local grammar of evaluation. In Chapter 5, I introduce the corpora that I will be using to evaluate FLAG, and discuss the relationship of each corpus with the task of appraisal expression extraction. In Chapter 6, I discuss the lexicon-based attitude extractor, and lexicon learning. In Chapter 7, I discuss the linkage associator, which applies local grammar patterns to each extracted attitude to turn them into candidate appraisal expressions. In Chapter 8, I introduce fully-supervised, and minimally-supervised techniques for local grammar patterns from a corpus. In Chapter 9, I describe a technique for unsupervised reranking of candidate appraisal expressions. In Chapter 10, I evaluate FLAG on five different corpora. In Chapter 11, I present my conclusions and discuss future work in this field.

# CHAPTER 2

# PRIOR WORK

This chapter gives a general background on applications and techniques that have been used to study evaluation for sentiment analysis, particularly those related to extracting individual evaluations from text. A comprehensive view of the field of sentiment analysis is given in a survey article by Pang and Lee [133]. This chapter also discusses local grammar techniques and information extraction techniques that are relevant to extracting individual evaluations from text.

## 2.1 Applications of Sentiment Analysis

Sentiment analysis has a number of interesting applications [133]. It can be used in recommendation systems (to recommend only products that consumers liked) [165], ad-placement applications (to avoid advertising a company alongside an article that is bad press for them) [79], and flame detection systems (to identify and remove message board postings that contain antagonistic language) [157]. It can also be used as a component technology in topical information retrieval systems (to discard subjective sections of documents and improve retrieval accuracy).

Structured extraction of evaluative language in particular can be used for multiple-viewpoint summarization, summarizing reviews and other social media for business intelligence [10, 98], for predicting product demand [120] or product pricing [5], and for political analysis.

One example of a higher-level task that depends on structured sentiment extraction is Archak et al.'s [5] technique for modeling the pricing effect of consumer opinion on products. They posit that demand for a product is driven by the price of the product and consumer opinion about the product. They model consumer opinion about a product by constructing, for each review, a matrix with product features

as rows and columns as sentiments where term-sentiment associations are found using a syntactic dependency parser (they don't specify in detail how this is done). They apply dimensionality reduction to this matrix using Latent Semantic Indexing, and apply the reduced matrix and other numerical data about the product and its reviews to a regression to determine how different sentiments about different parts of the product affect product pricing. They report a significant improvement over a comparable model that includes only the numerical data about the product and its reviews. Ghose et al. [59] apply a similar technique (without dimensionality reduction) to study how the reputation of a seller affects his pricing power.

## 2.2 Evaluation and other kinds of subjectivity

The terms "sentiment analysis" and "subjectivity" mean a lot of different things to different people. These terms are often used to cover a variety of different research problems that are only related insofar as they deal with analysing the non-factual information found in text. The following paragraphs describe a number of these different tasks and set out the terminology that I use to refer to these tasks elsewhere in the thesis.

Evaluation covers the ways in which a person communicates approval or disapproval of circumstances and objects in the world around him. Evaluation is one of the most commercially interesting fields in sentiment analysis, particularly when applied to product reviews, because it promises to allow companies to get quick summaries of why the public likes or dislikes their product, allowing them to decide which parts of the product to improve or which to advertise. Common tasks in academic literature have included review classification to determine whether reviewers like or dislike products overall, sentence classification to find representative positive or negative sentences for use in advertising materials, and "opinion mining" to drill down into what makes products succeed and fail.

Affect [2, 3, 20, 110, 156] concerns the emotions that people feel, whether in response to a trigger or not, whether positive, negative, or neither (e.g. surprise). Affect and evaluation have a lot of overlap, in that positive and negative emotions triggered by a particular trigger often constitute an evaluation of that trigger [110]. Because of this, affect is always included in studies of evaluation, and particular frameworks for classifying different types of affect (e.g. appraisal theory [110]) are particularly well suited for evaluation tasks. Affect can also have applications outside of evaluation, in fields like human-computer interaction [3, 189, 190], and also in applications outside of text analysis. Alm [3], for example, focused on identifying spans of text in stories which conveyed particular emotions, so that a computerized storyteller could vocalize those sections of a story with appropriately dramatic voices. Her framework for dealing with affect involved identifying the emotions "angry", "disgusted", "fearful", "happy", "sad", and "surprised." These emotion types are motivated (appropriately for the task) by the fact that they should be vocalized differently from each other, but because this framework lacks a unified concept of positive and negative emotions, it would not be appropriate for studying evaluative language.

There are many other non-objective aspects of texts that are interesting for different applications in the field of sentiment analysis, and a blanket term for these non-objective aspects of texts is "subjectivity". The most general studies of subjectivity have focused on how "private states", internal states that can't be observed directly by others, are expressed [174, 179]. More specific aspects of subjectivity include predictive opinions [90], speculation about what will happen in the future, recommendations of a course of action, and the intensity of rhetoric [158]. Sentiment analysis whose goal is to classify text for intensity of rhetoric, for example, can be used to identify flames (postings that contain antagonistic language) on a message board for moderator attention.

## 2.3 Review Classification

One of the earliest tasks in evaluation was review classification. A movie review, restaurant review, or product review consists of an article written by the reviewer, describing what he felt was particularly positive or negative about the product, plus an overall rating expressed as a number of stars indicating the quality of the product. In most schemes there are five stars, with low quality movies achieving one star and high quality movies achieving five. The stars provide a quick overview of the reviewer's overall impression of the movie. The task of review classification is to predict the number of stars, or more simply whether reviewer wrote a positive or negative review, based on an analysis of the text of the review.

The task of review classification derives its validity the fact that a review covers a single product, and that it is intended to be comprehensive and study all aspects of a product that are necessary to form a full opinion. The author of the review assigns a star rating indicating the extent to which they would recommend the product to another person, or the extent to which the product fulfilled the author's needs. The review is intended to convey the same rating to the reader, or at least justify the rating to the reader. The task, therefore, is to determine numerically how well the product which is the focus of the review satisfied the review author.

There have been many techniques for review classification applied in sentiment analysis literature. A brief summary of the highlights includes Pang et al. [134], who developed a corpus (which has since become standard) for evaluating review classification, using 1000 IMDB movie reviews with 4 or 5 stars as examples of positive reviews, and 1000 reviews with 1 or 2 stars as examples of negative reviews. Pang et al.'s [134] experiment in classification used bag-of-words features and bigram features in standard machine learning classifiers.

Turney [170] determined whether words are positive or negative and how strong the evaluation is by computing the words' pointwise mutual information for their co-occurrence with a positive seed word ("poor") and a negative seed word ("negative"). They call this value the word's *semantic orientation*. Turney's software scanned through a review looking for phrases that match certain part of speech patterns, computed the semantic orientation of those phrases, and added up the semantic orientation of all of those phrases to compute the orientation of a review. He achieved 74% accuracy classifying a corpus of product reviews. In his later work, [171] he applied semantic orientation to the task of lexicon building because of efficiency issues in using the internet to look up lots of unique phrases from many reviews. Harb et al. [65] performed blog classification by starting with the same seed adjectives and used Google's search engine to create association rules that find more. They then counted the numbers of positive versus negative adjectives in a document to classify the documents. They achieved 0.717 $F_1$ score identifying positive documents and 0.626 $F_1$ score identifying negative documents.

Whitelaw, Garg, and Argamon [173] augmented bag-of-words classification with a technique which performed shallow parsing to find opinion phrases, classified by orientation and by a taxonomy of attitude types from appraisal theory [110], specified by a hand-constructed attitude lexicon. Text classification was performed using a support vector machine, and the feature vector for each corpus included word frequencies (for the bag-of-words), and the percentage of appraisal groups that were classified at each location in the attitude taxonomy, with particular orientations. They achieved 90.2% accuracy classifying the movie reviews in Pang et al.'s [134] corpus.

Snyder and Barzilay [155] extended the problem of review classification to reviews that cover several different dimensions of the product being reviewed. They

use perceptron-based ordinal ranking model for ranking restaurant reviews from 1 to 5 along three dimensions: food quality, service, ambiance. They use three ordinal rankers (one for each dimension) to assign initial scores to the three dimensions, and additional binary classifier that tries to determine whether the three dimensions should really have the same score. They used unigram and bigram features in their classifiers. They report a 67% classification accuracy on their test set.

In a related (but affect oriented) task, Mishne and Rijke [121] predicted the mood that blog post authors were feeling at the time they wrote their post. They used n-grams with Pace regression to predict the author's "current mood" which is specified by the post author using an selector list when composing a post.

## 2.4 Sentence classification

After demonstrating the possibility of classifying reviews with high accuracy, work in sentiment analysis turned toward the task of classifying each sentence of a document as positive, negative, or neutral.

The sources of validity for a sentence-level view of sentiment vary, based on the application for which the sentences are intended. To Wiebe and Riloff [176], the purpose of recognizing objective and subjective sentences is to narrow down the amount of text that automated systems need to consider for other tasks by singling out (or removing) subjective sentences. They are not concerned in that paper with recognizing positive and negative sentences. To quote:

> There is also a need to explicitly recognize objective, factual information for applications such as information extraction and question answering. Linguistic processing alone cannot determine the truth or falsity of assertions, but we could direct the systems attention to statements that are objectively presented, to lessen distractions from opinionated, speculative, and evaluative language. (p. 1)

Because their goal is to help direct topical text analysis systems to objective

text, their data for sentence-level tasks is derived from the MPQA corpus [177, 179] (which annotates sub-sentence spans of subjective text), and considers a sentence subjective if the sentence has any subjective spans of sufficient strength within it. Thus, their sentence-level data derives its validity from the fact that it's derived from the corpus's finer-grained subjectivity annotations that they suppose an automated system would be interested in using or discarding.

Hurst and Nigam [73] write that recognizing sentences as having positive or negative polarity derives its validity from the goal of "[identifying] sentences that could be efficiently scanned by a marketing analyst to identify salient quotes to use in support of positive or negative marketing conclusions." [128, describing 73] They too perform sentiment extraction at a phrase level.

In the works described above, the authors behind each task have a specific justification for why sentence level sentiment analysis is valid, and the way in which they derive their sentence-level annotations from finer-grained annotations and the way in they approach the sentiment analysis task reflects the justification they give for the validity of sentence-level sentiment analysis. But somewhere int he development of the sentence-level sentiment analysis task, researchers lost their focus on the rather limited justifications of sentence-level sentiment analysis that I have discussed, and began to assume that whole sentences intrinsically reflect a single sentiment at a time or a single overall sentiment. (I do not understand why this assumption is valid, and I have yet to find a convincing justification in the literature.) In work that operates from this assumption, sentence-level sentiment annotations are not derived from finer-grained sentiment annotations. Instead, the sentence-level sentiment annotations are assigned directly by human annotators. For example, Jakob et al. [77] developed a corpus of finer-grained sentiment annotations by first having their annotators determine which sentences were topic-relevant and opinionated, working to

reconciling the differences in the sentence-level annotations, and then finally having the annotators identify individual opinions in only the sentences that all annotators agreed were opinionated and topic relevant.

The Japanese National Institute of Informatics hosted an opinion analysis shared task at their NTCIR conference for three years [91, 146, 147] that included a sentence-level sentiment analysis component on newswire text. Among the techniques that have been applied to this shared task are rule-based techniques that look at the main verb of a sentence, or various kinds of modality in the sentences [92, 122], lexicon-based techniques [28, 185], and techniques using standard machine-learning classifiers (almost invariably support vector machines) with various feature sets [22, 53, 100, 145]. The accuracy of all entries at the NTCIR conferences was low, due in part to low agreement between the human annotators of the NTCIR corpora.

McDonald et al. [115] developed a model for sentiment analysis at different levels of granularity simultaneously. They use graphical models in which a document-level sentiment is linked to several paragraph level sentiments, and each paragraph-level sentiment is linked to several sentence level sentiments (in addition to being linked sequentially). They apply the Viterbi algorithm to infer the sentiment of each text unit, constrained to ensure that the paragraph and document parts of the labels are always the same where they represent the same paragraph/document. They report 62.6% accuracy at classifying sentences when the orientation of the document is not given, and 82.8% accuracy at categorizing documents. When the orientation of the document is given, they report 70.2% accuracy at categorizing the sentences.

Nakagawa et al. [125] developed a conditional random field model structured like the dependency parse tree of the sentence they are classifying to determine the polarity of sentences, taking into account opinionated words and polarity shifters in the sentence. They report 77% to 86% accuracy at categorizing sentences, depending

on which corpus they tested against.

Neviarouskaya et al. [126] developed a system for computing the sentiment of a sentence based on the words in the sentence, using Martin and White's [110] appraisal theory and Izard's [74] affect categories. They used a complicated set of rules for composing attitudes found in different places in a sentence to come up with an overall label for the sentence. They achieved 62.1% accuracy at determining the fine-grained attitude types of each sentence in their corpus, and 87.9% accuracy at categorizing sentences as positive, negative, or neutral.

## 2.5 Structural sentiment extraction techniques

After demonstrating techniques for classifying full reviews or individual sentences with high accuracy, work in sentiment analysis turned toward deeper extraction methods, focused on determining parts of the sentiment structure, such as what a sentiment is about (the *target*), and who is expressing it (the *source*). Numerous researchers have performed work in this area, and there have been many different ways of evaluating structured sentiment analysis techniques. Table 2.1 highlights results reported by the some of the papers discussed in this section.

Among the techniques that focus specifically on evaluation, Nigam and Hurst [128] use part-of-speech extraction patterns and a manually-constructed sentiment lexicon to identify positive and negative phrases. They use a sentence-level classifier to determine whether each sentence of the document is relevant to a given topic, and assign all of the extracted sentiment phrases to that topic. They further discuss methods of assigning a sentiment score for a particular topic using the results of their system.

Most of the other techniques that have been developed for opinion extraction have focused on product reviews, and on finding product features and the opinions

that describe them. Indeed, when discussing opinion extraction in their survey of sentiment analysis, Pang and Lee [133] only discuss research relating to product reviews and product features. Most work on sentiment analysis in blogs, by contrast, has focused on document or sentence classification [37, 94, 121, 131].

The general setup of experiments in the product review domain has been to take a large number of reviews of the same product, and learn product features (and sometimes opinions) by taking advantage of the redundancy and cohesion between documents in the corpus. This works because although some people may see a product feature positively where others see it negatively, they are generally talking about the same product features.

Popescu and Etzioni [137] use the KnowItAll information extraction system [52] to identify and cluster product features into categories. Using dependency linkages, they then identify opinion phrases about those features, and lastly they determine the whether the opinions are positive or negative, and how strongly, using relaxation labeling. They achieve an 0.82 $F_1$ score extracting opinionated sentences, and they achieve 0.94 precision and 0.77 recall at identifying the set of distinct product feature names found in the corpus.

In a similar, but less sophisticated technique, Godbole et al. [61] construct a sentiment lexicon by using a WordNet based technique, and associate sentiments with entities (found using the Lydia information extraction system [103]) by assuming that a sentiment word found in the same sentence as an entity is describing that entity.

Hu and Liu [70] identify product features using frequent itemset extraction, and identify opinions about these product features by taking the closest opinion adjectives to each mention of a product feature. They use a simple WordNet synonymy/antonymy technique to determine orientation of each opinion word. They

Table 2.1. Comparison of reported results from past work in structured opinion extraction. The different columns report different techniques for evaluating opinion extraction, but even within a column, results may not be comparable since different researchers have evaluated their techniques on different corpora.

| Author | Opinionated Sentence Extraction | Attitudes given Features | Feature Names | Feature Mentions | Correct Pairings of provided annotations | Feature and Opinion pairs |
|---|---|---|---|---|---|---|
| Hu and Liu [70] | P=0.642<br>R=0.693 | | P=0.720<br>R=0.800 | | | |
| Ding et al. [44] | P=0.910<br>R=0.900 | | | | | |
| Kessler and Nicolov [87] | | | | | P=0.748<br>R=0.654 | |
| Popescu and Etzioni [137] | | P=0.79<br>R=0.76 | P=0.94<br>R=0.77 | | | |
| Popescu [136] | F1=0.82 | | | | | |
| Zhuang et al. [192] | | | | | | P=0.483<br>R=0.585 |
| Jakob and Gurevych [76] | | | | | | P=0.531<br>R=0.614 |
| Qiu et al. [138] | | | P=0.88<br>R=0.83 | | | |

achieve 0.642 precision and 0.693 recall at extracting opinionated sentences, and they achieve 0.72 precision and 0.8 recall at identifying the set of distinct product feature names found in the corpus.

Qiu et al. [138, 139] use a 4-step bootstrapping process for acquiring opinion and product feature lexicons, learning opinions from product features, and product features from opinions (using syntactic patterns for adjectival modification), and learning opinions from other opinions and product features from other product features (using syntactic patterns for conjunctions) in between these steps. They achieve 0.88 precision and .83 recall at identifying the set of distinct product feature names found in the corpus with their double-propagation version, and they achieve 0.94 precision and 0.66 recall with a non-propagation baseline version.

Zhuang et al. [192] learn opinion keywords and product feature words from the training subset of their corpus, selecting words that appeared in the annotations and eliminating those that appeared with low frequency. They use these words to search for both opinions and product features in the corpus. They learn a master list of dependency paths between opinions and product features from their annotated data, and eliminate those that appear with low frequency. They use these dependency paths to pair product features with opinions. It appears that they evaluate their technique for the task of feature-opinion pair mining, and they reimplemented and ran Hu and Liu's [70] technique as a baseline. They report 0.403 precision and 0.617 recall using Hu and Liu's [70] technique, and they report 0.483 precision and 0.585 recall using their own approach.

Jin and Ho [78] use HMMs to identify product features and opinions (explicit and implicit) with a series of 7 different entity types (3 for targets, and 4 for opinions). They start with a small amount of labeled data, and amplify it by adding unlabeled data in the same domain. They report precision and recall in the 70%–80% range

at finding entity-opinion pairs (depending which set of camera reviews they use to evaluate).

Li et al. [99] describe a technique for finding attitudes and product features using CRFs of various topologies. They then pair them by taking the closest opinion word for each product feature.

Jakob and Gurevych [75] extract opinion target mentions in their corpus of service reviews [77] using a linear CRF. Their corpus is publicly available and its advantages and flaws are discussed in Section 5.3.

Kessler and Nicolov [87] performed an experiment in which they had human taggers identify "sentiment expressions" as well as "mentions" covering all of the important product features in a particular domain, whether or not those mentions were the target of a sentiment expression, and had their taggers identify which of those mentions were opinion targets. They used SVM ranking to determine, from among the available mentions, which mention was the target of each opinion. Their corpus is publicly available and its advantages and flaws are discussed in Section 5.4.

Cruz et al. [40] complain that the idea of learning product features from a collection of reviews about a single product is too domain independent, and propose to make the task even more domain specific by using interactive methods to introduce a product-feature hierarchy, domain specific lexicon, and learning other resources from an annotated corpus.

Lakkaraju et al. [95] describe a graphical model for finding sentiments and the "facets" of a product described in reviews. The compare three models with different levels of complexity. FACTS is a sequence model, where each word is generated by 3 variables: a facet variable, a sentiment variable, and a selector variable (which determines whether to draw the word based on facet, sentiment, or as a non-sentiment

word). CFACTS breaks each document up into windows (which are 1 sentence long by default), treats the document as a sequence of windows, and each window as a sequence of words. More latent variables are added to assign each window a default facet and a default sentiment, and to model the transitions between the windows. This model removes the word-level facet and sentiment variables. CFACTS-R adds an additional variable for document-level sentiment to the CFACTS model. They perform a number of different evaluations – comparing the product facets their model identified with lists on Amazon for that kind of product, comparing sentence level evaluations, and identifying distinct facet-opinion pairs at the document and sentence level.

There has been minimal work in structured opinion extraction outside of the product review domain. The NTCIR-7 and NTCIR-8 Multilingual Opinion Annotation Tasks [147, 148] are the two most prominent examples, identifying opinionated sentences from newspaper documents, and finding opinion holders and targets in those sentences. No attempt was made to associate attitudes, targets, and opinion holders. I do not have any information about the scope of their idea of opinion targets. In each of these tasks, only one participant attempted to find opinion targets in English, though more made the attempt in Chinese and Japanese.

Janyce Wiebe's research team at the University of Pittsburgh has a large body of work on sentiment analysis, which has dealt broadly with subjectivity as a whole (not just evaluation), but many of her techniques are applicable to evaluation. Her team's approach uses supervised classifiers to learn tasks at many levels of the sentiment analysis problem, from the smallest details of opinion extraction such as contextual polarity inversion [180], up to discourse-level segmentation based on author point of view [175]. They have developed the MPQA corpus, a tagged corpus of opinionated text [179] for evaluating and training sentiment analysis programs, and

for studying subjectivity. The MPQA corpus is publicly available and it advantages and flaws are discussed in Section 5.1. They have not described an integrated system for sentiment extraction, and many of the experiments that they have performed have involved automatically boiling down the ground truth annotations into something more tractable for a computer to match. They've generally avoided trying to extract spans of text, preferring to take the existing ground truth annotations and classify them.

## 2.6 Opinion lexicon construction

Lexicon-based approaches to sentiment analysis often require large hand-built lexicons to identify opinion words. These lexicons can be time-consuming to construct, so there has been a lot of research into techniques for automatically building lexicons of positive and negative words.

Hatzivassiloglou and McKeown [66] developed a graph-based technique for learning lexicons by reading a corpus. In their technique, they find pairs of adjectives conjoined by conjunctions (e.g. "fair and legitimate" or "fair but brutal"), as well as morphologically related adjectives (e.g. "thoughtful" and "thoughtless"), and create a graph where the vertices represent words, and the edges represent pairs (marked as same-orientation or opposite-orientation links). They apply a graph clustering algorithm to cluster the adjectives found into two clusters of positive and negative terms. This technique achieved 82% accuracy at classifying the words found.

Another algorithm for constructing lexicons is that of Turney and Littman [171]. They determine whether words are positive or negative and how strong the evaluation is by computing the words' pointwise mutual information (PMI) for their co-occurrence with small set of positive seed words and a small set of negative seed words. Unlike their earlier work [170], which I mentioned in Section 2.3, the seed

sets contained seven representative positive and negative words each, instead of just one each. This technique had 78% accuracy classifying words in Hatzivassiloglou and McKeown's [66] word list. They also tried a version of semantic orientation that used latent semantic indexing as the association measure. Taboada and Grieve [164] used the PMI technique to classify words according to the three main attitude types laid out by Martin and White's [110] appraisal theory: affect, appreciation, and judgment. (These types are described in more detail in Section 4.1.) They did not develop any evaluation materials for attitude type classification, nor did they report accuracy. Many consider the semantic orientation technique to be a measure of force of the association, but this is not entirely well-defined, and it may make more sense to consider it as a measure of confidence in the result.

Esuli and Sebastiani [46] developed a technique for classifying words as positive or negative, by starting with a seed set of positive and negative words, then running WordNet synset expansion multiple times, and training a classifier on the expanded sets of positive and negative words. They found [47] that different amounts of WordNet expansion, and different learning methods had different properties of precision and recall at identifying opinionated words. Based on this observation, they applied a committee of 8 classifiers trained by this method (with different parameters and different machine learning algorithms) to create SentiWordNet [48] which assigns each WordNet synset a score for how positive the synset is, how negative the synset is, and how objective the synset is. The scores are graded in intervals of $1/8$, based on the binary results of each classifier, and for a given synset, all three scores sum to 1. This version of SentiWordNet was released as SentiWordNet 1.0. Baccianella, Esuli, and Sebastiani [12] improved upon SentiWordNet 1.0, by updating it to use WordNet 3.0 and the Princeton Annotated Gloss Corpus and by applying a random graph walk procedure so related synsets would have related opinion tags. They released this version of SentiWordNet as SentiWordNet 3.0. In other work [6, 49], they applied the

WordNet gloss classification technique to Martin and White's [110] attitude types.

## 2.7 The grammar of evaluation

There have been many different theories of subjectivity or evaluation developed by linguists, with different classification schemes and different scopes of inclusiveness. Since my work draws heavily on one of these theories, it is appropriate to discuss some of important theories here, though this list is not exhaustive. More complete overviews of different theoretical approaches to subjectivity are presented by Thompson and Hunston [166] and Bednarek [18]. The first theory that I will discuss, private states, deals with the general problem of subjectivity of all types, but the others deal with evaluation specifically. There is a common structure to all of the grammatical theories of evaluation that I have found: they each have a component dealing with the approval/disapproval dimension of opinions (most also have schemes for dividing this up into various types of evaluation), and they also each have a component that deals with the positioning of different evaluations, or the commitment that an author makes to an opinion that he mentions.

**2.7.1 Private States.** One influential framework for studying the general problem of subjectivity is the concept of a *private state*. The primary source for the definition of private states is Quirk et al. [140, §4.29]. In a discussion of stative verbs, they note that "many stative verbs denote 'private' states which can only be subjectively verified: i.e. states of mind, volition, attitude, etc." They specifically mention 4 types of private states expressed through verbs:

**intellectual states** e.g. *know, believe, think, wonder, suppose, imagine, realize, understand*

**states of emotion or attitude** e.g. *intend, wish, want, like, dislike, disagree, pity*

**states of perception** e.g. *see, hear, feel, smell, taste*

| | |
|---|---|
| Sentiment | Positive: Speaker looks favorably on target<br>Negative: Speaker looks unfavorably on target |
| Agreement | Positive: Speaker agrees with a person or proposition<br>Negative: Speaker disagrees with a person or proposition |
| Arguing | Positive: Speaker argues by presenting an alternate proposition<br>Negative: Speaker argues by denying the proposition he's arguing with |
| Intention | Positive: Speaker intends to perform an act<br>Negative: Speaker does not intend to perform an act |
| Speculation | Speaker speculates about the truth of a proposition |
| Other Attitude | Surprise, uncertainty, etc. |

Figure 2.1. Types of attitudes in the MPQA corpus version 2.0

**states of bodily sensation** e.g. *hurt, ache, tickle, itch, feel cold*

Wiebe [174] bases her work on this definition of private states, and the MPQA corpus [179] version 1.x focused on identifying private states and their sources, but did not subdivide these further into different types of private state.

**2.7.2 The MPQA Corpus 2.0 approach to attitudes.** Wilson [183] later extended the MPQA corpus more explicitly subdivide the different types of sentiment. Her classification scheme covers six types of attitude: *sentiment, agreement, arguing, intention, speculation,* and *other attitude*, shown in Figure 2.1. The first four of these types can appear in positive and negative forms, though the meaning of positive and negative is different for each of these attitude types. The sentiment attitude type is intended to correspond to the approval/disapproval dimension of evaluation, while the others correspond to other aspects of subjectivity.

In Wilson's tagging scheme, she also tracks whether attitudes are *inferred, sarcastic, contrast* or *repetition.* An example of an inferred attitude is that in the sentence "I think people are happy because Chavez has fallen," the negative sentiment of the people toward Chavez is an inferred attitude. Wilson tags it, but indicates that only very obvious inferences are used to identify inferred attitudes.

The MPQA 2.0 corpus is discussed in further detail in Section 5.1.

**2.7.3 Appraisal Theory.** Another influential theory of evaluative language is Martin and White's [110] *appraisal theory*, which studies the different types evaluative language that can occur, from within the framework of Systemic Functional Linguistics (SFL). They discuss three grammatical systems that comprise appraisal. ATTITUDE is concerned with the tools that an author uses to directly express his approval or disapproval of something. Attitude is further divided into three types: *affect* (which describes an internal emotional state), *appreciation* (which evaluates intrinsic qualities of an object), and *judgment* (which evaluates a person's behavior within a social context). GRADUATION is concerned with the resources which an author uses to convey the strength of that approval or disapproval. The ENGAGEMENT system is concerned with the resources which an author uses to position his statements relative to other possible statements on the same subject.

While Systemic Functional Linguistics is concerned with the types of constraints that different grammatical choices place on the expression of a sentence, Martin and White do not explore these constraints in detail. Other work by Bednarek [19] explores these constraints more comprehensively

There have been several applications of appraisal theory to sentiment analysis. Whitelaw et al. [173] applied appraisal theory to review classification, and Fletcher and Patrick [57] evaluated the validity of using attitude types for text classification by performing the same experiments with mixed-up versions of the hierarchy and the appraisal lexicon. Taboada and Grieve [164] automatically learned attitude types for words using pointwise mutual information, and Argamon et al. [6], Esuli et al. [49] learned attitude types for words using gloss classification.

Neviarouskaya et al. [126] performed related work on sentence classification

using the top-level attitude types of affect, appreciation, and judgment, and using Izard's [74] nine categories of emotion (anger, disgust, fear, guilt, interest, joy, sadness, shame and surprise) as subtypes of affect. The use of Izard's affect types introduced a major flaw into their work (which they acknowledge as an issue), in that negation no longer worked properly because Izard's attitude types didn't have correspondence between the positive and negative types. This problem might have been avoided by using Martin and White's [110] or Bednarek's [20] subdivisions of affect.

**2.7.4 A Local Grammar of Evaluation.** A more structurally focused approach to evaluation is that of Hunston and Sinclair [72], who studied the patterns by which adjectival appraisal is expressed in English. They look at these patterns from the point of view of local grammars (explained in Section 2.8), which in their view are concerned with applying a flat functional structure on top of the general grammar used throughout the English language. They analyzed a corpus of text using a concordancer and came up with a list of different textual frames in which adjectival appraisal can occur, breaking down representative sentences into different components of an appraisal expression (though they do not use that term). Some examples of these patterns are shown in Figure 2.2. Bednarek [19] used these patterns to perform a comprehensive text analysis of a small corpus of newspaper articles, looking for differences in the use of evaluation patterns between broadsheet and tabloid newspapers. While she didn't find any differences in the use of local grammar patterns, the pattern frequencies she reports are useful for other analyses. In later work, Bednarek [20] also developed additional local grammar patterns used to express emotions.

While Hunston and Sinclair's work does not address the relationship between the syntactic frames where evaluative language occurs and Martin and White's attitude types, Bednarek [21] studied a subset of Hunston and Sinclair's [72] patterns, to determine which local grammar patterns appeared in texts when the attitude had an

| Thing evaluated<br>**noun group** | Hinge<br>**link verb** | Evaluative Category<br>**evaluative group with "too" or "enough"** | Restriction on Evaluation<br>**to-infinitive or prepositional phrase with "for"** |
|---|---|---|---|
| He | looks | too young | to be a grandfather |
| Their relationship | was | strong enough | for anything |

| Hinge<br>***what* + link verb** | Evaluative Category<br>**adjective group** | Evaluating Context<br>**prep. phrase** | Hinge<br>**link verb** | Thing evaluated<br>**clause or noun group** |
|---|---|---|---|---|
| What's | very good | about this play | is | that it broadens people's view. |
| What's | interesting | | is | the tone of the statement. |

Figure 2.2. Examples of patterns for evaluative language in Hunston and Sinclair's [72] local grammar.

attitude type of affect, appreciation, or judgment. She found that appreciation and judgment were expressed using the same local grammar patterns, and that a subset of affect (which she called *covert affect*, consisting primarily of '-ing' participles) shared most of those same patterns as well. The majority of affect frames used a different set of local grammar patterns entirely, though a few patterns were shared between all attitude types. She also found that in some patterns shared by appreciation and judgment the *hinge* (linking verb) connecting parts of the pattern could be used to distinguish appreciation and judgment, and suggests that the kind of target could also be used to distinguish them.

**2.7.5 Semantic Differentiation.** Osgood et al. [132] developed the Theory of Semantic Differentiation, a framework for evaluative language in which they treat adjectives as a "semantic space" with multiple dimensions, and an evaluation represents a specific point in this space. They performed several quantitative studies, surveying subjects to look for correlations in their use of adjectives, and used factor analysis methods [167] to look for latent dimensions that best correlated the use of these adjectives. (The concept behind factor analysis is similar to Latent Semantic

Indexing [42], but rather than using singular value decomposition, other mathematical techniques are used.) They performed several different surveys with different factor analysis techniques.

From these studies, three dimensions consistently emerged as the strongest latent dimensions: the *evaluation* factor (exemplified by the adjective pair "good" and "bad"), the *potency* factor (exemplified by the adjective pair "strong" and "weak") and the *oriented activity* factor (exemplified by the adjective pair "active" and "passive"). They use their theory for experiments involving questionnaires, and also apply it to psycholinguistics to determine how combining two opinion words affects the meaning of the whole. They did not apply the theory to text analysis.

Kamps and Marx [84] developed a technique for scoring words according to Osgood et al.'s [132] theory, which rates words on the evaluation, potency, and activity axes. They define $MPL(w_1, w_2)$ (minimum path length) to be the number of WordNet [117] synsets needed to connect word $w_1$ to word $w_2$, and then compute

$$TRI(w_i; w_j, w_k) = \frac{MPL(w_i, w_k) - MPL(w_i, w_j)}{MPL(w_k, w_j)}$$

which gives the relative closeness of $w_i$ (the word in question) to $w_j$ (the positive example) versus $w_k$ (the negative example). 1 means the word is close to $w_j$ and -1 means the word is close to $w_k$. The three axes are thus computed by the following functions:

**Evaluation:** $EVA(w) = TRI(w, \text{'good'}, \text{'bad'})$

**Potency:** $POT(w) = TRI(w, \text{'strong'}, \text{'weak'})$

**Activity:** $ACT(w) = TRI(w, \text{'active'}, \text{'passive'})$

Kamps and Marx [84] present no evaluation of the accuracy of their technique against any gold standard lexicon. Mullen and Collier [124] use Kamps and Marx's

lexicon (among other lexicons and sentiment features) in a SVM-based review classifier. Testing on Pang et al.'s [134] standard corpus of movie reviews, they achieve 86.0% classification accuracy in their best configuration, but Kamps and Marx's lexicon causes only a minimal change in accuracy ($\pm 1\%$) when added to other feature sets. It seems, then, that Kamps and Marx's lexicon doesn't help in sentiment analysis tasks, though there has not been enough research to tell whether Osgood's theory is at fault, or whether the Kamps and Marx's lexicon construction technique is at fault.

**2.7.6 Bednarek's parameter-based approach to evaluation.** Bednarek [18] developed another approach to evaluation, classifying evaluations into several different evaluative parameters shown in Figure 2.3. She divides the evaluative parameters into two groups. The first group of parameters, *core evaluative parameters*, directly convey approval or disapproval, and consist of evaluative scales with two poles. The scope covered by these core evaluative parameters is larger than the scope of most other theories of evaluation. The second group of parameters, *peripheral evaluative parameters*, concerns the positioning of evaluations, and the level of commitment that authors have to the opinions they write.

**2.7.7 Asher's theory of opinion expressions in discourse.** Asher et al. [7, 8] developed an approach to evaluation intended to study how opinions combine with discourse structure to develop an overall opinion for a document. They consider how clause-sized units of text combine into larger discourse structures, where each clause is classified into types that convey approval/disapproval or interpersonal positioning, as shown in Figure 2.4 as well as the orientation, strength, and modality of the opinion or interpersonal positioning. They identify the discourse relations Contrast, Correction, Explanation, Result, and Continuation that make up the higher-level discourse units and compute opinion type, orientation, strength,

**Core Evaluative Parameters**

| | |
|---|---|
| Comprehensibility | Comprehensible: *plain, clear*<br>Incomprehensible: *mysterious, unclear* |
| Emotivity | Positive: *a polished speech*<br>Negative: *a rant* |
| Expectedness | Expected: *familiar, inevitably*<br>Unexpected: *astonishing, surprising*<br>Contrast: *but, however*<br>Contrast/Comparison: *not, no, hardly* |
| Importance | Important: *key, top, landmark*<br>Unimportant: *minor, slightly* |
| Possibility/Necessity | Necessary: *had to*<br>Not Necessary: *need not*<br>Possible: *could*<br>Not Possible: *inability, could not* |
| Reliability | Genuine: *real*<br>Fake: *choreographed*<br>High: *will, likely to*<br>Medium: *likely*<br>Low: *may* |

**Peripheral Evaluative Parameters**

| | |
|---|---|
| Evidentiality | Hearsay: *I heard*<br>Mindsay: *he thought*<br>Perception: *seem, visibly, betray*<br>General knowledge: *(in)famously*<br>Evidence: *proof that*<br>Unspecific: *it emerged that* |
| Mental State | Belief/Disbelief: *accept, doubt*<br>Emotion: *scared, angry*<br>Expectation: *expectations*<br>Knowledge: *know, recognize*<br>State-of-Mind: *alert, tired, confused*<br>Process: *forget, ponder*<br>Volition/Non-Volition: *deliberately, forced to* |
| Style | Self: *frankly,briefly*<br>Other: *promise,threaten* |

Figure 2.3. Evaluative parameters in Bednarek's theory of evaluation [from 18]

| | |
|---|---|
| Reporting | Inform: *inform, notify, explain*<br>Assert: *assert, claim, insist*<br>Tell: *say, announce, report*<br>Remark: *comment, observe, remark*<br>Think: *think, reckon, consider*<br>Guess: *presume, suspect, wonder* |
| Judgment | Blame: *blame, criticize, condemn*<br>Praise: *praise, agree, approve*<br>Appreciation: *good, shameful, brilliant* |
| Advise | Recommend: *advise, argue for*<br>Suggest: *suggest, propose*<br>Hope: *wish, hope* |
| Sentiment | Anger/CalmDown: *irritation, anger*<br>Astonishment: *astound, daze*<br>Love, fascinate: *fascinate, captivate*<br>Hate, disappoint: *demoralize, disgust*<br>Fear: *fear, frighten, alarm*<br>Offense: *hurt, chock*<br>Sadness/Joy: *happy, sad*<br>Bore/Entertain: *bore, distraction* |

Figure 2.4. Opinion Categories in Asher et al.'s [7] theory of opinion in discourse.

and modality of these discourse units based on the units being combined, and the relationship between those units. Their work in discourse relations is based on Segmented Discourse Representation Theory [9], an alternative theory to the Rhetorical Structure Theory more familiar to natural language processing researchers.

In this theory of evaluation, the Judgment, Sentiment, and Advise attitude types (Figure 2.4) convey approval or disapproval, and the Reporting type conveys positioning and commitment.

**2.7.8 Polar facts.** Some of the most useful information in product reviews consists of factual information that a person who has knowledge of the product domain can use to determine for himself that the fact is a positive or a negative thing for the product in question. This has been referred to in the literature as polar facts [168], evaluative factual subjectivity [128], or inferred opinion [183]. This is a kind

of evoked appraisal [20, 104, 108] requiring the same kind of inference as metaphors and subjectivity to understand. Thus, polar facts should be separated from explicit evaluation because of the inference and domain knowledge that it requires, and because of the ease with which people can disagree about the sentiment that is implied by these personal facts. Some work in sentiment analysis explicitly recognizes polar facts and treats them separately from explicit evaluation [128, 168]. However, most work in sentiment analysis has not made this distinction, and sought to include it in the sentiment analysis model through supervised learning or automatic domain adaptation techniques [11, 24].

## 2.8 Local Grammars

In general, the term "parsing" in natural language processing is used to refer to problem of parsing using a *general grammar*. A general grammar for a language is a grammar that is able to derive a complete parse of an arbitrary sentence in the language. General grammar parsing usually focuses on structural aspects of sentences, with little specialization toward the type of content which is being analyzed or the type of analysis which will ultimately be performed on the parsed sentences. General grammar parsers are intended to parse the whole of the language based on syntactic constituency, using formalisms such as probabilistic context free grammars (e.g. the annotation scheme of the Penn Treebank [106] and the parser by Charniak and Johnson [33]), head-driven phrase structure grammars [135], tree adjoining grammars [83], dependency grammars [130], link grammar [153, 154], or other similarly powerful models.

In contrast, there are several different notions of *local grammars* which aim to fill perceived gaps in the task of general grammar parsing:

- Analyzing constructions that should ostensibly be covered by the general gram-

mar, but have more complex constraints than are typically covered by a general grammar.

- Extracting constructions which appear in text, but can't easily be covered by the general grammar, such as street addresses or dates.
- Extracting pieces of text that can be analyzed with the general grammar, but discourse concerns demand that they be analyzed in another way at a higher level.

The relationships and development of all of these notions will be discussed shortly, but the one unifying thread that recurs in the literature about these disparate concepts of a local grammar is the idea that local grammars can or should be parsed using finite-state automata.

The first notion of a local grammar is the use of finite state automata to analyze constructions that should ostensibly be covered by the general grammar, but have more detailed and complex constraints than general grammars typically are concerned with. Similar to this is the notion of constraining idiomatic phrases to only match certain forms. This was introduced by Gross [62, 63], who felt that transformational grammars did not express many of the constraints and transformations used by speakers of a language, particularly when using certain kinds of idioms. He proposed [63] that:

> For obvious reasons, grammarians and theoreticians have always attempted to describe the general features of sentences. This tendency has materialized in sweeping generalizations intended to facilitate language teaching and recently to construct mathematical systems. But beyond these generalities lies an extremely rigid set of dependencies between individual words, which is huge in size; it has been accumulated over the millenia by language users, piece by piece, in micro areas such as those we began to analyze here. We have studied elsewhere what we call the lexicon-grammar of free sentences. The lexicon-grammar of French is a description of the argument structure of about 12,000 verbs. Each verbal entry

> has been marked for the transformations it accepts. It has been shown that every verb had a unique syntactic paradigm.

He proposes that the "rigid set of dependencies between individual words" can be modeled using local grammars, for example using a local grammar to model the argument structure of the French verbs.

Several other researchers have done work on this notion of local grammars, including Breidt et al. [29] who developed a regular expression language to parse these kinds of grammars, sun Choi and sun Nam [161] who constructed a local grammar to extract five contexts where proper nouns are found in Korean, and Venkova [172] who analyzes Bulgarian constructions that contain the *da-* conjunction. Other examples of this type of local grammar notion abound.

The next similar notion to Gross' definition of local grammars is the extraction of phrases that appear in text, but can't easily be covered by the general grammar, such as street addresses or dates. This is presented by Hunston and Sinclair [72] as the justification for local grammars. Hunston and Sinclair do not actually ever analyze a local grammar according to this second notion, nor have I found any other work that uses this notion of a local grammar. Instead, their work which I have cited presents a local grammar of appraisal based on the third notion of a local grammar: extracting pieces of text that can be analyzed with the general grammar, but particular applications demand that they be analyzed in another way at a higher level.

This third notion of local grammar was pioneered by Barnbrook [15, 16]. Barnbrook analyzed the Collins COBUILD English Dictionary [151] to study the form of definitions included in the dictionary, and to study the ability to extract different functional parts of the definitions. Since the Collins COBUILD English Dictionary is a learner's dictionary which gives definitions for words in the form of full sentences, it

| Text before headword | | Headword | Text after headword | | | |
|---|---|---|---|---|---|---|
| First part | | | | Second part | | |
| Hinge | Carrier | Headword | Object | Carrier Ref. | Explanation | Object Ref. |
| If | someone or something | **is geared** | to a particular purpose, | they | are organized or designed to be suitable | for it. |

Figure 2.5. A dictionary entry in Barnbrook's local grammar

could be parsed by general grammar parsers, but the result would be completely useless for the kind of analysis that Barnbrook wished to perform. Barnbrook developed a small collection of sequential patterns that the COBUILD definitions followed, and developed a parser to validate his theory by parsing the whole dictionary correctly.

An example of such a pattern can be applied to the definition:

> If someone or something **is geared** to a particular purpose, they are organized or designed to be suitable for it.

The definition is classified to be of type B2 in their grammar, and it is broken down into several components, shown in Figure 2.5.

Hunston and Sinclair's [72] local grammar of evaluation is based on the same framework. In their paper on the subject, they elaborate on the process for local grammar parsing. According to their process, parsing a local grammar consists of three steps: a parser must first detect which regions of the text it should parse, then it should determine which pattern to use. Finally, it should parse the text, using the pattern it has selected.

This notion of a local grammar is different from Gross's, but Hunston and Francis [71] have done grammatical analysis similar to Gross's as well. They called the formalism a *pattern grammar*. With pattern grammars, Hunston and Francis are concerned with cataloging the valid grammatical patterns for words which will

appear in the COBUILD dictionary, for example, the kinds of objects, complements, and clauses which verbs can operate on, and similar kinds of patterns for nouns, adjectives and adverbs. These are expressed as sequences of constituents that can appear in a given pattern. An example of some of these patterns are for one sense of the verb "fantasize": **V "about" n/-ing**, **V that**, **also V -ing**. The capitalized **V** indicates that the verb fills that slot, other pieces of a pattern indicate different types of structural components that can fill those slots. Hunston and Francis discuss the patterns from the standpoint of how to identify patterns to catalog them in the dictionary (what is a pattern, and what isn't a pattern), how clusters of patterns relate to similar meanings, and how patterns overlap each other, so a sentence can be seen as being made up of patterns of overlapping sentences. Since they are concerned with constructing the COBUILD dictionary Sinclair [152], there is no discussion of how to parse pattern grammars, either on their own, or as constraints overlaid onto a general grammar.

Mason [111] developed a local grammar parser applying the for studying COBUILD patterns to arbitrary text In his parser, a part-of-speech tagger is used to find all of the possible parts of speech that can be assigned to each token in the text. A finite state networks describing the permissible neighborhood for each word of interest is constructed by combining the different patterns for that word found in the Collins COBUILD Dictionary [152]. Additional finite state networks are defined to cover certain important constituents of the COBUILD patterns, such as noun groups, and verb groups. These finite state networks are applied using an RTN parser [38, 184] to parse the documents.

Mason's parser was evaluated to study how it fared at selecting the correct grammar pattern for occurrences of the words "blend" (where it was correct about 54 out of 56 occurrences), and "link" (where it was correct about 73 out of 116 oc-

currences). Mason and Hunston's [112] local grammar parser is only slightly different from Mason's [111]. It is likely an earlier version of the same parser.

## 2.9 Barnbrook's COBUILD Parser

Numerous examples of local grammars according to Gross's definition have been published. Many papers that describe a local grammar based on Gross's notion specify a full finite state automaton that can parse that local grammar [29, 62, 63, 111, 123, 161, 172]. Mason's [111] parser, described above, is more complicated but is still aimed at Gross's notion of a local grammar. On the other hand, the only parser developed according to Barnbrook's notion of a local grammar parser is Barnbrook's own parser. Because his formulation of a local grammar is closest to my own work, and because some parts of its operation are not described in detail in his published writings, I describe his parser in detail here. Barnbrook's parser is discussed in most detail in his Ph.D. thesis [15], but there is some discussion in his later book [16]. For some details that were not discussed in either place, I contacted him directly [17] to better understand the details.

Barnbrook's parser is designed to validate the theory behind his categorization of definition structures, so it is developed with full knowledge of the text it expects to encounter, and achieves nearly 100% accuracy in parsing the COBUILD dictionary. (The few exceptions are definitions that have typographical errors in them, and a single definition that doesn't fit any of the definition types he defined.) The parser would most likely have low accuracy if it encountered a different edition of the COBUILD dictionary with new definitions that were not considered while developing the parser, and its goal isn't to be a general example of how to parse general texts containing a local grammar phenomenon. Nevertheless, its operation is worth understanding.

Barnbrook's parser accepts as input a dictionary definition, marked to indicate

where the head word is located in the text of the definition, and augmented with a small amount of other grammatical information listed in the dictionary.

Barnbrook's parser operates in three stages. The first stage identifies which type of definition is to be parsed, according to Barnbrook's structural taxonomy of definition types. The definition is then dispatched to one of a number of different parsers implementing the second stage of the parsing algorithm, which is to break down the definition into functional components. There is one second-stage parser for each type of definition. The third stage of parsing further breaks down the explanation element of the definition, by searching for phrases which correspond to or co-refer to the head-word or its co-text (determined by the second stage), and assigning them to appropriate functional categories.

The first stage is a complex handwritten rule-based classifier, consisting of about 40 tests which classify definitions and provide flow control. Some of these rules are simple, trying to determine whether there is a certain word in a certain position of the text, for example:

> If field 1 (the text before the head word) ends with "is" or "are", mark as definition type F2, otherwise go on to the next test.

Others are more complicated:

> If field 1 contains "if" or "when" at the beginning or following a comma, followed by a potential verb subject, and field 1 does not end with an article, and field 1 does not contain "that" and field 5 (the part of speech specified in the dictionary) contains a verb grammar code, mark as definition type B1, otherwise go to the next test.

or:

> If field 1 contains a type J projection verb, mark as type J2, otherwise mark as type G3.

Many of these rules (such as the above example) depend on lists of words culled from the dictionary to fill certain roles. Stage 1 is painstakingly hand-coded and developed with knowledge of all of the definitions in the dictionary, to ensure that all of the necessary words to parse the dictionary are included in the word list.

Each second stage parser uses lists of words to identify functional components[2]. It appears that there are two types of functional components: short ones with relatively fixed text, and long ones with more variable text. Short functional components are recognized through highly rule-based searches for specific lists of words in specific positions. The remaining longer functional components contain more variable text, and are recognized by the short functional components (or punctuation) that they are located between. The definition taxonomy is structured so that it does not have two adjacent longer functional components — they are always separated by shorter functional components or punctuation.

The third stage of parsing (which Barnbrook actually presents as the second step of the second stage) then analyzes specific functional elements (typically the explanation element which actually defines the head word) identified by the second stage, and using lists of pronouns, and the text of other functional elements in the definition to identify elements which co-refer to these other elements in the definition.

The parser, as described, has two divergences from Hunston and Sinclair's framework for local grammar parsing. First, while most local grammar work assumes that a local grammar is suitable to be parsed using a finite state automaton, we see that it is not implemented as a finite state automaton, though it may be computationally equivalent to one. Second, while Barnbrook's parser is designed to determine

[2] The second stage parser is not well documented in any of Barnbrook's writings. After reading Barnbrook's writings, I emailed this description to Barnbrook, and he replied that my description of the recognition process was approximately correct.

which pattern to use to parse a specific definition, and to parse according to that pattern, his parser takes advantage of the structure of the dictionary to avoid having to determine which text matches the local grammar in the first place.

## 2.10 FrameNet labeling

FrameNet [144] is a resource that aims to document the semantic structure for each English word in each of its word senses, through annotations of example sentences. FrameNet frames have often been seen as starting point for extracting higher-level linguistic phenomena. To apply these kinds of techniques, first one must identify FrameNet frames correctly, and then one must correctly map the FrameNet frames to higher-level structures.

To identify FrameNet frames, Gildea and Jurafsky [60] developed a technique where they apply simple probabilistic models to pre-segmented sentences to identify semantic roles. It uses maximum likelihood estimation training and models that are conditioned on the target word, essentially leading to a different set of parameters for each verb that defines a frame. To develop an automatic segmentation technique, they used a classifier to identify which phrases in a phrase structure tree are semantic constituents. Their model decides this based on probabilities for the different paths between the verb that defines the frame, and the phrase in question. Fleischman et al. [56] improved on these techniques by using Maximum Entropy classifiers, and by extending the feature set for the role labeling task.

Kim and Hovy [89] developed a technique for extracting appraisal expressions by determining the FrameNet frame to be used for opinion words, and extracting the frames (filling their slots) and then selecting which slots in which frames are the opinion holder and the opinion topic. When run on ground truth FrameNet data (experiment 1), they report 71% to 78% on extracting opinion holders, and 66% to

70% on targets. When they have to extract the frames themselves (experiment 2), accuracy drops to 10% to 30% on targets and 30% to 40% on opinion holders, though they use very little data for this second experiment. These results suggest that the major stumbling block is in determining the frame correctly, and that there's a good mapping between a textual frame and an appraisal expression.

## 2.11 Information Extraction

The task of local grammar parsing is similar in some ways to the task of information extraction (IE), and techniques used for information extraction can be adapted for use in local grammar parsing.

The purpose of information extraction is to locate information in unstructured text which is topically related, and fill out a template to store the information in a structured fashion. Early research, particularly the early Message Understanding Conferences (MUC), focused on the task of template filling, building a whole system to fill in templates with tens of slots, by reading unstructured texts. More recent research specialized on smaller subtasks as researchers developed a consensus on the subtasks that were generally involved in template filling. These smaller subtasks include bootstrapping extraction patterns, named entity recognition, coreference resolution, relation prediction between extracted elements, and determining how to unify extracted slots and binary relations into multi-slot templates.

A full overview of information extraction is presented by Turmo et al. [169]. I will outline here some of the most relevant work to my own.

Template filling techniques are generally built as a cascade of several layers doing different tasks. While the exact number and function of the layers may vary, the functionality of the layers generally includes the following: document preprocessing, full or partial syntactic parsing, semantic interpretation of parsed sentences, discourse

analysis to link the semantic interpretations of different sentences, and generation of the output template.

An early IE system is that of Lenhert et al. [96], who use single word triggers to extract slots from a document. The entire document is assumed to describe a single terrorism event (in MUC-3's Latin American terrorism domain) so an entire document contains just a single template. Extraction is a matter of extracting text and determining which slot that text fills.

A template-filling IE system closest to the finite-state definition of local grammar parsing is FASTUS. FASTUS [4, 67, 68] is a template-filling IE system entered in MUC-4 and MUC-5 based on hand-built finite state technology. FASTUS uses five levels of cascaded finite-state processing. The lowest level looks to recognize and combine compound words and proper names. The next level performs shallow parsing, recognizing simple noun groups, verb groups, and particles. The third level uses the simple noun and verb groups to identify complex noun and verb groups, which are constructed by performing an number of operations such as attaching appositives to the noun group they describe, conjunction handling, and attachment of prepositional phrases. The fourth level looks for domain-specific phrases of interest, and creates structures containing the information found. The highest level merges these structures to create templates relevant to specific events. The structure of FASTUS is similar to Gross's local grammar parser, in that both spell out the complete structure of the patterns they are parsing.

It has recently become more desirable to develop information extraction systems that can learn extraction patterns, rather than being hand coded. While the machine-learning analogue of FASTUS's finite state automata would be to use hidden Markov models (HMMs) for extraction, or to use one of the models that have evolved from hidden Markov models, like maximum entropy tagging [142] or conditional ran-

dom fields (CRFs) [114], these techniques are typically not developed to operate like FASTUS or Gross's local grammar parser. Rather, the research on HMM and CRF techniques has been concerned with developing models to extract a single kind of reference, by tagging the text with "BEGIN-CONTINUE-OTHER" tags, then using other means to turn those into templates. HMM and CRF techniques have recently become the most widely used techniques for information extraction. Two typical examples of probabilistic techniques for information extraction are as follows.

Chieu and Ng [34] use two levels of maximum entropy learning to perform template extraction. Their system learns from a tagged document collection. First, they do maximum entropy tagging [142] to extract entities that will fill slots in the created template. Then, they perform maximum entropy classification on pairs of entities to determine which entities belong to the same template. The presence of positive relations between pairs of slots is taken as a graph, and the largest and highest-probability cliques in the graph are taken as filled-in templates. Another similar technique is that of Feng et al. [54], who use conditional random fields to segment the text into regions that each contain a single data record. Named entity recognition is performed on the text, and all named entities that appear in a single region of text are considered to fill slots in the same template. Both of these two techniques use features derived from a full syntactic parse as features for the machine learning taggers, but their overall philosophy does not depend on these features.

There are also techniques based directly on full syntactic parsing. One example is Miller et al. [119] who train an augmented probabilistic context free grammar to treat both the structure of the information to be extracted and the general syntactic structure of the text in a single unified parse tree. Another example is Yangarber et al.'s [186] system which uses a dependency-parsed corpus and a bootstrapping technique to learn syntactic-based patterns such as [Subject: Company, Verb: "appoint",

Direct Object: Person] or [Subject: Person, Verb: "resign"].

Some information extraction techniques aim to be domainless, looking for relations between entities in corpora as large and varied as the Internet. Etzioni et al. [51] have developed a the KnowItAll web information extractions system for extracting relationships in a highly unsupervised fashion. The KnowItAll system extracts relations given an ontology of relation names, and a small set of highly generic textual patterns for extracting relations, with placeholders in those patterns for the relation name, and the relationship's participants. An example of a relation would be the "country" relation, with the synonym "nation". An example extraction pattern would be `<Relation> [,] such as <List of Instances>`, which would be instantiated by phrases like "cities, such as San Francisco, Los Angeles, and Sacramento". Since KnowItAll is geared toward extracting information from the whole world wide web, and is evaluated in terms of the number of correct and incorrect relations of general knowledge that it finds, KnowItAll can afford to have very sparse extraction, and miss most of the more specific textual patterns that other information extractors use to extract relations.

After extracting relations, KnowItAll computes the probability of each extracted relation. It generates discriminator phrases using class names and keywords of the extraction rules to find co-occurrence counts, which it uses to compute probabilities. It determines positive and negative instances of each relation using PMI between the entity and both synonyms. Entities with high PMI to both synonyms are concluded to be positive examples, and entities with high PMI to only one synonym are concluded to be negative examples.

The successor to KnowItAll is Banko et al.'s [14] TextRunner system. Its goals are a generalization of KnowItAll's goals. In addition to extracting relations from the web, which may have only very sparse instances of the patterns that TextRunner

recognizes, and extracting these relations with minimal training, TextRunner adds the goal that it seeks to do this without any prespecified relation names.

TextRunner begins by training a naive Bayesian classifier from a small unlabeled corpus of texts. It does so by parsing those texts, finding all base noun phrases, heuristically determining whether the dependency paths connecting pairs of noun phrases indicate reliable relations. If so, it picks a likely relation name from the dependency path, and trains the Bayesian classifier using features that do not involve the parse. (Since it's inefficient to parse the whole web, TextRunner merely trains by parsing a smaller corpus of texts.)

Once trained, TextRunner finds relations in the web by part-of-speech tagging the text, and finding noun phrases using a chunker. Then, TextRunner looks at pairs of noun phrases and the text between them. After heuristically eliminating extraneous text from the noun phrases and the intermediate text, to identify relationship names, TextRunner feeds the noun phrase pair and the intermediate text to the naive Bayesian classifier to determine whether the relationship is trustworthy. Finally, TextRunner assigns probabilities to the extracted relations using the same technique as KnowItAll.

KnowItAll and TextRunner push the edges of information extraction towards generality, and have been referred to under heading of Open Information Extraction [14] or Machine Reading [50]. These are the opposite extreme from local grammar parsing. The goals of open information extraction are to compile a database of general knowledge facts, and at the same time learn very general patterns for how this knowledge is expressed in the world at large. Accuracy of open information extraction is evaluated in terms of the number of correct propositions extracted, and there is a very large pool of text (the Internet) from which to find these propositions. Local grammar parsing has the opposite goals. It is geared towards identifying and un-

derstanding the specific textual mentions of the phenomena it describes, and toward understanding the patterns that describe those specific phenomena. It may be operating on small corpora, and it is evaluated in terms of the textual mentions it finds and analyzes.

# CHAPTER 3

# FLAG'S ARCHITECTURE

## 3.1 Architecture Overview

FLAG's architecture (shown in Figure 3.1) is based on the three-step framework for parsing local grammars described in Chapter 1. These three steps are:

1. Detecting ranges of text which are candidates for local grammar parsing.
2. Finding entities and relationships between entities, and analyzing features of the possible local grammar parses, using all known local grammar patterns.
3. Choosing the best local grammar parse at each location in the text, based on information from the candidate parses and from contextual information.

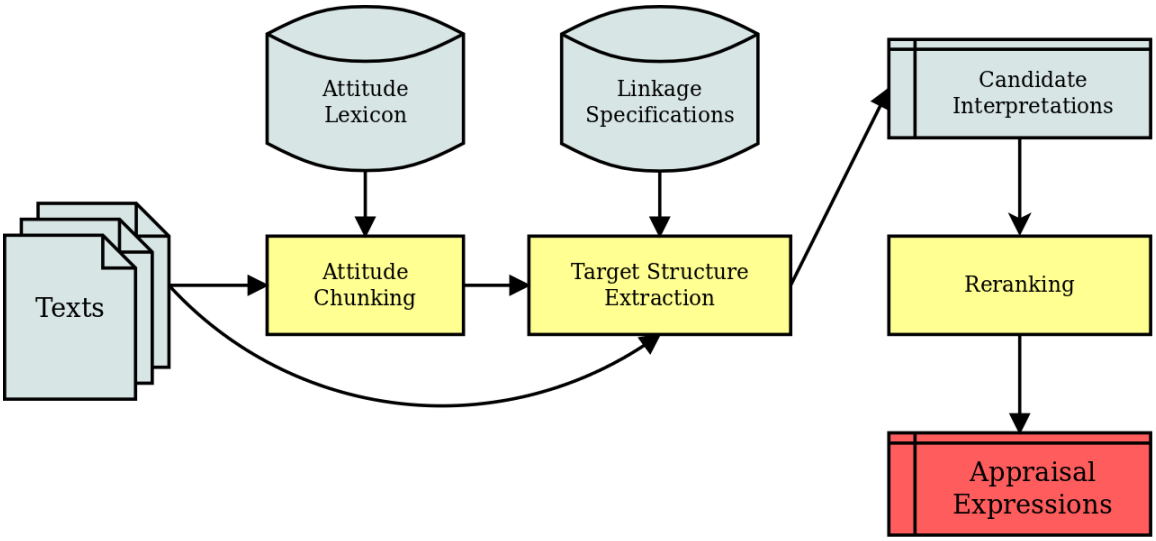



Figure 3.1. FLAG system architecture

FLAG's first step is to find attitude groups using a lexicon-based shallow parser, and to determine the values of several attributes which describe the attitude. The shallow parser, described in Chapter 6, finds a head word and takes that head

word's attribute values from the lexicon. It then looks leftwards to find modifiers, and modifies the values of the attributes based on instructions coded for that word in the lexicon. Because words may be double-coded in the lexicon, the shallow parser retains all of the codings, leading to multiple interpretations of the attitude group. The best interpretation will be selected in the last step of parsing, when other ambiguities will be resolved as well.

Starting with the locations of the extracted attitude groups, FLAG identifies appraisal targets, evaluators, and other parts of the appraisal expression by looking for specific patterns in a syntactic dependency parse, as described in Chapter 7. During this processing, multiple different matching syntactic patterns may be found, and these will be disambiguated in the last step.

The specific patterns used during this phase of parsing are called *linkage specifications*. There are several ways that these linkage specifications may be obtained. One set of linkage specifications was developed by hand, based on patterns described by Hunston and Sinclair [72]. Other sets of linkage specifications are learned using algorithms described in Chapter 8. The linkage specification learning algorithms reuse FLAG's attitude chunker and linkage associator in different configurations depending on the learning algorithm. Those configurations of FLAG are shown in Figures 8.6 and 8.8.

Finally, all of the extracted appraisal expression candidates are fed to a machine learning reranker to select the best candidate parse for each attitude group (Chapter 9). The various parts of the each appraisal expression candidate are analyzed create a feature vector for each candidate, and support vector machine reranking is used to select the best candidates. Alternatively, the machine-learning reranker may be bypassed, in which case the candidate with the most specific linkage specification is automatically selected as the correct linkage specification.

## 3.2 Document Preparation

Before FLAG can extract any appraisal expressions from a corpus, the documents have to be split into sentences, tokenized, and parsed. FLAG uses the Stanford NLP Parser version 1.6.1 [41] to perform all of this preprocessing work, and it stores the result in a SQL database for easy access throughout the appraisal expression extraction process.

**3.2.1 Tokenization and Sentence Splitting.** In three of the 5 corpora I tested FLAG on (the JDPA corpus, the MPQA corpus[3], and the IIT corpus), the text provided was not split into sentences or into tokens. On these documents, FLAG used Stanford's `DocumentPreprocessor` to split the document into sentences, and the `PTBTokenizer` class to split each sentence into tokens, and normalize the surface forms of some of the tokens, while retaining the start and end location of each token in the text.

The UIC Sentiment corpus's annotations are associated with particular sentences. For each product in the corpus, all of the reviews for that product are shipped in a single document, delimited by lines indicating the title of each review. For some products, the individual reviews are not delimited and there is no way to tell where one review ends and the next begins. The reviews come with one sentence per line, with product features listed at the beginning of each line, followed by the text of the sentence. To preprocess these documents, FLAG extracted the text of each sentence, and retained the sentence segmentation provided with the corpus, so that extracted appraisal targets could be compared against the correct annotations. FLAG used the `PTBTokenizer` class to split each sentence into tokens.

[3] Like the Darmstadt corpus, the MPQA corpus ships with annotations denoting the correct sentence segmentation, but because there are no attributes attached to these annotations, I saw no need to use them.

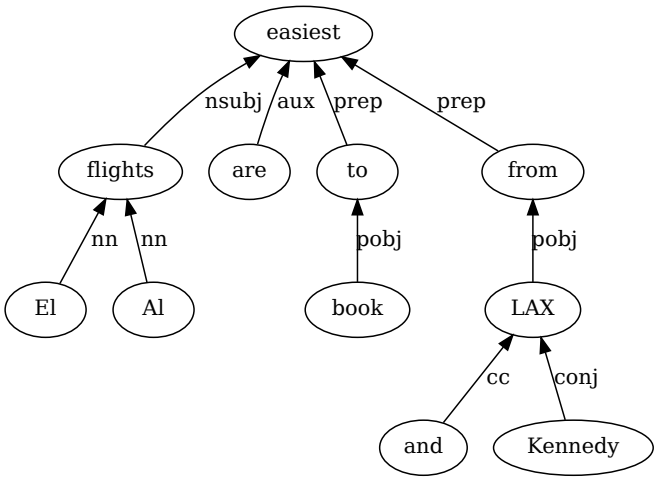
The Darmstadt Service Review Corpus is provided in plain-text format, with a separate XML file listing the tokens in the document (by their textual content). Separate XML files list the sentence level annotations and the sub-sentence sentiment annotations in each document. In the format that the Darmstadt Service Review corpus is provided, the start and end location of each of these annotations is given as a reference to the starting and ending token, not the character position in the plain-text file. To recover the character positions, FLAG aligned the provided listing of tokens against the plain text files provided to determine the start and end positions of each token, and then used this information to determine the starting and ending positions of the sentence and sub-sentence annotations. There were a couple of obvious errors in the sentence annotations that I corrected by hand — one where two words were omitted from the middle of a sentence, and another where two words were added to a sentence from an unrelated location in the same document — and I also hand-corrected the tokens files to fix some XML syntax problems. FLAG used the sentence segmentation provided with the corpus, in order to be able to omit non-opinionated sentences determining extraction accuracy, but used the Stanford Parser's tokenization (provided by the `PTBTokenizer` class) when working with the document internally, to avoid any errors that might be caused by systematic differences between the Stanford Parser's tokenization which FLAG expects, and the tokenization provided with the corpus.

**3.2.2 Syntactic Parsing.** After the documents were split into sentences and tokenized, they were parsed using the `englishPCFG` grammar provided with the Stanford Parser. Three parses were saved:

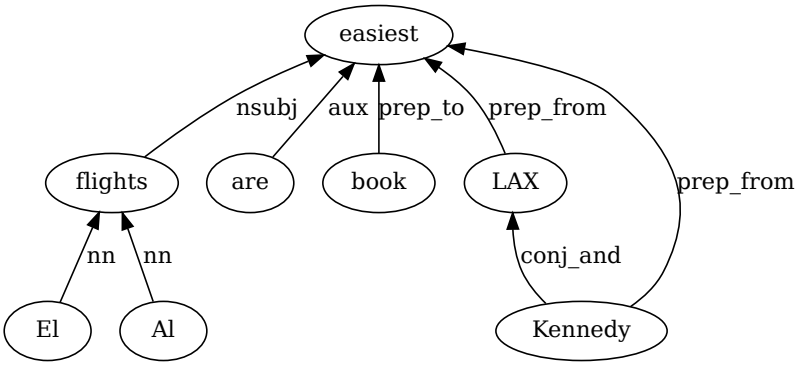
- The PCFG parse returned by `LexicalizedParser.getBestParse`, which was used by FLAG to determine the start and end of each slot extracted by the associator (Chapter 7).

- The typed dependency tree returned by `GrammaticalStructure.typedDependencies`, which was used by FLAG's linkage specification learner (Section 8.4).
- An augmented version of the collapsed dependency DAG returned by `GrammaticalStructure.typedDependenciesCCprocessed`, which was used by the associator (Chapter 7) to match linkage specifications.

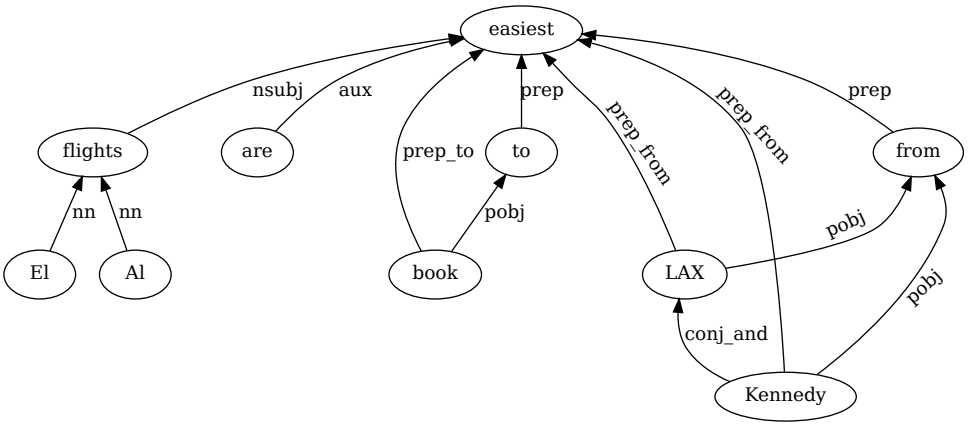
The typed dependency tree was ideal for FLAG's linkage specification learner, because each token (aside from the root) has only one token that governs it, as shown in Figure 3.2(a). The dependency tree has an undesirable feature of how it handles conjunctions, specifically that an extra link needs to be traversed in order to find the tokens on both sides of a conjunction, so different linkage specifications would be needed to extract each side of the conjunction. This is undesirable when actually extracting appraisal expressions using the learned linkage specifications in Chapter 7. The collapsed dependency DAG solves this problem, but adds another — where the uncollapsed tree represents prepositions with `prep` link and a `pobj` link, the DAG collapses this to a single link (`prep_for`, `prep_to`, etc.), and leaves the preposition token itself without any links. This is undesirable for two reasons. First, this is a potentially serious discrepancy between the uncollapsed dependency tree and the collapsed dependency DAG. Second, with the preposition specific links, it is impossible to create a single linkage specification one structural pattern but matches several different prepositions. Therefore, FLAG resolves this discrepancy by adding back the `prep` and `pobj` links and coordinating them across conjunctions, as shown in Figure 3.2(c).




(a) Uncollapsed dependency tree




(b) Collapsed dependency DAG generated by the Stanford Parser




(c) Collapsed dependency DAG, as augmented by FLAG.

Figure 3.2. Different kinds of dependency parses used by FLAG.

# CHAPTER 4
# THEORETICAL FRAMEWORK

## 4.1 Appraisal Theory

Appraisal theory [109, 110] studies language expressing the speaker or writer's opinion, broadly speaking, on whether something is good or bad. Based in the framework of systemic-functional linguistics [64], appraisal theory presents a grammatical system for appraisal, which presents sets of options available to the speaker or writer for how to convey their opinion. This system is pictured in Figure 4.1. The notation used in this figure is described in Appendix A. (Note that Taboada's [163] understanding of the APPRAISAL system differs from mine — in her version, the AFFECT TYPE and TRIGGERED systems apply regardless of the option selected in the REALIS system.)

There are four systems in appraisal theory which concern the expression of an attitude. Probably the most obvious and important distinction in appraisal theory is the ORIENTATION of the attitude, which differentiates between appraisal expressions that convey approval and those that convey disapproval — the difference between good and bad evaluations, or pleasant and unpleasant emotions.

The next important distinction that the appraisal system makes is the distinction between *evoked appraisal* and *inscribed appraisal* [104], contained in the EXPLICIT system. Evoked appraisal is expressed by evoking emotion in the reader by describing experiences that the reader identifies with specific emotions. Evoked appraisal includes such phenomena as sarcasm, figurative language, idioms, and polar facts [108]. An example of evoked appraisal would be the phrase "it was a dark and stormy night", which triggers a sense of gloom and foreboding in the reader. Another example would be the sentence "the SD card had very low capacity", which

not obviously negative to someone who doesn't know what an SD card is. Evoked appraisal can make even manual study of appraisal difficult and subjective, and is certainly difficult for computers to parse. Additionally, some of the other systems and constraints in Figure 4.1 do not apply to evoked appraisal.

By contrast, inscribed appraisal is expressed using explicitly evaluative lexical choices. The author tells the reader exactly how he feels, for example saying "I'm unhappy about this situation." These lexical expressions require little context to understand, and are easier for a computer to process. Whereas a full semantic knowledge of emotions and experiences would be required to process evoked appraisal, the amount of context and knowledge required to process inscribed appraisal is much less. Evoked appraisal, because of the more subjective element of its interpretation, is beyond the scope of appraisal expression extraction, and therefore beyond the scope of what FLAG attempts to extract. (One precedent for ignoring evoked appraisal is Bednarek's [20] work on affect. She makes a distinction between what she calls emotion talk (inscribed) and emotional talk (evoked) and studies only emotion talk.)[4]

A central contribution of appraisal theory is the ATTITUDE system. It divides attitudes into three main types (appreciation, judgment, and affect), and deals with the expression of each of these types.

*Appreciation* evaluates norms about how products, performances, and naturally occurring phenomena are valued, when this evaluation is expressed as being a property of the object. Its subsystems are concerned with dividing attitudes into

[4] Many other sentiment analysis systems do handle evoked appraisal, and have many ways of doing so. Some perform supervised learning on a corpus similar to their target corpus [192], some by finding product features first and then determining opinions about those product features by learning what the nearby words mean [136, 137], others by using very domain-specific sentiment resources [40], and others through learning techniques that don't particularly care about whether they're learning inscribed or evoked appraisals [170]. There has been a lot of research into domain adaptation to deal with the differences between what constitutes evoked appraisal in different domains and alleviate the need for annotated training data in every sentiment analysis domain of interest [24, 85, 143, 188].

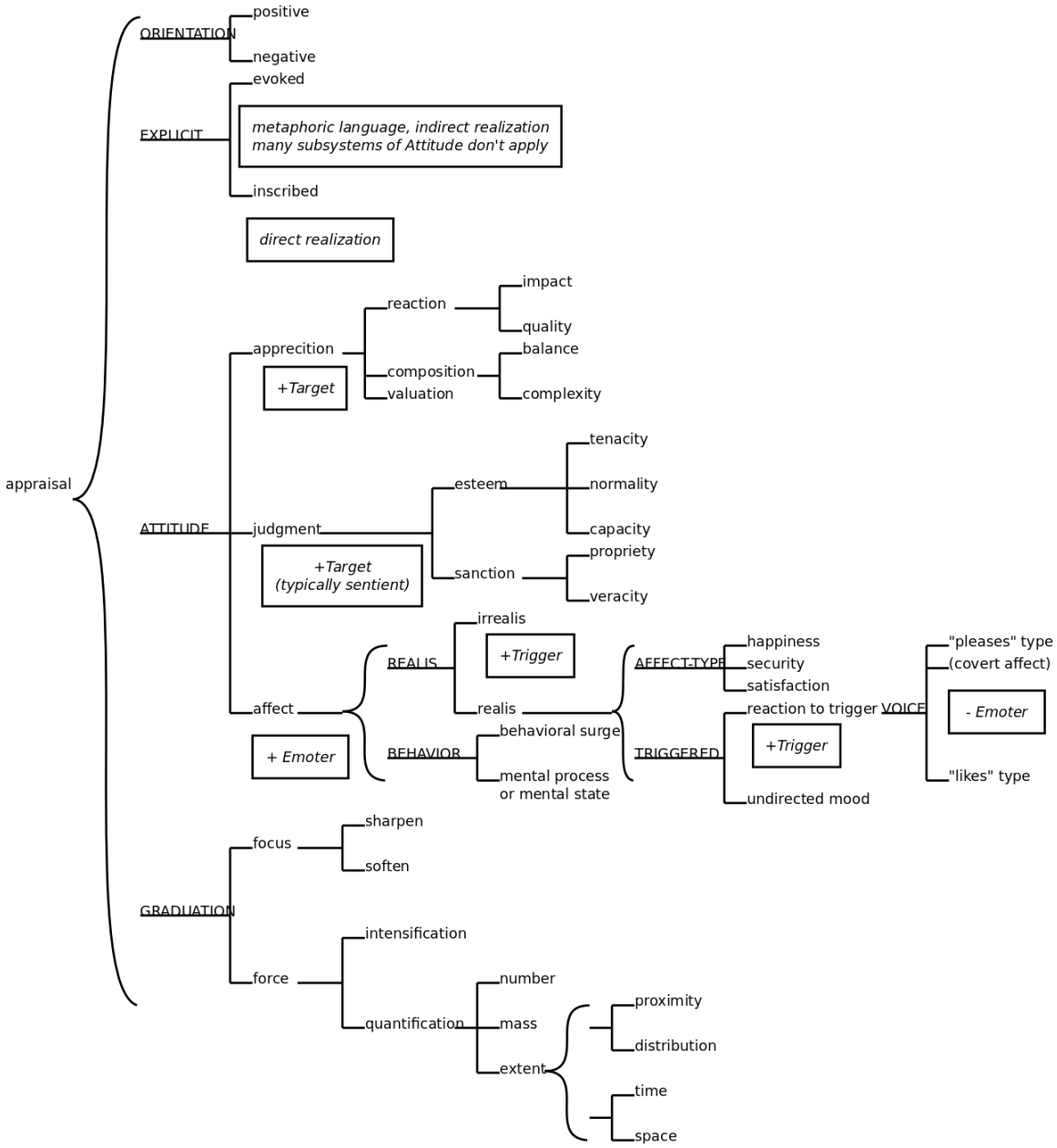



Figure 4.1. The Appraisal system, as described by Martin and White [110]. The notation used is described in Appendix A.

categories that identify their lexical meanings more specifically. The five types each answer different questions about the user's opinion of the object:

**Impact:** Did the speaker feel that the target of the appraisal grabbed his attention? Examples include the words "amazing", "compelling", and "dull."

**Quality:** Is the target good at what it was designed for? Or what the speaker feels it should be designed for? Examples include the words "beautiful", "elegant", and "hideous."

**Balance:** Did the speaker feel that the target hangs together well? Examples include the words "consistent", and "discordant."

**Complexity:** Is the target hard to follow, concerning the number of parts? Alternatively, is the target difficult to use? Examples include the words "elaborate", and "convoluted."

**Valuation:** Did the speaker feel that the target was significant, important, or worthwhile? Examples include the words "innovative", "profound", and "inferior".

*Judgment* evaluates a person's behavior in a social context. Like appreciation, its subsystems are concerned with dividing attitudes into a more fine-grained list of subtypes. Again, there are five subtypes answering different questions about the speaker's feelings about the target's behavior:

**Tenacity:** Is the target dependable or willing to put forth effort? Examples include the words "brave", "hard-working", and "foolhardy".

**Normality:** Is the target's behavior normal, abnormal, or unique? Examples include the words "famous", "lucky", and "obscure."

**Capacity:** Does the target have the ability to get results? How capable is the target? Examples include the words "clever", "competent", and "immature."

**Propriety:** Is the target nice or nasty? How far is he or she beyond reproach? Examples include the words "generous", "virtuous", and "corrupt."

**Veracity:** How honest is the target? Examples include the words "honest", "sincere", and "sneaky."

The ORIENTATION system doesn't necessarily correlate to whether to the presence or absence of the particular qualities for which these subcategories are named. It is concerned with whether the presence or absence of those qualities is a good thing. For example, as applied to normality, singling out someone as "special" or "unique" is different (positive) from singling them out as "weird" (negative), even though both indicate that a person is different from the social norm. Likewise, "conformity" is negative in some contexts, but being "normal" is positive in many, and both indicate that a person is in line with the social norm.

Both judgment and appreciation share in common that they have some kind of target, and that target is mandatory (although it may be elided or inferred from context). It appears that a major difference between judgment and appreciation is in what types of targets they can accept. Judgment typically only accepts conscious targets, like animals or other people, to appraise their behaviors. One cannot, for example, talk about "an evil towel" very easily because "evil" is a type of judgment, but a towel is an object that does not have behaviors (unless anthropomorphized). Propositions can also be evaluated using judgment, evaluating not just the person in a social context, but a specific behavior in a social context. Appreciation takes any kind of target, and treats them as things, so an appraisal of a "beautiful woman" typically speaks of her physical appearance.

The last major type of attitude is *affect*. Affect expresses a person's emotional state, and is a somewhat more complicated system than judgment and appreciation. Rather than having a target and a source, it has an emoter (the person who feels the emotion) and an optional trigger (the immediate reason he feels the emotion). Within the affect system, the first distinctions are whether the attitude is realis (a reaction to an existing trigger) or irrealis (a fear of or a desire for a not-yet existing trigger). There is also distinction as to whether the affect is a mental process ("He liked it") or a behavioral surge ("He smiled"). For realis affect, appraisal theory makes a distinction between different types of affect, and also whether the affect is the response to a trigger. Triggered affect can be expressed in several different lexical patterns: "It pleases him" (where the trigger comes first), "He likes it" (where the emoter comes first), or "It was surprising". (This third pattern, first recognized by Bednarek [21], is called *covert affect*, because of its similarity of expression to appreciation and judgment.)

Affect is also broken down into more specific attitude types based on the lexical meaning of appraisal words. These types, shown in Figure 4.2, were originally developed by Martin and White [110] and were improved by Bednarek [20] to resolve some correspondence issues between the subtypes of positive affect, and the subtypes of negative affect. The difference between their versions is primarily one of terminology, but the potential exists to categorize some attitude groups differently under one scheme than under the other scheme. Also, in Bednarek's scheme, surprise is treated as having neutral orientation (and is therefore not annotated in the IIT sentiment corpus described in Section 5.5). Inclination is the single attitude type for irrealis affect, and the other subtypes are all types of realis affect. In my research, I use Bednarek's version of the affect subtypes, because the positive and negative attitude subtypes correspond better in her version than in Martin and White's. I treat each pair of positive and negative subtypes as a single subtype, named after its positive

| Martin and White | | Bednarek | |
|---|---|---|---|
| General type | Specific type | General type | Specific type |
| **un/happiness** | cheer/misery<br>affection/antipathy | **un/happiness** | cheer/misery<br>affection/antipathy |
| **in/security** | confidence/disquiet<br>trust/surprise | **in/security** | quiet/disquiet<br>trust/distrust |
| **dis/satisfaction** | interest/ennui<br>pleasure/displeasure | **dis/satisfaction** | interest/ennui<br>pleasure/displeasure |
| **dis/inclination** | desire/fear | **dis/inclination** | desire/non-desire |
| | | **surprise** | |

Figure 4.2. Martin and White's subtypes of AFFECT versus Bednarek's

version. I have also simplified the system somewhat by not dealing directly with the other options in the AFFECT system described in the previous paragraph, because it is easier for annotators and for software to deal with a single hierarchy of attitude types, rather than a complex system diagram.

The GRADUATION system concerns the scalability of attitudes, and has two dimensions: *focus* and *force*. Focus deals with attitudes that are not gradable, and deals with how well the intended evaluation actually matches the characteristics of the head word used to convey the evaluation (for example "It was an apology of sorts" has softened focus because the sentence is talking about something that was not quite a direct apology.)

Force deals with attitudes that are gradable, and concerns the amount of that evaluation being conveyed. *Intensification* is the most direct way of expressing this using stronger language or emphasizing the attitude more (for example "He was very happy"), or using similar techniques to weaken the appraisal. *Quantification* conveys the force of an attitude by specifying how prevalent it is, how big it is, or how long

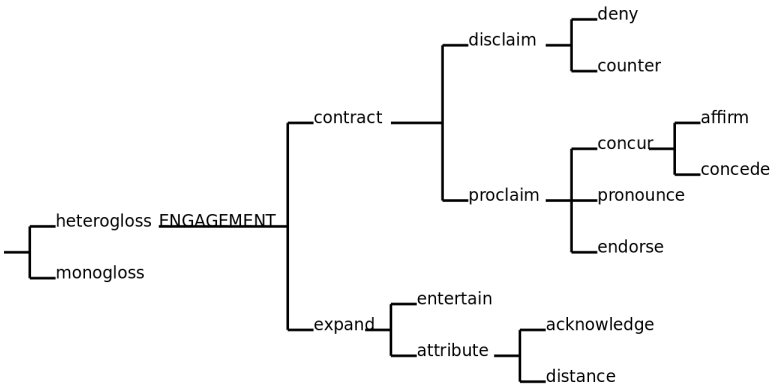



Figure 4.3. The ENGAGEMENT system, as described by Martin and White [110]. The notation used is described in Appendix A.

it has lasted (e.g. "a few problems", or "a tiny problem", or "widespread hostility").

Appraisal theory contains another system that does not directly concern the appraisal expression, and that is the ENGAGEMENT system (Figure 4.3), which deals with the way a speaker positions his statements with respect to other potential positions on the same topic. A statement may be presented in a *monoglossic* fashion, which is essentially a bare assertion with neutral positioning, or it may be presented in a *heteroglossic* fashion, in which case the ENGAGEMENT system selects how the statement is positioned with respect to other possibilities.

Within ENGAGEMENT, one may *contract* the discussion by ruling out positions. One may *disclaim* a position by stating it and rejecting it (for example "You don't need to give up potatoes to lose weight"). One may also *proclaim* a position with such certainty that it rules out other unstated positions (for example through the use of the word "obviously"). One may also *expand* the discussion by introducing new positions, either by tentatively *entertaining* them (as would be done by saying "it seems..." or "perhaps"), or by *attributing* them to somebody else and not taking direct credit.

My work models a subset of appraisal theory. FLAG is only concerned with

finding inscribed appraisal. It also uses simplified version of the AFFECT system (pictured in Figure 6.2). This version adopts some of Bednarek's modifications, and simplifies the system enough to sidestep the discrepancies with Taboada's version. My approach also vastly simplifies GRADUATION being concerned only with whether force is increased or decreased, and whether focus is sharpened or softened. The ENGAGEMENT system has no special application to appraisal expressions — it can be used to position non-evaluative propositions just as it can be used to position evaluations. Because of this, it is beyond the scope of this dissertation.

## 4.2 Lexicogrammar

Having explained the grammatical system of appraisal, which is an interpersonal system at the level of discourse semantics [110, p. 33], it is apparent that there are a lot of things that the APPRAISAL system is too abstract to specify completely on its own, in particular the specific parts of speech by which attitudes, targets, and evaluators are framed in the text. Collectively these pieces of the appraisal picture make up the lexicogrammar.

To capture these, I draw inspiration from Hunston and Sinclair [72], who studied the grammar of evaluation using local grammars, and from Bednarek [21] who studied the relationship between APPRAISAL and the local grammar patterns. Based on the observation that there are several different pieces of the target and evaluator (and comparisons) that can appear in an appraisal expression, I developed a set of names for other important components of an appraisal expression, with an eye towards capturing as much information as can usefully be related to the appraisal, and towards seeking reusability of the same component names across different frames for appraisal.

The components are as follows. The examples presented are illustrative of the

general concept of each component. More detailed examples can be found in the IIT sentiment corpus annotation manual in Appendix B.

**Attitude:** A phrase that indicates that evaluation is present in the sentence. The attitude also determines whether the appraisal is positive or negative (unless the polarity is shifted by a polarity marker), and it determines what type of appraisal is present (from among the types described by the APPRAISAL system).

(9) Her appearance and demeanor are $_{\text{attitude}}$ excellently suited to her role.

**Polarity:** A modifier to the attitude that changes the orientation of the attitude from positive to negative (or vice-versa).

There are many ways to change the orientation of an appraisal expression, or to divorce the appraisal expression from being factual. Words that resemble polarity can be used to indicate that the evaluator is specifically not making a particular appraisal or to deny the existence of any target matching the appraisal. Although these effects may be important to study, they are related to a more general problem of modality and engagement, which is beyond the scope of my work. They are not polarity, and do not affect the orientation of an attitude.

(10) I $_{\text{polarity}}$ couldn't bring myself to $_{\text{attitude}}$ like him.

**Target:** The object or proposition that is being evaluated. The target answers one of three questions depending on the type of the attitude. For appreciation, it answers the question "what thing or event has a positive/negative quality?" For judgment, it answers one of two questions: either "who has the positive/negative character?" or "what behavior is being considered as positive or negative?" For affect, it answers "what thing/agent/event was the cause of the good/bad feeling?" and is equivalent to the "trigger" shown in Figure 4.1.

(11) $_{\text{evaluator}}$ I $_{\text{attitude}}$ hate it $_{\text{target}}$ when people talk about me rather than to me.

**Superordinate:** A target can be evaluated concerning how well it functions as a particular kind of object, or how well it compares among a class of objects, in which case a *superordinate* will be part of the appraisal expression, indicating what class of objects is being considered.

(12) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

**Process:** When an attitude is expressed as an adverb, it frequently modifies a verb and serves to evaluate how well a target performs at that particular *process* represented by that verb.

(13) $_{\text{target}}$ The car $_{\text{process}}$ maneuvers $_{\text{attitude}}$ well, but $_{\text{process}}$ accelerates $_{\text{attitude}}$ sluggishly.

**Aspect:** When a target is being evaluated with regard to a specific behavior, or in a particular context or situation, this behavior, context, or situation is an aspect. An aspect serves to limit the evaluation in some way, or to better specify the circumstances under which the evaluation applies.

(14) There are a few $_{\text{attitude}}$ extremely sexy $_{\text{target}}$ new features $_{\text{aspect}}$ in Final Cut Pro 7.

**Evaluator:** The evaluator in an appraisal expression is the phrase that denotes whose opinion the appraisal expression represents. This can be grammatically accomplished in several ways, such as including the attitude in a quotation attributed to the evaluator, or indicating the evaluator as the subject of an attitude verb.

In some applications in the general problem of subjectivity, it can be important to keep track of several levels of attribution as Wiebe et al. [179] did in the MPQA corpus. This can be used to analyze things like speculation about other

people's opinions, disagreements between two people about what a third party thinks, or the motivation of one person in reporting another person's opinion. Though this undoubtedly has some utility to integrating evaluative language into applications concerned with the broader field of subjectivity, the innermost level of attribution is special inasmuch as it tells us who (allegedly) is making the evaluation expressed in the attitude[5]. In an appraisal expression, this person who is (allegedly) making the evaluation is the evaluator, and all other sources to whom the quotation is attributed are outside of the scope of the study of evaluation. They are therefore not included within the appraisal expression.

(15) $_{\text{target}}$ Zack would be $_{\text{evaluator}}$ my $_{\text{attitude}}$ hero $_{\text{aspect}}$ no matter what job he had.

**Expressor:** With expressions of affect, there may be an expressor, which denotes some instrument which conveys an emotion. Examples of expressors would include a part of a body, a document, a speech, or a friendly gesture.

(16) $_{\text{evaluator}}$ He opened with $_{\text{expressor}}$ greetings of gratitude and $_{\text{attitude}}$ peace.

(17) $_{\text{expressor}}$ His face at first wore the melancholy expression, almost despondency, of one who travels a wild and bleak road, at nightfall and alone, but soon $_{\text{attitude}}$ brightened up when he saw $_{\text{target}}$ the kindly warmth of his reception.

In non-comparative appraisal expressions, there can be any number of expressions of polarity (which may cancel each other out), at most one of each of the other components.

In comparative appraisal expressions, it is possible to compare how different targets measure up to a particular evaluation, to compare how two different evaluators

[5] The full attribution chain can also be important in understanding referent of pronominal evaluators, particularly in cases where the pronoun "I" appears in a quotation.

feel about a particular evaluation of a particular target, to compare two different evaluations of the same target, or even to compare two completely separate evaluations. A comparative appraisal expression, therefore, has a single *comparator* with two sides that are being compared. The comparator indicates the presence of a comparison, and also indicates which of the two things being compared is greater (is better described by the attitude) or whether the two are equal. Most English comparators have two parts (e.g. "more ... than"), and other pieces of the appraisal expression can appear between these two parts. Frequently an attitude appears between the two parts, but a superordinate or evaluator can appear as well, as in the comparison"more exciting to me than" (which contains both an attitude and an evaluator). Therefore, the "than" part of the comparator is annotated as a separate component of the appraisal expression, which I have named *comparator-than*. The forms of adjective comparators that concern me are discussed by Biber et al. [23, p. 527], specifically "more/less adjective ... than" "adjective-er ... than", and "as adjective ... as", as well as some verbs that can perform comparison.

Each side of the comparator can have all of the slots of a non-comparative appraisal expression (when two completely different evaluations are being compared), or some parts of the appraisal expression can be appear once, associated with the comparator and not associated with either of the sides (in any of the other three cases, for example when comparing how different targets measure up to a particular evaluation). I use the term *rank* to refer to which side of a comparison a particular component belongs to[6]. When the item has no rank (which I also refer to for short as "rank 0") this means that the component is shared between both sides of the comparator, and belongs to the comparator itself. Rank 1 means the component

[6] My decision to use integers for the ranks, rather than a naming scheme like "left", "right", and "both" is arbitrary, and is probably influenced by a computer-science predisposition to use integers wherever possible.

belongs to the left side of the comparator (the side that is "more" in a "more . . . than" comparison), and rank 2 means the it belongs to the right side of the comparator (the side that is "less" in a "more . . . than" comparison). This is a more versatile structure for a comparative appraisal (allowing one to express the comparison in example 18) than the structure usually assumed in sentiment analysis literature [55, 58, 77, 80] which only allows for comparing how two targets measure up to a single evaluation (as in example 19).

(18) Former Israeli prime minister Golda Meir said that "as long as the $_{\text{evaluator-1}}$ Arabs $_{\text{attitude-1}}$ hate $_{\text{target-1}}$ the Jews $_{\text{comparator}}$ more $_{\text{comparator-than}}$ than $_{\text{evaluator-2}}$ they $_{\text{attitude-2}}$ love $_{\text{target-2}}$ their own children, there will never be peace in the Middle East."

(19) $_{\text{evaluator}}$ I thought $_{\text{target-1}}$ they were $_{\text{comparator}}$ less $_{\text{attitude}}$ controversial $_{\text{comparator-than}}$ than $_{\text{target-2}}$ the ones I mentioned above.

Appraisal expressions involving superlatives are non-comparative. They frequently have a superordinate to indicate that the target being appraised is the best or worst in a particular class, as in example 12.

## 4.3 Summary

The definition of appraisal expression extraction is based on two primary linguistic studies of evaluation: Martin and White's [110] appraisal theory and Hunston and Sinclair's [72] local grammar of evaluation. Appraisal theory categorizes evaluative language conveying approval or disapproval into different types of evaluation, and characterizes the structural constraints these types of evaluation impose in general terms. The local grammar of evaluation characterizes the structure of appraisal expressions in detail. The definition of appraisal expressions introduced here breaks appraisal expressions down into a number of parts. Of these parts, evaluators, attitudes, targets, and various types of modifiers like polarity markers appear frequently

in appraisal expressions and have been recognized by many in the sentiment analysis community. Aspects, processes, superordinates, and expressors appear less frequently in appraisal expressions and are relatively unknown. The definition of appraisal expressions also provides a uniform method for annotating comparative appraisals.

# CHAPTER 5

# EVALUATION RESOURCES

There are several existing corpora for sentiment extraction. The most commonly used corpus for this task is the UIC Review Corpus (Section 5.2), which is annotated with product features and their sentiment in context (positive or negative). One of the oldest corpora that is annotated in detail for sentiment extraction is the MPQA Corpus (Section 5.1). Two other corpora have been developed and released more recently, but have not yet had time to attract as much interest as MPQA and UIC corpora. These newer corpora are the JDPA Sentiment Corpus (Section 5.4), and the Darmstadt Service Review Corpus (Section 5.3). I developed the IIT Sentiment Corpus (Section 5.5) to explore sentiment annotation issues that had not been addressed by these other corpora. I evaluate FLAG on all five of these corpora, and the nature of their annotations are analyzed in the following sections.

There is one other corpus described in the literature that has been developed for the purpose of appraisal expression extraction — that of Zhuang et al. [192]. I was unable to obtain a copy of this corpus, so I cannot discuss it here, nor could I use it to evaluate FLAG's performance.

Several other corpora have been used to evaluate sentiment analysis tasks, including Pang et al.'s [134] corpus of 2000 movie reviews, a product review corpus that I used in some previous work [27], and the NTCIR corpora [146–148]. Since these corpora are annotated with only document-level ratings or sentence-level annotations, I will not be using them to evaluate FLAG in this dissertation, and I will not be analyzing them further.

## 5.1 MPQA 2.0 Corpus

The Multi-Perspective Question Answering (MPQA) corpus [179] is a study in the general problem of subjectivity. The annotations on the corpus are based on a goal of identifying 'private states', a term which "covers opinions, beliefs, thought, feelings, emotions, goals, evaluations, and judgments" [179, p. 4]. The annotation scheme is very detailed, annotating ranges of text as being subjective, and identifying the source of the opinion. In the MPQA version 1.0, the annotation scheme focused heavily on identifying different ways in which opinions are expressed, and less on the content of those opinions. This is reflected in the annotation scheme, which annotates:

- Direct subjective frames which concern subjective speech events (the communication verb in a subjective statement), or explicit private states (opinions expressed as verbs such as "fears").
- Objective speech event frames, which indicate the communication verb used when someone states a fact.
- Expressive subjective element frames which contain evaluative language and the like.
- Agent frames which identify the textual location of the opinion source.

In version 2.0 of the corpus [183], annotations highlighting the content of these private states were added to the corpus, in the form of attitude and target annotations. A direct subjective frame may be linked to several attitude frames indicating its content, and each attitude can be linked to a target, which is the entity or proposition that the attitude is about. Each attitude has a type; those types are shown in Figure 5.1.

| | |
|---|---|
| Sentiment | Positive: Speaker looks favorably on target<br>Negative: Speaker looks unfavorably on target |
| Agreement | Positive: Speaker agrees with a person or proposition<br>Negative: Speaker disagrees with a person or proposition |
| Arguing | Positive: Speaker argues by presenting an alternate proposition<br>Negative: Speaker argues by denying the proposition he's arguing with |
| Intention | Positive: Speaker intends to perform an act<br>Negative: Speaker does not intend to perform an act |
| Speculation | Speaker speculates about the truth of a proposition |
| Other Attitude | Surprise, uncertainty, etc. |

Figure 5.1. Types of attitudes in the MPQA corpus version 2.0

The Sentiment attitude type covers text that addresses the approval/disapproval dimension of sentiment analysis (the Attitude and Orientation systems in appraisal theory), and the other attitude types cover aspects of stance (the Engagement system in appraisal theory.) Wilson contends that the structure of all of these phenomena can be adequately explained using the attitudes which indicate the presence of a particular type of sentiment or stance, and targets that indicate what that sentiment or stance is about. (Note that this means that Wilson's use of the term "attitude" is broader than I have defined it in Section 4.1, and I will be borrowing her definition of the term attitude when describing the MPQA corpus.)

Wilson [183] explains the process for annotating attitudes as:

> Annotate the span of text that expresses the attitude of the overall private state represented by the direct subjective frame. Specifically, for each direct subjective frame, first the attitude type(s) being expressed by the source of the direct subjective frame are determined by considering the text anchor of the frame and everything within the scope of the annotation attributed to the source. Then, for each attitude type identified, an attitude frame is created and anchored to whatever span of text completely captures the attitude type.

Targets follow a similar guideline.

This leads to an approach whereby annotators read the text, determine what kinds of attitudes are being conveyed, and then select long spans of text that express these attitudes. One advantage to this approach is that the annotators recognize when the target of an attitude is a proposition, and they tag the proposition accordingly. The IIT sentiment corpus (Section 5.5) is the only other sentiment corpora available today that does this. On the other hand, a single attitude can consist of several phrases consisting of similar sentiments, separated by conjunctions, where they should logically be two different attitudes. An example of both of these phenomena occurring in the same sentence is:

(20) That was what happened in 1998, and still today, Chavez gives constant demonstrations of $_{\text{attitude}}$ discontent and irritation at $_{\text{target}}$ having been democratically elected.

In many other places in the MPQA corpus, the attitude is implied through the use a polar fact or other evoked appraisal, for example:

(21) $_{\text{target}}$ He asserted, in these exact words, this barbarism: "4 February is not just any date, it is a historic date we can well compare to 19 April 1810, when that civic-military rebellion also opened a new path towards national independence." $_{\text{attitude}}$ No one had gone so far in the anthology of rhetorical follies, or in falsifying history.

Although the corpus allows an annotation to indicate inferred attitudes, many cases of inferred attitudes (including the one given in example 21) are not annotated as inferred.

Finally, the distinction between the Arguing attitude type (defined as "private states in which a person is arguing or expressing a belief about what is true or should be true in his or her view of the world"), and the Sentiment attitude type

(which corresponds more-or-less to evaluative language) was not entirely clear. It appears that "arguing" was often annotated based more on the context of the attitude than on its actual content. This can be attributed to the annotation instruction to "mark the arguing attitude on the span of text expressing the argument or what the argument is, and mark what the argument is about as the target of the arguing attitude."

(22) "We believe in the $_{\text{attitude-arguing}}$ sincerity of $_{\text{target-arguing}}$ the United States in promising not to mix up its counter-terrorism drive with the Taiwan Strait issue," Kao said, adding that relevant US officials have on many occasions reaffirmed similar commitments to the ROC.

(23) In his view, Kao said $_{\text{target-arguing}}$ the cross-strait balance of military power is $_{\text{attitude-arguing}}$ critical to the ROC's national security.

Both of these examples are classified as Arguing in the MPQA corpus. However both are clearly evaluative in nature, with the notion of an argument apparently arising from the context of the attitudes (expressed in phrases such as "We believe..." and "In his view..."). Indeed, both attitudes have very clear attitude types in appraisal theory ("sincerity" is veracity, and "critical" is valuation), thus it would seem that they could be considered Sentiment instead.

It appears that the best approach to resolving this would have been for MPQA annotators to use the rule "use the phrase indicating the presence of arguing as the attitude, and the entire proposition being argued as the target (including both the attitude and target of the Sentiment being argued, if any)" when annotating Arguing. Thus, the Arguing in these sentences should be tagged as follows. The annotations currently found in the MPQA corpus (which are shown above) would remain but would have an attitude type of Sentiment.

(24) "We $_{\text{attitude-arguing}}$ believe in the $_{\text{target-arguing}}$ $_{\text{attitude-sentiment}}$ sincerity of $_{\text{target-sentiment}}$ the United States in promising not to mix up its counter-terrorism drive with the Taiwan Strait issue," Kao said, adding that relevant US officials have on many occasions reaffirmed similar commitments to the ROC.

(25) $_{\text{attitude-arguing}}$ In his view, Kao said $_{\text{target-arguing}}$ $_{\text{target-sentiment}}$ the cross-strait balance of military power is $_{\text{attitude-sentiment}}$ critical to the ROC's national security.

In this scheme, attitudes indicate the evidential markers in the text, while the targets are the propositions thus marked.

In both of the above examples, we also see very long attitudes that contain much more information than simply the evaluation word. The additional phrases qualify evaluation and limit it to particular circumstances. The presence of these phrases makes it difficult to match the exact boundaries of an attitude when performing text extraction, and I contend that it would proper to recognize these qualifying phrases in a different annotation — the aspect annotation described in Section 4.2.

To date, the only published research of which I am aware that uses MPQA 2.0 attitude annotations for evaluation is Chapter 8 of Wilson's thesis [183], where she introduces the annotations. Her aim is to test classification accuracy to discriminate between Sentiment and Arguing. Stating that "the text spans of the attitude annotations do not lend an obvious choice for the unit of classification — attitude frames may be anchored to any span of words in a sentence" (p. 137), she automatically creates "attribution levels" based on the direct subjective and speech event frames in the corpus. She then associates these "attribution levels" with the attitude annotations that overlap them. The attitude types are then assigned from the attitudes to the attribution levels that contain them. Her classifiers then operate on the attribution levels to determine whether the attribution levels contain arguing

or sentiment, and whether they are positive or negative. The results derived using this scheme are not comparable to our own where we seek to extract attitude spans directly. As far as we know, ours is the first published work to attempt this task.

Several papers evaluating automated systems against the MPQA corpus use the other kinds of private state annotations on the corpus [1, 88, 177, 181, 182]. As with Wilson's work, many of these papers aggregate phrase-level annotations into simpler sentence-level or clause-level classifications and use those for testing classifiers.

## 5.2 UIC Review Corpus

Another frequently used corpus for evaluation opinion and product feature extraction is the product review corpus developed by Hu [69, introduced in 70], and expanded by Ding et al. [44]. They used the corpus to evaluate their opinion mining extractor; Popescu [136] also used Hu's subset of the corpus to evaluate the OPINE system. The corpus contains reviews for 14 products from Amazon.com and C|net.com. Reviews for five products were annotated by Hu, and reviews for an additional nine products were later tagged by Ding. I call this corpus the UIC Review corpus.

Human annotators read the reviews in the corpus, listed the product features evaluated in each sentence (they did not indicate the exact position in the sentence where the product features were found), and noted whether the user's opinion of that feature was positive or negative and the strength of that opinion (from 1 to 3, with the default being 1). Features are also tagged with certain opinion attributes when applicable: [u] if the product feature is implicit (not explicitly mentioned in the sentence), [p] if coreference resolution is needed to identify the product feature, [s] if the opinion is a suggestion or recommendation, or [cs] or [cc] when that the opinion is a comparison with a product from the same or a competing brand, respectively.

An example review from the corpus is given in Figure 5.2.

The UIC Review Corpus does not identify attitudes or opinion words explicitly, so evaluating the extraction of opinions can only be done indirectly, by associating them with product features and determining whether the orientations given in the ground truth match the orientations of the opinions that an extraction system associated with the product feature. Additionally, these targets themselves constitute only a subset of the appraisal targets found in the texts in the corpus, as the annotations only include product features. There are many appraisal targets in the corpus that are not product features. For example, it would be appropriate to annotate the following evaluative expression, which contains a proposition as a target which is not a product feature.

(26) ...what is $_{\text{attitude}}$ important is that $_{\text{target}}$ your Internet connection will never even reach the speed capabilities of this router...

One major difficulty in working with the corpus is that the corpus identifies *implicit features*, defined as features whose names do not appear as a substring of the sentence. For example, the phrase "it fits in a pocket nicely" is annotated with a positive evaluation of a feature called "size." As in this example, many, if not most, of the implicit features marked in the corpus are cases where an attitude or a target is referred to indirectly, via metaphor or inference from world knowledge (e.g., understanding that fitting in a pocket is a function of size and is a good thing). Implicit features account for 18% of the individual feature occurrences in the corpus. While identifying and analyzing such implicit features is an important part of appraisal expression extraction, this corpus lacks any ontology or convention for naming the implicit product features, so it is impossible to develop a system that matches the implicit feature names without learning arbitrary correspondences directly from in-domain training data.

| Tagged features | Sentence |
|---|---|
| | *This review had no title* |
| router[+2] | This router does everything that it is supposed to do, so i dont really know how to talk that bad about it. |
| setup[+2], installation[+2] | It was a very quick setup and installation, in fact the disc that it comes with pretty much makes sure you cant mess it up. |
| install[+3] | By no means do you have to be a tech junkie to be able to install it, just be able to put a CD in the computer and it tells you what to do. |
| works[+3] | It works great, i am usually at the full 54 mbps, although every now and then that drops to around 36 mbps only because i am 2 floors below where the router is. |
| | That only happens every so often, but its not that big of a drawback really, just a little slower than usual. |
| router[+2][p] | It really is a great buy if you are lookin at having just one modem but many computers around the house. |
| router[+2] | There are 3 computers in my house all getting wireless connection from this router, and everybody is happy with it. |
| | I do not really know why some people are tearing this router ! |
| | apart on their reviews, they are talking about installation problems and what not. |
| setup[+2], ROUTER[+][s] | Its the easiest thing to setup i thought, and i am only 16...So with all that said, BUY THE ROUTER!!!! |

Figure 5.2. An example review from the UIC Review Corpus. The left column lists the product features and their evaluations, and the right column gives the sentences from the review.

The UIC corpus is also inconsistent about what span of text it identifies as a product feature. Sometimes it identifies an opinion as a product feature (example 27), and sometimes an aspect (28) or a process (29).

(27) It is buggy, slow and basically frustrates the heck out of the user.

**Product feature:** "slow"

(28) This setup using the CD was about as easy as learning how to open a refrigerator door for the first time.

**Product feature:** "CD"

(29) This router works at 54Mbps that's megabyte not kilobyte.

**Product feature:** "works"

Finally, there are a number of inconsistencies in the corpus in selection of product feature terms; raters apparently made different decisions about what term to use for identical product features in different sentences. For example, in the first sentence in Figure 5.3, the annotator interpreted "this product" as a feature, but in the second sentence the annotator interpreted the same phrase as a reference to the product type ("router"). The prevalence of such inconsistencies is clear from a set of annotations on the product features indicating the presence of implicit features.

In the corpus, the `[u]` annotation indicates an implicit feature that doesn't appear in the sentence, and the `[p]` annotation indicates an implicit feature that doesn't appear in the sentence but which be found via coreference resolution. These annotations can be created or checked automatically; we find about 12% of such annotations in the testing corpus to be incorrect (as they are in all six sentences shown in Figure 5.3).

Hu and Liu evaluated their system's ability to extract product features by

| Tagged features | Sentence |
|---|---|
| product[+2][p] | It'll make life a lot easier, and preclude you from having to give this product a negative review. |
| router[-2][p] | However, this product performed well below my expectation. |
| Linksys[+2][u] | Even though you could get a cheap router these days, I'm happy I spent a little extra for the Linksys. |
| access[-2][u] | A couple of times a week it seems to cease access to the internet. |
| access[-2][u] | That is, you cannot access the internet at all. |
| model[+2][p] | This model appears to be especially good. |

Figure 5.3. Inconsistencies in the UIC Review Corpus

comparing the list of distinct feature names produced by their system with the list of distinct feature names derived from their corpus [101], as well as their system's ability to identify opinionated sentences and predict the orientation of those sentences.

As we examined the corpus, we also discovered some inconsistencies with published results using it. Counting the actual tags in their corpus (Table 5.1), we found that both the number of total individual feature occurrences and the number of unique feature names are different (usually much greater) than the numbers reported by Hu and Liu as "No. of manual features" in their published work. Liu [101] explained that the original work only dealt with "nouns and a few implicit features" and that the corpus was re-annotated after the original work was published. Unfortunately, this makes rigorous comparison to their originally published work impossible. I am unsure how others who have used this corpus for evaluation [43, 116, 136, 138, 139, 191] have dealt with the problem.

Table 5.1. Statistics for the Hu and Liu's corpus, comparing Hu and Liu's reported "No. of Manual Features" with our own computations of corpus statistics. We have assumed that Hu and Liu's "Digital Camera 1" is the Nikon 4300, and "Digital Camera 2" is the Canon G3, but even if reversed the numbers still do not match.

| Product | No. of manual features | Individual Feature Occurrences | Unique Feature Names |
|---|---|---|---|
| Digital Camera 1 | 79 | 203 | 75 |
| Digital Camera 2 | 96 | 286 | 105 |
| Nokia 6610 | 67 | 338 | 111 |
| Creative Nomad | 57 | 847 | 188 |
| Apex AD2600 | 49 | 429 | 115 |

## 5.3 Darmstadt Service Review Corpus

The Darmstadt Service Review corpus [77, 168] is an annotation study of how opinions are expressed in service reviews. The corpus consists of consists of 492 reviews about 5 major websites (eTrade, Mapquest, etc.), and 9 universities and vocational schools. All of the reviews were drawn from consumer review portals `www.rateitall.com` and `www.epinions.com`. Though their annotation manual [77] says they also annotated political blog posts, published materials about the corpus [168] only mention service reviews. There were no political blog posts present in the corpus which they provided to me.

The Darmstadt annotators annotated the corpus at the sentence level and then at the individual sentiment level. The first step in annotating the corpus was for the annotator to read the review and determine its topic (i.e. the service that the document is reviewing). Then the annotator looked at each sentence of the review and determined whether each it was on topic. If the sentence was on topic, the annotator determined whether it was objective, opinionated, or a polar fact. A sentence could not be considered opinionated if it was not on topic. This meant that the evaluation

"I made this mistake" in example 30, below, was not annotated as to whether it was opinionated, because it was not judged to be on-topic.

(30) Alright, word of advice. When you choose your groups, the screen will display how many members are in that group. If there are 200 members in every group that you join and you join 4 groups, it is very possible that you are going to get about 800 emails per day. WHAT?!! Yep, I am dead serious, you will get a MASSIVE quantity of emails. I made this mistake.

The sentences-level annotations were compared between all of the raters. For sentences that all annotators agreed were on-topic polar facts, the annotators tagged the polar targets found in the sentence, and annotated those targets with their orientations. For sentences that all annotators agreed were on-topic and opinionated, the annotators annotated individual opinion expressions, which are made up of the following components (called "markables" in the terminology of their corpus):

**Target:** annotates the target of the opinion in the sentence.

**Holder:** the person whose opinion is being expressed in the sentence.

**Modifier:** something that changes the strength or polarity of the opinion.

**OpinionExpression:** the expression from which we understand that a personal evaluation is being made. Each OpinionExpression has attributes referencing the targets, holders, and modifiers that it is related to.

The guidelines generally call for the annotators to annotate the smallest span of words that fully describes the target/holder/opinion. They don't include articles, possessive pronouns, appositives, or unnecessary adjectives in the markables. Although I disagree with this decision (because I think a longer phrase can be used

to differentiate different holders/targets) it seems they followed this guideline consistently, and in the case of nominal targets it makes little difference when evaluating extraction against the corpus, because one can simply evaluate by considering any annotations that overlap as being correct.

I looked through the 136 evaluative expressions found in the 20 documents that I set aside as a development corpus, to develop an understanding of the quality of the corpus, and to see how the annotation guidelines were applied in practice.

One very frequent issue I saw with their corpus concerned the method in which the annotators tagged propositional targets. The annotation guidelines specify that though targets are typically nouns, they can also be pronouns or complex phrases, and propositional targets would certainly justify annotating complex phrases as the target. The annotation manual includes an example of a propositional target by selecting the whole proposition, but since the annotation manual doesn't explain the example, propositional targets remained a subtlety that the annotators frequently missed. Rather than tag the entire target proposition as the target, annotators tended to select noun phrases that were part of the target, however the choice of noun phrase was not always consistent, and the relationship between the meaning of the noun phrase and the meaning of the proposition is not always clear. Examples 31, 32, and 33 demonstrate the inconsistencies in how propositions were annotated in the corpus. In these examples, three propositions have been annotated in three different ways. In example 31, an noun phrase in the proposition was selected as the target. In example 32, the verb in the proposition was selected. In example 33, the dummy "it" was selected as the target, instead of the proposition. Though this could be a sensible decision if the pronoun referenced the proposition, the annotations incorrectly claim that the pronoun references text in an earlier sentence.

(31) The $_{\text{attitude}}$ <u>positive</u> side of the egroups is that you will meet lots of new $_{\text{target}}$

people, and if you join an Epinions egroup, you will certainly see a change in your number of hits.

(32) $_{\text{attitude}}$ Luckily, eGroups allows you to $_{\text{target}}$ choose to moderate individual list members, or even ban those complete freaks who don't belong on your list.

(33) $_{\text{target}}$ It is much $_{\text{attitude}}$ easier to have it sent to your inbox.

Another frequent issue in the corpus concerns the way they annotate polar facts. The annotation manual presents 4 examples and uses them to show the distinction between polar facts (examples 34 and 35, which come from the annotation manual) and opinions such (examples 36 and 37).

(34) The double bed was so big that two large adults could easily sleep next to each other.

(35) The bed was blocking the door.

(36) The bed was too small.

(37) The bed was delightfully big.

The annotation manual doesn't clearly explain the distinction between polar facts and opinions. It explains example 34 by saying "Very little personal evaluation. We know that it's a good thing if two large adults can easily sleep next to each other in a double bed," and it explains example 36 by saying "No facts, just the personal perception of the bed size. We don't know whether the bed was just 1,5m long or the author is 2,30m tall."

It appears that there are two distinctions between these examples. First, the polar facts state objectively verifiable facts of which a buyer would either approve or disapprove based on their knowledge of the product and their intended use of the

product. Second, the opinions contain language that explicitly indicates a positive or negative polarity (specifically the words "too" and "delightfully"). It appears from their instructions that they did not intend the second distinction.

These examples miss a situation that falls into a middle ground between these two situations, demonstrated examples 38, 39, and 40, which I found in my development subset of their corpus. In these examples the opinion expressions annotated convey a subjective opinion about the size or length of something (i.e. it's big or small, compared to what the writer has experience with, or what he expects of this product), but it requires inference or domain knowledge to determine whether he approves of that or disapproves of the situation. By contrast, examples 34 and 35 do not even state the size or location of the bed in a subjective manner. I contend that it is most appropriate to consider these to be polar facts, because the approval or disapproval is not explicit from the text. However, the Darmstadt annotators marked these as opinionated expressions because the use of indefinite adjectives implies subjectivity. They appear to be pretty consistent about following this guideline — I did not see many examples like these annotated as polar facts.

(38) Yep, I am dead serious, you will get a $_{\text{attitude}}$ MASSIVE $_{\text{target}}$ quantity of emails.

(39) If you try to call them when this happens, there are already a million other people on the phone, so you have to $_{\text{target}}$ wait $_{\text{attitude}}$ forever.

(40) PROS: $_{\text{attitude}}$ small $_{\text{target}}$ class sizes

## 5.4 JDPA Sentiment Corpus

The JDPA Sentiment corpus [45, 86, 87] is a product-review corpus intended to be used for several different product related tasks, including product feature identification, coreference resolution, meronymy, and sentiment analysis. The corpus consists

of 180 camera reviews and 462 car reviews, gathered by searching the Internet for car and camera-related terms and restricting the search results to certain blog websites. They don't tell us which sites they used, though Brown [30] mentions the JDPA Power Steering blog (24% of the documents), Blogspot (18%) and LiveJournal (18%). The overwhelming majority of the documents have only a single topic (the product being reviewed), but they vary in formality. Some are comparable to editorial reviews, and others are more personal and informal in tone. I found that 67 of the reviews in the JDPA corpus are marketing reports authored by JDPA analysts in a standardized format. These marketing reports should be considered as a different domain from free-text product reviews that comprise the rest of the corpus, because they are likely to challenge any assumptions that an application makes about the meaning of the frequencies of different kinds of appraisal in product reviews.

The annotation manual [45] has very few surprises in it. The authors annotate a huge number of entities types related to the car and camera domains, and they annotate generic entity types from the ACE named entity task as well. Their primary guideline for identifying sentiment expressions is:

> Adjectival words and phrases that have inherent sentiment should always be marked as a Sentiment Expression. These words include: *ugly/pretty, good/bad, wonderful/terrible/horrible, dirty/clean.* There is also another type of adjective that doesn't have inherent sentiment but rather sentiment based on the context of the sentence. This means that these adjectives can take either positive or negative sentiment depending on the Mention that they are targeting and also other Sentiment Expressions in the sentence. For example, a *large salary* is positive whereas a *large phone bill* is negative. These adjectives should only be marked as Sentiment Expressions if the sentiment they are conveying is stated clearly in the surrounding context. In other cases these adjectives merely specify a Mention further instead of changing its sentiment.

They also point out that verbs and nouns can also be sentiment expressions when those nouns and verbs aren't themselves names for the particular entities that are being evaluated.

They annotate mentions for the opinion holder via the OtherPersonsOpinion entity. They annotate the reporting verb that associates the opinion holder with the attitude, and it refers to the entity who is the opinion holder, and the SentimentBearingExpression through attributes. In the case of verbal appraisal, they will annotate the same word as both the reporting verb and the SentimentBearingExpression.

Comparisons are reported by annotating either the word "less", "more" or a comparative adjective (ending in "-er") using a Comparison frame with 3 attributes: "less", "more", and "dimension". "Less" and "more" refer to the two entities (i.e. targets) being compared, and "dimension" refers to the sentiment expression along with they are being compared. (An additional attribute named "same" may be used to change the function of "less" and "more" when two entities are indicated to be equal.)

I reviewed the 515 evaluation expressions found in the 20 documents that I set aside as a development corpus.

The most common error I saw in the corpus (occurring 78 times) was a tendency to annotate outright objective facts as opinionated. The most egregious example of this was a list of changes in the new model of a particular car (example 41). There's no guarantee that a new feature in a car is better than the old one, and in some cases fact that something is new may itself be a bad thing (such as when the old version was so good that it makes no sense to change it). Additionally smoked taillight lenses are a particular kind of tinting for a tail light so the word "smoked" should not carry any particular evaluation.

(41) Here is what changed on the 2008 Toyota Avalon:

- $_{\text{attitude}}$ New $_{\text{target}}$ front bumper, $_{\text{target}}$ grille and $_{\text{target}}$ headlight design

- $_{\text{attitude}}$ Smoked $_{\text{target}}$ taillight lenses
- $_{\text{attitude}}$ Redesigned $_{\text{target}}$ wheels on Touring and Limited models
- Chrome door handles come standard
- $_{\text{attitude}}$ New $_{\text{target}}$ six-speed automatic with sequential shift feature
- $_{\text{attitude}}$ Revised $_{\text{target}}$ braking system with larger rear discs
- Power front passenger s seat now available on XL model
- XLS and Limited can be equipped with 8-way power front passenger s seat
- New multi-function display
- More chrome interior accents
- Six-disc CD changer with iPod auxiliary input jack
- Optional JBL audio package now includes Bluetooth wireless connectivity

This problem appears in other documents as well. Though examples 42, 43, and 44 each have an additional correct evaluation in it, I have only annotated the incorrectly annotated facts here.

(42) The rest of the interior is nicely done, with a lot of $_{\text{attitude}}$ soft touch $_{\text{target}}$ plastics, mixed in with harder plastics for controls and surfaces which might take more abuse.

(43) A good mark for the suspension is that going through curves with the Flex never caused $_{\text{target}}$ it to become $_{\text{attitude}}$ unsettled.

(44) In very short, this is an adaptable light sensor, whose way of working can be modified in order to get very $_{\text{attitude}}$ high $_{\text{target}}$ light sensibility and very low noise (by coupling two adjacent pixels, working like an old 6 megapixels SuperCCD), or to get a very $_{\text{attitude}}$ large $_{\text{target}}$ dynamic range, or to get a very $_{\text{attitude}}$ large $_{\text{target}}$ resolution (12 megapixels).

With 61 examples, the number of polar facts in the sample rivals the number of outright facts in the sample, and is the next most common error. These polar facts are allowed by their annotation scheme under specific conditions, but I consider them an error because, as I already have explained in Section 4.1, polar facts do not fall into the rubric of appraisal expression extraction. Many of these examples show inattention to grammatical structure, as in example 45 where the phrase "low contrast" should really be an adjectival modifier of the word "detail". A correct annotation of this sentence is shown in example 46. It's pretty clear that "low contrast detail" really is a product feature, specifically concerning the amount of detail found in pictures taken in low-contrast conditions, and that one should prefer a camera that can handle it better, all else being equal. The JDPA annotators did annotate "well" as an attitude, however they confused the process with the target, and used "handled" as the target.

(45) But they found that $_{\text{attitude}}$ low $_{\text{target}}$ contrast detail, a perennial problem in small sensor cameras, was not $_{\text{target}}$ handled $_{\text{attitude}}$ well.

(46) But they found that $_{\text{target}}$ low contrast detail, a perennial problem in small sensor cameras, was $_{\text{polarity}}$ not $_{\text{process}}$ handled $_{\text{attitude}}$ well.

Example 48 is another example of a polar fact with misunderstood grammar. In this example, the adverb "too" supposedly modifies adjectival targets "high up" and "low down". I am not aware of a case where adjectival targets should occur in correctly annotated opinion expressions, and it would have been more correct to select "electronic seat" as the target, though even this correction would still be a polar fact.

(47) The electronic seat on this car is not brilliant, its either $_{\text{attitude}}$ too $_{\text{target}}$ high up or way $_{\text{attitude}}$ too $_{\text{target}}$ low down.

Example 48 is another example of a polar fact with misunderstood grammar. The supposed target of "had to spend around 50k" is the word "mechanic" in an

earlier sentence. Though it is possible to have correct targets in a different sentence from the attitude (through ellipsis, or when the attitude is in a minor sentence that immediately follows the sentence with the target), the fact that they had to look several sentences back to find the target is a clue that this is a polar fact.

(48) The Blaupaunkt stopped working. disheartened, $_{\text{attitude}}$ had to spend around 50k to get it back in shape. [sic]

Examples 49 and 50 are another way in which polar facts may be annotated. These examples use highly domain-specific lexicon to convey the appraisal. In example 51, one should consider the word "short" to also be domain specific, because short can be positive or negative easily depending on the domain.

(49) We'd be looking at lots of $_{\text{attitude}}$ clumping in the Panny $_{\text{target}}$ image ... and some in the Fuji image too.

(50) You have probably heard this, but he $_{\text{target}}$ air conditioning is about ass big a $_{\text{attitude}}$ gas sucker that you have in your Jeep.

(51) The Panny is a serious camera with amazing ergonomics and a smoking good $_{\text{target}}$ lense, albeit way too $_{\text{attitude}}$ short (boooooo!)

Another category of errors that was roughly the same size as the mis-tagged facts and polar facts was number of times that the target was incorrect for various reasons. A process was selected instead of the correct target 20 times. A superordinate was selected instead of the correct target 16 times. A aspect was selected instead of the correct target 9 times. Propositional targets were incorrectly annotated 13 times. Between these and other places where either the opinion the target or the evaluator was incorrect for other reasons (usually one-off errors) 234 evaluations from the 515 turned out to be fully correct.

To date, there have been three papers performing evaluation against the JDPA sentiment corpus. Kessler and Nicolov [87] performed an experiment in associating opinions with the targets assuming that the ground truth opinion annotations and target annotations are provided to the system. Their experiment is intended to test a single component of the sentiment extraction process against the fine-grained annotations on the JDPA corpus. Yu and Kübler [187] created a technique for using cross-domain and semi-supervised training to learn sentence classifiers. They evaluated this technique against the sentence-level annotations on the JDPA corpus. Brown [30] has used the JDPA corpus for a meronymy task, and evaluated his technique on the corpus' fine-grained product feature annotations.

## 5.5 IIT Sentiment Corpus

To address the concerns that I've seen in the other corpora discussed thus far, I created a corpus with an annotation scheme that covers the lexicogrammar of appraisal described in Section 4.2. The texts in the corpus are annotated with appraisal expressions consisting of attitudes, evaluators, targets, aspects, processes, superordinates, comparators, and polarity markers. The attitudes are annotated with their orientations and attitude types.

The corpus consists of blog posts drawn from the LiveJournal blogs of the participants in the 2010 LJ Idol creative and personal writing blogging competition (`http://community.livejournal.com/therealljidol`). The corpus contains posts that respond to LJ Idol prompts alongside personal posts unrelated to the competition. The documents were selected from whatever blog posts were in each participant's RSS feed around late May 2010. Since a LiveJournal user's RSS feed contains the most recent 25 posts to the blog, the duration of time covered by these blog posts varies depending on the frequency with which the blogger posts new entries. I took the blog posts containing at least 400 words, so that they would be long enough to

have narrative content, and at most 2000 words, so that annotators would not be forced to spend too much time on any one post. I excluded some posts that were not narrative in nature (for example, lists and question-answer memes), and a couple of posts that were sexually explicit in nature. I sorted the posts into random order, and selected posts to annotate in order from the list.

I trained an IIT undergraduate to annotate documents, and updated the annotation manual based on feedback from the training process. During this training process, we annotated 29 blog entries plus a special document focused on teaching superordinates and processes. After I finished training this undergraduate, he did not stick around long enough to annotate any test documents. I wound up annotating 55 test documents myself. As the annotation manual was updated based on feedback from the training process, some example sentences appearing in the final annotation manual are drawn directly from the development subset of the corpus.

I split these documents to create a 21 document development subset and 64 document testing subset. The development subset comprises the first 20 documents used for rater training. Though these documents were annotated early in the training process, and the annotation guidelines were refined after they were annotated, these documents were rechecked later, after the test documents had been annotated, and brought up to date so that their annotations would match the standards in the final version of the annotation manual. The final 9 documents from the annotator training process, plus the 55 test documents I annotated myself were used to create the 64-document testing subset of the final corpus. Because the undergraduate didn't annotate any documents after the training process and the documents he annotated during the training process are presumed to be of lower quality, none of his annotations were included in the final corpus. All of the documents in the corpus use my version of the annotations.

In addition to the first 20 rater-training documents, the development subset also contains a document full of specially-selected sentences, which was created to give the undergraduate annotator focused practice at annotating processes and superordinates correctly. This document is part of the development subset everywhere that the development subset is used in this thesis, except for Section 10.5, which analyzes the effect on FLAG's performance when this document is not used.

The annotation manual is attached in Appendix B. Table 5.18 from *Emotion Talk Across Corpora* [20], and tables 2.6 thru 2.8 from *The Language of Evaluation* [110] were included with the annotation manual as guidelines for assigning attitude types to words. I also asked my annotator to read *A Local Grammar of Evaluation* [72] to familiarize himself with idea of annotating patterns in the text that were made up of the various appraisal components.

**5.5.1 Reflections on annotating the IIT Corpus.** When I first started training my undergraduate annotator, I began by giving him the annotation manual to read and 10 documents to annotate. I annotated the same 10 documents independently. After we both finished annotating these documents, I compared the documents, and made an appointment with him to go over the problems I saw. I followed this process again with the next 10 documents, but after I finished with these it was clear to me that this was a suboptimal process for annotator training. The annotator was not showing much improvement between the sets of documents, probably due to the delayed feedback, and the time constraints on our meetings that prevented me from going through every error. For the third set of documents, I scheduled several days where we would both annotate documents in the same room. In this process, we would each annotate a document independently (though he could ask me questions in the process) and then we would compare our results after each document. This proved to be a more effective way to train him, and his annotation skill improved

quickly.

While training this annotator, I also noticed that he was having a hard time learning about the rarer slots in the corpus, specifically processes, superordinates, and aspects. I determined that this was because these slots were too rare in the wild for him to get a good grasp on the concept. I resolved this problem by constructing a document that consisted of individual sentences automatically culled from other corpora (all corpora which I've used previously, but which were not otherwise used in this dissertation), where each sentence was likely to either contain a superordinate or a process, and worked with him on that document to learn to annotate these rarer slots. When annotating the focused document, I interrupted the undergraduate a number of times, so that we could compare our results at several points during the document, so he could improve at the task without more than one specially focused document. (This focused document was somewhat longer than the typical blog post in the corpus.)

When I started annotating the corpus, the slots that I was already aware of that needed to be annotated were the attitude, the comparator, polarity markers, targets, superordinates, aspects, evaluators, and expressors. During the training process, I discovered that adverbial appraisal expressions were difficult to annotate consistently, and determined that this was because of the presence of an additional slot that I had not accounted for — the process slot.

When I started annotating the corpus, I treated a comparator as a single slot that included the attitude group in the middle, like examples 52 and 53. Other examples like example 54, in which the evaluator can also be found in the middle of the comparator, suggested to me that it wasn't really so natural to treat a comparator as a single slot that includes the attitude. I resolved this by introducing the *comparator-than* slot, so that the two parts of the comparator could be annotated separately.

(52) $_{\text{comparator}}$ more $_{\text{attitude}}$ fun than

(53) $_{\text{comparator}}$ better than

(54) This storm is $_{\text{comparator}}$ so much more $_{\text{attitude}}$ exciting to $_{\text{evaluator}}$ me than the baseball game that it's delaying.

The superordinate slot was introduced by a similar process of observation, but this was well before the annotation manual was written.

After seeing the Darmstadt corpus, I went back and added evaluator-antecedent and target-antecedent slots, on the presumption that they might be useful for other users of the corpus who might later attempt techniques that were less strictly tied to syntax. I added these slots when the evaluator or target was a pronoun (like example 55), but not when the evaluator or target was a long phrase that happened to include a pronoun. I observed that pronominal targets didn't appear so frequently in the text; rather, pronouns were more frequently part of a longer target phrase (like the target in example 56), and could not be singled out for a target-antecedent annotation. For evaluators, the most common evaluator by far was "I", referring to the author of the document (whose name doesn't appear in the document), as is often required for affect or verb attitudes. No evaluator-antecedent was added for these cases. In sum, the evaluator-antecedent and target-antecedent slots are less useful than they might first appear, since they don't cover the majority of pronouns that need to be resolved to fully understand all of the targets in a document.

(55) $_{\text{target-antecedent}}$ Joel has carved something truly unique out of the bluffs for himself. . . . I've met him a few times now, and $_{\text{target}}$ he is a very open and $_{\text{attitude}}$ welcoming $_{\text{superordinate}}$ sort.

(56) $_{\text{evaluator}}$ I'm still $_{\text{attitude}}$ haunted when I think about $_{\text{target}}$ being there when she took

her last breath.

It appears to be possible for an attitude to be broken up into separate spans of text, one expressing the attitude, and the other expressing the orientation as in example 57. I didn't encounter this phenomenon in any of the texts I was annotating, so the annotation scheme does not deal with this, and may need to be extended in domains where this is a serious problem. According to the current scheme, phrase "low quality" would be annotated as the attitude, in a single slot, because its two pieces are adjacent to each other.

(57) The $_{\text{attitude-type}}$ quality of $_{\text{target}}$ the product was $_{\text{orientation}}$ very low.

The aspect slot appears to be more context dependent than the other slots in the annotation scheme. It corresponds to the "restriction on evaluation" slot used in Hunston and Sinclair's [72] local grammar of evaluation. In terms of the sentence structure, it often corresponds with one of the different types of circumstantial elements that can appear in a clause [see 64, section 5.6] such as location, manner, or accompaniment. Which, if any, of these is relevant as an aspect of an evaluation is very context dependent, and that probably makes the aspect a more difficult slot to extract than the other slots in this annotation scheme. It's also difficult to determine whether a prepositional phrase that post-modifies a target should be part of the target, or whether it should be an aspect.

The annotation process that I eventually settled on for annotating a document is slightly different from the one spelled out in Section B.9 of the annotation manual. I found it difficult to mentally switch between annotating the structure of an appraisal expression and selecting the attitude type. Instead of working on one appraisal expression all the way through to completion before moving on to the next, I ended up going through each document twice, first annotating the structure of each appraisal

expression while determining the attitude type only precisely enough to identify the correct evaluator and target. This involved only determining whether the attitude was affect or not. After completing the whole document, I went back and determined the attitude type and orientation for each attitude group, changing the structure of the appraisal expression if I changed my mind about the attitude type when I made this more precise determination. This could include deleting an appraisal expression completely if I decided that it no longer fit any attitude types well enough to actually be appraisal. This second pass also allowed me to correct any other errors that I had made in the first pass.

## 5.6 Summary

There are five main corpora for evaluating performance at appraisal expression extraction. FLAG is evaluated against all of these corpora.

- The MPQA Corpus is one of the earliest fine-grained sentiment corpora. It focuses on the general problem of subjectivity, and its attitude types evaluation as well as various aspects of stance.
- The UIC Review Corpus is a corpus of product reviews. Each sentence is annotated to name the product features evaluated in that sentence. Attitudes are not annotated.
- The JDPA Sentiment Corpus, and the Darmstadt Service Review Corpus are both made up of product or service reviews, and they are annotated with attitude, target, and evaluator annotations. Both have a focus on product features as sentiment targets.
- The IIT Sentiment Corpus consists of blogs annotated according to the theory introduced in Chapter 4 and the annotation guidelines given in Appendix B.

# CHAPTER 6

# LEXICON-BASED ATTITUDE EXTRACTION

The first phase of appraisal extraction is to find and analyze attitudes in the text. In this phase, FLAG looks for phrases such as "not very happy", "somewhat excited", "more sophisticated", or "not a major headache" which indicate the presence of a positive or negative evaluation, and the type of evaluation being conveyed.

Each attitude group realizes a set of options in the ATTITUDE system (described in Section 4.1). FLAG models a simplified version of the ATTITUDE system where it operates on the assumption that these options can be determined compositionally from values attached to the head word and its individual modifiers.

FLAG recognizes attitudes as phrases that consist of made up of a head word which conveys appraisal, and a string of modifiers which modify the meaning. It performs lexicon-based shallow parsing to find attitude groups. Since FLAG is designed to analyze attitude groups at the same time that it is finding them, FLAG combines the features of the individual words making up the attitude group as it encounters each word in the attitude group.

The algorithm and resources discussed in this chapter here were originally developed by Whitelaw, Garg, and Argamon [173]. I have expanded the lexicon, but have not improved upon the basic algorithm.

## 6.1 Attributes of Attitudes

One of the goals of attitude extraction is to determine the choices in the APPRAISAL system (described in Section 4.1) realized by each appraisal expression. Since the APPRAISAL system is a rather complex network of choices, FLAG uses a simplified version of this system which models the choices as a collection of orthogonal

$$\underset{\text{“happy”}}{\begin{bmatrix} \text{Attitude:} & \text{affect} \\ \text{Orientation:} & \text{positive} \\ \text{Force:} & \text{median} \\ \text{Focus:} & \text{median} \\ \text{Polarity:} & \text{unmarked} \end{bmatrix}} + \underset{\text{“very”}}{\begin{bmatrix} \text{Attitude:} & \\ \text{Orientation:} & \\ \text{Force:} & \text{increase} \\ \text{Focus:} & \\ \text{Polarity:} & \end{bmatrix}} \Rightarrow \underset{\text{“very happy”}}{\begin{bmatrix} \text{Attitude:} & \text{affect} \\ \text{Orientation:} & \text{positive} \\ \textbf{Force:} & \textbf{high} \\ \text{Focus:} & \text{median} \\ \text{Polarity:} & \text{unmarked} \end{bmatrix}}$$

Figure 6.1. An intensifier increases the force of an attitude group.

attributes for the type of attitude, its orientation and force. The attributes of the Appraisal system are represented using two different types of attributes, whose values can be changed in systematic ways by modifiers: clines to represent modifiable graded scales, and taxonomies to represent hierarchies of choices within the appraisal system.

A cline is expressed as a set of values with a flip-point, a minimum value, a maximum value, and a series of intermediate values. One can look at a cline as being a continuous graduation of values, but FLAG views it discretely to enable modifiers to increase and decrease the values of cline attributes in discrete chunks. There are several operations that can be performed by modifiers: flipping the value of the attribute around the flip-point, increasing it, decreasing it, maximizing it, and completely minimizing it. The *orientation* attribute, discussed below, is an example of a cline, that allows modifiers like “not” to flip the value between positive and negative. The *force* attribute is another example of a cline — intensifiers can increase the force from median to high to very high, as shown in Figure 6.1.

In taxonomies, a choice made at one level of the system requires another choice to be made at the next level. In Systemic-Functional systems, a choice made at one level could require two independent choices to be made at the next level. While this is expressed with a conjunction in SFL, this is simplified in FLAG by modeling some of these independent choices as separate root level attributes, and by ignoring some

of the extra choices to be made at lower levels of the taxonomy. There are no natural operations for modifying a taxonomic attribute in some way relative to the original value, but some rare cases exist where a modifier replaces the value of a taxonomic attribute from the head word with a value of its own. The *attitude type* attribute, described below, is a taxonomy of categorizing the lexical meaning of attitude groups.

The *Orientation* attribute is a cline which indicates whether an opinion phrase is considered to be positive or negative by most readers. This cline has two extreme values, positive and negative, and flip-point named neutral. *Orientation* can be flipped by modifiers such as "not" or made explicitly negative with the modifier "too". Along with *orientation*, FLAG keeps track of an additional *polarity* attribute, which is marked if the orientation of the phrase has been modified by a polarity marker such as the word "not". Much sentiment analysis work has used the term "polarity" to refer to what we call "orientation", but our usage follows the usage in Systemic-Functional Linguistics, where "polarity" refers to the presence of explicit negation [64].

*Force* is a cline taken from the Graduation system, which measures the intensity of the evaluation expressed by the writer. While this is frequently expressed by the presence of modifiers, it can also be a property of the appraisal head word. In FLAG, force is modeled as a cline of 7 discrete values (minimum, very low, low, median, high, very high, and maximum) intended to approximate a continuous system, because modifiers can increase and decrease the force of an attitude group and a quantum (one notch on the scale) is required in order to know how much to increase the force. Most of the modifiers that affect the force of an attitude group are intensifiers, for example "very", and "greatly."

*Attitude type* is a taxonomy made by combining a number of pieces of the Attitude system which deal with the dictionary definition and word sense of the

words in the attitude group. This taxonomy is pictured in Figure 6.2. Because the attitude type captures many of the distinctions in the ATTITUDE system (particularly the distinction of judgment vs. affect vs. appreciation), it has provided a useful model of the grammatical phenomena, while remaining simpler to store and process than the full attitude system. The only modifier currently in FLAG's lexicon to affect the attitude type of an attitude group is the word "moral" or "morally", which changes the attitude type of an attitude group to propriety from any other value (compare "excellence" which usually expresses quality versus "moral excellence" which usually expresses propriety).

An example of some of the lexicon entries is shown in Figure 6.3. This example depicts three modifiers and a head word. The modifier "too" makes any attitude negative, "not" flips the orientation of an attitude, "extremely" makes an attitude more forceful. These demonstrate the modification operations of `<modify type="increase"/>`, and `<modify type="flip"/>` which change an attribute value relative to the previous value, and `<set>` which unconditionally overwrites the old attribute value with a new one. The last entry presented is a head word which sets initial (`<base>`) values for all of the appraisal attributes. The `<constraints>` in the entries enforce part of speech tag restrictions — that "extremely" is an adverb and "entertained" is an adjective.

## 6.2 The FLAG appraisal lexicon

The words that convey attitudes are provided in a hand-constructed lexicon listing appraisal head-words with their attributes, and listing modifiers with the operations they perform on the attributes. I developed this lexicon by hand to be a domain-independent lexicon of appraisal words that are understood in most contexts to express certain kinds of evaluations. The lexicon lists head words along with values for the appraisal attributes, and lists modifiers with operations they perform on those

- Attitude Type
  - Appreciation
    - Composition
      - Balance: *consistent, discordant, ...*
      - Complexity: *elaborate, convoluted, ...*
    - Reaction
      - Impact: *amazing, compelling, dull, ...*
      - Quality: *beautiful, elegant, hideous, ...*
    - Valuation: *innovative, profound, inferior, ...*
  - Affect
    - Happiness
      - Cheer: *chuckle, cheerful, whimper ...*
      - Affection: *love, hate, revile ...*
    - Security
      - Quiet: *confident, assured, uneasy ...*
      - Trust: *entrust, trusting, confident in ...*
    - Satisfaction
      - Pleasure: *thrilled, compliment, furious ...*
      - Interest: *attentive, involved, fidget, stale ...*
    - Inclination: *weary, shudder, desire, miss, ...*
    - Surprise: *startled, jolted ...*
  - Judgment
    - Social Esteem
      - Capacity: *clever, competent, immature, ...*
      - Tenacity: *brave, hard-working, foolhardy, ...*
      - Normality: *famous, lucky, obscure, ...*
    - Social Sanction
      - Propriety: *generous, virtuous, corrupt, ...*
      - Veracity: *honest, sincere, sneaky, ...*

Figure 6.2. The *attitude type* taxonomy used in FLAG's appraisal lexicon.

```
<lexicon fileid="smallsample">
  <lexeme>
    <phrase>too</phrase>
    <entry domain="appraisal">
      <set att="orientation" value="negative"/>
    </entry>
  </lexeme>
  <lexeme>
    <phrase>not</phrase>
    <entry domain="appraisal">
      <set att="polarity" value="marked"/>
      <modify att="force" type="flip"/>
      <modify att="orientation" type="flip"/>
    </entry>
  </lexeme>
  <lexeme>
    <phrase>extremely</phrase>
    <entry domain="appraisal">
      <constraints> <pos>RB</pos> </constraints>
      <modify att="force" type="increase"/>
    </entry>
  </lexeme>
  <lexeme>
    <phrase>entertained</phrase>
    <entry domain="appraisal">
      <constraints> <pos>JJ</pos> </constraints>
      <base att="attitude" value="interest"/>
      <base att="orientation" value="positive"/>
      <base att="polarity" value="unmarked"/>
      <base att="force" value="median"/>
      <base att="focus" value="median"/>
    </entry>
  </lexeme>
</lexicon>
```

Figure 6.3. A sample of entries in the attitude lexicon.

attributes.

An adjectival appraisal lexicon was first constructed by Whitelaw et al. [173], using seed examples from Martin and White's [110] book on appraisal theory. WordNet [117] synset expansion and other thesauruses were used to expand this lexicon into a larger lexicon of close to 2000 head words. The head words were categorized according to the *attitude type* taxonomy, and assigned *force*, *orientation*, *focus*, and *polarity* values.

I took this lexicon and added nouns and verbs, and thoroughly reviewed both the adjectives and adverbs that were already in the lexicon. I also modified the *attitude type* taxonomy from the form in which it appeared in Whitelaw et al.'s [173] work, to the version in Figure 6.2, so as to reflect the different subtypes of affect.

To add nouns and verbs to the lexicon, I began with lists of positive and negative words from the General Inquirer lexicons [160], took all words with the appropriate part of speech, and assigned attitude types and orientations to the new words. I then used WordNet synset expansion to expand the number of nouns beyond the General Inquirer's more limited list. I performed a full manual review to remove the great many words that did not convey attitude, and to verify the correctness of the attitude types and orientations. During WordNet expansion, synonyms of a word in the lexicon were given the same attitude type and orientation, and antonyms were given the same attitude type with opposite orientation. Throughout the manual review stage, I consulted concordance lines from movie reviews and blog posts, to see how words were used in context.

I added modifiers for nouns and verbs to the lexicon by looking at words appearing near appraisal head words in sample texts and concordance lines. Most of the modifiers in the lexicon are intensifiers, but some are negation markers (e.g.

"not"). Certain function words, such as determiners and the preposition "of" were included in the lexicon as no-op modifiers to hold together attitude groups whose modifier chains cross constituent boundaries (for example "not a very good").

When I added nouns, I generally added only the singular (`NN`) forms to the lexicon, and used MorphAdorner 1.0 [31] to automatically generate lexicon entries for the plural forms with the same attribute values. When I added verbs, I generally added only the infinitive (`VB`) forms to the lexicon manually, and used MorphAdorner to automatically generate past (`VBD`), present (`VBZ` and `VBP`), present participle (`VBG`), gerund (`NN` ending in "-ing"), and past participle (`VBN` and `JJ` ending in "-ed") forms of the verbs. The numbers of automatically and manually generated lexicon entries are shown in Table 6.1.

FLAG's lexicon allows for a single word to have several different entries with different attribute values. Sometimes these entries are constrained to apply only to particular parts of speech, in which case I tried to avoid assigning different attribute values to different parts of speech (aside from the "part of speech" attribute). But many times a word appears in the lexicon with two entries that have different sets of attributes, usually because a word can be used to express two different attitude types, such as the word "good" which can indicate quality (e.g. "The Matrix was a good movie") or propriety ("good versus evil"). When a word appears in the lexicon with two different sets of attributes, this is done because the word is ambiguous. FLAG deals with this using the machine learning disambiguator described in Chapter 9 to determine which set of attributes is correct at the end of the appraisal extraction process.

Table 6.1. Manually and Automatically Generated Lexicon Entries.

| Part of speech | Manual | Automatic |
|---|---|---|
| JJ | 1419 | 632 |
| JJR | 46 | 0 |
| JJS | 40 | 0 |
| NN | 1155 | 635 |
| NNS | 21 | 1121 |
| RB | 376 | 0 |
| VB | 616 | 0 |
| VBD | 1 | 632 |
| VBG | 6 | 635 |
| VBN | 4 | 632 |
| VBP | 0 | 616 |
| VBZ | 1 | 629 |
| Multi-word | 169 | 5 |
| Modifiers | 191 | 12 |
| Total | 4045 | 5549 |

## 6.3 Baseline Lexicons

To evaluate the contribution of my manually constructed lexicon, I compared it against two automatically constructed lexicons of evaluative words. Both of these lexicons included only head words with no modifiers. Additionally, these lexicons only provide values for the orientation attribute. They do not list attitude types or force.

The first was the lexicon of Turney and Littman [171], where the words were hand-selected, but the orientations were assigned automatically. This lexicon was created by taking lists of positive and negative words from the General Inquirer corpus, and determining their orientations using the SO-PMI technique. The SO-PMI technique computes the semantic orientation of a word by computing the pointwise mutual information of the word with 14 positive and negative seed words, using co-occurrence information from the entire Internet discovered using AltaVista's NEAR operator.

The second was a sentiment lexicon I constructed based on SentiWordNet 3.0 [12], in which both the orientation and the set of terms included were determined automatically. The original SentiWordNet (version 1.0) was created using a committee of 8 classifiers that use gloss classification to determine whether a word is positive or negative [46, 47]. The results from the 8 classifiers were used to assign positivity, negativity, and objectivity scores based on how many classifiers placed the word into each of the 3 categories. These scores are assigned in intervals of 0.125, and the three scores always add up to 1 for a given synset. In SentiWordNet 3.0, they improved on this technique by also applying a random graph walk procedure so that related synsets would have related opinion tags. I took each word from each synset in SentiWordNet 3.0, and considered it to be positive if its positivity score was greater than 0.5 or negative if its negativity score was greater than 0.5. (In this way, each

word can only appear once in the lexicon for a given synset, but if the word appears in several synsets with different orientations, it can appear in the lexicon with both orientations.)

To get an idea of the coverage and accuracy of SentiWordNet, I compared it to the manually constructed General Inquirer's `Positiv`, `Negativ`, `Pstv`, and `Ngtv` categories [160], using different thresholds for the sentiment score. These results are shown in Table 6.2. When the SentiWordNet "positive" score is greater than or equal to the given threshold, then the word is considered positive, and it compared against the positive words in the General Inquirer for accuracy. When the "negative" score is greater than or equal to the given threshold, then the word was considered negative and it was compared against the negative words in the General Inquirer. For thresholds less than 0.625, it is possible for a word to be listed as both positive and negative, even when there's only a single synset — since the positivity, negativity, and objectivity scores all add up to 1, it's possible to have a positivity and a negativity score that both meet the threshold. The bold row with threshold 0.625 is the actual lexicon that I created for testing FLAG. The results show that there's little correlation between the content of the two lexicons.

## 6.4 Appraisal Chunking Algorithm

The FLAG attitude chunker is used to locate attitude groups in texts and compute their attribute values. The appraisal extractor is designed to deal with the common case with English adverbs and adjectives where the modifiers are pre-modifiers. Although nouns and verbs both allow for postmodifiers, I did not modify Whitelaw et al.'s [173] original algorithm to handle this. The chunker identifies attitude groups by searching to find attitude head-words in the text. When it finds one, it creates a new instance of an attitude group, whose attribute values are taken from the head word's lexicon entry. For each attitude head-word that the chunker

Table 6.2. Accuracy of SentiWordNet at Recreating the General Inquirer's Positive and Negative Word Lists.

| | Positiv | | | Negativ | | |
|---|---|---|---|---|---|---|
| Threshold | Prec | Rcl | $F_1$ | Prec | Rcl | $F_1$ |
| 0.000 | 0.011 | 0.992 | 0.022 | 0.013 | 0.990 | 0.027 |
| 0.125 | 0.052 | 0.779 | 0.097 | 0.059 | 0.773 | 0.110 |
| 0.250 | 0.071 | 0.667 | 0.128 | 0.074 | 0.676 | 0.133 |
| 0.375 | 0.096 | 0.571 | 0.165 | 0.089 | 0.557 | 0.154 |
| 0.500 | 0.128 | 0.446 | 0.199 | 0.103 | 0.448 | 0.167 |
| **0.625** | **0.180** | **0.270** | **0.216** | **0.123** | **0.318** | **0.178** |
| 0.750 | 0.252 | 0.134 | 0.175 | 0.161 | 0.188 | 0.173 |
| 0.875 | 0.323 | 0.043 | 0.076 | 0.251 | 0.070 | 0.110 |
| 1.000 | 0.278 | 0.003 | 0.006 | 0.733 | 0.005 | 0.011 |
| | Pstv | | | Ngtv | | |
| Threshold | P | R | $F_1$ | P | R | $F_1$ |
| 0.000 | 0.005 | 0.990 | 0.010 | 0.006 | 0.986 | 0.012 |
| 0.125 | 0.026 | 0.852 | 0.051 | 0.027 | 0.796 | 0.052 |
| 0.250 | 0.036 | 0.735 | 0.069 | 0.034 | 0.710 | 0.064 |
| 0.375 | 0.051 | 0.647 | 0.094 | 0.042 | 0.596 | 0.078 |
| 0.500 | 0.070 | 0.523 | 0.124 | 0.048 | 0.485 | 0.088 |
| **0.625** | **0.102** | **0.328** | **0.156** | **0.057** | **0.337** | **0.097** |
| 0.750 | 0.151 | 0.173 | 0.161 | 0.076 | 0.204 | 0.111 |
| 0.875 | 0.217 | 0.061 | 0.096 | 0.135 | 0.087 | 0.106 |
| 1.000 | 0.167 | 0.004 | 0.008 | 0.400 | 0.007 | 0.014 |

$$\begin{bmatrix} \text{Attitude:} & \text{affect} \\ \text{Orientation:} & \text{positive} \\ \text{Force:} & \text{median} \\ \text{Focus:} & \text{median} \\ \text{Polarity:} & \text{unmarked} \end{bmatrix} \Rightarrow \begin{bmatrix} \text{Attitude:} & \text{affect} \\ \text{Orientation:} & \text{positive} \\ \textbf{Force:} & \textbf{high} \\ \text{Focus:} & \text{median} \\ \text{Polarity:} & \text{unmarked} \end{bmatrix} \Rightarrow \begin{bmatrix} \text{Attitude:} & \text{affect} \\ \textbf{Orientation:} & \textbf{negative} \\ \textbf{Force:} & \textbf{low} \\ \text{Focus:} & \text{median} \\ \textbf{Polarity:} & \textbf{marked} \end{bmatrix}$$

"happy" "very happy" "not very happy"

Figure 6.4. Shallow parsing the attitude group "not very happy".

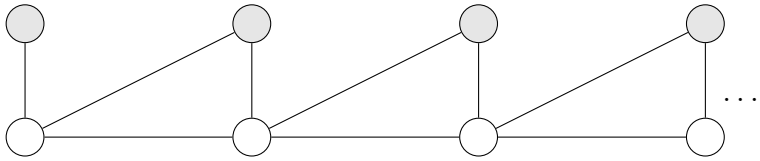
finds it moves leftwards adding modifiers until it finds a word that is not listed in the lexicon. For each modifier that the chunker finds, it updates the attributes of the attitude group under construction, according to the directions given for that word in the lexicon. An example of this technique is shown in Figure 6.4. When an ambiguous word, with two sets values for the appraisal attributes, appears in the attitude lexicon, the attitude chunker returns both versions of the attitude group, so that the disambiguator can choose the correct version later.

Whitelaw et al. [173] first applied this technique to review classification. I evaluated its precision in finding attitude groups in later work [27].

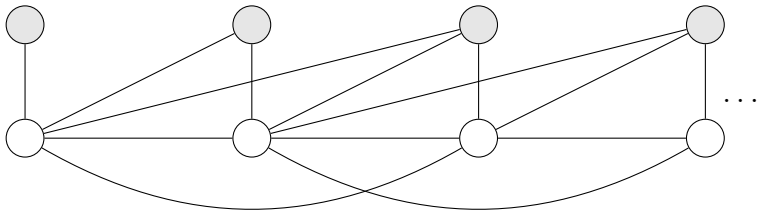
## 6.5 Sequence Tagging Baseline

To create a baseline to compare with lexicon-based opinion extraction, I employed the sequential Conditional Random Field (CRF) model from MALLET 2.0.6 [113].

**6.5.1 The MALLET CRF model.** The CRF model that MALLET uses is a sequential model with the structure shown in Figure 6.5. The nodes in the upper row of the model (shaded) represent the tokens in the order they appear in the document. The edges shown represent dependencies between the variables. Cliques in the graph structure represent feature functions. (They could also could represent overlapping n-grams in the neighborhood of the word corresponding to each node.) The model is conditioned on these nodes. Because CRFs can represent complex dependencies



(a) 1st order model.



(b) 2nd order model

Figure 6.5. Structure of the MALLET CRF extraction model.

between the variables that the model is conditioned on, they do not need to be represented directly in the graph.

The lower row of nodes represents the labels. When tagging unknown text, these variables are inferred using the CRF analog of the Viterbi algorithm [114].

When developing a model for the CRF model, the programmer defines a set of feature functions $f'_k(w_i)$ that is applied to each word node. These features can be real-valued or Boolean (which are trivially converted into real-valued features). MALLET automatically converts these internally into a set of feature functions $f_{k,l_1,l_2,\ldots}$

$$f_{k,l_1,l_2,\ldots}(w_i, label_i, label_{i-1}, \ldots) = f'_k(w_i) \times \begin{cases} 1 & \text{if } label_i = l_1 \wedge label_{i-1} = l_2 \wedge \ldots) \\ 0 & \text{otherwise} \end{cases}$$

where the number of labels used corresponds to the order of the model. Thus, if there are $n$ feature functions $f'$, and the model allows $k$ different state combinations, then there are $kn$ feature functions $f$ for which weights must be learned. In practice, there are somewhat less than $kn$ weights to learn since any feature function $f$ not seen in the training data does not need a weight.

It is possible to mark certain state transitions as being disallowed. In standard

NER BIO models, this is useful to prevent the CRF from ever predicting a state transition from OUT to IN without an intervening BEGIN.

MALLET computes features and labels from the raw training and testing data by using a pipeline of composable transformations to convert the instances from their raw form into the feature vector sequences used for training and testing the CRF.

**6.5.2 Labels.** The standard BIO model for extracting non-overlapping named entities operates by labeling each token with one of three labels:

**BEGIN:** This token is the first token in an entity reference

**IN:** This token is the second or later token in an entity reference

**OUT:** This token is not inside an entity reference

In a shallow parsing model or a NER model that extracts multiple entity types simultaneously, there is a single **OUT** label, and each entity type has two tags **B-***type* and **I-***type*. However, because the corpora I evaluate FLAG on contain overlapping annotations of different types, I only extracted a single type of entity at a time, so only the three labels **BEGIN**, **IN**, and **OUT** were used..

To convert BIO tags into individual spans, one must take each consecutive span matching the regular expression **BEGIN IN*** and treat it as an entity. Thus, the label sequence

BEGIN IN OUT BEGIN IN BEGIN

contains three spans: [1..2], [4..5], [6..6].

My test corpora use standoff annotations listing the start character and end character of each attitude and target span, and allows for annotations of the same

type to overlap each other, violating the assumption of the BIO model. To convert these to BIO tags, first FLAG converts them to token positions, assuming that if any character in a token was included in the span when expressed as start and end characters, then that token should be included in the span when expressed as start and end tokens. Then FLAG generates two labels **IN** and **OUT**, such that a token was marked as **IN** if it was in any span of the type being tested and **OUT** if it was not. FLAG then uses the MALLET pipe `Target2BIOFormat` to convert these to BIO tags. In addition to **OUT**–**IN** transitions which are already prohibited by the rules of the BIO model, this has the effect of prohibiting **IN**–**BEGIN** transitions since when there are two adjacent spans in the text, `Target2BIOFormat` can't tell where one ends and the next begins, so it considers them both to be one span.

**6.5.3 Features.** The features $f'_k$ used in the model were:

- The token text. The text was converted to lowercase, but punctuation was not stripped. This introduced a family of binary features $f'_w$

$$f'_w(token) = \begin{cases} 1 & \text{if } w = text(token) \\ 0 & \text{otherwise} \end{cases}$$

- Binary features indicating the presence of the token in each of in three lexicons. The first of these lexicons was the FLAG lexicon described in Section 6.2. The other lexicons used were the words from the `Pos` and `Neg` categories of the General Inquirer lexicon [160]. These two categories were treated as separate features. A version of the CRF was run which included these features, and another version was run which did not include these features.
- The part of speech assigned by the Stanford dependency parser. This introduced a family of binary features $f'_p$

$$f'_p(token) = \begin{cases} 1 & \text{if } p = postag(token) \\ 0 & \text{otherwise} \end{cases}$$

- For each token at position $i$, the features in a window from $i - n$ to $i + n - 1$ were included as features affecting the label of that token, using the `FeaturesInWindow` pipe. The length $n$ was tunable.

**6.5.4 Feature Selection.** When run on the corpus, the feature families above generate several thousand features $f'$. MALLET automatically multiplies these token features by the number of modeled relationships between states, as described in Section 6.5.1. For a first order model, there are 6 relationships between states (since IN can't come after an OTHER), and for second order models there are 29 different relationships between states.

Because MALLET can be very slow to train a model with this many different weights[7], I implemented a feature selection algorithm that retains only the $n$ features $f'$ with the highest information gain in discriminating between labels.

In my experiments I used a second-order model, and used feature selection to select the 10,000 features $f'$ with the highest information gain. The results are discussed in Section 10.2.

## 6.6 Summary

The first phase in FLAG's process to extract appraisal expressions is to find attitude groups, which it does using a lexicon-based shallow parser. As the shallow parser identifies attitude groups, it computes a set of attributes describing the attitude type, orientation, and force of each attitude group. These attributes are computed by starting with the attributes listed on the head-word entries in the lexicon, and applying operations listed on the modifier entries in the lexicon.

[7] When I first developed this model, certain single-threaded runs took upwards of 30 hours to do three-fold crossvalidation. Using newer hardware and multithreading seems to have improved this dramatically, possibly even without feature selection, but I haven't tested this extensively to determine what caused the slowness and why this improved performance so dramatically.

FLAG's ability to identify attitude groups is tested using 3 lexicons.

- FLAG's own manually constructed lexicon
- Turney and Littman's [171] lexicon, where the words were from the General Inquirer, and the orientations determined automatically.
- A lexicon based on SentiWordNet 3.0 [12] where both the words included and the orientations were determined automatically.
- An additional baseline is tested as well: a CRF-based extraction model.

The attitude groups that FLAG identifies are used as the starting points to identify appraisal expression candidates using the linkage extractor, which will be described in the next chapter.

# CHAPTER 7

# THE LINKAGE EXTRACTOR

The next step in extracting appraisal expressions is for FLAG to identify the other parts of each appraisal expression, relative to the location of the attitude group. Based on the ideas from Hunston and Sinclair's [72] local grammar, FLAG uses a syntactic pattern to identify all of the different pieces of the attitude group at once, as a single structure.

FLAG does not currently extract comparative appraisal expressions at all, since doing so would require identifying comparators from a lexicon, and potentially identfiying multiple attitudes. Adapting FLAG to identify comparative appraisal expressions is probably more of an engineering task than a research task — the conceptual framework described here should be able to handle comparative appraisal expressions adequately with only modifications to the implementation.

## 7.1 Do All Appraisal Expressions Fit in a Single Sentence?

Because FLAG treats an appraisal expression as a single syntactic structure, it necessarily follows that FLAG can only correctly extract appraisal expressions that appear in a single sentence. Therefore, it is important to see whether this assumption is justified.

Attitudes and their targets are generally connected grammatically, through well-defined patterns (as discussed by Hunston and Sinclair [72]). However there are some situations where this is not the case. One such case is where the target is connected to the attitude by an anaphoric reference. In this case, a pronoun appears in the proper syntactic location, and the pronoun can be considered the correct target (example 58). FLAG does not try to extract the antecedent at all. It just finds the pronoun, and the evaluations consider it correct that the extracted

appraisal expression contains the correct pronoun. Pronoun coreference is its own area of research, and I have not attempted handle it in FLAG. This works pretty well.

(58) It was $_{\text{target-antecedent}}$ a girl, and $_{\text{target}}$ she was $_{\text{attitude}}$ trouble.

Another case where syntactic patterns don't work so well is when the attitude is a surge of emotion, which is an explicit option in the affect system having no target or evaluator (example 59). FLAG can handle this by recognizing a local grammar pattern that consists of only an attitude group, and FLAG's disambiguator can select this pattern when the evidence supports it as the most likely local grammar pattern.

(59) I've learned a few things about pushing through $_{\text{attitude}}$ fear and $_{\text{attitude}}$ apprehension, this past year or so.

Another case is when a nominal attitude group also serves as an anaphoric reference to its own target (example 60). FLAG has difficulty with this case because the linkage extractor includes a requirement that each slot in a pattern has to cover a distinct span of text.

(60) I went on a date with a very hot guy, but $_{\text{target}}$ the $_{\text{attitude}}$ jerk said he had to go to the bathroom, disappeared, and left me with the bill.

Another case is when the target of an attitude appears in one sentence, but the attitude is expressed in a minor sentence that immediately follows the one containing the target (example 61). Only in this last case is the target in a different sentence from the attitude.

(61) It was a girl, and $_{\text{target}}$ she was trouble. $_{\text{attitude}}$ Big trouble.

The mechanisms to express evaluators are, in principle, more flexible than for

targets. One common way to indicate the evaluator in an appraisal expression is to quote the person whose opinion is stated, either through explicit quoting with quotation marks (as in example 62), or through attribution of an idea without quotation marks. These quotations can span multiple sentences, as in example 63. In practice, however, I have found that these two types of attribution are relatively rare in the product review domain and the blog domain. In these domains, evaluators appear in the corpus much more frequently in affective language, which tends to treat evaluators syntactically the way non-affective language treats targets, and verbal appraisal, which often requires that the evaluator be either subject or object of the verb (as in example 64). (Verbal appraisal often uses the pronoun "I" to indicate that a certain appraisal is the opinion of the author, where other parts of speech would indicate this by simply omitting any explicit evaluator.)

(62) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

(63) In addition, $_{\text{evaluator}}$ Barthelemy says, France's $_{\text{attitude}}$ pivotal role in the European Monetary Union and adoption of the euro as its currency have helped to bolster its appeal as a place for investment. "If you look at the $_{\text{attitude}}$ advantages of the euro — instant comparisons of retail or wholesale prices . . . If you deal with one currency you decrease your financial costs as you don't have to pay transaction fees. In terms of accounting and distribution strategy, it's $_{\text{attitude}}$ simpler to work with [than if each country had retained an individual currency]."

(64) $_{\text{evaluator}}$ I $_{\text{attitude}}$ loved it and $_{\text{attitude}}$ laughed all the way through.

It is easy to empirically measure how many appraisal expressions in my test corpora are contained in a single sentence. In the testing subset of the IIT Sentiment Corpus, only 9 targets out of 1426, 16 evaluators out of 814, and 1 expressor out of

28 appeared in a different sentence from the attitude. In the Darmstadt corpus, 29 targets out of 2574 appeared in a different sentence from the attitude.

Only in the JDPA corpus is the number of appraisal expressions that span multiple sentences significant — 1262 targets out of 19390 (about 6%) and 1075 evaluators out of 1836 (about 58%) appeared in a different sentence from the attitude. The large number of evaluators appearing in a different sentence is due to the presence of 67 marketing reports authored by JDPA analysts in a standardized format. In these marketing reports, the bulk of the report consists of quotations from user surveys, and the word "people" in the following introductory quote is marked as the evaluator for opinions in all of the quotations.

(65) In surveys that J.D. Power and Associates has conducted with verified owners of the 2008 Toyota Sienna, the people that actually own and drive one told us:

These marketing reports should probably be considered as a different domain from free-text product reviews like those found in magazines and on product review sites. Not only do they have very different characteristics in how evaluators are expressed, they are also likely to challenge any assumptions that an application makes about the meaning of the frequencies of different kinds of appraisal in product reviews.

Since the vast majority of attitudes in the other free-text reviews in the corpus do not have evaluators, but every attitude in a marketing report does, the increased concentration of evaluators in these marketing reports explains why the majority of evaluators in the corpus appear in a different sentence from the attitude, even though these marketing reports comprise only 10% of the documents in the JDPA corpus. However, the 6% of targets that appear in different sentences from the attitude indicate that JDPA's annotation standards were also more relaxed about where to identify evaluators and targets.

## 7.2 Linkage Specifications

FLAG's knowledge base of local grammar patterns for appraisal is stored as a set of linkage specifications that describe the syntactic patterns for connecting the different pieces of appraisal expressions, the constraints under which those syntactic patterns can be applied, and the priority by which these syntactic patterns are selected.

A linkage specification consists of three parts: a syntactic structure which must match a subtree of a sentence in the text, a list of constraints and extraction information for the words at particular positions in the syntactic structure, and a list of statistics about the linkage specification which can be used as features in the machine-learning disambiguator described in Chapter 9.

Two example linkage specifications are shown in Figure 7.1.

The first part of the linkage specification, the syntactic structure of the appraisal expression, is found on the first line of each linkage specification. This syntactic structure is expressed in a language that I have developed for specifying the links in a dependency parse tree that must be present in the appraisal expression's structure. Each link is represented as an arrow pointing to the right. The left end of each link lists a symbolic name for the dependent token, the middle of each link gives the name of the dependency relation that this link must match, and the right end of each link lists a symbolic name for the governing token. When two or more links refer to the same symbolic token name, these two links connect at a single token. The linkage language parser checks to ensure that links in the syntactic structure forms a connected graph.

Whether the symbolic name of a token constrains the word that needs to be found at that position is subject to the following convention:

```
#pattern 1
linkverb--cop->attitude target--dep->attitude
  target: extract=clause

#pattern 2:
attitude--amod->hinge target--pobj->target_prep target_prep--prep->hinge
  target_prep: extract=word word=(about,in)
  target: extract=np
  hinge: extract=shallownp word=(something,nothing,anything)

#pattern 3(iii)
evaluator--nsubj->attitude hinge--cop->attitude target--xcomp->attitude
  attitude: type=affect
  evaluator: extract=np
  target: extract=clause
  hinge: extract=shallowvp
  :depth: 3
```

Figure 7.1. Three example linkage specifications

1. The name `attitude` indicates that the word at that position needs to be the head word of an attitude group. Since the chunker only identifies pre-modifiers when identifying attitude groups, this is always the last token of the attitude group.

2. If the token at that position is to be extracted as one of the slots of the appraisal expression, then the symbolic name must be the name of the slot to be extracted. The constraints for this token will specify that the text of this slot that must be extracted and saved, and the constraints will specify the phrase type to be extracted.

3. Otherwise, there is no particular significance to the symbolic name for the token. Constraints can be specified for this token in the constraints section, including requiring a token to match a particular word, but the symbolic name does not have to hint at the nature of the constraints.

The second part of the linkage specification is the optional constraints and extraction instructions for each of the tokens. These are specified on a line that's indented, and which consists of the symbolic name of a token, followed by a colon, followed by the constraints. Three types of constraints are supported.

- A `extract` constraint indicates that the token is to be extracted and saved as a slot, and specifies the phrase type to use for that slot. The `attitude` slot does not need an `extract` constraint.
- A `word` constraint specifies that the token must match a particular word, or match one word from a set surrounded by parentheses and delimited by commas. (E.g. `word=to` or `word=(something,nothing,anything)`.)
- A `type` constraint applies to the `attitude` slot only, and indicates that the attitude type of the appraisal expression matched must be a subtype of the specified attitude type. (E.g. `type=affect` means that this linkage specification will only match attitude groups whose type is affect or a subtype of affect.)

Since the Stanford Parser generates both dependency parse trees, and phrase-structure parse trees, and FLAG saves both parse trees, the phrase types used by the `extract=` attribute are specified as groups of phrase types in the phrase structure parse tree. The following phrase types are supported:

- `shallownp` extracts contiguous spans of adjectives and nouns, starting up to 5 tokens to the left of the token matched by the dependency link, and continuing up to 1 token to the right of that token. It is intended to be used to find nominal targets when the nominal targets are named by compact noun phrases smaller than a full NP.

- `shallowvp` extracts continuous spans of modal verbs, adverbs, and verbs, starting up to 5 tokens to the left of the token matched by the dependency link, and continuing to the token itself. It is intended to be used to find verb groups, such as linking verbs and the hinges in Hunston and Sinclair's [72] local grammar.

- `np` extracts a full noun phrase (either NP or WHNP) from the PCFG tree to use to fill the slot. A command-line option can be passed to the associator to make `np` act like `shallownp`.

- `pp` extracts a full prepositional phrase (PP) from the PCFG tree to use to fill the slot. This is mostly used for extracting aspects.

- `clause` extracts a full clause (S) from the PCFG tree to use to fill the slot. This is intended to be used for extracting propositional targets.

- `word` uses only the token that was found to fill the slot. A command line option can be passed to the associator to make the associator ignore the phrase types completely and always extract just the token itself. This command-line option is intended to be used when extracting candidate appraisal expressions for the linkage specification learner described in Chapter 8.

The third part of the linkage specification is optional statistics about the linkage specification as a whole. These can be used as features of each appraisal expressions candidate in the machine learning reranker described in Chapter 9, and they can also be used for debugging purposes. These statistics are expressed on lines that start with colons, and the consist of the name of the statistic sandwiched between two colons, followed by the value of the statistic. Statistics are ignored by the associator.

The linkage specifications are stored in a text file in priority order. The linkage specifications that appear earlier in the file are given priority over those that appear

later. When an attitude group matches two or more linkage specifications, the one that appears earliest in the file is used. However, the associator also outputs all possible appraisal expressions for each attitude group, regardless of how many there are. This output is used as part of the process of learning linkage specifications (Chapter 8), and when the machine-learning disambiguator is used to select the best appraisal expressions (Chapter 9).

## 7.3 Operation of the Associator

**Algorithm 7.1** Algorithm for turning attitude groups into appraisal expression candidates

1: **for** each document $d$ and each linkage specification $l$ **do**
2: Find expressions $e$ in $d$ that meet the constraints specified in $l$.
3: **for** each extracted slot $s$ in each expression $e$ **do**
4: Identify the full phrase to be extracted for $s$, based on the `extract` attribute.
5: **end for**
6: **end for**
7: **for** each unassociated attitude group $a$ in the corpus **do**
8: Assign $a$ to the null linkage specification with lowest priority.
9: **end for**
10: Output the list of all possible appraisal expression parses.
11: **for** each attitude group $a$ in the corpus **do**
12: Delete all but the highest priority appraisal expression candidate for $a$.
13: **end for**
14: Output the list of the highest priority appraisal expression parses.

FLAG's associator is the component that turns each attitude group into a full appraisal expression using a list of linkage specifications, using the algorithm 7.1.

In the first phase of the associator's operation (line 2), the associator finds expressions in the corpus that match the structures given by the linkage specifications. In this phase the syntactic structure is checked using the augmented collapsed Stanford dependency tree described in Section 3.2.2 and the attitude position, attitude type, and word constraints are also checked. Expressions that match all of these constraints are returned, each one listing each the position of the single word where

that slot will be found.

In the second phase (line 4), FLAG determines the phrase boundaries of each extracted slot. For the `shallowvp` and `shallownp` phrase types, FLAG performs shallow parsing based on the part of speech tag. The algorithm looks for a contiguous string of words that have the allowed parts of speech, and it stops shallow parsing when it reaches certain boundaries or when it reaches the boundary of the attitude group. For the `pp`, `np` and `clause` phrase types, FLAG uses the largest matching constituent of the appropriate type that contains the head word, but does not overlap the attitude. If the only constituent of the appropriate type containing the head word overlaps the attitude group, then that constituent is used despite the overlap. If no appropriate constituent is found, then the head-word alone is used as the text of the slot. No appraisal expression candidate is discarded just because FLAG couldn't expand one of its slots to the appropriate phrase type.

When extracting candidate appraisal expressions for the linkage learner described in Chapter 8, this boundary-determination phase was skipped, so that spuriously overlapping annotations wouldn't cloud the accuracy of the individual linkage specification structures when selecting the best linkage specifications.

After determining the extent of each slot, each appraisal expression lists the slots extracted, and FLAG knows both the starting and ending token numbers, as well as the starting and ending character positions of each slot.

At the end of these two phases, each attitude group may have several different candidate appraisal expressions. Each candidate has a priority, based on the linkage specification that was used to extract it. Linkage specifications that appeared earlier in the list have higher priority, and linkage specifications that appeared later in the list have lower priority.

In the third phase (line 8), the associator adds a parse using the *null linkage specification* (a linkage specification that doesn't have any constraints, any syntactic links, or any extracted slots other than the attitude) for every attitude group. In this way, no attitude group is discarded simply because it didn't have any matching linkage specifications, and the disambiguator can select this linkage specification when it determines that an attitude group conveys a surge of emotion with no evaluator or target.

In the last phase (line 12), the associator selects the highest priority appraisal expression candidate for each attitude group, and assumes that it is the correct appraisal expression for that attitude group. The associator discards all of the lower priority candidates. The associator outputs the list of appraisal expressions both before and after this pruning phase. The list from before this pruning phase allows components like the linkage learner and disambiguator to have access to all of the candidates appraisal expressions for each attitude group, while the evaluation code sees only the highest-priority appraisal expression. The list from after this pruning phase is considered to contain the best appraisal expression candidates when the disambiguator is not used.

## 7.4 Example of the Associator in Operation

Consider the following sentence. Its dependency parse is shown in Figure 7.2, and its phrase structure parse is shown in Figure 7.3.

(66) It was an $_{\text{attitude}}$ <u>interesting</u> read.

The first linkage specification in the set is as follows:

```
attitude--amod->superordinate superordinate--dobj->t26
     target--dobj->t25 t25--csubj->t26
  target: extract=np
  superordinate: extract=np
```

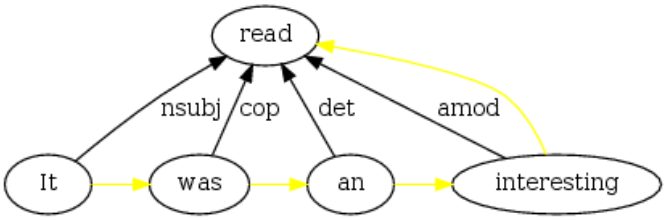



Figure 7.2. Dependency parse of the sentence "It was an interesting read."

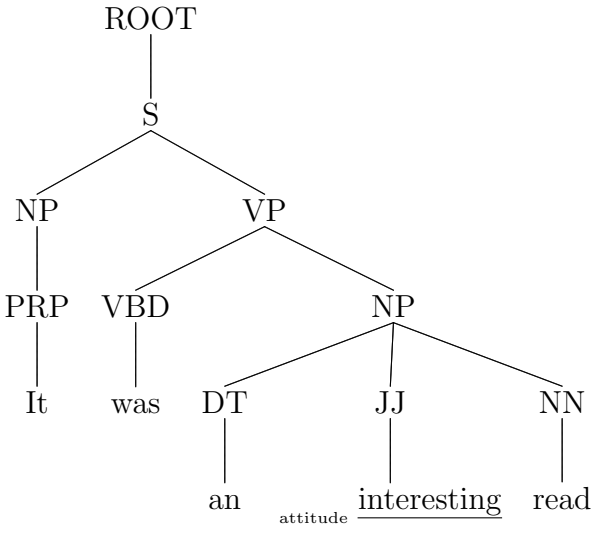



Figure 7.3. Phrase structure parse of the sentence "It was an interesting read."

```
  attitude: type=appreciation
```

The first link in the syntactic structure, `attitude--amod->superordinate` exists — there is an `amod` link leaving the head word of the attitude ("interesting"), connecting to another word in the sentence. FLAG takes this word and stores it under the name given in the linkage specification; here, it records the word "read" as the superordinate. The second link in the syntactic structure `superordinate--dobj->t26` does not exist. There is no `dobj` link leaving the word "read." Thus, this linkage specification does not match the syntactic structure in the neighborhood of the attitude "interesting", and any parts that have been extracted in the partial match are discarded.

The second linkage specification in the set is as follows:

```
attitude--amod->superordinate target--nsubj->superordinate
  target: extract=np
  attitude: type=appreciation
  superordinate: extract=np
```

The first link in the syntactic structure, `attitude--amod->superordinate` exists — it's the same as the first link matched in the previous linkage specification, and it connects to the word "read". FLAG therefore records the word "read" as the superordinate. The second link in the syntactic structure, `target--nsubj->superordinate` also exists — there is a word ("it") with an `nsubj` link connecting to the recorded superordinate "read". Therefore FLAG records the word "it" as the target.

Now FLAG applies the various constraints. The word attitude type "interesting" conveys impact, a subtype of appreciation, so the linkage specification satisfies the attitude type constraint. This is the only constraint in the linkage specification that needs to be checked.

The last step of applying a linkage specification is to extract the full phrase for each part of the sentence. The first extraction instruction is `target: extract=np`, so FLAG tries to find an `NP` or a `WHNP` constituent that surrounds the target word "it". It finds one, consisting of just the word "it", and uses that as the target. The next extraction instruction is `superordinate: extract=np`, so FLAG tries to find an `NP` or a `WHNP` constituent that surrounds the superordinate word "read". The only `NP` that FLAG can find happens to contain the attitude group, so FLAG can't use it. FLAG therefore takes just the word "read" as the superordinate.

FLAG is now done applying this linkage specification to the attitude group "interesting." Everything matched perfectly, so FLAG records this as one possible appraisal expression using the attitude group "interesting." Because this is the first linkage specification in the linkage specification set to match the attitude group, FLAG will consider it to be the best candidate when the discriminative reranker is not used. This happens to also be the correct appraisal expression.

There are still other linkage specifications in the linkage specification set, and FLAG continues on to apply linkage specifications, for the discriminative reranker or for linkage specification learning. The third and final linkage specification in this example is:

```
attitude--amod->evaluator
  evaluator: extract=word
```

This linkage specification starts from the word "interesting" as the attitude group, and finds the word "read" as the evaluator. Since the extraction instruction for the evaluator is `extract=word`, the phrase structure tree is not consulted, and the word "read" is used as the final evaluator.

| Priority | Appraisal Expression |
|---|---|
| 1 | ATTITUDE: "interesting" positive impact<br>SUPERORDINATE: "read"<br>TARGET: "It" |
| 2 | ATTITUDE: "interesting" positive impact<br>EVALUATOR: "read" |
| 3 | ATTITUDE: "interesting" positive impact |

Figure 7.4. Appraisal expression candidates found in the sentence "It was an interesting read."

After applying the linkage specifications, FLAG synthesizes final parse candidate using the null linkage specification. This final parse candidate contains only the attitude group "interesting." In total, FLAG has found all of the appraisal expression candidates in Figure 7.4.

## 7.5 Summary

After FLAG finds attitude groups, it determines the locations of the other slots in an appraisal expression relative to the position of each attitude group by using a set of linkage specifications that specify syntactic patterns to use to extract appraisal expressions. For each attitude group, the constraints specified in each linkage specification may or may not be satisfied by that attitude group. Those linkage specifications that the attitude group does match are extracted by the FLAG's linkage associator as possible appraisal expressions for that attitude group. Determining which of those appraisal expression candidates is correct is the job of the reranking disambiguator described in Chapter 9. Before discussing the reranking disambiguator, let us take a detour and see how linkage specifications can be automatically learned from an annotated corpus of appraisal expressions.

# CHAPTER 8

# LEARNING LINKAGE SPECIFICATIONS

I have experimented with several different ways of constructing the linkage specification sets used to find targets, evaluators, and the other slots of each appraisal expression.

## 8.1 Hunston and Sinclair's Linkage Specifications

The first set of linkage specifications I wrote for the associator is based on Hunston and Sinclair's [72] local grammar of evaluation. I took each example sentence shown in the paper, and parsed it using the Stanford Dependency Parser version 1.6.1 [41]. Using the uncollapsed dependency tree, I converted the slot names used in Hunston and Sinclair's local grammar to match those used in my local grammar (Section 4.2) and created trees that contained all of the required slots. The linkage specifications in this set were sorted using the topological sort algorithm described in Section 8.3. I refer to this set of linkage specifications as the **Hunston and Sinclair** linkage specifications. There are a total of 38 linkage specifications in this set.

The linkage language allows me to specify several types of constraints, including requiring particular positions in the tree to contain particular words or particular parts of speech, or restricting the linkage specification to matching only particular attitude types. I also had the option of adding additional links to the tree, beyond the bare minimum necessary to connect the slots that FLAG would extract. I took advantage of these features to further constrain the linkage specifications and prevent spurious matches. For example, in patterns containing copular verbs, I often added a "cop" link connecting to the verb. Additionally, I added some additional slots not required by the local grammar so that the linkage specifications would extract the hinge or the preposition that connects the target to the rest of the appraisal expression, so

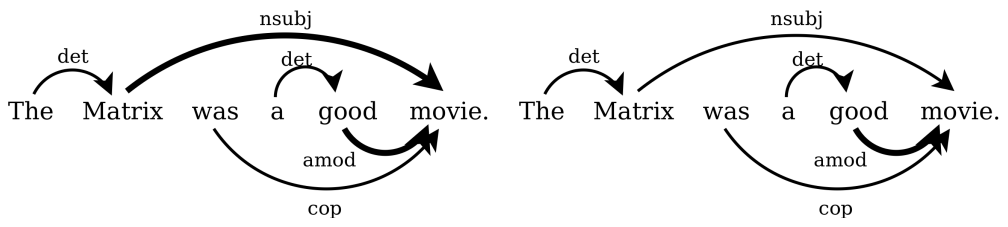
that the text of these slots could be used as features in the machine-learning disambiguator. (These extra constraints were unique to the manually constructed linkage specification sets. The linkage specification learning algorithms described later in this chapter don't know how to add any of them.)

## 8.2 Additions to Hunston and Sinclair's Linkage Specifications

Hunston and Sinclair's [72] local grammar of evaluation purports to be a comprehensive study of how adjectives convey evaluation, and to present some illustrative examples of how nouns convey evaluation (based only on the behavior of the word "nuisance"). Thus, verbs and adverbs that convey evaluation were omitted entirely, and the patterns that could be used by nouns were incomplete. I added additional patterns based on my own study of some examples of appraisal to fill in the gaps. Most of the example sentences that I looked at were from the annotation manual for my appraisal corpus (described in Section 5.5). I added 10 linkage specifications for when the attitude is expressed as a noun, adjective or adverb where individual patterns were missing from Hunston and Sinclair's study. I also added 27 patterns for when the attitude is expressed as a verb, since no verbs were studied in Hunston and Sinclair's work. Adding these to the 38 linkage specifications in the Hunston and Sinclair set, the set of **all manual linkage specifications** comprises 75 linkage specifications. These are also sorted using the topological sort algorithm described in Section 8.3.

## 8.3 Sorting Linkage Specifications by Specificity

It is often the case that multiple linkage specifications in a set can apply to the same attitude. When this occurs, a method is needed to determine which one is correct. Though I will describe a machine-learning approach to this problem in Chapter 9, a simple heuristic method for approaching this problem is to sort the




(a) "The Matrix" is the target. (b) "Movie" is the target.

Figure 8.1. "The Matrix is a good movie" matches two different linkage specifications. The links that match the linkage specification are shown as thick arrows. Other links that are not part of the linkage specification are shown as thin arrows.

linkage specifications into some order, and pick the first matching linkage specification as the correct one.

The key observation in developing a sort order is that some linkage specifications have a structure that matches a strict subset of the appraisal expressions matched by some other linkage specification. This occurs when the more general linkage specification's syntactic structure is a subtree of the less general linkage specification's syntactic structure. In Figure 8.1, linkage specification $a$ is more specific than linkage specification $b$, because $a$'s structure contains all of the links that $b$'s does, and more. If $b$ were to appear earlier in the list of linkage specifications, then $b$ would match every attitude group that $a$ could match, $a$ would match nothing, and there would be no reason for $a$ to appear in the list.

Thus, to sort the linkage specifications, FLAG creates a digraph where the vertices represent linkage specifications, and there is an edge from vertex $a$ to vertex $b$ if linkage specification $b$'s structure is a subtree of linkage specification $a$'s (this is computed by comparing the shape of the tree, and the edge labels representing the syntactic structure, but not the node labels that describe constraints on the words). Some linkage specifications can be isomorphic to each other with constraints on particular nodes or the position of the attitude differentiating them. These isomorphisms

**Algorithm 8.1** Algorithm for topologically sorting linkage specifications

```
procedure SORT-LINKAGE-SPECIFICATIONS
    g ← new graph with vertices corresponding to the linkage specifications.
    for v_1 ∈ Linkage Specifications do
        for v_2 ∈ Linkage Specifications (not including v_1) do
            if v_1 is a subtree of v_2 then
                add edge v_2 → v_1 to g
            end if
        end for
    end for
    cg ← condensation graph of g
        ▷ The vertices correspond to sets of linkage specifications with isomorphic
    structures (possibly containing only one element).
    for vs ∈ topological sort of cg do
        for v ∈ SORT-CONNECTED-COMPONENT(vs) do
            Output v
        end for
    end for
end procedure

function SORT-CONNECTED-COMPONENT(vs)
    g ← new graph with vertices corresponding to the linkage specifications in
    vs.
    for {v_1, v_2} ⊆ vs do
        f ← new instance of the FSA in Figure 8.2
        Compare all corresponding word positions in v_1, v_2 using f
        Add the edge, if any, indicated by the final state to g.
    end for
    Return topological sort of g
end function
```

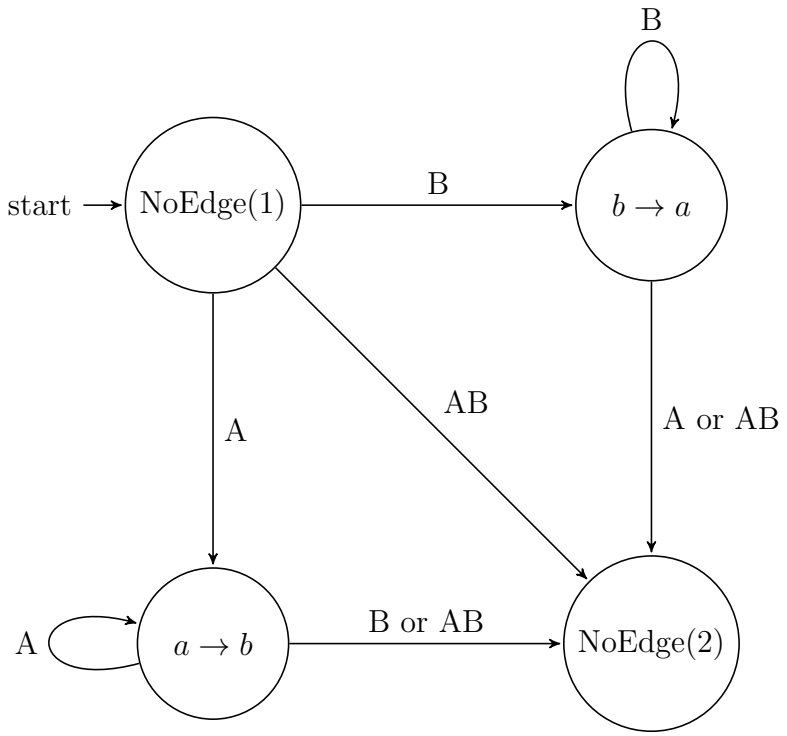



Figure 8.2. Finite state machine for comparing two linkage specifications $a$ and $b$ within a strongly connected component.

correspond to strongly connected components in the generated digraph. I compute the condensation of the graph (to represent each strongly connected component as a single vertex) and topologically sort the condensation graph. The linkage specifications are output in their topologically sorted order. This algorithm is shown in Algorithm 8.1.

To properly order the linkage specifications within each strongly connected component, another graph is created for that strongly connected component according to the constraints on particular words, and that graph topologically sorted. For each pair of linkage specifications $a$ and $b$, the finite state machine in Figure 8.2 is used to determine which linkage specification is more specific based on what constraints are present in each pair. Transition A indicates that at this particular word in position only linkage specification A has a constraint. Transition B indicates that at this

particular word in position only B has a constraint. Transition AB indicates that at this particular word position, both linkage specifications have constraints, and the constraints are different. If neither linkage specification has a constraint at this particular word position, or they both have the same constraint, no transition is taken. The constraints considered are

- The word that should appear in this location.
- The part of speech that should appear at this location.
- Whether this location links to the attitude group.
- The particular attitude types that this linkage specification can connect to.

An edge is added to the graph based on the final state of the automaton when the two linkage specifications have been completely compared. State "NoEdge(1)" indicates that we do not yet have enough information to order the two linkage specifications. If the FSA remains in state "NoEdge(1)" when the comparison is complete, it means that the two linkage specifications will match identical sets of attitude groups, though the two linkage specifications may have different slot assignments for the extracted text. State "NoEdge(2)" indicates that the two linkage specifications can appear in either order, because each has a constraint that makes it more specific than the other.

To better understand how isomorphic linkage specifications are sorted, here is an example. Consider the following three isomorphic linkage specifications shown in Figure 8.3. The three linkage specifications are sorted so that corresponding word positions are determined, as shown in figure Figure 8.4.

Then each pair is considered to determine which linkage specifications have ordering constraints.

```
target--nsubj->attitude hinge--cop->attitude
     evaluator--pobj->to to--prep->attitude
  evaluator: extract=np
  target: extract=np
  hinge: extract=shallowvp
  to: word=to

target--nsubj->attitude hinge--cop->attitude
     aspect--pobj->prep prep--prep->attitude
  target: extract=np
  hinge: extract=shallowvp
  aspect: extract=np

evaluator--nsubj->attitude hinge--cop->attitude
     target--pobj->target_prep target_prep--prep->attitude
  target_prep: extract=word
  attitude: type=affect
  target: extract=np
  evaluator: extract=np
  hinge:extract=shallowvp
```

Figure 8.3. Three isomorphic linkage specifications.

| Linkage Spec 1 | Linkage Spec 2 | Linkage Spec 3 |
|---|---|---|
| target | target | evaluator |
| attitude | attitude | attitude (`type=affect`) |
| hinge | hinge | hinge |
| evaluator | aspect | target |
| to (`word=to`) | prep | target_prep |

Figure 8.4. Word correspondences in three isomorphic linkage specifications.

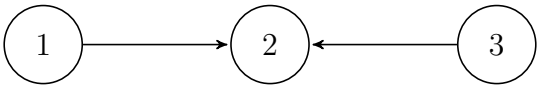



Figure 8.5. Final graph for sorting the three isomorphic linkage specifications.

First linkage specifications 1 and 2 are compared. The targets, attitudes, hinges, and the evaluator/aspect do not have constraints on them, so no transitions are made in the FSM. If these were the only slots in these linkage specifications, FLAG would conclude that they were identical, and not add any edge, because there would be no reason to prefer any particular ordering. However, there is the to/prep token, which does have a constraint in linkage specification 1. So the FSM transitions into the $1 \rightarrow 2$ state (the $A \rightarrow B$ state), because FLAG has now determined that linkage specification 1 is more specific than linkage specification 2, and should come before linkage specification 2 in the sorted list.

Then linkage specifications 1 and 3 are compared. The targets/evaluator has no constraint, but the attitude slot does — linkage specification 3 has an attitude type constraint, making it more specific than linkage specification 1. The FSM transitions into the $3 \rightarrow 1$ state (the $B \rightarrow A$ state). The hinge, and evaluator/target positions have no constraints, but the to/target_prep position does, namely the `word=` constraint on linkage specification 1. So the FSM transitions into the NoEdge(2) state. No ordering constraint is added between these two linkage specifications, because each is unique in its own way.

Then linkage specifications 2 and 3 are compared. The targets/evaluator has no constraint, but the attitude slot does — linkage specification 3 has an attitude type constraint, making it more specific than linkage specification 1. The FSM transitions into the $3 \rightarrow 2$ state (the $B \rightarrow A$ state). The hinge, evaluator/target, and prep/target_prep positions have no constraints, so the FSM remains in the $3 \rightarrow 2$ state as its final state. FLAG has now determined that linkage specification 3 is more specific than linkage specification 2, and should come before linkage specification 2 in the sorted list.

The final graph for sorting these three linkage specifications is shown in Fig-

ure 8.5. Linkage specifications 1 and 3 may appear in any order, so long as they appear before linkage specification 2.

The information obtained by sorting linkage specifications in this manner can also be used as a feature for the machine learning disambiguator. FLAG records each linkage specification's depth in the digraph as a statistic of that linkage specification for use by the disambiguator. The disambiguator also takes into account the linkage specification's overall ordering in the file. Consequently, this sorting algorithm (or the covering algorithm described in Section 8.9) must be run on linkage specification sets intended for use with the disambiguator.

## 8.4 Finding Linkage Specifications

To learn linkage specifications from a text, the linkage learner generates candidate appraisal expressions from the text (strategies for doing so are described in Sections 8.5 and 8.6), and then finds the grammatical trees that connect all of the slots.

Each candidate appraisal expression generated by the linkage learner consists of a list of distinct slot names, the position in the text at which each slot can be found, and the phrase type. For the attitude, the attitude type that the linkage specification should connect to may also be included. The following example would generate the linkage specification shown in Figure 8.1(a).

$$\{(\text{target}, \text{NP}, 2), (\text{attitude}, \text{attitude}, 5), (\text{superordinate}, \text{NP}, 6)\}$$

The uncollapsed Stanford dependency tree for the document is used for learning. It is represented in the form of a series of triples, each showing the relationships the integer positions of two words. The following example is the parse tree for the sentence shown in Figure 8.1. Each tuple has the form $(\textit{dependent}, \textit{relation}, \textit{governor})$. Since the dependent in each tuple is unique, the tuples are indexed by dependent in

a hash map or an array for fast lookup.

$$\{(1, det, 2), (2, nsubj, 6), (3, cop, 6), (4, det, 5), (5, amod, 6)\}$$

Starting from each slot in the candidate appraisal expression, the learning algorithm traces the path from the slot to the root of a tree, collecting the links it visits. Then the top of the linkage specification is pruned so that only links that are necessary to connect the slots are retained — any link that appears $n$ times in the resulting list (where $n$ is the number of slots in the candidate) is above the common intersection point for all of the paths, so it is removed from the list. The list is then filtered to make each remaining link appear only once. This list of link triples along with the slot triples that made up the candidate appraisal expression comprises the final linkage specification. This algorithm is shown in Algorithm 8.2

After each linkage specification is generated, it is checked for validity using a set of criteria specific to the candidate generator. At a minimum, it checks that the linkage specification is connected (that all of the slots came from the same sentence), but some candidate generators impose additional checks to ensure that the shape of the linkage specification is sensible. Candidates which generated invalid linkage specifications may have some slots removed to try a second time to learn a valid linkage specification, also depending on the policy of the candidate generator.

Each linkage specification learned is stored in a hash map counting how many times it appeared in the training corpus. Two linkage specifications are considered equal if their link structure is isomorphic, and if they have the same slot names in the same positions in the tree. (This is slightly more stringent than the criteria used for subtree matching and isomorphism detection in Section 8.3.) The phrase types to be extracted are not considered when comparing linkage specifications for equality; the phrase types that were present the first time the linkage specification appeared will

be the ones used in the final result, even if they were vastly outnumbered by some other combination of phrase types.

**Algorithm 8.2** Algorithm for learning a linkage specification from a candidate appraisal expression.

```
function LEARN-FROM-CANDIDATE(candidate)
    Let n be the number of slots in candidate.
    Let r be an empty list.
    for (slot = (name, d)) ∈ candidate do
        add slot to r
        while d ≠ NULL do
            Find the link l having dependent d.
            if l was found then
                Add l to r
                d ← governor of l.
            else
                d ← NULL
            end if
        end while
    end for
    Remove any link that appears n times in r.
    Filter r to make each link appear exactly once.
    Return r.
end function
```

The linkage learner does not learn constraints as to whether a particular word or part of speech should appear in a particular location.

After the linkage learner runs, it returns the $N$ most frequent linkage specifications. (I used $N = 3000$). The next step is to determine which of those linkage specifications are the best. I run the associator (Chapter 7) on some corpus, gather statistics about the appraisal expressions that it extracted, and use those statistics to select the best linkage specifications. Two techniques that I have developed for doing this by computing the accuracy of linkage specifications on a small annotated ground truth corpus are described in sections 8.8 and 8.9. In some previous work [25, 26], I discussed techniques for doing this by approximating the using ground truth annotations by taking advantage of the lexical redundancy of a large corpus that contains

documents about a single topic, however in the IIT sentiment corpus (Section 5.5) this redundancy is not available (and even in other corpora, it seems only to be available when dealing with targets, but not for the other parts of an appraisal expression), so now I use a small corpus with ground truth annotations instead of trying to rank linkage specifications in a fully unsupervised fashion.

## 8.5 Using Ground Truth Appraisal Expressions as Candidates

The ground truth candidate generator operates on ground truth corpora that are already annotated with appraisal expressions. It takes each appraisal expression that does not include comparisons[8] and creates one candidate appraisal expression from each annotated ground truth appraisal expression, limiting the candidate to the attitude, target, evaluator, expressor, process, aspect, superordinate, and comparator slots. If the ground truth corpus contains attitude types, then two identical candidates are created, one with an attitude type constraint, and one without.

For each slot, the candidate generator determines the phrase type by searching the Stanford phrase structure tree to find the phrase whose boundaries match the boundaries of the ground truth annotation most closely. It determines the token position for each slot as being the dependent node in a link that points from inside the ground truth annotation to outside the ground truth annotation, or the last token of the annotation if no such link can be found.

The validity check performed by this candidate generator checks to make sure

[8] FLAG does not currently extract comparisons, and therefore the linkage specification learners do not currently learn comparisons. This is because extracting comparisons would complicate some of the logic in the disambiguator, which would have to do additional work to determine whether two whether two non-comparative appraisal expressions should really be replaced by a single comparative appraisal expression with two attitudes. The details of how to adapt FLAG for this are probably not difficult, but they're probably not very technically interesting, so I did not focus on this aspect of FLAG's operation. There's no technical reason why FLAG couldn't be expanded to handle comparatives using the same framework by which FLAG handles all other types of appraisal expressions.

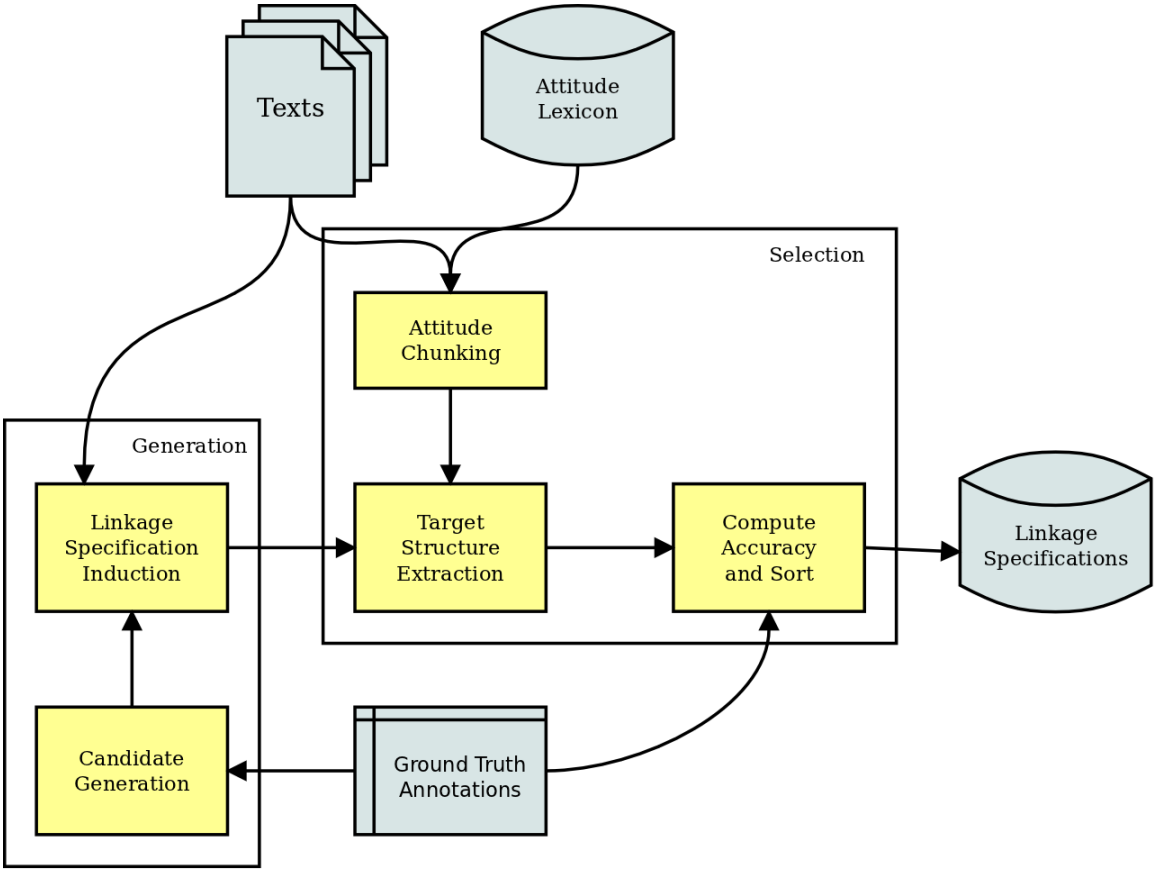



Figure 8.6. Operation of the linkage specification learner when learning from ground truth annotations

that the learned linkage specifications are connected, and that they don't have multiple slots at the same position in the tree. If a linkage specification is invalid, then the linkage learner removes the evaluator and tries a second time to learn a valid linkage specification. (The evaluator is removed because it can sometimes appear in a different sentence when the appraisal expression is inside a quotation and the evaluator is the person being quoted. Evaluators expressed through quotations should be found using a different technique, such as that of Kim and Hovy [88].)

Figure 8.6 shows the process that FLAG's linkage specification learner uses when learning linkage specifications from ground truth annotations.

## 8.6 Heuristically Generating Candidates from Unannotated Text

The unsupervised candidate generator operates by heuristically generating different slots and throwing them together in different combinations to create candidate appraisal expressions. It operates on a large unlabeled corpus. For this purpose, I used a subset of the ICWSM 2009 Spinn3r data set.

The ICWSM 2009 Spinn3r data set [32] is a set of 44 million blog posts made between August 1 and October 1, 2008, provided by Spinn3r.com. These blog posts weren't selected to cover any particular topics. The subset that I used for linkage specification learning consisted of 26992 documents taken from the corpus. This subset was large enough to distinguish common patterns of language use from uncommon patterns, but small enough that the Stanford parser could parse it in a reasonable amount of time, and FLAG could learn linkage specifications from it in a reasonable amount of time.

Candidate attitudes are found by using the results of the attitude chunker (Chapter 6), and then for each attitude, a set of potential targets is generated based on heuristic of finding noun phrases or clauses that start or end within 5 tokens of the attitude. For each attitude, and target pair, candidate superordinates, aspects, and processes are generated. The heuristic for finding superordinates is to look at all nouns in the sentence and select as superordinates any that WordNet identifies as being a hypernym of a word in the candidate target. (This results in a very low occurrence of superordinates in the learned linkage specifications.) The heuristic for finding aspects is to take any prepositional phrase that starts with 'in', 'on' or 'for' and starts or ends within 5 tokens of either the attitude or the target. The heuristic for finding processes is to take any verb phrase that starts or ends within 3 tokens of the attitude.

Additionally candidate evaluators are found by running the named entity recognition system in OpenNLP 1.3.0 [13] and taking named entities identified as organizations or people and personal pronouns appearing in the same sentence. No attempt is made to heuristically identify expressors.

Once all of these heuristic candidates are gathered for each appraisal expression, different combinations of them are taken to create candidate appraisal expressions, according to the list of patterns shown in Figure 8.7. Candidates that have two slots at the same position in the text are removed from the set. After the candidates for a document are generated, duplicate candidates are removed. Two versions of each candidate are generated — one with an attitude type (either appreciation, judgment, or affect), and one without.

The validity check performed by this candidate generator checks to make sure that each learned linkage specification is connected. Disconnected linkage specifications are completely thrown out. This candidate generator has no fallback mechanism, because suitable fallbacks are already generated by the component that takes different combinations of the slots to create candidate appraisal expressions.

Figure 8.8 shows the process that FLAG's linkage specification learner uses when learning linkage specifications a large unlabeled corpus.

## 8.7 Filtering Candidate Appraisal Expressions

In order to determine the effect of some of the conceptual innovations that FLAG implements — the addition of attitude types and extra slots beyond attitudes, targets, and evaluators, FLAG's linkage specification learner has optional filters implemented that allow one to turn off the innovations for comparison purposes.

One filter is used to determine the relative contribution of attitude types to FLAG's performance. This filter operates by taking the output from a candidate gen-

- attitude, target, process, aspect, superordinate
- attitude, target, superordinate, process
- attitude, target, superordinate, aspect
- attitude, target, superordinate
- attitude, target, process, aspect
- attitude, target, process
- attitude, target, aspect
- attitude, target
- attitude, target, evaluator, process, aspect, superordinate
- attitude, target, evaluator, process, superordinate
- attitude, target, evaluator, aspect, superordinate
- attitude, target, evaluator, superordinate
- attitude, target, evaluator, process, aspect
- attitude, target, evaluator, process
- attitude, target, evaluator, aspect
- attitude, target, evaluator
- attitude, evaluator

Figure 8.7. The patterns of appraisal components that can be put together into an appraisal expression by the unsupervised linkage learner.

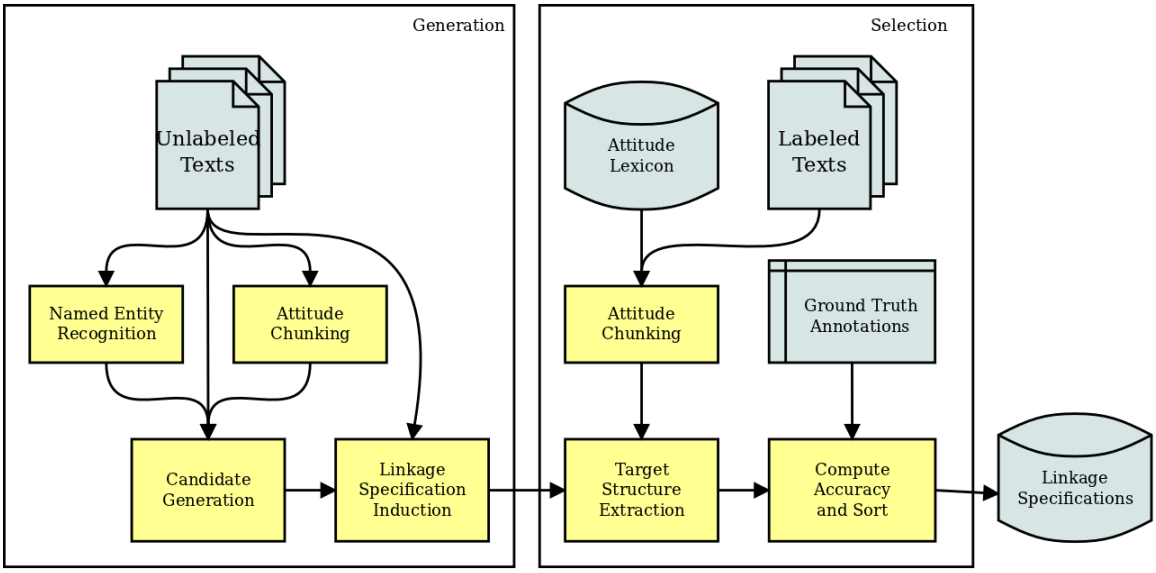



Figure 8.8. Operation of the linkage specification learner when learning from a large unlabeled corpus

erator (either the supervised or unsupervised candidate generator discussed above), and removes any candidates that have attitude type constraints. Since the candidate generators generate candidates in pairs — one with an attitude type constraint, and another that's otherwise identical, but without the attitude type constraint — this cause the linkage learner to find all of the same linkage specifications as would be found if the candidate generator were unfiltered, but without any attitude type constraints.

The other filter is used to determine the relative contribution of including aspect, process, superordinate, and expressor slots in the structure of the extracted linkage specifications. This filter operates by taking the output from a candidate generator, and modifies the candidates to restrict them to only attitude, target, and evaluator slots. It then checks the list of appraisal expression candidates from each document and removes any duplicates.

## 8.8 Selecting Linkage Specifications by Individual Performance

The first method for selecting linkage specifications that I implemented does so by considering both frequency with which the linkage structure appears in a corpus, and the frequency with which it is correct, independently of any other linkage specification. This technique is based on my previous work [25, 26] applying this technique in an unsupervised setting.

I run the associator (Chapter 7) on a small development corpus annotated with ground truth appraisal expressions, using the 3000 most frequent linkage specifications from the linkage-specification finder, and retain all extracted candidate interpretations (unlike target extraction where FLAG retains only the highest priority interpretation). Then, FLAG compares the accuracy of the extracted candidates with the ground truth.

In the first step of the comparison phase, the ground truth annotations and the extracted candidates are filtered to retain only expressions where the extracted candidate's attitude group overlaps a ground truth attitude group. Counting attitude groups where the attitude group is wrong when computing accuracy would penalize the linkage specifications for mistakes made by the attitude chunker (Chapter 6), so those mistakes are eliminated before comparing the accuracy of the linkage specifications.

After that, each linkage specification is evaluated to determine how many of the candidate interpretations it extracted are correct. The linkage specification is assigned a score

$$\frac{\log(correct + 1)}{\log(correct + incorrect + 2)}$$

The 100 highest scoring linkage specifications are selected to be learned for extraction and sorted according topologically using the algorithm described in Section 8.3.

(I've experimented with other scoring functions such as the Log-3 metric [26], defined as $\frac{correct}{[\log(correct+incorrect+2)]^3}$, and the Both$^2$ metric [25], defined as $\frac{correct^2}{correct+incorrect}$ but they turned out to be less accurate.)

The criteria used to decide whether an appraisal expression is correct can be changed depending on the corpus. On the IIT sentiment corpus (Section 5.5), all slots in the appraisal expression are considered. On the other corpora that do not define all of these slots, only the "attitude," "evaluator," and "target" slots need to be correct for the appraisal expression to be correct; in this situation, slots like superordinates or processes, if present in a linkage specification, are simply extra constraints to hopefully make the linking phase more accurate.

## 8.9 Selecting Linkage Specifications to Cover the Ground Truth

Another way to select the best linkage specifications is to consider how selecting one linkage specification removes the attitude groups that it matches from consideration by other linkage specifications. In this algorithm, I run the associator development corpus as described in Section 8.8, and remove extracted appraisal expressions where the attitude group doesn't match an attitude group in the ground truth.

Then Algorithm 8.3 is run. The precision of each linkage specification's appraisal expression interpretation candidates is computed, and the linkage specification with the highest precision is added to the result list. Then every appraisal expression that this linkage specification is marked as used (even the interpretations that were found a different linkage specification). The precision of the remaining linkage specifications is computed on the remaining appraisal expressions, iteratively until there are no remaining linkage specifications that found any correct interpretations.

The linkage specifications found by this algorithm do not need to be sorted using the algorithm described in Section 8.3 because this algorithm selects linkage specifications in topologically sorted order.

There is some room for variability in line 8 when breaking ties between two linkage specifications that have the same accuracy. FLAG resolves ties by always selecting the less frequent linkage specification (this is just for consistency — performance-wise, it makes little difference how the tie is broken).

## 8.10 Summary

The linkage specifications that FLAG uses to extract appraisal expressions can be manually constructed or automatically learned. Two sets of manually constructed linkage specifications that have been developed for FLAG are a set constructed only

**Algorithm 8.3** Covering algorithm for scoring appraisal expressions

```
function ACCURACY-BY-COVERING
    ls ← All linkage specifications.
    r ← empty results list.
    while There are unused appraisal expressions and there are linkage specifications remaining in ls do
        for l ∈ ls do
            Compute precision of l over unused appraisal expressions.
        end for
        next ← the linkage specification with the greatest accuracy.
        Remove any linkage specification from ls that had no correct matches.
        Remove next from ls
        Add next to r
        Mark all attitude groups matched by next as used.
    end while
    Return r
end function
```

from patterns found in Hunston and Sinclair's [72] local grammar of evaluation, and a set that starts with these patterns but adds more based on manual observations of a corpus to add coverage for parts of speech not considered by Hunston and Sinclair.

FLAG's linkage specification learner starts by learning a large set of potential linkage specifications from patterns that it finds in text. These linkage specifications can be learned from annotated ground truth, or from a large unannotated corpus using heuristics to identify linkage possible slots in appraisal expressions.

After this large set of potential linkage specifications, FLAG can apply one of two pruning methods to remove underperforming linkage specifications from the set.

After that it sorts the linkage specifications so that the most specific linkage specifications come first in the list. When the reranking disambiguator is not used, the first linkage specification in the list (the most specific) is considered to be the best candidate. When the reranking disambiguator is used, this sorting information is used as a feature in the reranking disambiguator.

# CHAPTER 9

# DISAMBIGUATION OF MULTIPLE INTERPRETATIONS

## 9.1 Ambiguities from Earlier Steps of Extraction

In the previous processing steps, a fundamental part of FLAG's operation was to create multiple candidates or interpretations of the appraisal expression being extracted. The last step of appraisal extraction is to perform machine learning disambiguation on each attitude group to select the extraction pattern and feature set that are most consistent with the grammatical constraints of appraisal theory. The idea that machine learning should be used to find the most grammatically consistent candidate appraisal expressions is based on Bednarek's [21] observation that attitude type of an appraisal, the local grammar pattern by which it is expressed, and features of the target and other slots extracted from the local grammar pattern impose grammatical constraints on each other.

Each of the earlier steps of the extractor each have the potential to introduce ambiguity. First, an attitude group extracted by the chunker described in Chapter 6 may be ambiguous as to attitude type, and consequently will be listed in the appraisal lexicon with both attitude types. This usually occurs when the word has multiple word senses, as in the word "good", which may indicate propriety (as good versus evil) or quality (e.g. reading a good book). The codings for these two word-senses are shown in Figure 9.1. Another example is the word "devious", defined by the American Heritage College Dictionary as "not straightforward; shifty; departing from the correct or accepted way; erring; deviating from the straight or direct course; roundabout". In the case of the word "devious", the different word senses can have different orientations for the different attitude types. "Devious" can be used in both a sense of "clever" (positive capacity) and a sense of "ethically questionable" (negative propriety); the attributes for both word senses are shown in Figure 9.2.

Second, it is possible for several different linkages to match an attitude group, connecting the attitude group to different targets. In some cases, this is incidental, but in most cases it is inevitable because some patterns are supersets of other more specific patterns.

The following two linkages are an example of this behavior. The second linkage will match the attitude group in any situation that the first will match, since the superordinate in linkage 1 is the target in linkage 2.

```
# Linkage Specification #1
target--nsubj->x superordinate--dobj->x attitude--amod->superordinate
  target: extract=np
  superordinate: extract=np

# Linkage Specification #2
attitude--amod->target
  target: extract=np
```

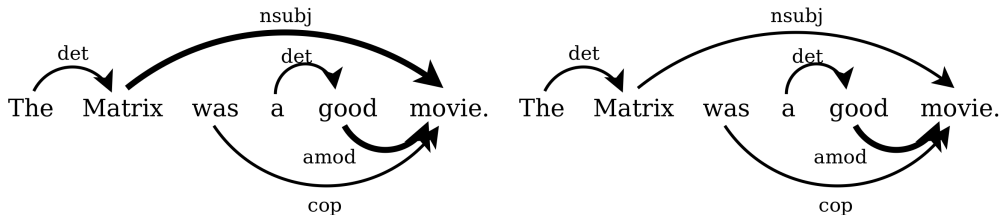
In this example, the first specification extracts a target that is the subject of the sentence, and a superordinate that is modified by the appraisal attitude. For example in the sentence "The Matrix is a good movie," it identifies "The Matrix" as the target, and "movie" as the superordinate. The second linkage specification extracts a target that is directly modified by the adjective group — the word "movie" in the example sentence. The application of these two linkage patterns to the sentence

| **good** girls | | a **good** camera | |
|---|---|---|---|
| Attitude: | **propriety** | Attitude: | **quality** |
| Orientation: | positive | Orientation: | positive |
| Force: | median | Force: | median |
| Focus: | median | Focus: | median |
| Polarity: | unmarked | Polarity: | unmarked |

Figure 9.1. Ambiguity in word-senses for the word 'good'

| **devious** (clever) | | **devious** (ethically questionable) | |
|---|---|---|---|
| Attitude: | **complexity** | Attitude: | **propriety** |
| Orientation: | **positive** | Orientation: | **negative** |
| Force: | high | Force: | high |
| Focus: | median | Focus: | median |
| Polarity: | unmarked | Polarity: | unmarked |

Figure 9.2. Ambiguity in word-senses for the word 'devious'




(a) "The Matrix" is the target, and "movie" is the superordinate.

(b) "Movie" is the target.

Figure 9.3. "The Matrix is a good movie" under two different linkage patterns

is shown in Figure 9.3. Disambiguation is necessary to choose which of these is the correct interpretation. In this example, the appropriate interpretation would be to recognize "The Matrix" as the target, and to recognize "movie" as the superordinate.

In Section 8.3, I resolved this behavior by sorting linkage specifications by their specificity and selecting the most specific one, but this doesn't always give the right answers for every appraisal expression, so I explore a more intelligent machine learning approach in this chapter.

A third area of ambiguity in FLAG's extraction is to determine whether an extracted appraisal expression is really appraisal or not. This ambiguity occurs often when extracting polar facts, where words in the which convey evoked appraisal in one domain do not convey appraisal in another domain. Domain adaptation techniques to deal with this problem have been an active area of research [24, 40, 85, 138, 139,

143, 188]. Although FLAG does not extract polar facts, this kind of ambiguity is still a problem because there are many generic appraisal words that have both subjective and objective word senses, including such words as "poor", "like", "just", and "able", and "low".

FLAG seeks to resolve the first two types of ambiguities by using a discriminative reranker to select the best appraisal expression for each attitude group, as described below. FLAG does not address the third type of ambiguity, though there has been work in resolving it elsewhere in sentiment analysis literature [1, 178].

## 9.2 Discriminative Reranking

Discriminative reranking [33, 35, 36, 39, 81, 88, 149, 150] is a technique used in machine translation and probabilistic parsing to select the best result from a collection of candidates, when those candidates were generated using a generative process that cannot support very complex dependencies between different parts of a candidate. Because discriminative learning techniques don't require independence between the different features in the feature set and because features can take into account the complete candidate answer at once, discriminative reranking is an ideal way to select the answer candidate that best fits a set of criteria that are more complicated than what the generative process can represent.

In Charniak and Johnson's [33] probabilistic parser, for example, the parse of a sentence is represented as a tree of constituents. The sentence itself is one single constituent, and that constituent has several non-overlapping children that break the entire sentence into smaller constituents. Those children have constituents within them, and so forth, until the level at which each word is a separate constituent. In the grammar used for probabilistic parsing, each constituent is assigned a small number of probabilities, based on the frequency with which it appeared in the training data that

was used to develop the grammar. The probability of a constituent of a particular type appearing at a particular place in the tree is conditioned only on things that are local to that constituent itself, such as the types of its children. In the first phase of parsing, the parser selects constituents to maximize the overall probability based on this limited set of dependencies. The first phase returns the 50 highest probability parses for the sentence. In the second phase of parsing, a discriminative reranker selects between these parses based on a set of more complex binary features that describe the overall shape of the tree. For example, English speakers tend to arrange their sentences so that more complicated constituents appear toward the end of the sentence, therefore the discriminative reranker has several features to indicate how well parse candidates reflect this tendency.

In FLAG, the problem of selecting the best appraisal expression candidates can be viewed as a problem of reranking. For each extracted attitude group, the previous steps in FLAG's extraction process created several different appraisal expression candidates, differing in their appraisal attitudes, and in the syntactic structure used to connect the different slots. FLAG can then use a reranker to select the best appraisal expression candidates for each attitude group.

Reranking problems differ from classification problems because they lack a fixed list of classes. In classification tasks, a learning algorithm is asked to assign each instance a class from a fixed list of classes. Because the list of possible classes is the same for all instances, it is easy to learn weights for each class separately. In reranking tasks, rather than selecting between different classes, the learning algorithm is asked to select between different instances of a particular "query". The list of instances varies between the queries, so it is not possible group them into classes and select the class with the highest score. Instead, for each query, the different instances considered in pairs and a classifier is trained to minimize the number of pairs that

are out of order. This turns out to be mathematically equivalent to training a binary classification problem to determine whether each pair of instances is in order or out of order, using a feature vector created by subtracting one instance's feature vector from the other's. Thus, one who is performing reranking trains a classifier to determine whether the difference between the feature vectors in each pair belongs in the class of in-order pairs, which will be assigned positive scores by the classifier, or the class of out-of-order pairs, which will be assigned negative scores. Pairs of vectors that come from different queries are not compared with each other. When reranking instances, the classifier takes the dot product of the weight vector that it learned and a single instance's feature vector, just as a binary classifier would, to assign each instance a score. For each query, the highest-scoring instance is selected as the correct one. This formulation of the discriminative reranking problem been applied to several learning algorithms, including Support Vector Machines [81], and perceptron [149].

## 9.3 Applying Discriminative Reranking in FLAG

FLAG uses $SVM^{rank}$ [81, 82] as its reranking algorithm.

To train the discriminative reranker, FLAG runs the attitude chunker and the associator on the a labeled corpus, and saves the full list of candidate appraisal expressions (including the candidate with the null linkage specification). The set of candidates appraisal expressions for each attitude group is considered a single query, and ranks are assigned — rank 1 to any candidates that are correct, and rank 2 to any candidates that are not correct. A vector file in constructed from the candidates, and the SVM reranker is trained with a linear kernel using that vector file. FLAG does not have any special rankings for partially correct candidates — they're simply incorrect, and are given rank 2. Learning from partially correct candidates is a possible future improvement for FLAG's reranker.

change of state, basic cognitive process, higher cognitive process, natural phenomenon, period of time, ability, animal, organization, statement, creation, mental process, social group, idea, device, status, quality, natural event, subject matter, group, substance, state, activity, knowledge, communication, person, living thing, human activity, object, physical entity, abstract entity

Figure 9.4. WordNet hypernyms of interest in the reranker.

To use the disambiguator to determine select the best appraisal expression for each attitude group, FLAG runs the attitude chunker and the associator on the a labeled corpus, and saves the full list of candidate appraisal expressions. The set of candidates appraisal expressions for each attitude group is considered a single query, but no ranks need to be assigned in the vector file when using the model to rank instances. The SVM model is used to assign scores to each candidate, and for each appraisal expression. Since the scores returned by $SVM^{rank}$ parallel the ranks, (something with rank 1 will have a lower score than something with rank 2), for each attitude group, the candidate with the lowest score is considered to be the best one.

FLAG's reranker uses the following features to characterize appraisal expression candidates. These features are all binary features unless otherwise noted.

- Whether each of the following slots is present in the linkage specification: the evaluator, the target, the aspect, the expressor, the superordinate, and the process.
- Each of the words in the evaluator, target, aspect, expressor, superordinate, and process slots is checked using WordNet to determine all of its ancestors in the WordNet hypernym hierarchy. If any of the terms shown in Figure 9.4 is found, then a feature $f(slotname, hypernym)$ is included in the feature vector.
- The `extract=` phrase type specifier from the linkage specification of the evaluator, target, aspect, expressor, superordinate, and process slots.

- The preposition connecting the target to the attitude, if there is one, and if the linkage specification extracts this as a slot. (Only the manual linkage specifications recognize and extract this as a slot.)

- The part of speech of the attitude head word.

- The type of the attitude group at all levels of the attitude type hierarchy.

- The depth of the linkage specification graph created by the topological sort algorithm (Section 8.3). This is a numeric feature ranging between 0 and 1, where 0 is the depth of the lowest linkage specification in the file, and 1 is the depth of the highest specification in the linkage file. There can be many linkage specifications that have the same depth, since the sort tree is not very deep and many linkage specifications do not have a specific order with regard to each other.

- The priority of the linkage specification — the absolute order in which it appears in the file. This is a family of binary features, with one binary feature for each linkage specification in the file. This allows the SVM to consider specific linkage specifications as being more likely or less likely.

- The priority of the linkage specification as a numeric feature, normalized to range from 0 (for the highest priority linkage) to 1 (for the null linkage specification, which is considered the lowest priority). This allows the SVM to consider the absolute order of the linkage specifications that would be used by the learner if the disambiguator were not applied.

- A family of binary features corresponding to many of the above features, that combine those features with the attitude type of the attitude group, and its part of speech. Specifically, for each feature relating to a particular slot in the appraisal expression (whether that slot is present, what its hypernyms are, what

phrase type is extracted), a second binary feature is generated that is true if the original feature true, and the attitude conveys a particular appraisal type. A third binary feature is generated that is true if the original feature true, the attitude conveys a particular appraisal type, and the attitude head word had a particular part of speech.

## 9.4 Summary

FLAG's final step is to use a discriminative reranker to select the best appraisal expression candidate for each attitude group. Ambiguities in the attributes of an attitude group are resolved at this stage, and the best syntactic structure is chosen from the candidate parses generated by the linkage extractor.

FLAG does not apply machine learning to determine whether an identified attitude group is correct — FLAG currently assumes that all identified attitude groups are conveying evaluation in context — but other work has been done that addresses this problem.

# CHAPTER 10
# EVALUATION OF PERFORMANCE

## 10.1 General Principles

In the literature, there have been a lot of different ways to evaluate sentiment extraction, each with its own pluses and minuses. Different evaluations that have been performed in the literature include:

*Review classification.* Review classification is intended to determine whether a review has an overall positive or negative orientation, usually determined by the number of starts the reviewer assigned to the product being reviewed. When applied to a technique like that of Whitelaw et al. [173], which identifies attitude groups and then uses their attributes as a feature for a review classifier, it is as though the correctness of the appraisal expressions is being evaluated by evaluating a summary of those appraisal expressions.

*Opinionated sentence identification.* Some work [44, 69, 70, 95, 101, 136] evaluates opinion extraction by using the identified attitude groups to determine whether a sentences as being opinionated or not, and to determine the orientation of each opinionated sentence. This is usually performed under either the incorrect assumption that a single sentence conveys a single kind of opinion, or performed with the goal of summarizing all of the individual evaluative expressions in a sentence.

*Opinion lexicon building* [138] and accuracy at identifying distinct product feature names in a document [95] or corpus [69, 102], are both concerned with the idea of finding the different kinds of opinions that exist in a document, counting them once whether they appear only once in the text, or whether they appear many times in the text. Finding distinct opinion words in a corpus makes sense when the goal is to construct an opinion lexicon, and identifying product feature names is useful as

an information extraction task for learning about the mechanics of a type of product, but this kind of evaluation doesn't appear to be very useful when the goal is to study the opinions that people have about a product, because it doesn't take into account how frequently opinions were found in the corpus.

All of these techniques can mask error in the individual attitudes extracted, because a minority of incorrect attitudes can be canceled out against a majority of correct ones.

Kessler and Nicolov [87] performed a unique evaluation of their system. Their goal was to study a particular part of the process of extracting appraisal expressions — connecting attitudes with product features — so they provided their system with both the attitudes and potential product features from ground-truth annotations, and evaluated accuracy only based on how well their system could connect them. This evaluation technique was not intended to be an end-to-end evaluation of opinion extraction.

My primary goal is to evaluate FLAG's ability to extract every appraisal expression in a corpus correctly, and to measure FLAG's ability to run end-to-end to extract appraisal expressions, starting with nothing.

To perform an end-to-end evaluation of FLAG's performance, while also being able to understand the overall contribution of various parts of FLAG's operation, I have focused on three primary evaluations. The first is to evaluate how accurately FLAG identifies individual attitude occurrences in the text, and how accurately FLAG assigns them the right attributes. This evaluation appears in Section 10.2.

The second is to evaluate how often FLAG's associator finds the correct structure of the full appraisal expression. In this evaluation, FLAG's appraisal lexicon is used to find all of the attitude groups in a particular corpus. Then different sets

of linkage specifications are used to associate these attitude groups with the other slots that belong in appraisal expressions. The ground truth and extraction results are both filtered so that only appraisal expressions with correct attitude groups are considered. Then from these lists, the accuracy of the full appraisal expressions is computed, and this is reported as the percentage accuracy. This evaluation is performed in several upcoming sections in this chapter (Sections 10.4 thru 10.8). Those sections compare different sets of linkage specifications against each other on different corpora, with and without the use of the disambiguator, in order to study the effect of different learning algorithms and variations on the types of linkage specifications learned.

Although this evaluation focuses on the performance of a particular aspect of appraisal extraction, end-to-end extraction accuracy can be computed (exactly) by simply multiplying precision and recall from this evaluation by the precision and recall of finding attitude groups using FLAG's appraisal lexicon because the tests that measure the accuracy of the FLAG's associator are conditioned on using only attitude groups that were correctly found by FLAG's attitude chunker. This end-to-end extraction accuracy using FLAG's appraisal lexicon with selected sets of linkage specifications is reported explicitly in Section 10.9 for the best performing variations. One can perform similar multiplication to estimate what the end-to-end extraction accuracy would be if one of the baseline lexicons was used to find attitude groups instead of FLAG's appraisal lexicon.

I also report the end-to-end extraction accuracy at identifying particular slots in an appraisal expression in Section 10.9. In that evaluation, FLAG's appraisal lexicon appraisal lexicon was used to find attitude groups, and a particular set of linkage specifications was used to find linkage specifications. Then, for each particular type of slot (e.g. targets), all of the occurrences of that slot in the ground truth were

compared against all of the extracted occurrences of that slot. This was done without regard for whether the attitude groups in these appraisal expressions were correct, and without regard for whether any other slot in these appraisal expressions was correct.

In the UIC Review corpus, since the only available annotations are product features, there is no way to test the separate components of FLAG individually to study how different components contribute to FLAG's performance. I therefore perform end-to-end extraction using linkage specifications learned on the IIT sentiment corpus, and present precision and recall at finding individual product feature mentions in a separate section from the other experiments, Section 10.11. This is different from the evaluations performed by Hu [69], Popescu [136], and the many others whom I have already mentioned in Section 5.2, due to my contention that the correct method of evaluation is to determine how well FLAG finds individual product feature mentions (not distinct product feature names), and due to the inconsistencies in how the corpus is distributed that have already been discussed in Section 5.2.

**10.1.1 Computing Precision and Recall.** In the tests that study FLAG's accuracy at finding attitude groups, and in the tests that study FLAG's end-to-end appraisal expression extraction accuracy, I present results that show FLAG's precision, recall, and $F_1$.

In all of the tests, a slot was considered correct if FLAG's extracted slot overlapped with the ground truth annotation. Because the ground truth annotations on all of the corpora may list an attitude multiple times if it has different targets[9], duplicates are removed before comparison. Attitude groups and appraisal expressions

[9] The IIT sentiment corpus is annotated this way, but the other corpora are not annotated this way by default. The algorithms that process their annotations created the duplicate attitudes when multiple targets were present, so that FLAG could treat all of the various annotation schemes in a uniform manner.

extracted by FLAG that had ambiguous sets of attributes are also de-duplicated before comparison.

Since the overlap criteria don't enforce a one-to-one match between ground truth annotations and extracted attitude groups, precision was computed by determining which extracted attitude groups matched any ground truth attitude groups, and then recall was computed separately by determining which ground truth attitude groups matched any extracted attitude groups. This meant that the number of true positives in the ground truth could be different from the number of true positives in the extracted attitudes.

$$P = \frac{\text{correctly extracted}}{\text{correctly extracted} + \text{incorrectly extracted}} \tag{10.1}$$

$$R = \frac{\text{ground truth annotations found}}{\text{ground truth found} + \text{ground truth not found}} \tag{10.2}$$

$$F_1 = \frac{2}{P^{-1} + R^{-1}} \tag{10.3}$$

**10.1.2 Computing Percent Accuracy.** In the tests that study the accuracy of FLAG's associator, which operate only on appraisal expressions where the attitude group was already known to be correct, I present results that show percentage of appraisal expressions that FLAG's associator got correct. In principle, the number of ground-truth appraisal expressions should be the same as the number of appraisal expressions that FLAG found. In practice, however, these numbers can vary slightly, for two reasons. First, the presence of conjunctions in a sentence can cause FLAG to extract multiple appraisal expressions for a single attitude group. For the same reason, there can be multiple appraisal expressions in the ground truth for a single attitude group. Second, a single FLAG attitude group can overlap multiple ground truth attitude groups (or vice-versa), in which case all attitude groups involved were considered correct.

These slight differences in counts can cause precision and recall to be slightly different from each other, though in principle they should be the same. However, the differences are very small, so I simply use the precision as though it was the percentage accuracy. This means that the percentage accuracy reported for FLAG's associator is the number of extracted appraisal expressions where all concerned slots are correct, divided by the total number of extracted appraisal expressions where the attitude was correct. Throughout this chapter, this number is reported as a proportion between 0 and 1, unless it is followed by a percent sign (as it appears in a couple of places in the text).

This is the same way in which appraisal expressions were selected when computing accuracy while learning linkage specifications in Sections 8.8 and 8.9.

$$\text{Percent accuracy} = \frac{\text{correctly extracted}}{\text{correctly extracted} + \text{incorrectly extracted}} \tag{10.4}$$

## 10.2 Attitude Group Extraction Accuracy

The accuracy of each attitude lexicon and the accuracy of the sequence tagging baseline at finding individual attitudes occurrences in the various evaluation corpora is reported in Tables 10.1 thru 10.4.

All of the attitude groups used in this comparison are taken from the results generated by the chunker before any attempt is made to link them to the other slots that appear in appraisal expressions. Deduplication is performed to ensure that when multiple attitude groups cover the same span of text (either associated with different targets in the ground truth, or having different attributes in the extraction results), those attitude groups are not counted twice. The associator and the disambiguator do not remove any attitude groups, so the results before linking are exactly the same

as they would be after linking, though distributions of attitude types and orientations can change.

In these tests, the CRF model baseline was run on the testing subset of the corpus only, under 10-fold cross validation, a second-order model with a window size of 6 tokens, and with feature selection to select the best 10,000 features $f'$.

These tables also report the accuracy of the different lexicons at determining orientation in context of each attitude group that appeared in both the ground truth and in FLAG's extraction results. The CRF model does not attempt to identify the orientation of the attitude groups it extracts, but if such a model were to be deployed, there are several ways to address this deficiency, like training separate models to identify positive and negative attitude groups or applying Turney's [170] or Esuli and Sebastiani's [46] classification techniques to the spans of text that the CRF identified as being attitude groups. In the run named **CRF baseline**, the FLAG and General Inquirer lexicons were not used as features in the CRF model. The **CRF + Lexicons** explores what happens when the CRF baseline can also take into account the presence of words in the FLAG and General Inquirer lexicons.

On the IIT sentiment corpus (Table 10.1), lexicon-based extraction using FLAG's lexicon achieved higher overall accuracy than the baselines. The CRF baseline (without the lexicon features) had higher precision, and FLAG's lexicon achieved higher recall. The SentiWordNet lexicon and Turney's lexicon (that is, the General Inquirer, since Turney's method did not perform automatic selection of the sentiment words) both achieved lower recall and lower precision than the FLAG lexicon, but both achieved higher recall than the CRF Model baseline. The CRF model achieved higher precision than any of the lexicon-based approaches, a result that was repeated on all four corpora.

Table 10.1. Accuracy of Different Methods for Finding Attitude Groups on the IIT Sentiment Corpus.

| Lexicon | Prec | Rcl | $F_1$ | Orientation |
|---|---|---|---|---|
| FLAG | 0.490 | 0.729 | 0.586 | 0.915 |
| SentiWordNet | 0.187 | 0.604 | 0.286 | 0.817 |
| Turney | 0.239 | 0.554 | 0.334 | 0.790 |
| CRF baseline | 0.710 | 0.402 | 0.513 | - |
| CRF + Lexicons | 0.693 | 0.512 | 0.589 | - |

Table 10.2. Accuracy of Different Methods for Finding Attitude Groups on the Darmstadt Corpus.

| | All sentences | | | | Opinionated sentences only | | | |
|---|---|---|---|---|---|---|---|---|
| Lexicon | Prec | Rcl | $F_1$ | Ori. | Prec | Rcl | $F_1$ | Ori. |
| FLAG | 0.226 | 0.618 | 0.331 | 0.882 | 0.568 | 0.611 | 0.589 | 0.883 |
| SentiWordNet | 0.090 | 0.552 | 0.155 | 0.737 | 0.288 | 0.544 | 0.377 | 0.735 |
| Turney | 0.120 | 0.482 | 0.192 | 0.856 | 0.360 | 0.477 | 0.410 | 0.856 |
| CRF baseline | 0.627 | 0.377 | 0.471 | - | 0.753 | 0.557 | 0.653 | - |
| CRF + Lexicons | 0.620 | 0.397 | 0.484 | - | 0.764 | 0.599 | 0.671 | - |

Table 10.3. Accuracy of Different Methods for Finding Attitude Groups on the JDPA Corpus.

| Lexicon | Prec | Rcl | $F_1$ | Orientation |
|---|---|---|---|---|
| FLAG | 0.422 | 0.405 | 0.413 | 0.885 |
| SentiWordNet | 0.216 | 0.413 | 0.283 | 0.692 |
| Turney | 0.248 | 0.357 | 0.292 | 0.852 |
| CRF baseline | 0.665 | 0.332 | 0.443 | - |
| CRF + Lexicons | 0.653 | 0.357 | 0.462 | - |

Table 10.4. Accuracy of Different Methods for Finding Attitude Groups on the MPQA Corpus.

| | Overlapping | | | | Exact Match | | |
|---|---|---|---|---|---|---|---|
| Lexicon | Prec | Rcl | $F_1$ | Ori. | Prec | Rcl | $F_1$ |
| FLAG | 0.531 | 0.485 | 0.507 | 0.738 | 0.057 | 0.058 | 0.057 |
| SentiWordNet | 0.417 | 0.535 | 0.469 | 0.679 | 0.020 | 0.035 | 0.025 |
| Turney | 0.414 | 0.510 | 0.457 | 0.681 | 0.025 | 0.041 | 0.031 |
| CRF baseline | 0.819 | 0.294 | 0.433 | - | 0.226 | 0.081 | 0.119 |
| CRF + Lexicons | 0.826 | 0.321 | 0.462 | - | 0.320 | 0.129 | 0.184 |

When the CRF baseline was augmented with features that indicate the presence of each word in the FLAG and General Inquirer lexicons, recall on the IIT corpus increased 9%, and precison only fell 2%, causing a significant increase in extraction accuracy, to the point that **CRF + Lexicons** very slightly beat the lexicon-based chunker's accuracy using the FLAG lexicon. (The Darmstadt and JDPA corpora demonstrated a similar, but less pronounced effect of decreased precision and increased recall and $F_1$ when the lexicons were added to the CRF baseline.)

The FLAG lexicon, which takes into account the effect of polarity shifters performed best at determining the orientation of each attitude group. It noticeably outperformed both Turney's method and SentiWordNet.

The FLAG lexicon achieved 54.1% accuracy at identifying the attitude type of each attitude group at the leaf level, and 75.8% accuracy at distinguishing between the 3 main attitude types: appreciation, judgment, and affect. There are no baselines to compare this performance against, since the other lexicons and other corpora did not include attitude type data. The techniques of Argamon et al. [6], Esuli et al. [49] or Taboada and Grieve [164] could potentially be applied to these lexicons to automatically determine the attitude types of either lexicon entries or extracted attitude groups in context but more research is necessary to improve these techniques.

(Taboada and Grieve's [164] SO-PMI approach has never been evaluated to determine its accuracy at classifying individual lexicon entries.)

Table 10.2 shows the results of the same experiments on the Darmstadt corpus. All of the lexicon-based approaches demonstrate low precision because unlike the Darmstadt annotators, FLAG makes no attempt to determine whether a sentence is on topic and opinionated before identifying attitude groups in the sentence. The CRF model has a similar problem, but it compensated for this by learning a higher-precision model that achieves lower recall. This strategy worked well, and the CRF model achieved the highest accuracy overall.

To account for the effect of the off-topic and non-opinionated sentences on the low precision, I restricted the test results to include only attitude groups that had been found in sentences deemed on-topic and opinionated by the Darmstadt annotators. I did not restrict the ground truth annotations, because in theory there should be no attitude groups annotated in the off-topic and non-opinionated sentences. When the extracted attitude groups are restricted this way, all of the lexicons perform even better on the Darmstadt corpus than they performed on the IIT corpus. This is likely because some opinionated words have both opinionated and non-opinionated word senses. The lexicon-based extraction techniques will spuriously extract opinionated words with non-opinionated word senses, because they have no way to determine which word sense is used. Removing the non-opinionated sentences removes more non-opinionated word senses than opinionated word senses, decreasing the number of false positives and increasing precision.

The slight drop in recall between the two experiments indicates that my assumption that there should be no attitude groups annotated in the off-topic and non-opinionated sentences was incorrect. This indicates that the Darmstadt annotators made a few errors when annotating their corpus, and they annotated opinion

expressions in a few sentences that were not marked as opinionated.

Table 10.3 shows the results of the same experiments on the JDPA corpus. In this experiment, the CRF model performed best overall (achieving the best precision and $F_1$), probably because it could learn to identify polar facts (and outright facts) from the corpus annotations. The lexicon-based methods achieved better recall.

On the MPQA corpus (Table 10.4), the results are more complicated. The FLAG lexicon performs achieves the highest $F_1$, but the SentiWordNet lexicon achieves the best recall, and the CRF baseline achieves the best precision. Looking at the performance when an exact match is required, the three lexicons all perform poorly. The CRF baseline also performs poorly, but less poorly than the lexicons.

In my observations of FLAG's results on the MPQA corpus, the long ground truth annotations encourage accidental true positives in an overlap evaluation, and there are frequently cases where the words that FLAG identifies as an attitude do not have any connection to the overall meaning of the ground truth annotation with which they overlap. At the same time, any more strict comparison is prevented from correctly identifying all matches where the annotations do have the same meaning, because the boundaries do not match. This problem affects target annotations as well. Because of this, I concluded that the MPQA corpus is not very well suited for evaluating direct opinion extraction.

## 10.3 Linkage Specification Sets

In this rest of this chapter I will be comparing different sets of linkage specifications that differ in one or two aspects of how they were generated (with and without the disambiguator), in order to demonstrate the gains that come from modeling different aspects of the appraisal grammar in FLAG's extraction process. In the interest of clarity and uniformity, I will be referring to different sets of linkage

specifications by using abbreviations that concisely explain different aspects of how they were generated. The linkage specification names are of the form:

Candidate Generator + Selection Algorithm +
Slots Included + Attitude Type Constraints used

The *candidate generator* is either **Sup** or **Unsup**. **Sup** means the linkage specifications were generated using the supervised candidate generator discussed in Section 8.5. **Unsup** means the linkage specifications were generated using the unsupervised candidate generator discussed in Section 8.6.

The *selection algorithm* is either **All**, **MC#**, **LL#**, or **Cover**. **All** means that all of the linkage specifications returned by the candidate generator were used, no matter how infrequently the appeared, and no algorithm was used to prune the set of linkage specifications. **MC#** means that the linkage specifications were selected by taking the linkage specifications that were most frequently learned from candidates returned by the candidate generator. The **All** and **MC#** linkage specifications were sorted by the topological sort algorithm in section Section 8.3. **LL#** means that the linkage specifications were selected by their LogLog score, as discussed in Section 8.8. In both of these abbreviations, the pound sign is replaced with a number indicating how many linkage specifications were selected using this method. **Cover** means that the linkage specifications were selected using the covering algorithm discussed in Section 8.9. It is worth noting that when the covering or LogLog selection algorithms are run, they are applied to a set of **All** linkage specifications, so a set of **Cover** linkage specifications can be said to be derived from a corresponding set of **All** linkage specifications.

The *slots included* are either **ES**, **ATE** or **AT**. **ES** means that the linkage specifications included all of the slots that could be generated by the candidate generator

(these are discussed in Section 8.5 and Section 8.6.) **ATE** means the linkage specifications include only attitudes, evaluators, and targets, and **AT** means the linkage specifications include only attitudes and targets. When using the unsupervised candidate generator, and when using the supervised candidate generator on the IIT blog corpus, the **ATE** and **AT** linkage specifications were obtained by using the filter in Section 8.7 to remove the extraneous slots from the appraisal expression candidates used for learning. When using the supervised candidate generator on the JDPA, Darmstadt, and MPQA corpora, the ground truth annotations didn't include any of the other slots of interest, so this filter did not need to be applied.

The *attitude type constraints used* is either **Att** or **NoAtt**. **Att** indicates that the linkage specification set has attitude type constraints on some of its linkage specifications. **NoAtt** indicates that the specification set does not have attitude type constraints, either because the ground truth annotations in the corpus don't have attitude types so the ground truth candidate generator couldn't generate linkage specifications with constraints, or because attitude type constraints were filtered out by the filter discussed in Section 8.7.

There isn't a part of the linkage specification set's name that indicates whether or not the disambiguator was used for extraction. Rather, the use of the disambiguator is clearly indicated, in the sections where it is used.

The two sets of manually constructed linkage specifications don't follow this naming scheme. The linkage specifications based on Hunston and Sinclair's [72] local grammar of evaluation (described in Section 8.1) are referred to as **Hunston and Sinclair**. The full set of manual linkage specifications described in Section 8.2, which includes the Hunston and Sinclair linkage specifications as a subset, are referred to as **All Manual LS**.

## 10.4 Does Learning Linkage Specifications Help?

The first questions that need to be addressed in evaluating FLAG's performance with different sets of linkage specifications are very basic.

- What is the baseline accuracy using linkage specifications developed from Hunston and Sinclair's [72] linguistic study on the subject?
- Are automatically-learned linkage specifications an improvement over those manually constructed linkage specifications at all?
- Is it better to generate candidate linkage specifications from ground truth annotations (Section 8.5), or from unsupervised heuristics (Section 8.6)?
- After learning a list of linkage specifications this way, is it necessary to prune the list remove less accurate linkage specifications (using one of the algorithms in Sections 8.8 and 8.9)?
- If so, which pruning algorithm is better?

The first set of results I ran that deals with these is presented in Table 10.5 for the IIT Sentiment Corpus, and Table 10.6 for the Darmstadt and JDPA corpora.

Table 10.5. Performance of Different Linkage Specification Sets on the IIT Sentiment Corpus.

| | Linkage Specifications | All Slots | Target Eval. | Target |
|---|---|---|---|---|
| 1. | Hunston and Sinclair | 0.239 | 0.267 | 0.461 |
| 2. | All Manual LS | 0.362 | 0.396 | 0.545 |
| 3. | Unsup+LL50+ES+Att | 0.367 | 0.394 | 0.521 |
| 4. | Unsup+LL100+ES+Att | **0.405** | 0.431 | **0.557** |
| 5. | Unsup+LL150+ES+Att | 0.388 | 0.408 | 0.515 |
| 6. | Unsup+Cover+ES+Att | 0.383 | 0.419 | 0.530 |
| 7. | Sup+All+ES+Att | 0.180 | 0.238 | 0.335 |
| 8. | Sup+MC50+ES+Att | 0.346 | 0.383 | 0.528 |
| 9. | Sup+LL50+ES+Att | 0.368 | 0.407 | 0.538 |
| 10. | Sup+LL100+ES+Att | 0.384 | 0.422 | 0.545 |
| 11. | Sup+LL150+ES+Att | 0.377 | 0.415 | 0.547 |
| 12. | Sup+Cover+ES+Att | **0.406** | **0.454** | **0.555** |

Table 10.6. Performance of Different Linkage Specification sets on the Darmstadt and JDPA Corpora.

| | | JDPA Corpus | | Darmstadt Corpus | |
|---|---|---|---|---|---|
| | Linkage Specifications | Target Eval. | Target | Target Eval. | Target |
| 1. | Hunston and Sinclair | 0.423 | 0.487 | 0.398 | 0.417 |
| 2. | All Manual LS | 0.419 | 0.522 | 0.460 | 0.506 |
| 3. | Unsup+LL50+ES+Att | **0.500** | **0.571** | 0.445 | 0.455 |
| 4. | Unsup+LL100+ES+Att | 0.486 | 0.555 | 0.407 | 0.416 |
| 5. | Unsup+LL150+ES+Att | 0.485 | 0.569 | 0.421 | 0.431 |
| 6. | Unsup+Cover+ES+Att | **0.502** | **0.573** | 0.476 | 0.485 |
| 7. | Sup+All+ATE+NoAtt | 0.213 | 0.273 | 0.460 | 0.492 |
| 8. | Sup+MC50+ATE+NoAtt | 0.409 | 0.476 | 0.324 | 0.363 |
| 9. | Sup+LL50+ATE+NoAtt | 0.466 | 0.535 | 0.455 | 0.464 |
| 10. | Sup+LL100+ATE+NoAtt | 0.309 | 0.400 | 0.397 | 0.408 |
| 11. | Sup+LL150+ATE+NoAtt | 0.328 | 0.413 | 0.412 | 0.421 |
| 12. | Sup+Cover+ATE+NoAtt | 0.484 | 0.558 | **0.525** | **0.536** |

These results demonstrate that the best sets of learned linkage specifications outperform the manual linkage specification (lines 1 and 2) on all of three corpora.

They're much less conclusive about whether the supervised candidate generator performs better or the unsupervised candidate generator performs better. Linkage specifications learned using the supervised candidate generator perform better on the IIT corpus and the Darmstadt corpus, but linkage specifications learned using the unsupervised candidate generator perform better on the JDPA corpus. It is unlikely that this unusual result on the JDPA corpus is caused by appraisal expressions that span multiple sentences being discarded in the learning process (a potential problem for this corpus in particular, discussed in Section 7.1), as only 10% of the candidates considered have this problem. Whether the presence of attitude types and slots not found in the JDPA corpus annotations contributed to this performance is discussed in Section 10.6.

The topological sort algorithm described in Section 8.3 is used for sorting linkage specifications by their specificity. This algorithm basically ensures that the linkage specifications meet the minimum requirement to ensure that none of the linkage specifications in a set is completely useless, shadowed by some more general linkage specification. The results in lines 3 and 4 of these two tables demonstrate good accuracy showing that using the LogLog pruning algorithm with the topological sorting algorithm is a reasonably good method for accurate extraction. The accuracy is close to that of the covering algorithm (results shown on lines 6 and 12), and both are reasonably close to the best FLAG can achieve without using the disambiguator, demonstrating that ordering the linkage specifications appropriately is a reasonably good method for selecting the correct appraisal expression candidates, even without the disambiguator.

It is worth investigating whether this is a sufficient condition to achieve good performance when extracting appraisal expressions, even without some method of selecting worthwhile linkage specifications. The assumption here is that if a particular linkage specification doesn't apply to a given attitude group, then its syntactic pattern won't be found in the sentences. To test this, FLAG was run with a set of linkage specifications learned using the supervised candidate generator, and then directly topologically sorted by specificity, without any pruning of the linkage specification set. The results in line 5 show that this assumption should not be relied on. Though it performed well on the Darmstadt corpus, it achieved the lowest accuracy of any experiment in this section when tried on the JDPA and IIT corpora. (The results in Section 10.7, however, demonstrate that the machine-learning disambiguator can obviate the need for some kind of pruning of the learned linkage specification set, and with the machine-learning disambiguator, this set of linkage specifications performed the best.)

Having established the necessity of some algorithm for pruning a learned linkage-specification set, it is now necessary to determine which is better. It turns out that there's no clear answer to this question either. While the covering algorithm performs better on the Darmstadt corpus than the Log-Log scoring function, the unsupervised candidate generator performs as well with the Log-Log scoring function as the supervised candidate generator performs with the covering algorithm on the IIT corpus, and there's a virtual tie between the two methods when using the unsupervised candidate generator on the JDPA corpus. That said, using the supervised candidate generator with the covering algorithm doesn't perform too much worse on the JDPA corpus, and the justification for its operation is less arbitrary — it doesn't need an arbitrarily chosen scoring method to score the linkage specifications — so it is generally good choice as a method for learning linkage specifications when not using the machine-learning disambiguator.

Table 10.7. Performance of Different Linkage Specification Sets on the MPQA Corpus.

| | Linkage Specifications | MPQA Target |
|---|---|---|
| 1. | Hunston and Sinclair | 0.266 |
| 2. | All Manual LS | 0.349 |
| 3. | Unsup+LL150+ES+Att | 0.346 |
| 4. | Unsup+Cover+ES+Att | 0.350 |
| 5. | Sup+All+AT+NoAtt | **0.355** |
| 6. | Sup+MC50+AT+NoAtt | 0.340 |
| 7. | Sup+Cover+AT+NoAtt | 0.338 |

On the MPQA corpus, all of the different linkage specification sets tested (aside from the ones based on Hunston and Sinclair's [72] local grammar, which are adapted mostly for adjectival attitudes) perform approximately equally well. However, it turns out that the MPQA corpus isn't really such a good corpus for evaluating FLAG. If I had required FLAG to find an exact match for the MPQA annotation in order for it to be considered correct, then the scores would have been so low, owing to the very long annotations frequently found in the corpus, that nothing could be learned from them. But in the evaluations performed here, requiring only that spans overlap in order to be considered correct, I have found that many of the true positives reported by the evaluation are essentially random — FLAG frequently picks an unimportant word from the attitude and an unimportant word from the target, and happens to get a correct answer. So while FLAG performed better on the MPQA corpus with the manual linkage specifications (line 2) than with the Sup+Cover+AT+NoAtt linkage specifications (line 7), this isn't enough to disprove the conclusion that learning linkage specifications works better than using manually constructed linkage specifications. The best performance on the MPQA corpus was achieved by the Sup+All+AT+NoAtt linkage specifications (line 5).

## 10.5 The Document Emphasizing Processes and Superordinates

While training an undergraduate annotator to create the IIT Sentiment Corpus, I noticed that he was having a hard time learning about the rarer slots in the corpus, which included processes, superordinates, and aspects. I determined that this was because these slots were too rare in the wild for him to get a good grasp on the concept. I constructed a document consisting of individual sentences automatically culled from other corpora, where each sentence was likely to either contain a superordinate or a process, and worked with him on that document to learn to annotate these rarer slots.

FLAG had problem similar to that of the undergraduate annotator. When FLAG learned linkage specifications on the development subset made up of only 20 natural blog posts (similar to the ones in the testing subset), it had a hard time identifying processes, superordinates, and aspects. I therefore created an additional version of the development subset that contained the same 20 blog posts, plus the document I developed for focused training on superordinates and processes.

Table 10.8. Comparison of Performance when the Document Focusing on Appraisal Expressions with Superordinates and Processes is Omitted.

| | | With Focused Document | | | Without Focused Doc. | | |
|---|---|---|---|---|---|---|---|
| | Linkage Specifications | All Slots | Target and Eval. | Target | All Slots | Target and Eval. | Target |
| 1. | Unsup+LL150+ES+Att | 0.388 | 0.408 | 0.515 | 0.360 | 0.383 | 0.486 |
| 2. | Unsup+Cover+ES+Att | 0.383 | 0.419 | 0.530 | 0.394 | 0.430 | 0.537 |
| 3. | Sup+All+ES+Att | 0.180 | 0.238 | 0.335 | 0.198 | 0.256 | 0.338 |
| 4. | Sup+MC50+ES+Att | 0.346 | 0.383 | 0.528 | 0.363 | 0.406 | 0.538 |
| 5. | Sup+Cover+ES+Att | 0.406 | 0.454 | 0.555 | 0.393 | 0.439 | 0.543 |

The results in Table 10.8 indicate that there's no clear advantage or disadvantage in including the document for focused training on superordinates and processes

in the data set. It improved the overall accuracy in line 5 (the best run without the disambiguator on the IIT corpus), but hurt accuracy when it was used to learn several other sets of linkage specifications (lines 2–4).

## 10.6 The Effect of Attitude Type Constraints and Rare Slots

As the presence of attitude type constraints and additional slots (aspects, processes, superordinates, and expressors) present the potential for an increase in accuracy when compared to basic linkage specifications that don't include these features, it is important to see whether these features actually improve accuracy. To do this, I generated linkage specifications that exclude attitude type constraints and linkage specifications that include only attitudes, targets, and evaluators, using the filters described in Section 8.7, and compared their performance on my corpora.

Table 10.9. The Effect of Attitude Type Constraints and Rare Slots in Linkage Specifications on the IIT Sentiment Corpus.

| | Linkage Specifications | All Slots | Target and Eval. | Target |
|---|---|---|---|---|
| 1. | Sup+Cover+ATE+NoAtt | 0.386 | 0.420 | 0.529 |
| 2. | Sup+Cover+ES+NoAtt | 0.370 | 0.425 | 0.538 |
| 3. | Sup+Cover+ATE+Att | **0.414** | 0.448 | **0.559** |
| 4. | Sup+Cover+ES+Att | **0.406** | **0.454** | **0.555** |
| 5. | Unsup+Cover+ATE+NoAtt | 0.382 | 0.413 | 0.531 |
| 6. | Unsup+Cover+ES+NoAtt | 0.384 | 0.418 | 0.532 |
| 7. | Unsup+Cover+ATE+Att | 0.382 | 0.412 | 0.524 |
| 8. | Unsup+Cover+ES+Att | 0.383 | 0.419 | 0.530 |

Table 10.10. The Effect of Attitude Type Constraints and Rare Slots in Linkage Specifications on the Darmstadt, JDPA, and MPQA Corpora.

| | Linkage Specifications | JDPA Corpus | | Darmstadt Corpus | |
|---|---|---|---|---|---|
| | | Target and Eval. | Target | Target and Eval. | Target |
| 1. | Sup+Cover+ATE+NoAtt | 0.484 | 0.558 | **0.525** | **0.536** |
| 2. | Unsup+Cover+ATE+NoAtt | 0.495 | 0.565 | 0.524 | 0.535 |
| 3. | Unsup+Cover+ES+NoAtt | 0.498 | 0.569 | 0.482 | 0.491 |
| 4. | Unsup+Cover+ATE+Att | 0.496 | 0.565 | 0.524 | 0.535 |
| 5. | Unsup+Cover+ES+Att | **0.502** | **0.573** | 0.476 | 0.485 |

It seems clear from the results in Tables 10.9 and 10.10 that the inclusion of the extra slots in the linkage specifications neither hurts nor helps FLAG's accuracy in identifying appraisal expressions. On the IIT Corpus, they hurt extraction slightly with supervised linkage specifications, and cause no significant gain or loss when used in unsupervised linkage specifications. They hurt performance noticeably on the Darmstadt corpus, and cause no significant gain or loss on the JDPA corpus.

The inclusion of attitude type constraints on particular linkage specifications also does not appear to hurt or help extraction accuracy. On the IIT Corpus, they help extraction with supervised linkage specifications, and cause no significant gain or loss when used in unsupervised linkage specifications. They cause no significant gain or loss on either the Darmstadt corpus, or the JDPA corpus.

## 10.7 Applying the Disambiguator

To test the machine learning disambiguator (Chapter 9), FLAG first learned linkage specifications from the development subset of each corpus using several different varieties of linkage specifications. 10-fold crossvalidation of the disambiguator was then performed on the test subset of each corpus. The support vector machine

trade-off parameter $C$ was manually fixed at 10, though in a real-life deployment, another round of crossvalidation should be used to select the best value of $C$ each time the model is trained.

Table 10.11. Performance with the Disambiguator on the IIT Sentiment Corpus.

| | Linkage Specifications | Highest Priority | | | Disambiguator | | |
|---|---|---|---|---|---|---|---|
| | | All Slots | Target and Eval. | Target | All Slots | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.239 | 0.267 | 0.461 | 0.250 | 0.279 | 0.476 |
| 2. | All Manual LS | 0.362 | 0.396 | 0.545 | 0.400 | 0.438 | **0.573** |
| 3. | Unsup+LL150+ES+Att | 0.388 | 0.408 | 0.515 | 0.430 | 0.461 | **0.572** |
| 4. | Unsup+Cover+ES+Att | 0.383 | 0.419 | 0.530 | 0.391 | 0.429 | 0.538 |
| 5. | Sup+All+ES+Att | 0.180 | 0.238 | 0.335 | **0.437** | **0.478** | **0.571** |
| 6. | Sup+Cover+ES+Att | 0.406 | 0.454 | 0.555 | 0.433 | 0.473 | 0.580 |

Table 10.12. Performance with the Disambiguator on the Darmstadt Corpus.

| | Linkage Specifications | Highest Priority | | Disambiguator | |
|---|---|---|---|---|---|
| | | Target and Eval. | Target | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.398 | 0.417 | 0.427 | 0.435 |
| 2. | All Manual LS | 0.460 | 0.506 | **0.523** | **0.537** |
| 3. | Unsup+LL150+ES+Att | 0.421 | 0.431 | 0.520 | 0.530 |
| 4. | Unsup+Cover+ES+Att | 0.476 | 0.485 | 0.497 | 0.507 |
| 5. | Sup+All+ATE+NoAtt | 0.460 | 0.492 | **0.527** | **0.538** |
| 6. | Sup+Cover+ATE+NoAtt | **0.525** | **0.536** | **0.523** | **0.534** |

Table 10.13. Performance with the Disambiguator on the JDPA Corpora.

| | Linkage Specifications | Highest Priority | | Disambiguator | |
|---|---|---|---|---|---|
| | | Target and Eval. | Target | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.423 | 0.487 | 0.442 | 0.498 |
| 2. | All Manual LS | 0.419 | 0.522 | 0.494 | 0.569 |
| 3. | Unsup+LL150+ES+Att | 0.485 | 0.569 | 0.539 | 0.613 |
| 4. | Unsup+Cover+ES+Att | 0.502 | 0.573 | 0.529 | 0.600 |
| 5. | Sup+All+ATE+NoAtt | 0.213 | 0.273 | **0.557** | **0.631** |
| 6. | Sup+Cover+ATE+NoAtt | 0.484 | 0.558 | 0.541 | 0.617 |

In all three corpora, the machine learning disambiguator caused a noticeable increase in accuracy compared to techniques that simply selected the most specific linkage specification for each attitude group. Additionally, the best performing set of linkage specifications was always the Sup+All variant, though some other variants often achieved similar performance on particular corpora.

The Sup+All linkage specifications (line 5) performed the worst without the disambiguator, when linkage specifications had to be selected by their priority. FLAG would very often pick an overly-specific linkage specification that had been seen only a couple of times in the training data. With the disambiguator, FLAG can use much more information to select the best linkage specification, and the disambiguator can learn conditions under which rare linkage specifications should and should not be used. With the disambiguator, Sup+All linkage specifications became the best performers.

## 10.8 The Disambiguator Feature Set

To explore the contribution of the attitude types in the disambiguator feature set, I ran an experiment in which attitude types were excluded from the feature set. To study the effect of the associator, and not the linkage specifications, I ran this

on just the automatically learned linkage specifications that did not include attitude type constraints. (The Hunston and Sinclair and All Manual LS sets still do include attitude type constraints.)

Table 10.14. Performance with the Disambiguator on the IIT Sentiment Corpus.

| | Linkage Specifications | Without Attitude Types | | | With Attitude Types | | |
|---|---|---|---|---|---|---|---|
| | | All Slots | Target and Eval. | Target | All Slots | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.249 | 0.279 | 0.478 | 0.251 | 0.279 | 0.477 |
| 2. | All Manual LS | 0.391 | 0.428 | 0.560 | 0.401 | 0.437 | 0.572 |
| 3. | Unsup+LL150+ES+NoAtt | 0.401 | 0.430 | 0.522 | 0.429 | 0.464 | 0.572 |
| 4. | Unsup+Cover+ES+NoAtt | 0.380 | 0.411 | 0.523 | 0.396 | 0.435 | 0.550 |
| 5. | Sup+All+ES+NoAtt | 0.408 | 0.429 | 0.518 | **0.446** | **0.484** | **0.576** |
| 6. | Sup+Cover+ES+NoAtt | 0.382 | 0.427 | 0.535 | 0.389 | 0.435 | 0.539 |

Table 10.15. Performance with the Disambiguator on the Darmstadt Corpus.

| | Linkage Specifications | Without Att. Types | | With Att. Types | |
|---|---|---|---|---|---|
| | | Target and Eval. | Target | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.423 | 0.431 | 0.429 | 0.437 |
| 2. | All Manual LS | 0.527 | 0.536 | 0.524 | 0.538 |
| 3. | Unsup+LL150+ES+NoAtt | 0.518 | 0.528 | 0.524 | 0.535 |
| 4. | Unsup+Cover+ES+NoAtt | 0.507 | 0.518 | 0.505 | 0.516 |
| 5. | Sup+All+ATE+NoAtt | **0.533** | **0.543** | 0.524 | 0.534 |
| 6. | Sup+Cover+ATE+NoAtt | 0.523 | 0.534 | 0.525 | 0.537 |

Table 10.16. Performance with the Disambiguator on the JDPA Corpus.

| | Linkage Specifications | Without Att. Types | | With Att. Types | |
|---|---|---|---|---|---|
| | | Target and Eval. | Target | Target and Eval. | Target |
| 1. | Hunston and Sinclair | 0.439 | 0.495 | 0.441 | 0.498 |
| 2. | All Manual LS | 0.490 | 0.558 | 0.493 | 0.566 |
| 3. | Unsup+LL150+ES+NoAtt | 0.551 | 0.626 | 0.552 | 0.626 |
| 4. | Unsup+Cover+ES+NoAtt | 0.524 | 0.595 | 0.520 | 0.591 |
| 5. | Sup+All+ATE+NoAtt | 0.556 | 0.630 | **0.557** | **0.631** |
| 6. | Sup+Cover+ATE+NoAtt | 0.542 | 0.617 | 0.541 | 0.617 |

The end results show a small improvement on the IIT corpus when attitude types are modeled in the disambiguator's feature set, but no improvement on the JDPA or Darmstadt corpora. This may be because the IIT attitude types where considered when IIT sentiment corpus was being annotated, leading to a cleaner separation between the patterns for the different attitude types.

This may also be because of different distributions of attitude types between the corpora, shown in Table 10.17. The first part of this table shows the incidence of the three main attitude types in attitude groups found by FLAG's chunker, regardless of whether the attitude group found was actually correct. The second part of this table shows the incidence of the three main attitude types in attitude groups found by FLAG's chunker, when the attitude group correctly identified a span of text that denoted an attiutude, regardless of whether the identified attitude type was correct. FLAG's identified attitude type is not checked for correctness in this table because the JDPA and Darmstadt corpora don't have attitude type information in their ground truth annotations, and because FLAG's identified attitude type is the one used by the disambiguator to select the correct linkage specification.

FLAG's chunker found that the IIT corpus contains an almost 50/50 split

Table 10.17. Incidence of Extracted Attitude Types in the IIT, JDPA, and Darmstadt Corpora.

| All extracted attitude groups | | | |
|---|---|---|---|
| | Affect | Appreciation | Judgment |
| IIT | 1353 (42.2%) | 995 (31.0%) | 855 (26.7%) |
| JDPA | 4974 (22.4%) | 10813 (48.7%) | 6429 (28.9%) |
| Darmstadt | 2974 (28.5%) | 4738 (45.3%) | 2743 (26.2%) |
| Correct attitude groups | | | |
| | Affect | Appreciation | Judgment |
| IIT | 766 (47.1%) | 557 (34.2%) | 305 (18.7%) |
| JDPA | 1871 (19.6%) | 5349 (56.2%) | 2302 (24.2%) |
| Darmstadt | 335 (13.8%) | 1520 (62.7%) | 570 (23.5%) |

of affect versus appreciation and judgment (the split that Bednarek [21] found to be most important when determining which attitude types go with which syntactic patterns). On the JDPA and Darmstadt corpora, there is much less affect (and extracted attitudes that convey affect are more likely to be in error), making this primary attitude type distinction less helpful.

One particularly notable result on the IIT corpus is that FLAG's best performing configuration is the version that uses Sup+All+ES+NoAtt (shown in Table 10.14), slightly edging out the Sup+All+ES+Att variation (which included attitude types in the linkage specifications, as well as the disambiguator's feature set) recorded in Table 10.11. This suggests that pushing more decisions off to the disambiguator gives better accuracy in general.

## 10.9 End-to-end extraction results

This set of results takes into account both the accuracy of FLAG's attitude chunker at identifying attitude groups, and FLAG's associator's accuracy at finding all of the other slots involved.

Table 10.18. End-to-end Extraction Results on the IIT Sentiment Corpus

| | | Target and Evaluator | | | All Slots | | |
|---|---|---|---|---|---|---|---|
| | Linkage Specifications | P | R | $F_1$ | P | R | $F_1$ |
| | | Without the Disambiguator | | | | | |
| 1. | Hunston and Sinclair | 0.131 | 0.194 | 0.156 | 0.117 | 0.175 | 0.140 |
| 2. | All Manual LS | 0.194 | 0.293 | 0.233 | 0.177 | 0.269 | 0.214 |
| 3. | Unsup+LL150+ES+Att | 0.200 | 0.303 | 0.241 | 0.190 | 0.289 | 0.229 |
| 4. | Unsup+Cover+ES+Att | 0.205 | 0.305 | 0.245 | 0.187 | 0.279 | 0.224 |
| 5. | Sup+All+ES+Att | 0.118 | 0.184 | 0.143 | 0.089 | 0.140 | 0.108 |
| 6. | Sup+MC50+ES+Att | 0.187 | 0.279 | 0.224 | 0.169 | 0.253 | 0.202 |
| 7. | Sup+Cover+ES+Att | 0.224 | 0.349 | 0.273 | 0.201 | 0.313 | 0.245 |
| | | With the Disambiguator | | | | | |
| 8. | Hunston and Sinclair | 0.136 | 0.202 | 0.163 | 0.122 | 0.181 | 0.146 |
| 9. | All Manual LS | 0.215 | 0.319 | 0.257 | 0.196 | 0.292 | 0.234 |
| 10. | Unsup+LL150+ES+Att | 0.226 | 0.338 | 0.271 | 0.211 | 0.315 | 0.253 |
| 11. | Unsup+Cover+ES+Att | 0.210 | 0.311 | 0.251 | 0.192 | 0.285 | 0.229 |
| 12. | Sup+All+ES+Att | 0.234 | 0.348 | 0.280 | 0.214 | 0.319 | 0.256 |
| 13. | Sup+Cover+ES+Att | 0.232 | 0.344 | 0.277 | 0.212 | 0.315 | 0.253 |
| 14. | Sup+All+ES+NoAtt | **0.237** | **0.352** | **0.284** | **0.218** | **0.325** | **0.261** |

Table 10.19. End-to-end Extraction Results on the Darmstadt and JDPA Corpora

| | | JDPA Corpus | | | Darmstadt Corpus | | |
|---|---|---|---|---|---|---|---|
| | Linkage Specifications | P | R | $F_1$ | P | R | $F_1$ |
| | | Without the Disambiguator | | | | | |
| 1. | Hunston and Sinclair | 0.179 | 0.159 | 0.169 | 0.091 | 0.251 | 0.133 |
| 2. | All Manual LS | 0.177 | 0.161 | 0.169 | 0.105 | 0.294 | 0.154 |
| 3. | Unsup+LL150+ES+Att | 0.204 | 0.188 | 0.196 | 0.096 | 0.272 | 0.142 |
| 4. | Unsup+Cover+ES+Att | 0.211 | 0.190 | 0.200 | 0.108 | 0.297 | 0.158 |
| 5. | Sup+All+ATE+NoAtt | 0.089 | 0.083 | 0.086 | 0.104 | 0.290 | 0.153 |
| 6. | Sup+MC50+ATE+NoAtt | 0.172 | 0.158 | 0.165 | 0.073 | 0.205 | 0.108 |
| 7. | Sup+Cover+ATE+NoAtt | 0.203 | 0.183 | 0.193 | 0.118 | **0.328** | **0.174** |
| | | With the Disambiguator | | | | | |
| 8. | Hunston and Sinclair | 0.186 | 0.165 | 0.175 | 0.096 | 0.263 | 0.141 |
| 9. | All Manual LS | 0.208 | 0.184 | 0.196 | 0.118 | 0.324 | 0.173 |
| 10. | Unsup+LL150+ES+Att | 0.227 | 0.202 | 0.214 | 0.117 | 0.321 | 0.172 |
| 11. | Unsup+Cover+ES+Att | 0.223 | 0.197 | 0.209 | 0.112 | 0.308 | 0.165 |
| 12. | Sup+All+ATE+NoAtt | **0.235** | **0.208** | **0.221** | **0.119** | 0.325 | **0.174** |
| 13. | Sup+Cover+ATE+NoAtt | 0.228 | 0.202 | 0.214 | 0.118 | 0.324 | 0.173 |

The overall best performance on the IIT sentiment corpus is 0.261 $F_1$ at finding full appraisal expressions, and 0.284 $F_1$ when only the attitude, target, and evaluator need to be correct, achieved when the Sup+All+ES+NoAtt linkage specifications are used with the disambiguator (line 14). The overall best performance on the JDPA corpus is 0.221 $F_1$. The overall best performance on the Darmstadt corpus is 0.174 $F_1$. Both of these corpora achieved their best performance when Sup+All+ATE+NoAtt linkage specifications were used with the disambiguator (line 12). The performance on the Darmstadt is lower than the other corpora, because FLAG was allowed to find attitudes in sentences that Darmstadt annotators had marked as off-topic or not-opinionated, as explained in Section 10.2.

These results do indicate a low overall accuracy at the task of appraisal expres-

sion extraction, and more research is necessary to improve accuracy to the point where an application working with automatically extracted appraisal expressions could reasonably expect that those appraisal expressions are correct. Nonetheless, this accuracy is an achievement an appraisal expression extraction system subjected to this kind end-to-end evaluation.

This kind of end-to-end evaluation that I have performed to evaluate FLAG was reasonably expected to be more difficult than the other evaluations that have been performed in the literature, because of the emphasis it places on finding each appraisal expression correctly. Review classification and sentence classification can tolerate some percentage of incorrect appraisal expressions, which may be masked from the final evaluation score by the process of summarizing the opinions into an overall sentence or document classification before computing the overall accuracy. The kind of end-to-end extraction I perform cannot mask the incorrect appraisal expressions. Kessler and Nicolov's [87] provides correct ground truth annotations as a starting point for their algorithm to operate on, and measures only the accuracy at connecting these annotations correctly. An algorithm that performs this kind of end-to-end extraction must discover the same information for itself. Thus, it is reasonable to expect lower accuracy numbers for an end-to-end evaluation than for the other kinds of evaluations that can be found in the literature.

The NTCIR multilingual opinion extraction task [91, 146, 147], subtasks to identify opinion targets and opinion holders are examples of tasks comparable to the end-to-end evaluation I have performed here. An almost directly comparable measure is FLAG's ability to identify targets and evaluators regardless of whether the rest of the appraisal expression is correct, and results for such an evaluation of FLAG's performance (using the disambiguator and Sup+All+ES+NoAtt linkage specifications on the IIT Corpus) is shown in Table 10.20, along with the lenient

Table 10.20. FLAG's results at finding evaluators and targets compared to similar NTCIR subtasks.

| System | Evaluation | P | R | $F_1$ |
|---|---|---|---|---|
| | | | Targets | |
| ICU | NTCIR-7 | 0.106 | 0.176 | 0.132 |
| KAIST | NTCIR-8 | 0.231 | 0.346 | 0.277 |
| **FLAG** | **IIT Corpus** | **0.352** | **0.511** | **0.417** |
| | | | Evaluators | |
| IIT | NTCIR-6 | 0.198 | 0.409 | 0.266 |
| TUT | NTCIR-6 | 0.117 | 0.218 | 0.153 |
| Cornell | NTCIR-6 | 0.163 | 0.346 | 0.222 |
| NII | NTCIR-6 | 0.066 | 0.166 | 0.094 |
| GATE | NTCIR-6 | 0.121 | 0.349 | 0.180 |
| ICU-IR | NTCIR-6 | 0.303 | 0.404 | 0.346 |
| KLE | NTCIR-7 | 0.400 | 0.508 | 0.447 |
| TUT | NTCIR-7 | 0.392 | 0.283 | 0.329 |
| KLELAB | NTCIR-8 | 0.434 | 0.278 | 0.339 |
| **FLAG** | **IIT Corpus** | **0.433** | **0.494** | **0.461** |

evaluation results[10] of all of the NTCIR participants to attempt these subtasks in English. The best result on the NTCIR opinion holders subtask was 0.45 $F_1$ and the best result on the opinion target subtask was 0.27 $F_1$. One should note that the NTCIR task used a different corpus than we do here, so these results are only a ballpark figure for how hard we might expect this task to be.

## 10.10 Learning Curve

To understand FLAG's performance when trained on corpora of different sizes, and perhaps find an optimal size for the training set, I generated a learning curve

[10] NTCIR's lenient evaluation results required 2 of the 3 human annotators to agree that a particular phrase was an opinion holder or opinion target to be included in the ground truth. Their strict evaluation required all 3 human annotators to agree. Participants performed much worse on the strict evaluation than on the lenient evaluation, achieving 0.05 to 0.10 $F_1$, but these lowered results reflect on the low interrater agreement on the NTCIR corpora rather than on the quality of the systems to attempt the task.

for FLAG on each of the testing corpora. To do this, I took the documents in the test subset of each corpus, sorted them randomly, and then created document subsets from the first $n, 2n, 3n, \ldots$ documents in the list. FLAG learned linkage specifications (Sup+Cover+ES+Att for the IIT corpus, and Sup+Cover+ATE+NoAtt for the Darmstadt corpus), for each of these subsets. I then tested FLAG against the development subset of each corpus using all of the linkage specifications, and computed the accuracy. I repeated this for 50 different orderings of the documents on each corpus.

The learning curves for each of these corpora are shown in Figures 10.1 and 10.2. In each plot, the box plot shows the five quartiles for the performance of the different document orderings. The $x$-coordinate of each box-plot shows the number of documents used for that box plot. The whisker plot offset slightly to the right of each box plot shows the mean $\pm$ 1 standard deviation.

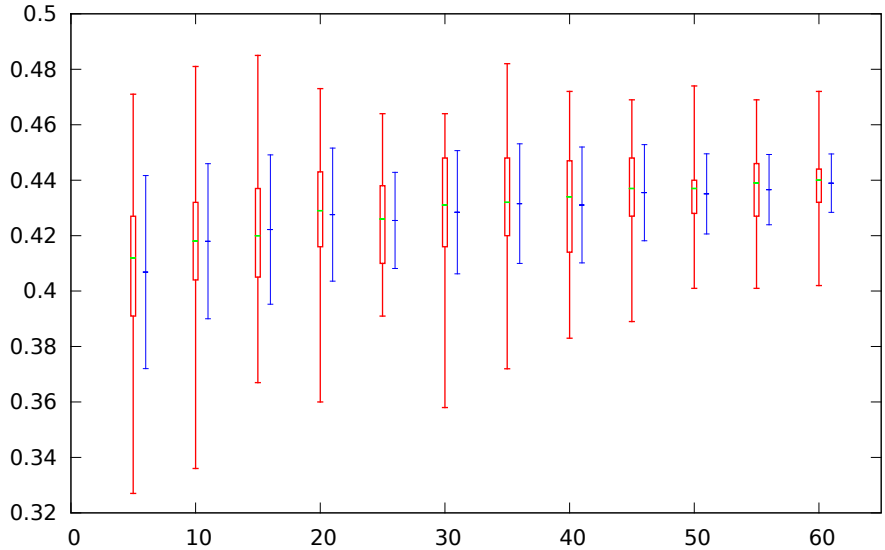



Figure 10.1. Learning curve on the IIT sentiment corpus. Accuracy at finding all slots in an appraisal expression.

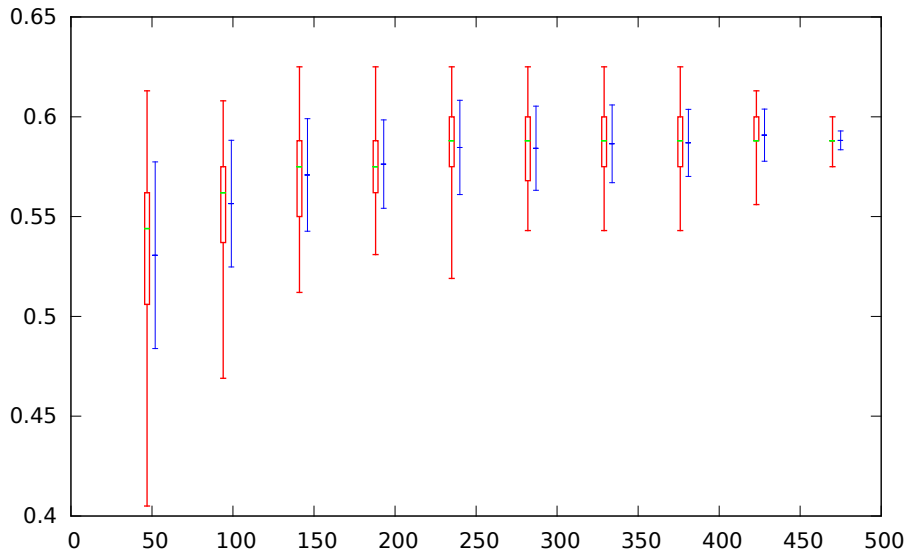



Figure 10.2. Learning curve on the Darmstadt corpus. Accuracy at finding evaluators and targets.

The mean accuracy on the IIT sentiment corpus shows an upward trend from 0.406 to 0.438 as the learning curve ranges from 5 documents to 60 documents in intervals of 5 documents. Although the range of accuracies by the different runs decreases considerably as the number of documents, it is likely that a lot of this decrease is due to increasing overlap between training sets, since the IIT corpus's test subset only contains 64 documents. The Darmstadt corpus's learning curve shows a much more pronounced increase in accuracy over the first 150 documents, and the mean accuracy stops increasing (at 0.585) once the test set consists of 235 documents. The range of accuracies achieved by the different runs also stops decreasing once the test set consists of 235 documents, settling down at a point where one can usually expect accuracy greater than 0.55 once the linkage specifications have been trained on 235 documents.

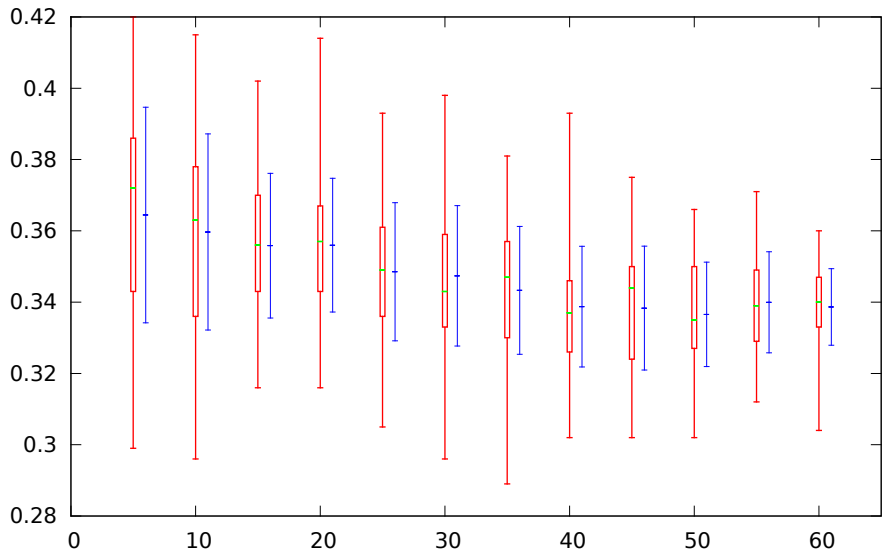



Figure 10.3. Learning curve on the IIT sentiment corpus with the disambiguator. Accuracy at finding all slots in an appraisal expression.

Figure 10.3 shows a learning curve for the disambiguator on the IIT corpus. Sets of Sup+All+ES+Att linkage specifications (the same ones that were learned for the other learning curve) were learned on corpora of various sizes, as for the other learning curves, and then a disambiguation model was trained on the same corpus subset. The trained model and the the linkage specifications were then tested on the development subset of the corpus.

Unlinke the learning curves without the disambiguator, this learning curve shows much less of a trend, and the trend it does show points slightly downward (the mean accuracy decreases from 0.36 to 0.34). It's difficult to say why there's no good trend here. It's possible that there is simply not enough training data to observe a significant upward trend, however it's more likely that the increasingly-large sets of linkage specifications make it more difficult to train an accurate model. The Sup+All+ES+Att linakge specifications contain about 1200 linkage specifications when trained on 60 documents, but applying the covering algorithm to prune this set

shrank the set of linakge specifications to approximately 200 for the other learning curves in this section. It is therefore possible that in order to achieve good accuracy when training on lots of documents, the linkage specification set needs to be pruned back to prevent too many linkage specifications from decreasing the accuracy of the disambiguator.

### 10.11 The UIC Review Corpus

As explained in Section 5.2, the UIC review corpus does not have attitude annotations. It only has product feature annotations (on a per-sentence level) with a notation indicating whether the product feature was evaluated positively or negatively in context. As a result of this, the experiment performed on the UIC review corpus is somewhat different from the experiment performed on the other corpora. FLAG was evaluated for its ability to find individual product feature mentions in the UIC review corpus, and its ability to determine whether they are evaluated positively or negatively in context. (This is different from Hu and Liu's [70] evaluation which evaluated their system's ability to identify distinct product feature names.)

In this evaluation, FLAG assumes that each appraisal target is a product feature, and compiles a list of unique appraisal targets (by textual position) to compare against the ground truth. Since the ground truth annotations do not indicate the textual position of the product features, except to indicate which sentence they appear in, I compared the appraisal targets found in each sentence against the ground truth annotations of the same sentence, and considered a target to be correct if any of the product features in the sentence was a substring of the extracted appraisal target. I computed precision and recall at finding individual product feature mentions this way.

To determine the orientation in context of each appraisal target, I used the

majority vote of the different appraisal expressions that included that target, so if a given target appeared in 3 appraisal expressions, 2 positive and 1 negative, then the target was considered to be positive in context. This is an example of the kind of "boiling-down" the extraction results to simpler annotations that I hope to eliminate from appraisal evaluation, but the nature of the UIC corpus annotations (along with its popularity) gives me no choice but to use it in this manner.

Since there are no attitude annotations in the UIC corpus, all of the automatically-learned linkage specifications used were learned on the development subset of the IIT sentiment corpus. The disambiguator was not used for these experiments either.

Table 10.21. Accuracy at finding distinct product feature mentions in the UIC review corpus

| | Linkage Specifications | P | R | $F_1$ | Ori |
|---|---|---|---|---|---|
| 1. | Hunston and Sinclair | 0.216 | 0.214 | 0.215 | 0.86 |
| 2. | All Manual LS | 0.206 | 0.245 | 0.224 | 0.86 |
| 3. | Unsup+LL150+ES+Att | 0.180 | 0.234 | 0.204 | 0.85 |
| 4. | Unsup+Cover+ES+Att | 0.181 | 0.245 | 0.208 | 0.86 |
| 5. | Sup+All+ES+Att | 0.109 | 0.161 | 0.130 | 0.84 |
| 6. | Sup+MC50+ES+Att | 0.168 | 0.220 | 0.191 | 0.85 |
| 7. | Sup+Cover+ES+Att | 0.187 | 0.237 | 0.209 | 0.86 |

FLAG's performance on the UIC review corpus is shown in Table 10.21.

The best performing run on the UIC review corpus is the run using all manually constructed linkage specifications, which tied for the highest recall, and achieved the second highest precision (behind a run that used only the Hunston and Sinclair linkage specifications). All of the automatically-learned linkage specifications sets have noticeably worse precision, with varying recall. This appears to demonstrate the automatically-learned linkage specifications capture patterns specific to the corpus they're trained on. The IIT sentiment corpus may actually be a worse match for the

UIC corpus than the Darmstadt or JDPA corpora are, because those two corpora are focused finding product features in product reviews.

Based on this corpus-dependence, and based on the fact that the UIC review corpus doesn't include attitude annotations, I would not consider this a serious challenge to my conclusion (discussed in Section 10.4) that learning linkage specifications improves FLAG's performance in general.

# CHAPTER 11
# CONCLUSION

## 11.1 Appraisal Expression Extraction

The field of sentiment analysis has turned to structured sentiment extraction in recent years as a way of enabling new applications for sentiment analysis that deal with opinions and their targets. The goal of this disseration has been to redefine the problem of structured sentiment analysis, to recognize and eliminate the assumptions that have been made in previous research, and to analyze opinions in a fine-grained way that will allow more progress to be made in the field.

The first problem that the this dissertation addresses is a problem with how structured sentiment analysis technqiues have been evaluated. There have been a number of ways to evaluate the accuracy of different techniques at performing this task. Much of the past work in structured sentiment extraction has been evaluated through applications that summarize the output of a sentiment extraction technique, or through other evaluation techniques that can mask a lot of errors without significantly impacting the bottom-line score achieved by the sentiment extraction system. In order to get a true picture of how accurate a sentiment extraction system is, however, it is important to evaluate how well it performs at finding individual mentions of opinions in a corpus. The resources for performing this kind of evaluation have not been around for very long, and those that are now available have been saddled with an annotation scheme that is not expressive enough to capture the full structure of evaluative language. This lack of expressiveness has caused problems with annotation consistency in these corpora.

Based on linguistic research into the structure of evaluative language, I have constructed a definition for the task of appraisal expression extraction that more

clearly defines the boundaries of the task, and which provides a vocabulary to discuss the relative accuracy of existing sentiment analysis resources. The key aspects of this definition are the attitude type hierarchy, which makes it clear what kinds of opinions fit into the rubric of appraisal expression extraction, the focus on the approval/disapproval dimension of opinion, and the 10 different slots that unambiguously capture the structure of appraisal expressions.

The IIT sentiment corpus, annotated according to this definition of appraisal expression extraction, demonstrates the proper application of this definition, and provides a resource against which to evaluate appraisal expression extraction techniques.

## 11.2 Sentiment Extraction in Non-Review Domains

Most existing academic work in structured sentiment analysis has focused on mining opinion/product-feature pairs from product reviews found on review sites like Epinions.com. This exclusive focus on product reviews has lead to academic sentiment analysis systems relying on several assumptions that cannot be relied upon in other domains. Among these is the assumption that the same product features recur frequently in a corpus, so that sentiment analysis systems can look for frequently occurring phrases to use as targets for the sentiments found in reviews. Another assumption in the product review domain is that each document concerns only a single topic, the product that review is about. Product reviews also bring with them a particular distribution of attitude types that is heavier on appreciation and lighter on affect than in other genres of text.

As sentiment analysis consumers want to mine a wider variety of texts to find opinions in them, these assumptions are no longer justified. Financial blog posts that discuss stocks often discuss multiple stocks in a single post [131], and it is very difficult to find commonalities between posts in arbitrary personal blog posts.

In the IIT sentiment corpus, consists of a collection of personal blog posts, annotated to identify the appraisal expressions that appear in the posts. The documents in the corpus present new challenges for those seeking to find opinion targets, because each post discusses a different topic and the posts don't share the same targets from document to document. The corpus presents a different distribution of attitude types than product reviews, which also presents a new challenge to sentiment extraction systems.

## 11.3 FLAG's Operation

FLAG, an appraisal expression extraction system, operates by a 3 step process:

1. Detect which regions of text potentially contain appraisal expressions by identifying attitude groups using a lexicon-based shallow parser.
2. Apply linkage specifications, patterns in a sentence's dependency parse tree identifying the other parts of an appraisal expression, to find potential appraisal expressions containing each attitude group.
3. Select the best appraisal expression for each attitude group using a discriminative reranker.

This 3 step process is reasonably effective at finding appraisal expressions, achieving 0.261 $F_1$ on the IIT sentiment corpus. Although it's clear that any application working with extracted appraisal expressions at this point will need to wade through a lot more errors than correct appraisal expressions, this performance is comparable to the techniques that have been attempted on the most similar sentiment extraction evaluation that's been performed to date: the NTCIR Multilingual Opinion Analysis Tasks' subtasks in identifying opinion holders and opinion targets [91, 146, 147].

The linkage specifications that FLAG uses to extract full appraisal expressions can be manually constructed, or they can be automatically learned from ground truth data. When they're manually constructed, the logical source to use to construct these linkage specifications is Hunston and Sinclair's [72] local grammar of evaluation, on which the task of appraisal expression extraction is based. However, given that there are many more patterns for appraisal expressions that can appear in an annotated corpus, automatically learning linkage specifications performs better than using manually-constructed linkage specifications.

Similarly, though FLAG can be run in a mode where linkage specifications are sorted by how specific their structure is, and where only this ordering of linkage specifications is used to select the best appraisal expression candidates, it is better to use a discriminative reranker, so that FLAG's overall operation is governed by the principle of least commitment.

The definition of appraisal expressions introduced in this dissertation introduces many new slots not seen before in structured sentiment analysis literature. One advantage in extracting these new slots is that sentiment analysis applications can take advantage of the information contained in them to better understand the evaluation being conveyed and the target being evaluated. Another potential advantage was that these slots could help FLAG to more accurately extract appraisal expression, on the theory that these slots were present in the structure appraisal expression of annotated corpora, even if they weren't explicitly recognized by the corpora's annotation scheme. This second advantage did not turn out to be the case — extracting the aspect, process, superordinate, and expressor didn't consistently increase accuracy at extracting targets, evaluators, and attitudes on any of the test corpora.

The definition of appraisal expressions also introduces (to a computational setting) the concept of dividing up evaluations into the three main attitude types of affect, appreciation, and judgment. While these attitude types may be useful for applications (as Whitelaw et al. [173] showed), they should also be useful for selecting the correct linkage specification to use to extract an appraisal expression (as Bednarek [21] discussed). The attitude type helped on the IIT corpus, which was annotated with attitude types in mind, but not on the JDPA or Darmstadt corpora. (The higher proportion of affect found in the IIT corpus may also have helped.) On the IIT corpus, attitude types improve performance when they are used as hard constraints on applying linkage specifications, but they improve performance even more when FLAG adheres to the principle of least commitment, lets the machine-learning disambiguator use them as a feature, and doesn't use them as a hard constraint on individual linkage specifications.

## 11.4 FLAG's Best Configuration

Appraisal expression extraction is a difficult task. FLAG achieves 0.586 $F_1$ at finding attitude groups, which is comparable to a CRF baseline, and better than other existing automatically constructed lexicons. FLAG achieves 44.6% accuracy at identifying the correct linkage structure for each attitude group (57.6% for applications that only care about attitudes and targets). Since this works out to an overall accuracy of 0.261 $F_1$, there are still a lot of errors that need to be resolved before applications can assume that all of the appraisal expressions found are correct. This is due to the nature of information extraction from text, and due to the fact that the IIT sentiment corpus eliminated some assumptions specific to the domain of product reviews that simplified the task of sentiment analysis. Given these changes in the goals of sentiment analysis, it could have reasonably been expected that FLAG's results under this evaluation would appear less accurate than the kinds of evaluations

that others have been concerned with in the past.

It appears from learning curves generated by learning linkage specifications from different numbers of documents that the best overall performance for applying this technique can be achieved by annotating a corpus of about 200 to 250 documents, though since this is roughly half the size of the corpus these learning curves were generated from, it is nevertheless possible that there is not yet enough data to really know how many documents are necessary to achieve the best performance.

## 11.5 Directions for Future Research

Appraisal expressions are a useful and consistent way to understand the inscribed evaluations in text. The work that I have done in defining them, developing the IIT sentiment corpus, and developing FLAG presents a number of new directions for future research.

First, there is a lot of research that can be done to improve FLAG's performance, while staying within FLAG's paradigm of finding attitudes, fining candidate appraisal expressions, and selecting the best ones. Research into FLAG's attitude extraction step should focus on methods for improving both recall and precision. To improve the precision of the lexicon-based attitude chunker, a technique should be developed for determining which attitude groups really convey attitude in context, either by integrating this into the existing reranking scheme used for the disambiguator, or by creating a separate classifier to determine whether words are positive or negative. If the CRF model is used as a a springboard for future improvements, then it is necessary to find ways to improve the recall of this technique, in addition to developing automatic techniques to identify the attitude type of identified attiude groups accurately.

The unsupervised candidate generator in the linkage specification learner is

intended to be a first step in developing a fully unsupervised linkage-specification learner, but it's clear that in its current iteration any linkage-specification that does not appear in a small annotated corpus of text will be pruned by the supervised pruning algorithms FLAG currently employs. To make the linkage specificaion learner fully unsupervisd, new techniques are needed for estimating accuracy of linkage specifications on an unlabled corpus, or for bootstrapping a reasonable corpus even when the textual redundancy found in product review corpora is not available.

When a set of linkage specifications is learned using the supervised candidate generator, then pruned using an algorithm like the covering algorithm, the most common error in selecting the right appraisal expression is that the correct linkage specification was pruned from the set of linakge specifications by the covering algorithm. This problem was solved by not pruning the linkage specifications, instead applying the disambiguator directly to the full set of linkage specifications. It appears from the rather small learning curve in Figure 10.3 that the accuracy of the reranking disambiguator drops as more linkage specifications appear in the set, suggesting that this does not scale. Consequently, better pruning algorithms must be developed that provide a set of linkage specifications that's specifically useful to the disambiguator, or ways of factoring linkage specifications for better generality must be developed.

FLAG's linkage extractor could also be improved by giving it the ability to identify comparative appraisal expressions, appraisal expressions that do not have a target or where the target is the same phrase as the atttiude, and by augmenting it with a way to identify appraisal expressions that span multiple sentences (where the attitude is in a minor sentence which follows the sentence containing the target).

To improve the ranking disambiguator it would be appropriate to explore ways to incorporate other parts of the Appraisal system that FLAG does not yet model. Identifying whether there are better features that could be included in the disambigua-

tor's feature set is another area of research that would improve the disambiguator. Some starting points might be to model "please"-type versus "likes"-type distinction in the ATTITUDE system or or to incorporate verb patterns more generally using a system like VerbNet [93], and using named entity recognition to identify whether evaluators are correct.

In the broader field of sentiment analysis, the most obvious area of future research concerns the addition of aspects, processes, superordinates, and expressors to the sentiment analysis arsenal. It is important for researchers in the field to understand these slots in an appraisal expression, to be able to identify them, and to be able to differentiate them from the more commonly recognized parts of the sentiment analysis picture: targets and evaluators. The presence of aspects, processes, superordinates, and expressors also presents opportunities for further research into how and when to consider the contents of these slots in applications that have until now only been concerned with attitudes and targets.

A more important task in the field of sentiment analysis is to evaluate other new and existing structured sentiment extraction techniques against the IIT, Darmstadt, and JDPA corpora, evaluating them to study their accuracy at identifying individual appraisal expression occurrences, as I have done here. In the existing literature on structured sentiment extraction, evaluation techniques have been inconsistent, and some of the literature has been unclear about exactly what types of evaluations have been performed. For structured sentiment extraction research to continue, it is important to establish an agreed standard evaluation method, and to develop high quality resources to use for this evaluation. Appraisal expression and the IIT sentiment corpus present a way forward for this evaluation, but more annotated text is needed.

# APPENDIX A

# READING A SYSTEM DIAGRAM IN SYSTEMIC FUNCTIONAL LINGUISTICS

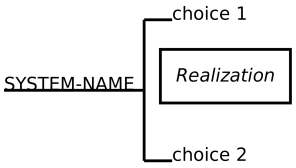
Systemic Functional Linguistics treats grammatical structure as a series of choices that a speaker or writer makes about the meaning he wishes to convey. These choices can grow to contain many complex dependencies, so Halliday and Matthiessen [64] have developed notation for diagramming these dependencies, called a "system diagram" or a "system network". The choices in a system network are called "features".

System diagrams are related to AND/OR graphs [97, 105, 129, 159, and others], which are directed graphs whose nodes are labeled AND or OR. In an AND/OR graph, a node labeled AND is considered solved if all of its successor nodes are solved, and a node labeled OR is considered solved if exactly one of its successor nodes is solved.

## A.1 A Simple System

The basic unit making up a system diagram is a simple system, such as the one shown below. This represents a grammatical choice between the options on the right side. In the figure below the speaker or writer must choose between "choice 1" or "choice 2", and may not choose both, and may not choose neither.

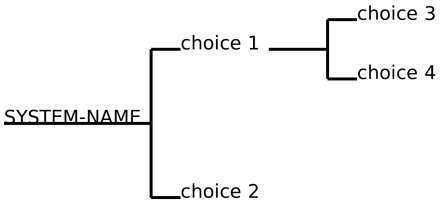
The *realization* of a feature is shown in a box below that feature. This indicates how the choice manifests itself in the actual utterance. The box will include a plain English explanation of the effect of this feature on the sentence, though Halliday and Matthiessen [64] have developed some special notation for some of the more common kinds of realizations. This notation is described in Section A.4.




A simple system may optionally have a system name (here System-Name)

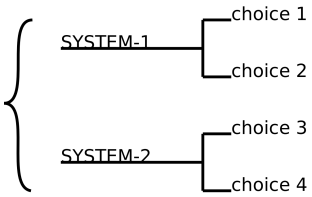
describing the choice to be made, but this may be omitted if it is obvious or irrelevant.

Depending on which feature the speaker chooses, he may be presented with other choices to make. This is represented by cascading another system. In the diagram that follows, if the speaker chooses "choice 1", then he is presented with a choice between "choice 3" and "choice 4", and must choose one. If he chooses "choice 2", then he is not presented with a choice between "choice 3" and "choice 4". In this way, "choice 1" is considered the "entry condition" for choices 3 and 4.




## A.2 Simultaneous Systems

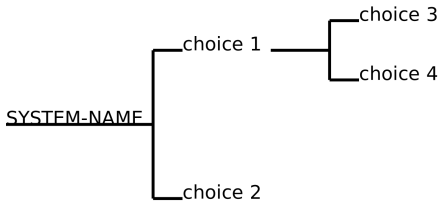
In some cases, selecting a certain feature in a system diagram presents multiple independent choices. These are shown in the diagram by using simultaneous systems, represented with a big left bracket enclosing the entry side of the system diagram, as in the following diagram:



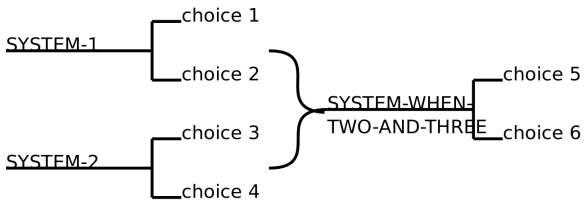

The speaker must choose between "choice 1" and "choice 2" as well as between "choice 3" and "choice 4". He may match either feature from System-1 with either feature of System-2

## A.3 Entry Conditions

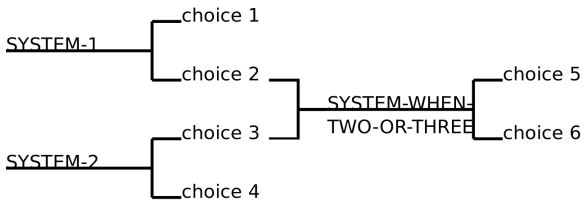
One enters a system diagram starting at the left size, where there is a single entry point. All other systems in the diagram have entry conditions based on previous features selected in the system. The simplest entry condition that the presence of a single feature requires more choices to refine its use. This is shown by having the additional system to the right of the feature that requires it, as shown below:




A system may also be applicable based on combinations of different features. Some systems may only apply when multiple features are all selected. This is a *conjunctive entry condition* and is shown below. The speaker makes a choice between "choice 5" and "choice 6" only when he has chosen both "choice 2" and "choice 3".




Some systems may only apply when multiple features are all selected. This is a *disjunctive entry condition* and is shown below. The speaker makes a choice between "choice 5" and "choice 6" when he has chosen either "choice 2" or "choice 3". (It does not matter whether the features involved in the disjunctive entry condition occur in simultaneous systems, or they are mutually exclusive, since either one is sufficient to require the additional choice.)




## A.4 Realizations

The realization of a feature in the sentence is written in a box below that feature, and is usually written in plain English. However, Halliday and Matthiessen [64] have developed some notation al shortcuts for describing realizations in a system diagram. The notation $+Subject$ in a realization would indicate the sentence must have a subject. This doesn't require it to appear textually, since it may be elided if it can be inferred from context. Such ellipsis is typically not explicitly accounted for by options in a system network.

The notation $-Subject$ would indicate that a subject required by an earlier decision is now no longer required (and is in fact forbidden, because anywhere that there would be an "optional" realization, this would have to be represented by another simple system).

The notation $Subject\hat{}Verb$ indicates that the subject and verb must appear in that order.

# APPENDIX B

# ANNOTATION MANUAL FOR THE IIT SENTIMENT CORPUS

## B.1 Introduction

We are creating a corpus of documents tagged to indicate the structure of opinions in English, specifically in the field of evaluation, which conveys a person's approval or disapproval of circumstances and objects in the world around him. We deal with the structure of an opinion by introducing the concept of an *appraisal expression* which is a structured unit of text expressing a single evaluation (*attitude*) of a single primary object or proposition being evaluated (or a single comparison between two attitudes, or between attitudes about two targets). The corpus that we develop will be used to develop computerized systems that can extract opinions, and to test those systems.

## B.2 Attitude Groups

The core of an appraisal expression is an *attitude group*. An attitude group is a phrase that indicates that approval or disapproval is happening in the sentence. Besides this, it has two other functions: it determines whether the appraisal is positive or negative, and it differentiates several different types of appraisal.

Some examples of attitude groups include the phrases "crying", "happy", "good", "romantic", "not very hard", and "far more interesting." In context in a sentence, we might expect to see

(67) What is $_{\textbf{attitude}}$ <u>far more interesting</u> than the title, is that the community is prepared to pay hard earned cash for this role to be filled in this way.

(68) ...then it is $_{\textbf{attitude}}$ <u>not very hard</u> to see the public need that is arguably being addressed.

(69) The $_{\textbf{attitude}}$ <u>romantic</u> pass of the Notch is a great artery.

(70) And it used to $_{\text{attitude}}$ bother me quite a lot, that I was so completely out there on my own, me and the sources, and no rabbi or even teacher within sight.

There is typically a single word that carries most of the meaning of the attitude group ("interesting" in example 67), then other words modify it by making it stronger or weaker, or changing the orientation of the appraisal expression. When tagging appraisal expressions, you should include all of these words, including articles that happen to be in the way (as in example 70), and including the modifier "so" (as in "so nice"). You should not include linking verbs in an attitude group unless they change the meaning of the appraisal in some way.

There are situations when you will find terms that ordinarily convey appraisal, but in context they do not convey appraisal. The most important question to ask yourself when you identify a potential attitude in the text is "does this convey approval or disapproval?" If it conveys neither approval nor disapproval, then it is not appraisal. For example, the word "creative" in example 71 does not convey approval or disapproval of any "world." (With direct emotions, the affect attitude types, the question to ask "is this a positive or negative emotion?" which generally corresponds to approval or disapproval of the target.)

(71) I could not tap his shoulder and intrude on his private, creative world.

A common way in which this happens is when the attitude is a *classifier* which indicates that the word it modifies is of a particular kind. For example the word "funny" usually conveys appraisal, but in example 72 it talks about a kind of gene instead, so it is not appraisal.

(72) No ideas for a topic, no funny genes in my body.

You can test whether this is the case by rephrasing the sentence to include an in-

tensifier such as "more". If this cannot be done, then the word is a classifier, and therefore isn't appraisal. In example 72 this can't be done.

* No ideas for a topic, no funnier genes in my body.

Another common confusion with attitudes is determining whether a word is an appraisal head word which should be tagged as its own attitude group or whether it is a modifier that is part of another attitude group, for example the word "excellently" in example 74:

(74) Her appearance and demeanor are $_{\text{attitude}}$ <u>excellently suited</u> to her role.

You can test to determine whether a word conveys appraisal on its own by trying to remove it from the sentence to see whether the attitude is still being conveyed. When we remove "excellently", we are left with

(75) Her appearance and demeanor are $_{\text{attitude}}$ <u>suited</u> to her role.

since both sentences convey positive quality (in the sense of appropriateness for a given task), the words "excellently" and "suited" are part of the same attitude group.

Attitude groups need to be tagged with their orientation and attitude type. The *orientation* of an attitude group indicates whether it is positive or negative. You should tag this taking any available contextual clues into account (*including* polarity markers described below). Once you have assigned both an orientation and an attitude type, you will note that orientation doesn't necessarily correlate to the presence or absence of the particular qualities for which the attitude type subcategories are named. It is concerned with whether the presence or absence of those qualities is a good thing.

The *attitude type* is explained in section B.2.2.

**B.2.1 Polarity Marker.** Sometimes there is a word (a *polarity marker*) elsewhere in the sentence, that is not attached directly to the attitude group, which changes the orientation of the attitude group.

(76) I $_{\textbf{polarity}}$ don't feel $_{\text{attitude}}$ good.

(77) I $_{\textbf{polarity}}$ couldn't bring myself to $_{\text{attitude}}$ like him.

In example 76, the word "don't" should be tagged as a polarity marker, and the attitude group for the word "good" should be marked as having a negative orientation, even though the word "good" is ordinarily positive. Similarly, in example 77, the word "couldn't" should be tagged as a polarity marker, and the attitude group for the word "like" should be marked as having a negative orientation.

Polarity markers be tagged even when they're already part of an attitude group.

In example 78, we see a situation where the polarity marker isn't a form of the word "not."

(78) Telugu film stars $_{\textbf{polarity}}$ failed to $_{\text{attitude}}$ shine in polls.

A polarity word should only be tagged when it affects the orientation of the appraisal expression, as in examples 76 or 81.

A polarity word should not be tagged when it indicates that the evaluator is specifically not making a particular appraisal (as in example 79), or when it is used to deny the existence of any target matching the appraisal (as in example 80, which shows two appraisal expressions sharing the same target). Although these effects may be important to study, they are complicated and beyond the scope of our work. You should tag the rest of the appraisal expression, and be sure you assign the *orientation*

as though the polarity word has no effect (so the *orientation* will be positive, in both examples 79 and 80).

(79) Here $_{\text{evaluator}}$ I though that it was a good pick up from where we left off but not a $_{\text{attitude}}$ brilliant $_{\text{target}}$ one.

(80) Some things just don't have a $_{\text{attitude}}$ logical, $_{\text{attitude}}$ rational $_{\text{target}}$ explanation.

Polarity markers have an attribute called *effect* which indicates whether they change the polarity of the attitude (the value flip) or not (the value same). The latter value is used when a string of words appears, each of which individually changes the orientation of the attitude, but when used in combination they cancel each other out. This is the case in example 81, where "never" and "fails" cancel each other out. We tag both as a single polarity element, and set *effect* to same.

(81) Hollywood $_{\textbf{polarity}}$ never fails to $_{\text{attitude}}$ astound me.

**B.2.2 Attitude Types.** There are three basic types of attitudes that are divided into several subtypes. The basic types are *appreciation*, *judgment*, and *affect*. *Appreciation* evaluates norms about how products, performances, and naturally occurring phenomena are valued, when this evaluation is expressed as being a property of the object. *Judgment* evaluates a person's behavior in a social context. *Affect* expresses a person's internal emotional state.

You will be tagging attitudes to express the individual subtypes of these attitude types. The subtypes and their definitions are presented in Figure B.1. The ones you will be tagging are marked in bold. Please see the figure in detail. I will describe only a few specific points of confusion here in the body of the text.

Examples of words conveying examples of appreciation and judgement are on pages 53 and 56 of "The Language of Evaluation" by James R. Martin and Peter

- Attitude Type
  - Appreciation
    - Composition
      - **Balance** — Did the speaker feel that the target hangs together well?
      - **Complexity** — Is the focus of the evaluation about a multiplicity of interrelating parts, or the simplicity of something?
    - Reaction
      - **Impact** — Did the speaker feel that the target of the appraisal grabbed his attention?
      - **Quality** — Is the target good at what it was designed for? Or what the speaker feels it should be designed for?
    - **Valuation** — Did the speaker feel that the target was significant, important, or worthwhile?
  - Judgment
    - Social Esteem
      - **Capacity** — Does the target have the ability to get results? How capable is the target?
      - **Tenacity** — Is the target dependable or willing to put forth effort?
      - **Normality** — Is the target's behavior normal, abnormal, or unique?
    - Social Sanction
      - **Propriety** — Is the target nice or nasty? How far is he or she beyond reproach?
      - **Veracity** — How honest is the target?
  - Affect
    - Happiness
      - **Cheer** — Does the evaluator feel happy?
      - **Affection** — Does the evaluator feel or desire a sense of closeness with the target?
    - Satisfaction
      - **Pleasure** — Does the evaluator feel that the target met or exceeded his expectations? Does the evaluator feel gratified by the target?
      - **Interest** — Does the evaluator feel like paying attention to the target?
    - Security
      - **Quiet** — Does the evaluator have peace of mind?
      - **Trust** — Does the evaluator feel he can depend on the target?
    - **Surprise** — Does the evaluator feel that the target was unexpected?
    - **Inclination** — Does the evaluator want to do something, or want the target to occur?

Figure B.1. Attitude Types that you will be tagging are marked in bold, with the question that defines each attitude type.

R. R. White, and examples of words conveying affect affect are on page 173–175 of "Emotion Talk Across Corpora". Both of these sets of pages are attached to this tagging manual.

When determining the attitude type, you should first determine whether the attitude is affect, appreciation, or judgement, and then you should determine the specific subtype of attitude. Some attitude subtypes are easily confused, for example impact and interest, so a careful determination of whether the attitude is affect or appreciation goes a long way toward determining the correct attitude type.

A few notes about particular attitude types:

Surprise appears to usually be neutral in text. Since we are concerned with only the approval/disapproval dimension most instances of surprise should not be tagged. You should only tag appraisal expressions of surprise if they clearly convey approval or disapproval.

Inclination can easily be confused for non-evaluative intention (something that happens frequently with the word "want") or for a need for something to happen. An appraisal expression should only be for inclination if it's clearly expresses a desire for something to happen. In example 82, the word "need" does not express inclination, nor does "had better" in example 83.

(82) Do you see dead people or do you think those who claim to are in need of serious mental health care?

(83) Then I thought that I had better get my ass into gear.

The word "want" in example 84 does express inclination.

(84) "Seriously, Debra, I don't want to burn, get my back and shoulders, okay?"

To differentiate between cheer and pleasure, use the following rule: cheer is for evaluations that concern a state of mind, while pleasure is related to an experience. Thus example 85 is cheer, while example 86 is pleasure

(85) I was $_{\text{attitude}}$ happy about my purchase.

(86) I found the walk $_{\text{attitude}}$ enjoyable.

Propriety includes examples where a character trait is described that's generally considered as positive or negative. Any evaluation of morality or ethics should be categorized as propriety.

Propriety and quality can easily be confused in situations where the attitude conveys "appropriateness." When this is the case, appraisal expressions evaluating the appropriateness of a behavior in a certain social situation should be categorized as propriety, but those evaluating the appropriateness of an object for a particular task should be categorized as quality.

Appraisal expressions evaluating the monetary value of an object should be categorized as valuation (as in examples 87 and 88, both of which convey positive valuation.).

(87) I bought it because it was $_{\text{attitude}}$ cheap.

(88) She was wearing what must have been a $_{\text{attitude}}$ very expensive necklace.

Veracity only concerns evaluations about people. Attitudes that concern the truth of a particular fact do not fall under the rubric of attitude.

**B.2.3 Inscribed versus Evoked Appraisal.** There are two ways that attitude groups can be expressed in documents: implicitly or explicitly. Linguistically, these are referred to as *inscribed* and *evoked* appraisal.

Inscribed appraisal uses explicitly evaluative language to convey emotions or evaluation. This (roughly) means that looking up the word in a dictionary should give you a good idea of whether it is opinionated, and whether the word is usually positive or negative. The simplest example of this is the word "good" which readers agree on as conveying a positive evaluation of something.

Evoked appraisal is expressed by evoking emotion in the reader by describing experiences that the reader identifies with specific emotions. Evoked appraisal includes such phenomena as sarcasm, figurative language, and idioms. A simple example of evoked appraisal would be the use of the phrase "it was a dark and stormy night", to triggers a sense of gloom and foreboding. Evoked appraisal can be difficult to analyze and is particularly subjective in its interpretation, so we are not interested in trying to tag it here.

Examples 89–91 demonstrate evoked appraisals.

(89) "I am quite benumbed; for the Notch is $_{\textbf{not attitude}}$ <u>just like the pipe of a great pair of bellows</u>;"

(90) But Im a sports lover at heart and the support for those (aside from Nascar, shamefully) is $_{\textbf{not attitude}}$ <u>seriously a joke</u>.

(91) $_{\textbf{not attitude}}$ <u>Who can resist</u> men with permanent black eyes and missing teeth?

Example 92 conveys two attitudes: the inscribed "happy", and the evoked sadness of "the smile doesn't quite reach my eyes".

(92) At least, I seem $_{\textbf{attitude}}$ <u>happy</u>, but can they tell $_{\textbf{not attitude}}$ <u>the smile doesn't quite reach my eyes?</u>

We are interested in tagging only inscribed appraisal. This means that we will

not be tagging most metaphoric uses of language.

If you are in doubt as to whether something is inscribed or evoked appraisal, look in a dictionary to see whether the attitude is listed in the dictionary. This will help you to identify common idioms that are always attitude (and are thus considered inscribed appraisal). This will also help you to identify when a typically non-evaluative word also has an evaluative word sense.

For example, the word dense typically refers to very heavy materials or very thick fog or smoke, but in example 93 it refers to a person who's very slow to learn, and this meaning is listed in the dictionary. The later word sense is a negative evaluation of a person's intellectual capacity and should be tagged as inscribed appraisal.

(93) ...so $_{\textbf{attitude}}$ dense he never understands anything I say to him

Conversely, the word "slow" has a word sense for being slow to learn (similar to "dense"), and another word sense for being uninteresting, and both of these word senses are inscribed appraisal. However, in example 94 the word "slow" is being used in its simplest sense of taking a comparatively long time, so this example would be considered evoked appraisal (if it is evaluative at all) and would not be tagged.

(94) Despite the cluttered plot and the $_{\textbf{attitude}}$ slow wait for things to get moving, it's not bad.

If a dictionary does not help you determine whether appraisal is inscribed or evoked, then you should be conservative, assume that it is evoked, and not tag it.

Attitudes that are domain-sensitive but have well understood meanings within the particular domain should be tagged as inscribed appraisal (as in example 95 where faster computers are always evaluations of positive capacity.)

(95) So, if you have a $_{\textbf{attitude}}$ fairly fast $_{\text{target}}$ computer (1 gig or better) with plenty of ram (512) and your not gaming online or running streaming video continually, you should be fine.

**B.2.4 Appraisals that aren't the point of the sentence.** Sometimes an appraisal will be offered, in a by-the-way fashion, where the sentence is intended to convey something else, and appraisal isn't really the point of the sentence, as in examples 96 and 97. We are interested in finding appraisal expressions, even when they're found in unlikely places, so even in these cases, you still need to tag appraisal expression.

(96) Kaspar Hediger, master tailor of Zurich, had reached the age at which an $_{\textbf{attitude}}$ industrious $_{\text{target}}$ craftsman begins to allow himself a brief hour of rest after dinner.

(97) So it happened that one $_{\textbf{attitude}}$ beautiful $_{\text{target}}$ March day, he was sitting not in his manual but his mental workshop, a small separate room which for years he had reserved for himself.

Likewise, irrealis appraisals and hypothetical appraisals and queries about a person's opinion should also be tagged.

## B.3 Comparative Appraisals

In some appraisal expressions, we may be comparing different targets, aspects, or even attitudes with each other. In this case any of the slots in the normal appraisal expression structure may be doubled, which. We represent this by adding the indexes 1 and 2 on the slots that are doubled. The first instance of a particular slot gets index 1, and the second gets index 2. Almost all comparative attitudes have some textual slot in common that gets tagged without indices. Examples 98, 99, 100, and 101 all

demonstrate this. In examples 102 and 103, have entities used in comparison that are the same, but the textual references for those entities are different, so the they are tagged as separate slots. It is possible for a comparative appraisal expression to have no entities in common between the two sides.

A *comparator* describes the relationship between two things being compared. A comparator is annotated with a single attribute indicating the *relationship* between the two appraisals that are being compared. This can have the values greater, less, and equal. A comparator can (and frequently does) overlap the attitude in the appraisal expression.

Since most of these comparators have two parts, we have two annotations: *comparator*, and *comparator-than*. The comparator is used to annotate the first part of the text, and you should tag the part that tells you what the relationship is between the two items being compared. This should usually be a comparative adjective ending in "-er" (which will also be tagged as an attitude) or the word "more" or "less" (in which case, the attitude should not be tagged as part of the comparator). The comparator-than is used to annotate the word 'than' or 'as' which separates the two items being compared. Even when these two parts of the comparator are adjacent to each other, tag both parts. If there is a polarity marker somewhere else in the sentence that reverses the relationship between two items being compared, annotate that as polarity with no rank.

Some examples of comparators:

- "$_{\textbf{comparator}}$ $\underset{\text{attitude}}{\underline{\text{better}}}$ $_{\textbf{comparator-than}}$ $\underline{\text{than}}$" is an example of a greater relationship.
- "$_{\textbf{comparator}}$ $\underset{\text{attitude}}{\underline{\text{worse}}}$ $_{\textbf{comparator}}$ $\underline{\text{than}}$" is also an example of a greater relationship. Since the attitude here is negative, this indicates that the first thing being compared has a more negative evaluation than the second thing being compared.

- "$_{\textbf{comparator}}$ more $_{\text{attitude}}$ exciting $_{\textbf{comparator-than}}$ than" is also an example of a greater relationship
- "$_{\textbf{comparator}}$ less $_{\text{attitude}}$ exciting $_{\textbf{comparator-than}}$ than" is an example of a less relationship
- "$_{\textbf{comparator}}$ as $_{\text{attitude}}$ good $_{\textbf{comparator-than}}$ as" is an example of an equal relationship
- "$_{\textbf{comparator}}$ twice as $_{\text{attitude}}$ bad $_{\textbf{comparator-than}}$ as" is an example of a greater relationship.
- "$_{\textbf{comparator}}$ not as $_{\text{attitude}}$ bad $_{\textbf{comparator-than}}$ as" is an example of a less relationship.

(An authoritative English grammar tells us that the list above pretty much covers all of the textual forms for a comparator. However, if you see something else that fits the bill, tag it.)

Some examples of how to tag comparative appraisal expressions are as follows:

(98) $_{\text{target}}$ The Lost World was a $_{\textbf{comparator}}$ $_{\text{attitude}}$ better $_{\text{superordinate-1}}$ book $_{\textbf{comparator-than}}$ than $_{\text{superordinate-2}}$ movie.

(99) $_{\text{target-1}}$ Global warming may be $_{\textbf{comparator}}$ twice as $_{\text{attitude}}$ bad $_{\textbf{comparator-than}}$ as $_{\text{target-2}}$ previously expected.

Examples 100 and 101 show how a particular evaluators compare their evaluations of two different targets. Example 100 demonstrates an equal relationship, and 101 demonstrates a less relationship.

(100) $_{\text{evaluator}}$ Cops : $_{\text{target-1}}$ Imitation pot $_{\textbf{comparator}}$ as $_{\text{attitude}}$ bad $_{\textbf{comparator-than}}$ as $_{\text{target-2}}$ the real thing

(101) $_{\text{evaluator}}$ I thought $_{\text{target-1}}$ they were $_{\textbf{comparator}}$ less $_{\text{attitude}}$ controversial $_{\textbf{comparator-than}}$ than $_{\text{target-2}}$ the ones I mentioned above.

When multiple slots are duplicated, the slots tagged with index 1 form a coherent structure which is usually paralleled by the structure of the slots tagged with index 2. In examples 102 and 103, the appraisal expressions compare full appraisals of their own. In example 102, the two attitudes being compared are the same, but they are still tagged separately.

(102) "$_{\text{evaluator-1}}$ I $_{\text{attitude-1}}$ love $_{\text{target-1}}$ them $_{\textbf{comparator}}$ more $_{\textbf{comparator-than}}$ than $_{\text{evaluator-2}}$ they $_{\text{attitude-2}}$ love $_{\text{target-2}}$ me," Jamison said.

In example 103, two opposite attitudes are being compared. (Although this might be like comparing apples and oranges, it's still grammatically allowed. The irony of the comparison is a rhetorical device that makes the quote memorable.)

(103) Former Israeli prime minister Golda Meir said that "as long as the $_{\text{evaluator-1}}$ Arabs $_{\text{attitude-1}}$ hate $_{\text{target-1}}$ the Jews $_{\textbf{comparator}}$ more $_{\textbf{comparator-than}}$ than $_{\text{evaluator-2}}$ they $_{\text{attitude-2}}$ love $_{\text{target-2}}$ their own children, there will never be peace in the Middle East."

Example 104 contains two separate appraisal expressions. The first appraisal expression (meant to be read ironically) should be tagged as it would be if the second part were not present (a less relationship), and the second should be tagged as a greater relationship.

(104) No, $_{\text{target-1}}$ It's $_{\textbf{comparator}}$ Not As $_{\text{attitude}}$ Bad $_{\textbf{comparator-than}}$ As $_{\text{target-2}}$ Was Feared – It's Worse

(105) ... $_{\text{target-1}}$ It's $_{\textbf{comparator}}$ $_{\text{attitude}}$ worse.

If there is a comparator in the sentence, but there aren't two things being compared in the sentence, then you should not tag a comparator (as in examples 106 and 107.

(106) $_{\text{target}}$ Idon't have to contort my face with a smile to be $_{\text{attitude}}$ more pleasing to $_{\text{evaluator}}$ people.

(107) The stricter the conditions under which they were creating, the $_{\text{attitude}}$ more miserable $_{\text{evaluator}}$ they were with $_{\text{target}}$ the process.

However if it is clear that one of the slots being compared has been elided from the sentence (but should be there), then you should tag a comparator as in example 105. A common sign that something has been elided from a sentence is when the sentence is a sentence fragment. It's up to you to fill in the part that's missing and determine whether that's the missing slot in the comparison.

### B.4 The Target Structure

The primary slot involved in framing an attitude group is the *target*. The target of an evaluation is the object that the attitude group evaluates.

(108) He could have borne to live an $_{\text{attitude}}$ undistinguished $_{\textbf{target}}$ life, but not to be forgotten in the grave.

The target answers one of three questions depending on the type of the attitude:

**Appreciation:** What thing or event has a positive/negative quality.

**Judgment:** Who has the positive/negative character? Or what behavior is being considered as positive or negative.

**Affect:** What thing/agent/event was the cause of the good/bad feeling?

The target (and other slots such as the process, superordinate, or aspect) *must* be in the same sentence as the attitude.

A target can also being a proposition that is being evaluated, as in examples 109 and 110.

(109) $_{\textbf{target}}$ A real rav muvhak ends up knowing you very well, very intimately one might say - in a way that I am not sure is actually $_{\text{attitude}}$ very appropriate or easy to negotiate $_{\text{aspect}}$ when the sexes differ.

(110) $_{\text{evaluator}}$ I $_{\text{attitude}}$ hate it $_{\textbf{target}}$ when people talk about me rather than to me.

When the attitude is a noun phrase that refers in the real world to the thing that's being appraised, and there is no other phrase that refers to the same target, then the attitude should be tagged as its own target, as in examples 111 and 112 (where the attitude is an anaphoric reference to the target which appears in another sentence). Where here's no target, and the attitude does not refer to an entity in the real world, we tag the attitude without a target, as in example 113.

(111) On the other hand, I am aware of women who seem to manage to find male mentors, so clearly some people do manage to negotiate $_{\textbf{target}}$ the $_{\text{attitude}}$ perils $_{\text{aspect}}$ that might be found in such a relationship.

(112) Rick trusts Cyrus. $_{\textbf{target}}$ The $_{\text{attitude}}$ idiot.

(113) Though the average person might see a cute beagle-like dog in an oversized suit, I see $_{\text{attitude}}$ bravery and $_{\text{attitude}}$ persistence.

**B.4.1 Pronominal Targets.** If the target is a pronoun (as in example 114), you should tag the pronoun as the target, and tag the antecedent of the pronoun as

the *target-antecedent*. The target-antecedent should be the closest non-pronominal mention of the antecedent. It should precede the target if the pronoun is an anaphor (references something you've already referred to in the text), and come after the target if the pronoun is a cataphor (forward reference). Both the target and target-antecedent should have the same id and rank. Even if the antecedent of the pronoun appears in the same sentence (as in example 115) you should tag the pronoun as the target.

(114) Heading off to dinner at $_{\textbf{target-antecedent}}$ Villa Sorriso in Pasadena. I hear $_{\text{target}}$ it's $_{\text{attitude}}$ good. Any opinions?

(115) I had $_{\textbf{target-antecedent}}$ a voice lesson with Jona today, and $_{\text{target}}$ it was $_{\text{attitude}}$ awesome.

When the antecedent of the pronoun is a complex situation described over the course of several sentences, do not tag a target-antecedent.

When the evaluator is the pronoun "I" or "me", you need only find an antecedent phrase when the antecedent is not the author of the document.

If the pronoun 'it' appears as a dummy pronoun (as in example 116), you should not tag 'it' as the target. In this case, there will be no target-antecedent.

(116) Anyone else think $_{\textbf{not target}}$ it was $_{\text{attitude}}$ strange that $_{\textbf{target}}$ during the Olympics they played "Party in the USA" over the PA system considering they are in Canada?

**B.4.2 Aspects.** When a target is being evaluated with regard to a specific behavior, or in a particular context or situation, this behavior, context, or situation should be annotated as an *aspect*. An aspect serves to limit the evaluation in some way, or to better specify the circumstances under which the evaluation applies. An example

of this is example 117.

(117) $_{\text{target}}$ Zack would be my $_{\text{attitude}}$ hero $_{\textbf{aspect}}$ no matter what job he had.

When the target (or superordinate) and aspect are adjacent, it can be difficult to tell whether the entire phrase is the target (example 119), or whether it should be split into a target and an aspect (example 118).

(118) There are a few $_{\text{attitude}}$ extremely sexy $_{\text{target}}$ new features $_{\textbf{aspect}}$ in Final Cut Pro 7.

(119) I $_{\text{attitude}}$ like $_{\text{target}}$ the idea of the Angels.

We must resolve these questions by looking for ways to rephrase the sentence to determine whether the potential aspect modifies the target or the verb phrase. Example 118 can be rephrased to move the prepositional phrase "in Final Cut Pro 7" to the beginning of the sentence.

In Final Cut Pro 7, there are a few extremely sexy new features.

Thus, the phrase "in Final Cut Pro 7" is not part of the target, and should be tagged as an aspect.

To contrast, in example 119, the phrase "of the Angels" cannot be moved to the beginning of the sentence – the following does not make any sense:

* Of the Angels, I liked the idea.

Thus the phrase "of the angels" is part of the target.

(122) $_{\text{target}}$ A real rav muvhak ends up knowing you very well, very intimately one might say - in a way that I am not sure is actually $_{\text{attitude}}$ very appropriate or easy to negotiate $_{\textbf{aspect}}$ when the sexes differ.

In example 122, we are faced with a different uncertainty as to whether the phrase "when the sexes differ" is an aspect — it is not easy to tell whether the phrase concerns the attitude only the attitude "easy to negotiate" or whether it concerns"very appropriate" as well. In this case, It depends on the context. Since the document from which this sentence is drawn deals with the subject of women's' relationships with rabbis, I can remove the phrase "or easy to negotiate" and the sentence will still make sense in context. Thus, "when the sexes differ" is an aspect of the appraisal expression for "very appropriate."

An appraisal expression can only have an aspect if there is a separate span of text tagged as the target. If you think you see an aspect without a separate target, you should tag the aspect as the target. However if it is clear that the target has been elided from the sentence, you should tag the aspect without tagging a target. A common sign that something has been elided from a sentence is when the sentence is a sentence fragment. It's up to you to fill in the part that's missing and determine whether that's the missing target.

**B.4.3 Processes.** When an attitude is expressed as an adverb, it frequently modifies a verb and serves to evaluate how well a target performs at that particular *process* (the verb). Several examples demonstrate the appearances of processes in the appraisal expression:

(123) $_{\text{target}}$ The car $_{\textbf{process}}$ handles $_{\text{attitude}}$ really well, but it's ugly.

(124) $_{\text{target}}$ We're still $_{\textbf{process}}$ working $_{\text{attitude}}$ hard.

(125) However, since $_{\text{target}}$ the night seemed to be $_{\textbf{process}}$ going $_{\text{attitude}}$ so well I wanted to hang out a little bit longer.

The general pattern for these seems to be that an adverbial attitude modifies the process, and the target is the subject of the process.

The same target can be evaluated in different processes, as in example 126 which shows two appraisal expressions sharing the same target. (The attitude "sluggishly" is an evoked appraisal, so you won't tag that appraisal expression, but it's included for illustration.)

(126) $_{\text{target}}$ The car $_{\textbf{process}}$ maneuvers $_{\text{attitude}}$ well, but $_{\textbf{process}}$ accelerates $_{\text{attitude}}$ sluggishly.

You should tag the process, even when it's noninformative, as in example 127.

(127) $_{\text{target}}$ We arranged via e-mail to meet for dinner last night, which $_{\textbf{process}}$ went $_{\text{attitude}}$ really well.

An appraisal expression does not have a process when the target isn't doing anything, as in example 128.

(128) $_{\text{evaluator}}$ She turns to him and looks at $_{\text{target}}$ him $_{\text{attitude}}$ funny.

An appraisal expression does not have a process when the attitude modifies the whole clause, as in example 129. However, example 130's attitude modifies a single verb in the clause and so that verb is the process.

(129) $_{\text{attitude}}$ Hopefully $_{\text{target}}$ we'll be able to hang out more.

(130) $_{\text{attitude}}$ Sluggishly, $_{\text{target}}$ the car $_{\textbf{process}}$ accelerated.

An appraisal expression can only have a process if there is a separate span of text tagged as the target. If you think you see a process without a separate target, you should tag the process as the target. However if it is clear that the target has been elided from the sentence, you should tag the process without tagging a target,

as in example 131. A common sign that something has been elided from a sentence is when the sentence is a sentence fragment. It's up to you to fill in the part that's missing and determine whether that's the missing target.

(131) $_{\textbf{process}}$ Works $_{\text{attitude}}$ great!

**B.4.4 Superordinates.** A target can also be evaluated as how well it functions as a particular kind of object, in which case a *superordinate* will be part of the appraisal expression. Examples 132, and 134 demonstrate sentences with both a superordinate and an aspect. Example 133 demonstrates a sentence with only a superordinate. (In example 133, the word 'It' refers to the previous sentence, so it is the target.)

(132) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\textbf{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

(133) $_{\text{target}}$ It was a $_{\text{attitude}}$ good $_{\textbf{superordinate}}$ pick up from where we left off.

(134) $_{\text{target}}$ She is the $_{\text{attitude}}$ perfect $_{\textbf{superordinate}}$ companion $_{\text{aspect}}$ for this Doctor.

These three examples demonstrate a very common pattern involving a superordinate: "*target* is an *attitude superordinate*." It is such a common pattern that you should memorize this pattern so that when you see it you can tag it consistently.

The general rule to differentiate between a superordinate and an aspect is that an aspect is generally a prepositional phrase, which can be deleted from the sentence without requiring that the sentence be significantly rephrased. A superordinate is generally a noun phrase, and it cannot be deleted from the sentence so easily.

An appraisal expression can only have a superordinate if there is a separate span of text tagged as the target. If you think you see a superordinate without a separate target, you should tag the superordinate as the target. (Example 135 shows

a similar pattern to the examples given above, but the beginning of the sentence is no longer the target, so the phrase "new features" becomes a target instead of a superordinate.)

(135) There are a few $_{\text{attitude}}$ extremely sexy $_{\text{target}}$ new features $_{\text{aspect}}$ in Final Cut Pro 7.

However if it is clear that the target has been elided from the sentence, you should tag the superordinate without tagging a target. A common sign that something has been elided from a sentence is when the sentence is a sentence fragment. It's up to you to fill in the part that's missing and determine whether that's the missing target.

### B.5 Evaluator

The evaluator in an appraisal expression is the phrase that denotes whose opinion the appraisal expression represents. Unlike the target structure, which generally appears in the same sentence, the evaluator may appear in other places in the document as well. A frequent mechanism for indicating evaluators is through quotations of speech or thought. One example is in sentence 136.

(136) "$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\textbf{evaluator}}$ he cried, and clinched his hands.

Thinking (as in example 137) is similar to quoting even though there are no quotations marks to denote the quoted text.

(137) $_{\textbf{evaluator}}$ I thought $_{\text{target-1}}$ they were $_{\text{comparator}}$ less $_{\text{attitude}}$ controversial than $_{\text{target-2}}$ the ones I mentioned above.

A possessive phrase may also indicate the evaluator, as in example 138.

(138) $_{\text{target}}$ Zack would be $_{\textbf{evaluator}}$ my $_{\text{attitude}}$ hero $_{\text{aspect}}$ no matter what job he had.

An evaluator always refers to a person or animate object (except when personification is involved). If you are prepared to tag an inanimate object as an evaluator, consider whether it would be more appropriate to tag it as an expressor (section B.5.2).

Some simple inference is permitted in determining who the evaluator is. In example 139, because the boss appreciates the target's diligence, we can conclude that he's also the evaluator responsible for the evaluation of diligence in the first place. In example 140, we assign the generic attitude "comfortable" to the person who said it.

(139) $_{\textbf{evaluator}}$ The boss appreciates you for $_{\text{target}}$ your $_{\text{attitude}}$ diligence.

(140) "It is better to sit here by this fire," answered $_{\textbf{evaluator}}$ the girl, blushing, "and be $_{\text{attitude}}$ comfortable and contented, though nobody thinks about us."

When the sentence contains the pronoun "I", it is easy to be confused whether "I" is the evaluator or whether there is no evaluator (meaning the author of the document is the evaluator). An easy test to determine which is the case is to try replacing the "I" with another person (perhaps "he" or "she"). In example 141, when we replace "I" with "he", it becomes clear that the author of the document thinks the camera is decent, and that "He/I" is just the owner of the camera. Therefore no evaluator should be tagged.

(141) I had a $_{\text{attitude}}$ decent $_{\text{target}}$ camera.
He had a $_{\text{attitude}}$ decent $_{\text{target}}$ camera.

The evaluator tagged should be the span of text which indicates to whom *this* attitude is attributed. Even though a evaluator's name may appear many times in a single document, and some of these may provide a more complete version the

evaluator's name, the phrase you're looking for is the one associated with *this* attitude group, even if it is a pronoun. (For information on how to tag pronominal evaluators, see section B.5.1)

There may be several levels of attribution explaining how one person's opinion is reported (or distorted) by another person, but these other levels of attribution concern the veracity of the information chain leading to the appraisal expression, and they are beyond scope of the appraisal expression. Only the person who (allegedly) made the evaluation should be tagged as the evaluator. This is evident in example 142, where the appraisal expression expresses women's' evaluation of Judaism. In fact, this sentence appears in a discussion of whether the alienation Rabbi Weiss sees is true, and what to do about it if it is true. From this example, we see that these other levels of attribution are important to the broader question of subjectivity, but they're not directly relevant to appraisal expressions.

Rabbi Weiss would tell you that looking around at the community he serves, he sees too many $_{\textbf{evaluator}}$ girls and women who are $_{\text{attitude}}$ alienated from $_{\text{target}}$ Judaism

When the attitude conveys affect, the evaluator evaluates the target by feeling some emotion about it. We find, in affect, that the evaluator is usually linked syntactically with the attitude (and not through quoting as is commonly the case with appreciation and judgement.) In these cases, the attitude may *describe* the evaluator, while nevertheless *evaluating* the target. The target may in fact be unimportant to the evaluation being described, and may be omitted.

(143) $_{\textbf{evaluator}}$ He is $_{\text{attitude}}$ very happy today.

(144) $_{\text{target}}$ He was $_{\text{attitude}}$ very honest today.

In example 143, the evaluator, "he" makes an evaluation of some unknown target by virtue of the fact that he is very happy (attitude type cheer, a subtype of affect) about or because of it. In example 144, some unknown evaluator (presumably the author of the text or quotation in which this sentence is found) makes an evaluation of "he" that he is very honest (attitude type veracity, a subtype of judgement). These two sentences share the same sentence structure, and in both the attitude group (conveying adjectival appraisal) describes "He", however the structure of the appraisal expression is different.

Some additional examples the evaluator in a situation concerning affect.

(145) $_{\text{target}}$ The president's frank language and references to Islam's historic contributions to civilization and the U.S. also inspired $_{\text{attitude}}$ respect and hope among $_{\textbf{evaluator}}$ American Muslims.

(146) The daughter had just uttered $_{\text{target}}$ some simple jest that filled $_{\textbf{evaluator}}$ them all with $_{\text{attitude}}$ mirth

(147) For a moment $_{\text{target}}$ it $_{\text{attitude}}$ saddened $_{\textbf{evaluator}}$ them, though there was nothing unusual in the tones.

**B.5.1 Pronominal Evaluators.** When the evaluator is a pronoun, in addition to tagging the pronoun with the evaluator slot, you should tag the antecedent of the pronoun with the *evaluator-antecedent* slot — this should be the closest non-pronominal mention of the antecedent. It should precede the evaluator if the pronoun is an anaphor (references something you've already referred to in the text), and come after the evaluator if the pronoun is a cataphor (forward reference). Both the evaluator and evaluator-antecedent should have the same id and rank.

A larger excerpt of text around example 136 (quoted as example 148) shows

a situation where we choose the pronoun subject of the word "said", rather than the phrase "the young man" or his name "Mr. Morpeth" introduced by direct address earlier in the conversation.

(148) "Dropped again, $_{\textbf{evaluator-antecedent}}$ Mr. Morpeth?"

. . .

"Your sister," replied the young man with dignity, "was to have gone fishing with me; but she remembered at the last moment that she had a prior engagement with Mr. Brown."

"She hadn't," said the girl."I heard them make it up last evening, after you went upstairs."

The young man clean forgot himself.

"$_{\text{target}}$ She's the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world," $_{\text{evaluator}}$ he cried, and clinched his hands.

When the evaluator is the pronoun "I" or "me", you need only find an antecedent phrase when the antecedent is not the author of the document.

**B.5.2 Expressor.** With expressions of affect, there may be an *expressor*, which denotes some instrument (a part of a body, a document, a speech, etc.) which conveys an emotion.

(149) $_{\text{evaluator}}$ He opened with $_{\textbf{expressor}}$ greetings of gratitude and $_{\text{attitude}}$ peace.

(150) $_{\text{evaluator}}$ She viewed $_{\text{target}}$ him with an $_{\text{attitude}}$ appreciative $_{\textbf{expressor}}$ gaze.

(151) $_{\textbf{expressor}}$ His face at first wore the melancholy expression, almost despondency, of one who travels a wild and bleak road, at nightfall and alone, but soon $_{\text{attitude}}$ brightened up when he saw $_{\text{target}}$ the kindly warmth of his reception.

In example 151, the possessive "his" is part of the expressor (applications which use appraisal extraction may process the expressor to find such possessive expressions, to treat them as an evaluator.)

An expressor is never a person or animate object. If you are prepared to tag a reference to a person as an expressor, you should consider tagging it as an evaluator instead.

## B.6 Which Slots are Present in Different Attitude Types?

In this section, I present some guidelines that may help in determining the attitude type of an appraisal expression based on different target structures. Use your judgement when applying these guidelines, as there may be exceptions that we have not yet discovered.

Judgement and appreciation generally require a target, but not an evaluator.

(152) He could have borne to live an $_{\text{attitude}}$ undistinguished $_{\text{target}}$ life, but not to be forgotten in the grave.

(153) $_{\text{target}}$ Kaspar Hediger, $_{\text{attitude}}$ master $_{\text{superordinate}}$ tailor $_{\text{aspect}}$ of Zurich, had reached the age at which an industrious craftsman begins to allow himself a brief hour of rest after dinner.

(154) So it is entirely possible to get a $_{\text{attitude}}$ solid $_{\text{target}}$ 1U server $_{\text{aspect}}$ from Dell or HP for far less than what youd spend on an Xserve.

When judgement and appreciation have an evaluator, that evaluator is usually expressed through the use of a quotation.

(155) "It is $_{\text{attitude}}$ better $_{\text{target}}$ to sit here by this fire," answered $_{\text{evaluator}}$ the girl, blushing, "and be comfortable and contented, though nobody thinks about us."

(156) “$_{\text{target}}$ She’s the $_{\text{attitude}}$ most heartless $_{\text{superordinate}}$ coquette $_{\text{aspect}}$ in the world,” $_{\text{evaluator}}$ he cried, and clinched his hands.

Direct affect generally requires an evaluator, but the target is not required (though it is often present, and less directly linked to the attitude).

(157) $_{\text{evaluator}}$ He is $_{\text{attitude}}$ very happy today.

(158) $_{\text{evaluator}}$ He opened with $_{\text{expressor}}$ greetings of gratitude and $_{\text{attitude}}$ peace.

An expressor always indicates affect.

(159) $_{\text{expressor}}$ His face at first wore the melancholy expression, almost despondency, of one who travels a wild and bleak road, at nightfall and alone, but soon $_{\text{attitude}}$ brightened up when he saw $_{\text{target}}$ the kindly warmth of his reception.

Covert affect occurs when an attitude group’s lexical meaning is a kind of affect, but its target structure is like that of attitude or judgement. Usually the most obvious sign of this is when the emoter is omitted. Another sign of this is the presence of an aspect or a superordinate. Covert affect usually means that a particular target has the capability to cause someone to feel a particular emotion, or it causes someone to feel a particular emotion with regularity.

We will not be singling out covert affect to tag it specially in any way, but awareness of its existence can help in determining the correct attitude type.

Examples 160, 161, and 162 are examples of covert interest, a subtype of affect, and are not impact, a subtype of appreciation. Example 163 is an example of negative pleasure.

(160) It’s $_{\text{attitude}}$ interesting that $_{\text{target}}$ somebody thinks that death and tragedy makes me happy.

(161) $_{\text{target}}$ Today was an $_{\text{attitude}}$ interesting $_{\text{superordinate}}$ day.

(162) Some men seemed proud that they weren't romantic, viewing $_{\text{target}}$ it as $_{\text{attitude}}$ boring.

(163) It was $_{\text{attitude}}$ irritating of $_{\text{target}}$ me $_{\text{aspect}}$ to whine.

Active verbs frequently come with both an evaluator and a target, closely associated with the verb. It may seem that the verb describes both the evaluator and the target in different ways. Nevertheless, you should tag them as a single appraisal expression, and determine the attitude type based on the lexical meaning of the verb.

(164) Then I discovered that $_{\text{evaluator}}$ they $_{\text{attitude}}$ wanted $_{\text{target}}$ me $_{\text{aspect}}$ for her younger sister.

(165) $_{\text{evaluator}}$ I $_{\text{attitude}}$ admire $_{\text{target}}$ you $_{\text{aspect}}$ as a composer.

(166) $_{\text{evaluator}}$ Everyone $_{\text{attitude}}$ loves $_{\text{target}}$ being complimented.

Example 166 conveys pleasure, not affection determined based on the fact that the target is not a person, but the fact it is some subtype of affect is determined lexically.

Sometimes appraisal expressions have no evaluator structure, and no target structure. In example 167, "complimented" is an appraisal expression because it concerns evaluation, but it speaks of a general concept, and it's not clear who the target or evaluator. In these cases, you need to determine the attitude type based on the lexical meaning of the verb.

(167) Everyone loves being $_{\text{attitude}}$ complimented.

(168) "It is better to sit here by this fire," answered the girl, blushing, "and be $_{\text{attitude}}$

comfortable and contented, though nobody thinks about us."

## B.7 Using Callisto to Tag

We will be tagging using MITRE's Callisto[11] software. The software isn't perfect, but it appears to be significantly less clumsy than the other software we've explored for tagging. Callisto allows us to tag individual slots and assign attributes to them. To group these slots into appraisal expressions, you must manually assign all of the parts of the appraisal expression the same ID.

The procedure for tagging individual appraisal expressions is spelled out in Section B.9 on the tagging procedure quick reference sheet.

**B.7.1 Tagging Conjunctions.** When there is a conjunction in an attitude or target (or any other slot), you should tag two appraisal expressions, creating duplicate annotations (with different id numbers) for the parts that are shared in common.

(169) $_{\text{evaluator}}$ I've $_{\text{attitude}}$ doubted $_{\text{target}}$ myself, $_{\text{target}}$ my looks, $_{\text{target}}$ my success (or lack of it).

(170) $_{\text{evaluator}}$ You're more than welcome to call $_{\text{target}}$ me $_{\text{attitude}}$ crazy, $_{\text{attitude}}$ nuts or $_{\text{attitude}}$ wacko but I know what I know, know what I've seen and know what I've experienced.

The slots from example 169 should be tagged as shown in Table B.1(a). Since the parenthetical quote "(or lack of it)" explains the target "my success" rather than adding a new entity, it should be tagged as part of the same target as "my success". The slots from example 170 should be tagged as shown in Table B.1(b).

[11] http://callisto.mitre.org/

Table B.1. How to tag multiple appraisal expressions with conjunctions.

(a) Example 169.

| **Type** | **Text** | **ID** |
|---|---|---|
| Evaluator | I | 3 |
| Evaluator | I | 4 |
| Evaluator | I | 5 |
| Attitude | doubted | 3 |
| Attitude | doubted | 4 |
| Attitude | doubted | 5 |
| Target | myself | 3 |
| Target | my looks | 4 |
| Target | my success (or lack of it) | 5 |

(b) Example 170.

| **Type** | **Text** | **ID** |
|---|---|---|
| Evaluator | You | 10 |
| Evaluator | You | 11 |
| Evaluator | You | 12 |
| Attitude | crazy | 10 |
| Attitude | nuts | 11 |
| Attitude | wacko | 12 |
| Target | me | 10 |
| Target | me | 11 |
| Target | me | 12 |

## B.8 Summary of Slots to Extract

| Slot | Possible Textual Forms | Attributes |
|---|---|---|
| Attitude | VP, NP, AdjP, Adverbial | *attitude-type*, *orientation* |
| Comparator | "more/less . . . than", "as Adj as" usually overlapping attitude | *relationship* |
| Polarity | "not", contractions ending in "-n't", "no", verbs such as "failed" | *effect* |
| Target | NP, VP, Clause | |
| Aspect | Prep. phrase, Clause | |
| Process | VP | |
| Superordinate | NP, VP | |
| Evaluator | NP (human/animate object) | |
| Expressor | NP (inanimate) | |

## B.9 Tagging Procedure

1. Find the attitude.

2. Ask yourself whether the attitude conveys approval or disapproval. If it does not convey approval or disapproval, don't tag it!

3. Verify that the attitude is inscribed appraisal by checking the word in a dictionary. If the dictionary doesn't convey

4. Tag the attitude and assign the it the next consecutive unused ID number. You will use this ID number to identify all of the other parts of the appraisal expression.

5. Determine the attitude's orientation.

6. If there is a polarity marker tag it and assign it the same ID.

7. If the attitude is involved in a comparison, tag the comparator and assign it the same ID.

8. If two attitudes are being compared, find the second attitude, and assign it the same ID. Assign the second attitude it rank 2, and go back and assign the first attitude rank 1.

9. Determine the target of the attitude, and any other target slots that are available, and assign them all the ID of the attitude group. (If multiple instances of a slot are being compared, assign the first instance rank 1, and the second instance rank 2.)

10. Determine the evaluator (and expressor) if they are available in the text.

11. Determine the attitude type of each attitude. Start by determining whether it is affect, judgement, or appreciation. (Knowing evaluator and target help with this process, see Section B.6.) Then determine which subtype it belongs to.

# BIBLIOGRAPHY


[1] Akkaya, C., Wiebe, J., and Mihalcea, R. (2009). Subjectivity word sense disambiguation. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, pp. 190–199. URL `http://www.aclweb.org/anthology/D/D09/D09-1020.pdf`.

[2] Alm, C. O. (2010). Characteristics of high agreement affect annotation in text. In *Proceedings of the Fourth Linguistic Annotation Workshop*. Uppsala, Sweden: Association for Computational Linguistics, pp. 118–122. URL `http://www.aclweb.org/anthology/W10-1815`.

[3] Alm, E. C. O. (2008). *Affect in Text and Speech*. Ph.D. thesis, University of Illinois at Urbana-Champaign.

[4] Appelt, D. E., Hobbs, J. R., Bear, J., Israel, D. J., and Tyson, M. (1993). FASTUS: A finite-state processor for information extraction from real-world text. In *IJCAI*. pp. 1172–1178. URL `http://www.isi.edu/~hobbs/ijcai93.pdf`.

[5] Archak, N., Ghose, A., and Ipeirotis, P. G. (2007). Show me the money: deriving the pricing power of product features by mining consumer reviews. In *KDD '07: Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: ACM, pp. 56–65. URL `http://doi.acm.org/10.1145/1281192.1281202`.

[6] Argamon, S., Bloom, K., Esuli, A., and Sebastiani, F. (2009). Automatically determining attitude type and force for sentiment analysis. In Z. Vetulani and H. Uszkoreit (Eds.), *Human Language Technologies as a Challenge for Computer Science and Linguistics*. Springer.

[7] Asher, N., Benamara, F., and Mathieu, Y. (2009). Appraisal of opinion expressions in discourse. *Lingvisticæ Investigationes*, 31.2, 279–292. URL `http://www.llf.cnrs.fr/Gens/Mathieu/AsheretalLI2009.pdf`.

[8] Asher, N., Benamara, F., and Mathieu, Y. Y. (2008). Distilling opinion in discourse: A preliminary study. In *Coling 2008: Companion volume: Posters*. Manchester, UK: Coling 2008 Organizing Committee, pp. 7–10. URL `http://www.aclweb.org/anthology/C08-2002`.

[9] Asher, N. and Lascarides, A. (2003). *Logics of conversation*. Studies in natural language processing. Cambridge University Press. URL `http://books.google.com.au/books?id=VD-8yisFhBwC`.

[10] Attensity Group (2011). Accuracy matters: Key considerations for choosing a text analytics solution. URL `http://www.attensity.com/wp-content/uploads/2011/05/Accuracy-MattersMay2011.pdf`.

[11] Aue, A. and Gamon, M. (2005). Customizing sentiment classifiers to new domains: A case study. In *Proceedings of Recent Advances in Natural Language Processing (RANLP)*. URL `http://research.microsoft.com/pubs/65430/new_domain_sentiment.pdf`.

[12] Baccianella, S., Esuli, A., and Sebastiani, F. (2010). Sentiwordnet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining. In N. Calzolari, K. Choukri, B. Maegaard, J. Mariani, J. Odijk, S. Piperidis, M. Rosner, and D. Tapias (Eds.), *LREC*. European Language Resources Association. URL `http://nmis.isti.cnr.it/sebastiani/Publications/LREC10.pdf`.

[13] Baldridge, J., Bierner, G., Cavalcanti, J., Friedman, E., Morton, T., and Kottmann, J. (2005). OpenNLP. URL `http://sourceforge.net/projects/opennlp/`.

[14] Banko, M., Cafarella, M. J., Soderland, S., Broadhead, M., and Etzioni, O. (2007). Open information extraction from the web. In M. M. Veloso (Ed.), *IJCAI*. pp. 2670–2676. URL `http://www.ijcai.org/papers07/Papers/IJCAI07-429.pdf`.

[15] Barnbrook, G. (1995). *The Language of Definition*. Ph.D. thesis, University of Birmingham.

[16] Barnbrook, G. (2002). *Defining Language: a local grammar of definition sentences*. John Benjamins Publishing Company.

[17] Barnbrook, G. (2007). Re: Your PhD thesis: The language of definition. Email to the author.

[18] Bednarek, M. (2006). *Evaluation in Media Discourse: Analysis of a Newspaper Corpus*. London/New York: Continuum.

[19] Bednarek, M. (2007). Local grammar and register variation: Explorations in broadsheet and tabloid newspaper discourse. *Empirical Language Research*. URL `http://ejournals.org.uk/ELR/article/2007/1`.

[20] Bednarek, M. (2008). *Emotion Talk Across Corpora*. New York: Palgrave Macmillan.

[21] Bednarek, M. (2009). Language patterns and ATTITUDE. *Functions of Language*, 16(2), 165 – 192.

[22] Bereck, E., Choi, Y., Stoyanov, V., and Cardie, C. (2007). Cornell system description for the NTCIR-6 opinion task. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 286–289.

[23] Biber, D., Johansson, S., Leech, G., Conrad, S., and Finegan, E. (1999). *Longman Grammar of Spoken and Written English (Hardcover)*. Pearson ESL.

[24] Blitzer, J., Dredze, M., and Pereira, F. (2007). Biographies, bollywood, boomboxes and blenders: Domain adaptation for sentiment classification. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*. Prague, Czech Republic: Association for Computational Linguistics, pp. 440–447. URL `http://www.aclweb.org/anthology-new/P/P07/P07-1056.pdf`.

[25] Bloom, K. and Argamon, S. (2009). Automated learning of appraisal extraction patterns. In S. T. Gries, S. Wulff, and M. Davies (Eds.), *Corpus Linguistic Applications: Current Studies, New Directions*. Amsterdam: Rodopi.

[26] Bloom, K. and Argamon, S. (2010). Unsupervised extraction of appraisal expressions. In A. Farzindar and V. Kešelj (Eds.), *Advances in Artificial Intelligence*, *Lecture Notes in Computer Science*, vol. 6085. Springer Berlin / Heidelberg, pp. 290–294. URL http://dx.doi.org/10.1007/978-3-642-13059-5_31.

[27] Bloom, K., Garg, N., and Argamon, S. (2007). Extracting appraisal expressions. *Proceedings of Human Language Technologies/North American Association of Computational Linguists*. URL http://lingcog.iit.edu/doc/bloom_naacl2007.pdf.

[28] Bloom, K., Stein, S., and Argamon, S. (2007). Appraisal extraction for news opinion analysis at NTCIR-6. In *NTCIR-6*. URL http://lingcog.iit.edu/doc/bloom_ntcir2007.pdf.

[29] Breidt, E., Segond, F., and Valetto, G. (1996). Local grammars for the description of multi-word lexemes and their automatic recognition in texts. In *Proceedings of 4th Conference on Computational Lexicography and Text Research*. URL http://citeseer.ist.psu.edu/breidt96local.html.

[30] Brown, G. (2011). An error analysis of relation extraction in social media documents. In *Proceedings of the ACL 2011 Student Session*. Portland, OR, USA: Association for Computational Linguistics, pp. 64–68. URL http://www.aclweb.org/anthology/P11-3012.

[31] Burns, P. R., Norstad, J. L., and Mueller, M. (2009). MorphAdorner (version 1.0) [computer software]. URL http://morphadorner.northwestern.edu/.

[32] Burton, K., Java, A., and Soboroff, I. (2009). The ICWSM 2009 Spinn3r dataset. In *Third Annual Conference on Weblogs and Social Media (ICWSM 2009)*. San Jose, CA: AAAI.

[33] Charniak, E. and Johnson, M. (2005). Coarse-to-fine N-best parsing and MaxEnt discriminative reranking. In *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL'05)*. Ann Arbor, Michigan: Association for Computational Linguistics, pp. 173–180. URL http://www.aclweb.org/anthology/P/P05/P05-1022.pdf.

[34] Chieu, H. L. and Ng, H. T. (2002). A maximum entropy approach to information extraction from semi-structured and free text. In *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI 2002)*. pp. 786–791. URL http://citeseer.ist.psu.edu/chieu02maximum.html.

[35] Collins, M. (2000). Discriminative reranking for natural language parsing. In *Proc. 17th International Conf. on Machine Learning*. Morgan Kaufmann, San Francisco, CA, pp. 175–182. URL http://citeseer.ist.psu.edu/collins00discriminative.html.

[36] Collins, M. J. and Koo, T. (2005). Discriminative reranking for natural language parsing. *Computational Linguistics*, 31(1), 25–70. URL http://dx.doi.org/10.1162/0891201053630273.

[37] Conrad, J. G. and Schilder, F. (2007). Opinion mining in legal blogs. In *Proceedings of the 11th International Conference on Artificial intelligence and Law*, ICAIL '07. New York, NY, USA: ACM, pp. 231–236. URL http://doi.acm.org/10.1145/1276318.1276363.

[38] Conway, M. E. (1963). Design of a separable transition-diagram compiler. *Commun. ACM*, 6, 396–408. URL http://doi.acm.org/10.1145/366663.366704.

[39] Crammer, K. and Singer, Y. (2002). Pranking with ranking. In *Advances in Neural Information Processing Systems 14*. MIT Press, pp. 641–647. URL http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.20.378.

[40] Cruz, F. L., Troyano, J. A., Enríquez, F., Ortega, F. J., and Vallejo, C. G. (2010). A knowledge-rich approach to feature-based opinion extraction from product reviews. In *Proceedings of the 2nd international workshop on Search and mining user-generated contents*, SMUC '10. New York, NY, USA: ACM, pp. 13–20. URL http://doi.acm.org/10.1145/1871985.1871990.

[41] de Marneffe, M.-C. and Manning, C. D. (2008). *Stanford Typed Dependencies Manual*. URL http://nlp.stanford.edu/software/dependencies_manual.pdf.

[42] Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., and Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391–407. URL http://lsi.argreenhouse.com/lsi/papers/JASIS90.ps.

[43] Ding, X. and Liu, B. (2007). The utility of linguistic rules in opinion mining. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '07. New York, NY, USA: ACM, pp. 811–812. URL http://doi.acm.org/10.1145/1277741.1277921.

[44] Ding, X., Liu, B., and Yu, P. S. (2008). A holistic lexicon-based approach to opinion mining. In M. Najork, A. Z. Broder, and S. Chakrabarti (Eds.), *First ACM International Conference on Web Search and Data Mining (WSDM)*. ACM, pp. 231–240. URL http://doi.acm.org/10.1145/1341531.1341561.

[45] Eckert, M., Clark, L., and Kessler, J. (2008). *Structural Sentiment and Entity Annotation Guidelines*. J. D. Power and Associates. URL https://www.cs.indiana.edu/~jaskessl/annotationguidelines.pdf.

[46] Esuli, A. and Sebastiani, F. (2005). Determining the semantic orientation of terms through gloss classification. In O. Herzog, H.-J. Schek, N. Fuhr, A. Chowdhury, and W. Teiken (Eds.), *Proceedings of the 2005 ACM CIKM International Conference on Information and Knowledge Management*. ACM, pp. 617–624. URL http://doi.acm.org/10.1145/1099554.1099713.

[47] Esuli, A. and Sebastiani, F. (2006). Determining term subjectivity and term orientation for opinion mining. In *EACL*. The Association for Computer Linguistics. URL http://acl.ldc.upenn.edu/E/E06/E06-1025.pdf.

[48] Esuli, A. and Sebastiani, F. (2006). SentiWordNet: A publicly available lexical resource for opinion mining. In *Proceedings of LREC-06, the 5th Conference on Language Resources and Evaluation*. Genova, IT. URL http://tcc.itc.it/projects/ontotext/Publications/LREC2006-esuli-sebastiani.pdf.

[49] Esuli, A., Sebastiani, F., Bloom, K., and Argamon, S. (2007). Automatically determining attitude type and force for sentiment analysis. In *LTC 2007*. URL http://lingcog.iit.edu/doc/argamon_ltc2007.pdf.

[50] Etzioni, O., Banko, M., and Cafarella, M. J. (2006). Machine reading. In *Proceedings of The Twenty-First National Conference on Artificial Intelligence and the Eighteenth Innovative Applications of Artificial Intelligence Conference*. AAAI Press. URL `http://www.cs.washington.edu/homes/etzioni/papers/aaai06.pdf`.

[51] Etzioni, O., Cafarella, M., Downey, D., Kok, S., Popescu, A.-M., Shaked, T., Soderland, S., Weld, D. S., and Yates, A. (2004). Web-scale information extraction in KnowItAll (preliminary results). In *Proceedings of the Thirteenth International World Wide Web Conference*. URL `http://wwwconf.ecs.soton.ac.uk/archive/00000552/01/p100-etzioni.pdf`.

[52] Etzioni, O., Cafarella, M. J., Downey, D., Popescu, A.-M., Shaked, T., Soderland, S., Weld, D. S., and Yates, A. (2005). Unsupervised named-entity extraction from the web: An experimental study. *Artif. Intell*, 165(1), 91–134. URL `http://dx.doi.org/10.1016/j.artint.2005.03.001`.

[53] Evans, D. K. (2007). A low-resources approach to opinon analysis: Machine learning and simple approaches. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 290–295.

[54] Feng, D., Burns, G., and Hovy, E. (2007). Extracting data records from unstructured biomedical full text. In *Proceedings of the 2007 Joint Conference on Emperical Methods in Natural Language Processing and Computational Natural Language Learning*. URL `http://acl.ldc.upenn.edu/D/D07/D07-1088.pdf`.

[55] Fiszman, M., Demner-Fushman, D., Lang, F. M., Goetz, P., and Rindflesch, T. C. (2007). Interpreting comparative constructions in biomedical text. In *Proceedings of the Workshop on BioNLP 2007: Biological, Translational, and Clinical Language Processing*, BioNLP '07. Stroudsburg, PA, USA: Association for Computational Linguistics, pp. 137–144. URL `http://portal.acm.org/citation.cfm?id=1572392.1572417`.

[56] Fleischman, M., Kwon, N., and Hovy, E. (2003). Maximum entropy models for framenet classification. In *EMNLP '03: Proceedings of the 2003 conference on Empirical methods in natural language processing*. Morristown, NJ, USA: Association for Computational Linguistics, pp. 49–56.

[57] Fletcher, J. and Patrick, J. (2005). Evaluating the utility of appraisal hierarchies as a method for sentiment classification. In *Proceedings of the Australasian Language Technology Workshop*. URL `http://alta.asn.au/events/altw2005/cdrom/pdf/ALTA200520.pdf`.

[58] Ganapathibhotla, M. and Liu, B. (2008). Mining opinions in comparative sentences. In D. Scott and H. Uszkoreit (Eds.), *COLING*. pp. 241–248. URL `http://www.aclweb.org/anthology/C08-1031.pdf`.

[59] Ghose, A., Ipeirotis, P. G., and Sundararajan, A. (2007). Opinion mining using econometrics: A case study on reputation systems. In *ACL*. The Association for Computer Linguistics. URL `http://aclweb.org/anthology-new/P/P07/P07-1053.pdf`.

[60] Gildea, D. and Jurafsky, D. (2002). Automatic labeling of semantic roles. URL `http://www.cs.rochester.edu/~gildea/gildea-cl02.pdf`.

[61] Godbole, N., Srinivasaiah, M., and Skiena, S. (2007). Large-scale sentiment analysis for news and blogs. In *Proceedings of the International Conference on Weblogs and Social Media (ICWSM)*. URL http://www.icwsm.org/papers/5--Godbole-Srinivasaiah-Skiena-demo.pdf.

[62] Gross, M. (1993). Local grammars and their representation by finite automata. In M. Hoey (Ed.), *Data, Description, Discourse: Papers on the English Language in honour of John McH Sinclair*. London: HarperCollins.

[63] Gross, M. (1997). The construction of local grammars. In E. Roche and Y. Schabes (Eds.), *Finite State Language Processing*. Cambridge, MA: MIT Press.

[64] Halliday, M. A. K. and Matthiessen, C. M. I. M. (2004). *An Introduction to Functional Grammar*. London: Edward Arnold, 3rd ed.

[65] Harb, A., Plantié, M., Dray, G., Roche, M., Trousset, F., and Poncelet, P. (2008). Web opinion mining: How to extract opinions from blogs? In *CSTST '08: Proceedings of the 5th International Conference on Soft Computing as Transdisciplinary Science and Technology*. New York, NY, USA: ACM, pp. 211–217. URL http://doi.acm.org/10.1145/1456223.1456269.

[66] Hatzivassiloglou, V. and McKeown, K. (1997). Predicting the semantic orientation of adjectives. In *ACL*. pp. 174–181. URL http://acl.ldc.upenn.edu/P/P97/P97-1023.pdf.

[67] Hobbs, J. R., Appelt, D., Bear, J., Israel, D., Kameyama, M., Stickel, M., and Tyson, M. (1997). FASTUS: A cascaded finite-state transducer for extracting information from natural-language text. In E. Roche and Y. Schabes (Eds.), *Finite State Language Processing*. Cambridge, MA: MIT Press. URL http://www.ai.sri.com/natural-language/projects/fastus-schabes.html.

[68] Hobbs, J. R., Appelt, D. E., Bear, J., Israel, D., and Tyson, M. (1992). *FASTUS: A System For Extracting Information From Natural-Language Text*. Tech. Rep. 519, AI Center, SRI International, 333 Ravenswood Ave., Menlo Park, CA 94025. URL http://www.ai.sri.com/pub_list/456.

[69] Hu, M. (2006). *Feature-based Opinion Analysis and Summarization*. Ph.D. thesis, University of Illinois at Chicago. URL http://proquest.umi.com/pqdweb?did=1221734561&sid=2&Fmt=2&clientId=2287&RQT=309&VName=PQD.

[70] Hu, M. and Liu, B. (2004). Mining and summarizing customer reviews. In *KDD '04: Proceedings of the tenth ACM SIGKDD international conference on Knowledge discovery and data mining*. New York, NY, USA: ACM, pp. 168–177. URL http://doi.acm.org/10.1145/1014052.1014073.

[71] Hunston, S. and Francis, G. (2000). *Pattern Grammar: A Corpus-Driven Approach to the Lexical Grammar of English*. Amsterdam: John Benjamins. URL http://citeseer.ist.psu.edu/hunston00pattern.html.

[72] Hunston, S. and Sinclair, J. (2000). A local grammar of evaluation. In S. Hunston and G. Thompson (Eds.), *Evaluation in Text: authorial stance and the construction of discourse*. Oxford, England: Oxford University Press, pp. 74–101.

[73] Hurst, M. and Nigam, K. (2004). Retrieving topical sentiments from online document collections. URL http://www.kamalnigam.com/papers/polarity-DRR04.pdf.

[74] Izard, C. E. (1971). *The Face of Emotion*. Appleton-Century-Crofts.

[75] Jakob, N. and Gurevych, I. (2010). Extracting opinion targets in a single and cross-domain setting with conditional random fields. In *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing*. Cambridge, MA: Association for Computational Linguistics, pp. 1035–1045. URL `http://www.aclweb.org/anthology/D10-1101`.

[76] Jakob, N. and Gurevych, I. (2010). Using anaphora resolution to improve opinion target identification in movie reviews. In *Proceedings of the ACL 2010 Conference Short Papers*. Uppsala, Sweden: Association for Computational Linguistics, pp. 263–268. URL `http://www.aclweb.org/anthology/P10-2049`.

[77] Jakob, N., Toprak, C., and Gurevych, I. (2008). *Sentiment Annotation in Consumer Reviews and Blogs*. Distributed with the Darmstadt Service Review Corpus.

[78] Jin, W. and Ho, H. H. (2009). A novel lexicalized HMM-based learning framework for web opinion mining. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09. New York, NY, USA: ACM, pp. 465–472. URL `http://doi.acm.org/10.1145/1553374.1553435`.

[79] Jin, X., Li, Y., Mah, T., and Tong, J. (2007). Sensitive webpage classification for content advertising. In *Proceedings of the 1st international workshop on Data mining and audience intelligence for advertising*, ADKDD '07. New York, NY, USA: ACM, pp. 28–33. URL `http://doi.acm.org/10.1145/1348599.1348604`.

[80] Jindal, N. and Liu, B. (2006). Mining comparative sentences and relations. In *AAAI*. AAAI Press. URL `http://www.cs.uic.edu/~liub/publications/aaai06-comp-relation.pdf`.

[81] Joachims, T. (2002). Optimizing search engines using clickthrough data. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. pp. 133–142. URL `http://www.cs.cornell.edu/People/tj/publications/joachims_02c.pdf`.

[82] Joachims, T. (2006). Training linear SVMs in linear time. In *KDD '06: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. New York, NY, USA: ACM, pp. 217–226. URL `http://www.cs.cornell.edu/People/tj/publications/joachims_06a.pdf`.

[83] Joshi, A. K. (1986). *An Introduction to Tree Adjoining Grammars*. Tech. Rep. MS-CIS-86-64, Department of Computer and Information Science, University of Pennsylvania.

[84] Kamps, J. and Marx, M. (2002). Words with attitude. In *1st International WordNet Conference*. Mysore, India, pp. 332–341. URL `http://staff.science.uva.nl/~kamps/papers/wn.pdf`.

[85] Kanayama, H. and Nasukawa, T. (2006). Fully automatic lexicon expansion for domain-oriented sentiment analysis. In *Proceedings of the 2006 Conference on Empirical Methods in Natural Language Processing*, EMNLP '06. Morristown, NJ, USA: Association for Computational Linguistics, pp. 355–363. URL `http://portal.acm.org/citation.cfm?id=1610075.1610125`.

[86] Kessler, J. S., Eckert, M., Clark, L., and Nicolov, N. (2010). The 2010 icwsm jdpa sentment corpus for the automotive domain. In *4th Int'l AAAI Conference on Weblogs and Social Media Data Workshop Challenge (ICWSM-DWC 2010)*. URL `http://www.cs.indiana.edu/~jaskessl/icwsm10.pdf`.

[87] Kessler, J. S. and Nicolov, N. (2009). Targeting sentiment expressions through supervised ranking of linguistic configurations. In *3rd Int'l AAAI Conference on Weblogs and Social Media (ICWSM 2009)*. URL `http://www.cs.indiana.edu/~jaskessl/icwsm09.pdf`.

[88] Kim, S.-M. and Hovy, E. (2005). Identifying opinion holders for question answering in opinion texts. In *Proceedings of AAAI-05 Workshop on Question Answering in Restricted Domains*. Pittsburgh, US. URL `http://ai.isi.edu/pubs/papers/kim2005identifying.pdf`.

[89] Kim, S.-M. and Hovy, E. (2006). Extracting opinions, opinion holders, and topics expressed in online news media text. In *Proceedings of ACL/COLING Workshop on Sentiment and Subjectivity in Text*. Sidney, AUS. URL `http://www.isi.edu/~skim/Download/Papers/2006/Topic_and_Holder_ACL06WS.pdf`.

[90] Kim, S.-M. and Hovy, E. H. (2007). Crystal: Analyzing predictive opinions on the web. In *EMNLP-CoNLL*. ACL, pp. 1056–1064. URL `http://www.aclweb.org/anthology/D07-1113`.

[91] Kim, Y., Kim, S., and Myaeng, S.-H. (2008). Extracting topic-related opinions and their targets in NTCIR-7. In *Proceedings of NTCIR-7*. URL `http://research.nii.ac.jp/ntcir/workshop/OnlineProceedings7/pdf/NTCIR7/C2/MOAT/09-NTCIR7-MOAT-KimY.pdf`.

[92] Kim, Y. and Myaeng, S.-H. (2007). Opinion analysis based on lexical clues and their expansion. In *Proceedings of NTCIR-6 Workshop Meeting*. URL `http://research.nii.ac.jp/ntcir/ntcir-ws6/OnlineProceedings/NTCIR/53.pdf`.

[93] Kipper-Schuler, K. (2005). *VerbNet: a broad-coverage, comprehensive verb lexicon*. Ph.D. thesis, Computer and Information Science Department, Universiy of Pennsylvania, Philadelphia, PA. URL `http://repository.upenn.edu/dissertations/AAI3179808/`.

[94] Ku, L.-W., Lee, L.-Y., and Chen, H.-H. (2006). Opinion extraction, summarization and tracking in news and blog corpora. In *Proceedings of AAAI-2006 Spring Symposium on Computational Approaches to Analyzing Weblogs*. URL `http://nlg18.csie.ntu.edu.tw:8080/opinion/SS0603KuLW.pdf`.

[95] Lakkaraju, H., Bhattacharyya, C., Bhattacharya, I., and Merugu, S. (2011). Exploiting coherence for the simultaneous discovery of latent facets and associated sentiments. In *SIAM International Conference on Data Mining*. URL `http://mllab.csa.iisc.ernet.in/html/pubs/FINAL.pdf`.

[96] Lenhert, W., Cardie, C., Fisher, D., Riloff, E., and Williams, R. (1991). Description of the CIRCUS system as used for MUC-3. Morgan Kaufmann. URL `http://acl.ldc.upenn.edu/M/M91/M91-1033.pdf`.

[97] Levi, G. and Sirovich, F. (1976). Generalized AND/OR graphs. *Artificial Intelligence*, 7(3), 243–259.

[98] Lexalytics Inc. (2011). Social media whitepaper. URL http://img.en25.com/Web/LexalyticsInc/lexalytics-social_media_whitepaper.pdf.

[99] Li, F., Han, C., Huang, M., Zhu, X., Xia, Y.-J., Zhang, S., and Yu, H. (2010). Structure-aware review mining and summarization. In *Proceedings of the 23rd International Conference on Computational Linguistics (Coling 2010)*. Beijing, China: Coling 2010 Organizing Committee, pp. 653–661. URL http://www.aclweb.org/anthology/C10-1074.

[100] Li, Y. and amd Hamish Cunningham, K. B. (2007). Experiments of opinion analysis on the corpora MPQA and NTCIR-6. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 323–329.

[101] Liu, B. (2009). Re: Sentiment analysis questions. Email to the author.

[102] Liu, B., Hu, M., and Cheng, J. (2005). Opinion observer: analyzing and comparing opinions on the web. In *WWW '05: Proceedings of the 14th international conference on World Wide Web*. New York, NY, USA: ACM, pp. 342–351. URL http://doi.acm.org/10.1145/1060745.1060797.

[103] Lloyd, L., Kechagias, D., and Skiena, S. (2005). Lydia: A System for Large-Scale News Analysis. In M. Consens and G. Navarro (Eds.), *String Processing and Information Retrieval*, *Lecture Notes in Computer Science*, vol. 3772, chap. 18. Berlin, Heidelberg: Springer Berlin / Heidelberg, pp. 161–166. URL http://dx.doi.org/10.1007/11575832_18.

[104] Macken-Horarik, M. (2003). Appraisal and the special instructiveness of narrative. *Text – Interdisciplinary Journal for the Study of Discourse*. URL http://www.grammatics.com/appraisal/textSpecial/macken-horarik-narrative.pdf.

[105] Mahanti, A. and Bagchi, A. (1985). AND/OR graph heuristic search methods. *J. ACM*, 32(1), 28–51. URL http://doi.acm.org/10.1145/2455.2459.

[106] Marcus, M. P., Santorini, B., and Marcinkiewicz, M. A. (1994). Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19. URL http://www.aclweb.org/anthology-new/J/J93/J93-2004.pdf.

[107] Marr, D. C. (1975). *Early Processing of Visual Information*. Tech. Rep. AIM-340, MIT Artificial Intelligence Laboratory. URL http://dspace.mit.edu/handle/1721.1/6241.

[108] Martin, J. H. (1996). Computational approaches to figurative language. *Metaphor and Symbolic Activity*, 11(1).

[109] Martin, J. R. (2000). Beyond exchange: Appraisal systems in English. In S. Hunston and G. Thompson (Eds.), *Evaluation in Text: authorial stance and the construction of discourse*. Oxford, England: Oxford University Press, pp. 142–175.

[110] Martin, J. R. and White, P. R. R. (2005). *The Language of Evaluation: Appraisal in English*. London: Palgrave. (http://grammatics.com/appraisal/).

[111] Mason, O. (2004). Automatic processing of local grammar patterns. In *Proceedings of the 7th Annual Colloquium for the UK Special Interest Group for Computational Linguistics*. URL http://www.cs.bham.ac.uk/~mgl/cluk/papers/mason.pdf.

[112] Mason, O. and Hunston, S. (2004). The automatic recognition of verb patterns: A feasibility study. *International Journal of Corpus Linguistics*, 9(2), 253–270. URL http://www.corpus4u.org/forum/upload/forum/2005062303222421.pdf.

[113] McCallum, A. (2002). MALLET: A machine learning for language toolkit. URL http://mallet.cs.umass.edu.

[114] McCallum, A. and Sutton, C. (2006). An introduction to conditional random fields for relational learning. In L. Getoor and B. Taskar (Eds.), *Introduction to Statistical Relational Learning*. MIT Press. URL http://www.cs.umass.edu/~mccallum/papers/crf-tutorial.pdf.

[115] McDonald, R. T., Hannan, K., Neylon, T., Wells, M., and Reynar, J. C. (2007). Structured models for fine-to-coarse sentiment analysis. In *ACL*. The Association for Computer Linguistics. URL http://aclweb.org/anthology-new/P/P07/P07-1055.pdf.

[116] Miao, Q., Li, Q., and Dai, R. (2008). An integration strategy for mining product features and opinions. In *Proceeding of the 17th ACM conference on Information and knowledge management*, CIKM '08. New York, NY, USA: ACM, pp. 1369–1370. URL http://doi.acm.org/10.1145/1458082.1458284.

[117] Miller, G. A. (1995). WordNet: A lexical database for English. *Commun. ACM*, 38(11), 39 –41. URL http://doi.acm.org/10.1145/219717.219748.

[118] Miller, M. L. and Goldstein, I. P. (1976). *PAZATN: A Linguistic Approach to Automatic Analysis of Elementary Programming Protocols*. Tech. Rep. AIM-388, MIT Artificial Intelligence Laboratory. URL http://dspace.mit.edu/handle/1721.1/6263.

[119] Miller, S., Crystal, M., Fox, H., Ramshaw, L., Schwartz, R., Stone, R., Weischedel, R., and Technologies), T. A. G. B. (1998). BBN: Description of the Sift system as used for MUC-7. In *Proceedings of the Seventh Message Understanding Conference (MUC-7)*. URL http://acl.ldc.upenn.edu/muc7/M98-0009.pdf.

[120] Mishne, G. and Glance, N. (2006). Predicting movie sales from blogger sentiment. *AAAI 2006 Spring Symposium on Computational Approaches to Analysing Weblogs (AAAI-CAAW 2006)*. URL http://staff.science.uva.nl/~gilad/pubs/aaai06-linkpolarity.pdf.

[121] Mishne, G. and Rijke, M. d. (2006). Capturing global mood levels using blog posts. In *AAAI 2006 Spring Symposium on Computational Approaches to Analysing Weblogs*. AAAI Press. URL http://ilps.science.uva.nl/Teaching/PIR0506/Projects/P8/aaai06-blogmoods.pdf.

[122] Mizuguchi, H., Tsuchida, M., and Kusui, D. (2007). Three-phase opinion analysis system at NTCIR-6. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 330–335.

[123] Mohri, M. (2005). Local grammar algorithms. In A. Arppe, L. Carlson, K. Lindèn, J. Piitulainen, M. Suominen, M. Vainio, H. Westerlund, and A. Yli-Jyrä (Eds.), *Inquiries into Words, Constraints, and Contexts. Festschrift in Honour of Kimmo Koskenniemi on his 60th Birthday*. Stanford University: CSLI Publications, pp. 84–93. URL http://www.cs.nyu.edu/~mohri/postscript/kos.pdf.

[124] Mullen, A. and Collier, N. (2004). Sentiment analysis using support vector machines with diverse information source. In *Proceedings of the 42nd Meeting of the Association for Computational Linguistics*. URL http://research.nii.ac.jp/~collier/papers/emnlp2004.pdf.

[125] Nakagawa, T., Inui, K., and Kurohashi, S. (2010). Dependency tree-based sentiment classification using crfs with hidden variables. In *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*. Los Angeles, California: Association for Computational Linguistics, pp. 786–794. URL http://www.aclweb.org/anthology/N10-1120.

[126] Neviarouskaya, A., Prendinger, H., and Ishizuka, M. (2010). Recognition of affect, judgment, and appreciation in text. In *Proceedings of the 23rd International Conference on Computational Linguistics (Coling 2010)*. Beijing, China: Coling 2010 Organizing Committee, pp. 806–814. URL http://www.aclweb.org/anthology/C10-1091.

[127] New York Times Editorial Bord (2011). The nation's cruelest immigration law. *New York Times*. URL http://www.nytimes.com/2011/08/29/opinion/the-nations-cruelest-immigration-law.html.

[128] Nigam, K. and Hurst, M. (2004). Towards a robust metric of opinion. In *Proceedings of the AAAI Spring Symposium on Exploring Attitude and Affect in Text*. URL http://www.kamalnigam.com/papers/metric-EAAT04.pdf.

[129] Nilsson, N. J. (1971). *Problem-solving methods in artificial intelligence*. New York: McGraw-Hill.

[130] Nivre, J. (2005). *Dependency Grammar and Dependency Parsing*. Tech. Rep. 05133, Växjö University: School of Mathematics and Systems Engineering. URL http://stp.lingfil.uu.se/~nivre/docs/05133.pdf.

[131] O'Hare, N., Davy, M., Bermingham, A., Ferguson, P., Sheridan, P., Gurrin, C., and Smeaton, A. F. (2009). Topic-dependent sentiment analysis of financial blogs. In *Proceeding of the 1st International CIKM Workshop on Topic-Sentiment Analysis for Mass Opinion Mining*, TSA '09. New York, NY, USA: ACM, pp. 9–16. URL http://doi.acm.org/10.1145/1651461.1651464.

[132] Osgood, C. E., Suci, G. J., and Tannenbaum, P. H. (1957). *The Measurement of Meaning*. University of Illinois Press. URL http://books.google.com/books?id=Qj8GeUrKZdAC.

[133] Pang, B. and Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2. URL http://dx.doi.org/10.1561/1500000011.

[134] Pang, B., Lee, L., and Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. In *Proceedings of EMNLP-02, the Conference on Empirical Methods in Natural Language Processing*. Philadelphia, US: Association for Computational Linguistics, pp. 79–86. URL http://www.cs.cornell.edu/home/llee/papers/sentiment.pdf.

[135] Pollard, C. and Sag, I. (1994). *Head-Driven Phrase Structure Grammar*. Chicago, Illinois: Chicago University Press.

[136] Popescu, A.-M. (2007). *Information extraction from unstructured web text*. Ph.D. thesis, University of Washington, Seattle, WA, USA. URL http://turing.cs.washington.edu/papers/popescu.pdf.

[137] Popescu, A.-M. and Etzioni, O. (2005). Extracting product features and opinions from reviews. In *Proceedings of HLT-EMNLP-05, the Human Language Technology Conference/Conference on Empirical Methods in Natural Language Processing*. Vancouver, CA. URL http://www.cs.washington.edu/homes/etzioni/papers/emnlp05_opine.pdf.

[138] Qiu, G., Liu, B., Bu, J., and Chen, C. (2009). Expanding domain sentiment lexicon through double propagation. In C. Boutilier (Ed.), *IJCAI*. pp. 1199–1204. URL http://ijcai.org/papers09/Papers/IJCAI09-202.pdf.

[139] Qiu, G., Liu, B., Bu, J., and Chen, C. (2011). Opinion word expansion and target extraction through double propagation. *Computational Linguistics*. To appear, URL http://www.cs.uic.edu/~liub/publications/computational-linguistics-double-propagation.pdf.

[140] Quirk, R., Greenbaum, S., Leech, G., and Svartvik, J. (1985). *A Comprehensive Grammar of the English Language*. Longman.

[141] Ramakrishnan, G., Chakrabarti, S., Paranjpe, D., and Bhattacharya, P. (2004). Is question answering an acquired skill? In *WWW '04: Proceedings of the 13th international conference on World Wide Web*. New York, NY, USA: ACM, pp. 111–120. URL http://doi.acm.org/10.1145/988672.988688.

[142] Ratnaparkhi, A. (1996). A maximum entropy model for part-of-speech tagging. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. URL http://citeseer.ist.psu.edu/581830.html.

[143] Riloff, E. (1996). An empirical study of automated dictionary construction for information extraction in three domains. URL http://www.cs.utah.edu/~riloff/psfiles/aij.ps.

[144] Ruppenhofer, J., Ellsworth, M., Petruck, M. R. L., Johnson, C. R., and Scheffczyk, J. (2005). *FrameNet II: Extended Theory and Practice*. Tech. rep., ICSI. URL http://framenet.icsi.berkeley.edu/book/book.pdf.

[145] Seki, Y. (2007). Crosslingual opinion extraction from author and authority viewpoints at NTCIR-6. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 336–343.

[146] Seki, Y., Evans, D. K., Ku, L.-W., Chen, H.-H., Kando, N., and Lin, C.-Y. (2007). Overview of opinion analysis pilot task at NTCIR-6. In *Proceedings of NTICR-6*. URL http://nlg18.csie.ntu.edu.tw:8080/opinion/ntcir6opinion.pdf.

[147] Seki, Y., Evans, D. K., Ku, L.-W., Sun, L., Chen, H.-H., and Kando, N. (2008). Overview of multilingual opinion analysis task at NTCIR-7. In *Proceedings of NTCIR-7*. URL http://research.nii.ac.jp/ntcir/workshop/OnlineProceedings7/pdf/revise/01-NTCIR-OV-MOAT-SekiY-revised-20081216.pdf.

[148] Seki, Y., Ku, L.-W., Sun, L., Chen, H.-H., and Kando, N. (2010). Overview of multilingual opinion analysis task at NTCIR-8. In *Proceedings of NTICR-8*. URL http://research.nii.ac.jp/ntcir/workshop/OnlineProceedings8/NTCIR/01-NTCIR8-OV-MOAT-SekiY.pdf.

[149] Shen, L. and Joshi, A. K. (2005). Ranking and reranking with perceptron. *Mach. Learn.*, 60, 73–96. URL http://libinshen.net/Documents/mlj05.pdf.

[150] Shen, L., Sarkar, A., and Och, F. J. (2004). Discriminative reranking for machine translation. In *HLT-NAACL*. pp. 177–184. URL http://acl.ldc.upenn.edu/hlt-naacl2004/main/pdf/121_Paper.pdf.

[151] Sinclair, J. (Ed.) (1995). *Collins COBUILD English Dictionary*. Glasgow: HarperCollins, 2nd ed.

[152] Sinclair, J. (Ed.) (1995). *Collins COBUILD English Dictionary for Advanced Learners*. Glasgow: HarperCollins.

[153] Sleator, D. and Temperley, D. (1991). *Parsing English with a Link Grammar*. Tech. Rep. CMU-CS-91-196, Carnegie-Mellon University. URL http://www.cs.cmu.edu/afs/cs.cmu.edu/project/link/pub/www/papers/ps/tr91-196.pdf.

[154] Sleator, D. and Temperley, D. (1993). Parsing English with a link grammar. In *Third International Workshop on Parsing Technologies*. URL http://www.cs.cmu.edu/afs/cs.cmu.edu/project/link/pub/www/papers/ps/LG-IWPT93.pdf.

[155] Snyder, B. and Barzilay, R. (2007). Multiple aspect ranking using the good grief algorithm. In *Human Language Technologies 2007: The Conference of the North American Chapter of the Association for Computational Linguistics; Proceedings of the Main Conference*. Rochester, New York: Association for Computational Linguistics, pp. 300–307. URL http://www.aclweb.org/anthology/N/N07/N07-1038.pdf.

[156] Sokolova, M. and Lapalme, G. (2008). Verbs as the most affective words. In *Proceedings of the International Symposium on Affective Language in Human and Machine*. UK, Scotland, Aberdeen, pp. 73–76. URL http://rali.iro.umontreal.ca/Publications/files/VerbsAffect2.pdf.

[157] Spertus, E. (1997). Smokey: automatic recognition of hostile messages. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence and Ninth Conference on Innovative Applications of Artificial Intelligence*, AAAI'97/IAAI'97. AAAI Press, pp. 1058–1065. URL http://portal.acm.org/citation.cfm?id=1867406.1867616.

[158] Spertus, E. (1997). Smokey: Automatic recognition of hostile messages. In *Proceedings of the 14th National Conference on Artificial Intelligence and 9th Innovative Applications of Artificial Intelligence Conference (AAAI-97/IAAI-97)*. Menlo Park: AAAI Press, pp. 1058–1065. URL http://www.ai.mit.edu/people/ellens/smokey.ps.

[159] Stoffel, D., Kunz, W., and Gerber, S. (1995). *AND/OR Graphs*. Tech. rep., University of Potsdam. URL http://www.mpag-inf.uni-potsdam.de/reports/MPI-I-95-602.ps.gz.

[160] Stone, P. J., Dunphy, D. C., Smith, M. S., and Ogilvie, D. M. (1966). *The General Inquirer: A Computer Approach to Content Analysis*. MIT Press. URL http://www.webuse.umd.edu:9090/.

[161] sun Choi, K. and sun Nam, J. (1997). A local-grammar based approach to recognizing of proper names in Korean texts. In J. Zhou and K. Church (Eds.), *Proceedings of the Fifth Workshop on Very Large Corpora*. URL http://citeseer.ist.psu.edu/551967.html.

[162] Swartout, W. R. (1978). *A Comparison of PARSIFAL with Augmented Transition Networks*. Tech. Rep. AIM-462, MIT Artificial Intelligence Laboratory. URL http://dspace.mit.edu/handle/1721.1/6289.

[163] Taboada, M. (2008). Appraisal in the text sentiment project. URL http://www.sfu.ca/~mtaboada/research/appraisal.html.

[164] Taboada, M. and Grieve, J. (2004). Analyzing appraisal automatically. In *Proceedings of the AAAI Spring Symposium on Exploring Attitude and Affect in Text*. URL http://www.sfu.ca/~mtaboada/docs/TaboadaGrieveAppraisal.pdf.

[165] Tatemura, J. (2000). Virtual reviewers for collaborative exploration of movie reviews. In *Proceedings of the 5th international conference on Intelligent user interfaces*, IUI '00. New York, NY, USA: ACM, pp. 272–275. URL http://doi.acm.org/10.1145/325737.325870.

[166] Thompson, G. and Hunston, S. (2000). Evaluation: An introduction. In S. Hunston and G. Thompson (Eds.), *Evaluation in Text: Authorial Stance and the Construction of Discourse*. Oxford, England: Oxford University Press, pp. 1–27.

[167] Thurstone, L. (1947). *Multiple-factor analysis: a development and expansion of the vectors of the mind*. The University of Chicago Press. URL http://books.google.com/books?id=p4swAAAAIAAJ.

[168] Toprak, C., Jakob, N., and Gurevych, I. (2010). Sentence and expression level annotation of opinions in user-generated discourse. In *ACL '10: Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*. Morristown, NJ, USA: Association for Computational Linguistics, pp. 575–584. URL http://www.ukp.tu-darmstadt.de/fileadmin/user_upload/Group_UKP/publikationen/2010/CameraReadyACL2010OpinionAnnotation.pdf.

[169] Turmo, J., Ageno, A., and Català, N. (2006). Adaptive information extraction. *ACM Computing Surveys*, 38(2), 4. URL http://doi.acm.org/10.1145/1132956.1132957.

[170] Turney, P. D. (2002). Thumbs up or thumbs down? Semantic orientation applied to unsupervised classification of reviews. In *ACL*. pp. 417–424. URL http://www.aclweb.org/anthology/P02-1053.pdf.

[171] Turney, P. D. and Littman, M. L. (2003). Measuring praise and criticism: Inference of semantic orientation from association. *ACM Trans. Inf. Syst.*, 21(4), 315–346. URL http://doi.acm.org/10.1145/944013.

[172] Venkova, T. (2001). A local grammar disambiguator of compound conjunctions as a pre-processor for deep analysers. In *Proceedings of Workshop on Linguistic Theory and Grammar Implementation*. URL http://citeseer.ist.psu.edu/459916.html.

[173] Whitelaw, C., Garg, N., and Argamon, S. (2005). Using appraisal taxonomies for sentiment analysis. In *ACM SIGIR Conference on Information and Knowledge Management*. URL http://lingcog.iit.edu/doc/appraisal_sentiment_cikm.pdf.

[174] Wiebe, J. (1994). Tracking point of view in narrative. *Computational Linguistics*, 20(2), 233–287. URL http://acl.ldc.upenn.edu/J/J94/J94-2004.pdf.

[175] Wiebe, J. and Bruce, R. (1995). Probabilistic classifiers for tracking point of view. In *Working Notes of the AAAI Spring Symposium on Empirical Methods in Discourse Interpretation*. URL http://citeseer.ist.psu.edu/421637.html.

[176] Wiebe, J. and Riloff, E. (2005). Creating subjective and objective sentence classifiers from unannotated texts. In A. F. Gelbukh (Ed.), *Proceedings of the Sixth International Conference on Computational Linguistics and Intelligent Text (CICLing)*, *Lecture Notes in Computer Science*, vol. 3406. Springer, pp. 486–497. URL http://www.cs.pitt.edu/~wiebe/pubs/papers/cicling05.pdf.

[177] Wiebe, J. and Riloff, E. (2005). Creating subjective and objective sentence classifiers from unannotated texts. In *Proceeding of CICLing-05, International Conference on Intelligent Text Processing and Computational Linguistics.*, *Lecture Notes in Computer Science*, vol. 3406. Mexico City, MX: Springer-Verlag, pp. 475–486. URL http://www.cs.pitt.edu/~wiebe/pubs/papers/cicling05.pdf.

[178] Wiebe, J. and Wilson, T. (2002). Learning to disambiguate potentially subjective expressions. In *COLING-02: proceedings of the 6th conference on Natural language learning*. Morristown, NJ, USA: Association for Computational Linguistics, pp. 1–7. URL http://dx.doi.org/10.3115/1118853.1118887.

[179] Wiebe, J., Wilson, T., and Cardie, C. (2005). Annotating expressions of opinions and emotions in language. *Language Resources and Evaluation*, 39(2–3), 165–210. URL http://www.cs.pitt.edu/~wiebe/pubs/papers/lre05withappendix.pdf.

[180] Wilson, T., Wiebe, J., and Hoffmann, P. (2005). Recognizing contextual polarity in phrase-level sentiment analysis. In *Proceedings of Human Language Technologies Conference/Conference on Empirical Methods in Natural Language Processing (HLT/EMNLP 2005)*. Vancouver, CA. URL http://www.cs.pitt.edu/~twilson/pubs/hltemnlp05.pdf.

[181] Wilson, T., Wiebe, J., and Hoffmann, P. (2009). Recognizing contextual polarity: An exploration of features for phrase-level sentiment analysis. *Computational Linguistics*. http://www.mitpressjournals.org/doi/pdf/10.1162/coli.08-012-R1-06-90, URL http://www.mitpressjournals.org/doi/abs/10.1162/coli.08-012-R1-06-90.

[182] Wilson, T., Wiebe, J., and Hwa, R. (2006). Recognizing strong and weak opinion clauses. *Computational Intelligence*, 22(2), 73–99. URL http://www.cs.pitt.edu/~wiebe/pubs/papers/ci06.pdf.

[183] Wilson, T. A. (2008). *Fine-grained Subjectivity and Sentiment Analysis: Recognizing the Intensity, Polarity, and Attitudes of Private States*. Ph.D. thesis, University of Pittsburgh. URL http://homepages.inf.ed.ac.uk/twilson/pubs/TAWilsonDissertationApr08.pdf.

[184] Woods, W. A. (1970). Transition network grammars for natural language analysis. *Commun. ACM*, 13, 591–606. URL http://doi.acm.org/10.1145/355598.362773.

[185] Wu, Y. and Oard, D. (2007). NTCIR-6 at Maryland: Chinese opinion analysis pilot task. In *Proceedings of NTCIR-6 Workshop Meeting*. pp. 344–349. URL http://research.nii.ac.jp/ntcir/ntcir-ws6/OnlineProceedings/NTCIR/44.pdf.

[186] Yangarber, R., Grishman, R., Tapanainen, P., and Huttunen, S. (2000). Automatic acquisition of domain knowledge for information extraction. In *COLING*. Morgan Kaufmann, pp. 940–946. URL http://acl.ldc.upenn.edu/C/C00/C00-2136.pdf.

[187] Yu, N. and Kübler, S. (2011). Filling the gap: Semi-supervised learning for opinion detection across domains. In *CoNLL '11: Proceedings of the Fifteenth Conference on Computational Natural Language Learning*. pp. 200–209. URL http://www.aclweb.org/anthology/W11-0323.

[188] Zagibalov, T. and Carroll, J. (2008). Unsupervised classification of sentiment and objectivity in Chinese text. In *Proceedings of the Third International Joint Conference on Natural Language Processing (IJCNLP)*. URL http://www.aclweb.org/anthology-new/I/I08/I08-1040.pdf.

[189] Zhang, L. (2009). An intelligent agent with affect sensing from metaphorical language and speech. In *Proceedings of the International Conference on Advances in Computer Enterntainment Technology*, ACE '09. New York, NY, USA: ACM, pp. 53–60. URL http://doi.acm.org/10.1145/1690388.1690398.

[190] Zhang, L., Barnden, J., Hendley, R., and Wallington, A. (2006). Developments in affect detection from text in open-ended improvisational e-drama. In Z. Pan, R. Aylett, H. Diener, X. Jin, S. Gbel, and L. Li (Eds.), *Technologies for E-Learning and Digital Entertainment*, *Lecture Notes in Computer Science*, vol. 3942. Springer Berlin / Heidelberg, pp. 368–379. URL http://dx.doi.org/10.1007/11736639_48.

[191] Zhang, Q., Wu, Y., Li, T., Ogihara, M., Johnson, J., and Huang, X. (2009). Mining product reviews based on shallow dependency parsing. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, SIGIR '09. New York, NY, USA: ACM, pp. 726–727. URL http://doi.acm.org/10.1145/1571941.1572098.

[192] Zhuang, L., Jing, F., and Zhu, X.-Y. (2006). Movie review mining and summarization. In *CIKM '06: Proceedings of the 15th ACM international conference on Information and knowledge management*. New York, NY, USA: ACM, pp. 43–50. URL http://doi.acm.org/10.1145/1183614.1183625.